

SPCA107N

POSTGRADUATE COURSE
Master of Computer Applications

FIRST YEAR

FIRST SEMESTER

ELECTIVE PAPER - I

OPERATING SYSTEMS



INSTITUTE OF DISTANCE EDUCATION
UNIVERSITY OF MADRAS

WELCOME

Warm Greetings.

It is with a great pleasure to welcome you as a student of Institute of Distance Education, University of Madras. It is a proud moment for the Institute of Distance education as you are entering into a cafeteria system of learning process as envisaged by the University Grants Commission. Yes, we have framed and introduced as per AICTE Choice Based Credit System(CBCS) in Semester pattern from the academic year 2020-21. You are free to choose courses, as per the Regulations, to attain the target of total number of credits set for each course and also each degree programme. What is a credit? To earn one credit in a semester you have to spend 30 hours of learning process. Each course has a weightage in terms of credits. Credits are assigned by taking into account of its level of subject content. For instance, if one particular course or paper has 4 credits then you have to spend 120 hours of self-learning in a semester. You are advised to plan the strategy to devote hours of self-study in the learning process. You will be assessed periodically by means of tests, assignments and quizzes either in class room or laboratory or field work. In the case of PG (UG), Continuous Internal Assessment for 20(25) percentage and End Semester University Examination for 80 (75) percentage of the maximum score for a course / paper. The theory paper in the end semester examination will bring out your various skills: namely basic knowledge about subject, memory recall, application, analysis, comprehension and descriptive writing. We will always have in mind while training you in conducting experiments, analyzing the performance during laboratory work, and observing the outcomes to bring out the truth from the experiment, and we measure these skills in the end semester examination. You will be guided by well experienced faculty.

I invite you to join the CBCS in Semester System to gain rich knowledge leisurely at your will and wish. Choose the right courses at right times so as to erect your flag of success. We always encourage and enlighten to excel and empower. We are the cross bearers to make you a torch bearer to have a bright future.

With best wishes from mind and heart,

DIRECTOR

**MCA
FIRST YEAR - FIRST SEMESTER**

**ELECTIVE PAPER - I
OPERATING SYSTEMS**

COORDINATION & EDITING

Dr. S. Sasikala

Assistant Professor in Computer Science
Institute of Distance Education
University of Madras
Chepauk Chennai - 600 005.

MCA
FIRST YEAR
FIRST SEMESTER
Elective Paper - I
OPERATING SYSTEMS
SYLLABUS

Objective of the course: This course introduces the fundamental concepts of operating systems.

Course Outcomes: After successful completion of this course, the students should be able to understand the behavior of the operating system.

Unit-I: Introduction – Multiprogramming - Time sharing - Distributed system - Real-Time systems - I/O structure - Dual-mode operation - Hardware protection _ General system architecture - Operating system services - System calls - System programs - System design and implementation. Process Management: Process concept - Concurrent process - Scheduling concepts - CPU scheduling - Scheduling algorithms, Multiple processor Scheduling

Unit-II: Process Management: Process Synchronization - Critical section - Synchronization hardware - Semaphores, classical problem of synchronization, Interprocess communication. Deadlocks: Characterization, Prevention, Avoidance, and Detection.

Unit-III: Storage management - Swapping, single and multiple partition allocation - paging - segmentation - paged segmentation, virtual memory - demand paging - page replacement and algorithms, thrashing. Secondary storage management - disk structure - free space management - allocation methods – disk scheduling - performance and reliability improvements - storage hierarchy.

Unit-IV: Files and protection - file system organization - file operations - access methods - consistency semantics - directory structure organization - file protection - implementation issues - security – encryption

Unit-V: Case Studies: UNIX and Windows operating systems.

Recommended Texts

- 1) A. Silberschatz P.B. Galvin, Gange, 2002, Operating System Concepts, 6th Edn., Addison-Wesley Publishing Co., Boston.

Reference Books

- 1) H.M. Deitel, 1990, An Introduction to Operating Systems, Addison Wesley Publishing Co., Boston.
- 2) D.M. Dhamdhare , 2002, Operating System, Tata McGraw-Hill, New Delhi.
- 3) A.S. Tanenbaum , Operating Systems: Design and Implementation, Prentice-Hall of India, New Delhi.
- 4) Nutt, 2005, Operating Systems, 3 rd Edition, Pearson Education, Delhi.

MCA
FIRST YEAR
FIRST SEMESTER
Elective Paper - I
OPERATING SYSTEMS
SCHEME OF LESSONS

Sl.No.	Title	Page
1.	History of Operating Systems	1
2.	The File System, Device Driver and Terminal I/O	16
3.	Process Management and Inter-Process Management	37
4.	Dead-locks	71
5.	Memory Management (MM)	81
6.	Paging	98
7.	Security Protection	117
8.	Encryption	142
9.	Windows Technology	155
10.	Requirements of a windows based GUI, MS Windows and Windows NT	167
11.	UNIX - History & Overview	177
12.	Data Structures for Process / Memory Management	210
13.	Process Scheduling	242

LESSON - 1

HISTORY OF OPERATING SYSTEMS

Structure

- 1.1 Introduction
- 1.2 Objectives
- 1.3 Zeroeth Generation: Mechanical Parts
- 1.4 First Generation: Vacuum Tubes
- 1.5 Second Generation: Transistors
- 1.6 Third Generation: Integrated Circuits
- 1.7 Fourth Generation: Large Scale Integration
- 1.8 Summary
- 1.9 Review Questions

1.1 Introduction

An operating system acts as an intermediary between the user of a computer and computer hardware. The purpose of an operating system is to provide an environment in which a user can execute programs in a convenient and efficient manner.

An operating system is a software that manages the computer hardware. The hardware must provide appropriate mechanisms to ensure the correct operation of the computer system and to prevent user programs from interfering with the proper operation of the system.

Operating Systems have evolved over the years. So, their evolution through the years can be mapped using generations of operating systems. There are four generations of operating systems. In this lesson, we will discuss about the generation, development and some basic concepts of Operating Systems.

1.2 Objectives

After studying this lesson you should be able to

- * Discuss about the origin of the operating system and their subsequent developments.
- * Describe developments on the basis of generations of computers
- * Introduces the concepts of Spooling, Timesharing, Multiprogramming, IOCS and so on.

The history of the operating systems is inextricable linked with the history and development of various generations of computer system.

1.3 Zeroeth Generation : Mechanical Parts

The first digital computer was designed by Charles Babbage (1732-1871), an English mathematician. This machine has a mechanical design where wheels, gears, cogs and so on were used.

As this computer was slow and unreliable, this design could not really become very popular. There was no question of any operating system of any kind for this machine.

1.4 First Generation (1945-1955): Vacuum Tubes

Several decades later, a solution evolved which was electronic rather than mechanical. This solution emerged out of the concerted research carried out as part of the war effort during the Second World War. Around 1945, However Aiken at Harvard, John Von Neumann at Princeton, J. Eckert and William Manchely at the University of Pennsylvania and K. Zuse in Germany succeeded in designing calculating machines with vacuum tubes as the central components.

These machines were huge and their continued usage generated a great deal of heat. The vacuum tubes also used to get burnt very fast (During one computer run, as many as 10000-20000 tubes could be wasted). The programming was done only in machine language, not any higher level language. Again, there was no operating system for these machines. These were single-user machines, which were extremely unfriendly to users/programmers.

1.5 Second Generation (1955-1965): Transistors

Around 1955, transistors were introduced in the U.S.A. at AT & T. The problems associated with vacuum tubes vanished overnight. The size and the cost of the machine dramatically dwindled. The reliability improved. For the first time, new categories of professionals called system analysts, designers, programmers and operators came into being as distinct entries. Until then, the functions handled by these categories of people had been managed by a single individual.

Assembly language, as a second generation language, and FORTRAN as a high level language (third generation language) emerged and the programmer's job was extremely simplified.

However, these were batch systems, The IBM-1401 belonged to that era. There was no question of having multiple terminals attached to the machine, carrying out different inquiries. The operator was continuously busy loading or unloading cards and tapes before and after the jobs. At a time, only one job could run. At the end of one job, the operator had to dismount the tapes, take out the cards ('teardown Operation') load the decks of cards and mount the tapes for the new job ('setup operation'). This entailed the usage of a lot of computer time, Valuable CPU time was therefore, wasted. An improvement came when IBM-7094 - a faster and larger computer was used in conjunction with IBM-1401, which then was used as a 'satellite computer' . The scheme used to work as follows:

- i) There used to be 'control cards' giving information about the job, the user, and so on, sequentially stacked. For instance, \$JOB specified the job to be done, the user who is doing it and may be some other information \$LOAD signified that what would follow were the cards with executable machine instructions punched onto them and that they were to be loaded in the main memory before it could be executed. These cards were therefore, collectively known as an 'object deck' or an 'object program'. When the programmer wrote his program in an assembly language called a 'source program', the assembly process carried out by a special program called 'assembler' would convert it into an object program before it could be executed. The assembler would also punch these machine instructions on the cards in a predefined format. For instance, each card had a sequence number to help it to be rearranged in case it fell out by mistake. The column in which the 'op code' of the machine instruction started was also fixed (e.g. column 16 in the case of Autocoder), so that the loader could do its job easily and quickly . The &LOAD card would essentially signify that the object cards following it should then be loaded in the memory. Obviously,

the object card would specify that the program just then loaded should be executed by branching to the first executable instruction specified by the programmer in the 'ORG' statement. The program might need some data cards which then followed. SEND specified the end of the data cards and &JOB specified the beginning of a new job again.

- ii) An advantage of stacking these cards together to reduce the efforts of the operator is set up and teardown' operations and therefore to save precious CPU time. Therefore, many such jobs were stacked together one after the other.
- iii) All these cards were then read one by one and copied onto a tape using a 'card to tape' utility program. This was done on an IBM 1401 which was used as a satellite computer. Controls such as 'total number of card read' were developed and printed by the utility program at the end of the job to ensure that all cards were read.
- iv) The prepared tape was taken to the main 7094 computer and processed. The printed reports were not actually printed on 7094, but the print image was dumped onto the tape which was carried to slower 1401 computer again, which did the final printing. Due to this procedure, 7094 computer, which was a faster and more expensive machine was not locked up for a long time unnecessarily.

The logic of splitting the operation of printing into two stages here was simple. The CPU of a computer was quite fast as compared to any I/O operation. This was so, because the CPU was a purely electronic device, whereas I/O involved electromechanical operations. Secondly, within two types of I/O operations, writing on a tape was faster than printing a line on paper. Therefore, the time of the more powerful, more expensive 7094 was saved. This is because, the CPU can execute only one instruction at a time.

1.6 Third Generation (1965-1980): Integrated circuits

IBM announced system / 360 series of computers in 1964. IBM had designed various computers in this series which were mutually compatible so that the conversion efforts for programs from one machine to the other in the same family were minimal. This is how the concept of "family of computers" came into being. IBM-370, 43xx, and 30xx systems belong to the same family of computers. IBM faced the problem of converting the existing 1401 users to system / 360, and there were many. IBM provided the customers with utilities such as 'simulators' (totally software driven and therefore, a little slow) and 'emulators' (using hardware modifications to enhance the speed at extra cost) to enable the old 1401 based software to run on the IBM 360 family of computers.

Initially, IBM had plans for delivering only one operating system for all the computers in the family. However, this approach proved to be practically difficult and cumbersome. The operating system for the larger computer in the family meant to manage larger resources was found to create far more burden and overhead if used on the smaller computers. Again, the operating system that could run efficiently on a smaller computer would not manage the resources for a large computer effectively. At least, IBM thought so at that time. Therefore, IBM was forced to deliver four operating systems within the same range of computers. These were:

cp-67/CMS for the powerful 360/67,
 using virtual storage
 OS/MVT for the bigger 360 systems
 OS/MFT for the medium 360 systems
 DOS/360 for the small 360 systems

The major advantage/ features and problems of this computer family and its operating systems were as follows:

(a) Integrated Circuits

The system/360 was based on Integrated Circuits (ICs)' rather than transistors. With ICs, the cost and the size of the computer shrank substantially, and yet the performance improved.

(b) Portability

The operating systems for the system/360 were written in assembly language. The routines were therefore, complex and time-consuming to write and maintain. Many bugs persisted for a long time. As these were written for a specific machine and in the assembly language of that machine, they were tied to the hardware. They were not easily 'portable' to machines with a different architecture not belonging to the same family.

(c) Job Control Language

Despite these problems, the user found them acceptable, because, the operator intervention (for setup and teardown) decreased. A JOB Control Language (JCL) was developed to allow communication between the user / programmer and the computer along with its operating system. By using the JCL, a user programmer could instruct the computer and its operating system. By using the JCL, a user/programmer could instruct the computer and its operating system. By using the JCL, user/programmer could instruct the computer and its operating system to perform certain tasks, in a specific sequence of creating a file, running a job or sorting a file.

(d) Multiprogramming

The operating systems supported mainly batch programs but it made multiprogramming very popular. This was a major contribution. The physical memory was divided into many partitions, each holding a separate program. One of these partitions was holding the operating system as shown in Fig. 1.1.

However, because, there was only one CPU, at the time only one program could be executed. Therefore, there was a need for a mechanism to switch the CPU from one program to the next. This is exactly what the operating system provided. One of the major advantages of this scheme was the increase in the throughput. If the same three programs shown in Fig. 1.1 were to run one after the other, the total elapsed time would have been much more than under a scheme which used multiprogramming. The reason was simple. In a uniprogramming environment, the CPU was idle when any I/O for any program was going on the CPU was 'switched' to another program. This allowed for the overlapped operations of I/O for one program and the other processing for some other program by the CPU, thereby increasing the throughput.

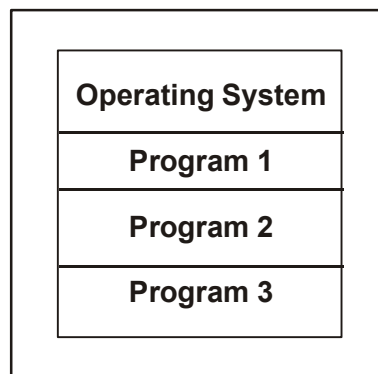


Fig. 1.1 – Physical memory in multiprogramming

(e) Spooling

The concept of 'simultaneously Peripheral Operations On-Line (spool)' was fully developed during this period. This was the outgrowth of the same principle that was used in the scheme discussed earlier. The only advantage of spooling was that you no longer had to carry tapes to and fro the 1401 and 7049 machines. Under the new operating system, all jobs in the form of cards could be, read in to the disk first (shown as 'a' in fig.1.2) and later on, the operating system would load as many jobs in the memory, one after the other, until the available memory could accommodate them, the CPU was switched from one program to another to achieve

multiprogramming. We will later see different policies used to achieve this switching. Similarly, whenever any program printed something, it was not written directly onto the disk in the area reserved for spooling. At any convenient time later, the actual printing from this disk file could be undertaken. Spooling had two distinct advantages. In the first place, it allowed smooth multiprogramming operations. Imagine if two programs, say, Store's Ledger and Payslips printing, were allowed to issue simultaneous instructions to write directly on the printer, the kind of hilarious report that would be produced with intermingled lines from both the reports on the same page. Instead, the print images of both the reports were written on to the disk at two different locations of the spool file first, and spooler program subsequently printed them one by one. Therefore, while printing, the printer was allocated only to the spooler program. In order to guide this subsequent printing process, the print image copy of the report on the disk also contained some preknown special characters such as for skipping a page. These were interpreted by the Spooler program at the time of producing the actual report. Spooling had another advantage. All the I/O of all the jobs was essentially pooled together in the spooling method and therefore, this could be overlapped with the CPU bound computations of all the jobs at an appropriate time chosen by the operating system to improve the throughput.

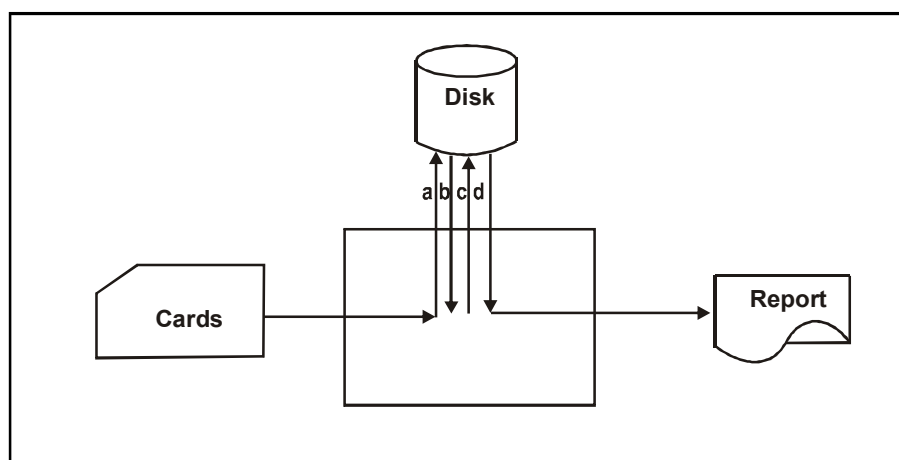


Fig. 1.2 Spooling

f) Time Sharing

The system / 360 with its Operating systems enhanced multiprogramming, but the operating systems were not geared to meet the requirements of interactive users. They were not very suitable for the query systems for example.

The reason was simple. In interactive systems, the operating system needs to recognize a terminal as an input medium. In addition, the operating system has to give priority to the interactive processes over batch processes. For instance, if you fire a query on the terminal, “what is the flight time of Flight SQ024?”, and the passenger has to be serviced within a brief time interval, the operating system must give higher priority to this process than, say, for a payroll program running in the batch mode. The classical ‘multiprogramming batch’ operating system did not provide for this kind of scheduling of various processes.

A change was needed. IBM responded by giving its users a program called ‘Customer Information Control System (CICS)’ which essentially provided “Data communication (DC)” facility between the terminal and the computer. It also scheduled various interactive users’ jobs on top of the operating system. Therefore, CICS functioned not only as a Transaction Processing (TP) monitor but also took over some functions of the operating system such as scheduling. IBM also provided the users with the “Time Sharing Option (TSO) Software” later to deal with the situation.

Many other vendors came up with “Time Sharing Operating Systems” during the same period. DEC came up with TOPS-10 on the DEC-10 machine. RSTS/E and RSX-11M for the PDP-11 family of computers and VMS for the VAX-11 family of computers. Data General produced AOS for its 16 bit minicomputers and VMS for the VAX-11 family of computers. Data General produced AOS for its 16 bit minicomputers and AOS/VS for its 32 bit Super-mini-computers.

These operating systems could learn from the good/bad points of the operating system running on the system/360. Most of these were far more user/programmer friendly. Terminal handling was in-built in the operating system. These operating systems provided for batch as well as on-line jobs by allowing both to coexist and compete for the resources, but giving higher performance for servicing the on-line requests.

One of the first time sharing systems was ‘Compatible Time Sharing System (CTSS)’ developed at the Massachusetts Institute of technology (MIT). It was used on the IBM 7094 and it supported a large number of interactive users. Time sharing became popular at once.

“Multiplexed Information and Computing Service (MULTICS)” was the next one to follow. It was a joint effort of MIT, BELL Labs and General Electric. The aim was to create a ‘computer utility’ which could support hundreds of simultaneous time sharing users.

MULTICS was a crucible which generated and tested almost all the important ideas and algorithms which were to be used repeatedly over several years in many operating systems. But the development of MULTICS itself was very painful and expensive. Finally, Bell Labs withdrew from the project. In fact, in the process, GE gave up its computer business altogether. Despite its relative failure, MULTICS had a tremendous influence on the design of an operating system for many years to come.

One of the computer scientists Ken Thompson - working on the MULTICS project through Bell Labs subsequently got hold of a PDP-7 Machine which was unused. Bell labs had already withdrawn from MULTICS> Another computer scientist Brian Kernighan - started calling this system “unics”. Later on the name UNIX was adopted. The UNIX operating system was later ported to a larger machine, PDP 11/45.

There were, however, major problems in this porting. The problems arose because UNIX was written in the assembly language. A more adventurous idea struck another computer scientist - Dennis Ritchie that of writing UNIX in a higher level languages (HLLs) and found none suitable for this task. He, in fact designed and implemented a language called C for this purpose. Finally, UNIX was written in C. Only 10 % of the kernel and hardware dependent routines, where the architecture and the speed matter were written in the assembly language for that machine. All the rest was written in C. This made the job of ‘porting’ the operating system far easier.

Office automation systems language compilers and so on could also then be easily ported, once the system calls under UNIX were known and available. After this, porting application programs also became a relatively easier task.

1.7 Fourth Generation (1980-1990) Large Scale Integration

“Control Program for Microcomputers (CP/M)” was almost the first operating system on the microcomputer platform. It was developed on Intel 8080 in 1974 as a file system by Gary Kindall. Intel Corporation had decided to use PL/M instead of the assembly language for the development of systems software and needed a compiler for it badly. CP/M was initially only a file system to support a resident PL/M compiler. This was done at ‘Digital Research Inc. (DRI)’.

After the commercial licensing of CP/M in 1975, other utilities such as editors debuggers etc. were developed and CP/M became very popular. CP/M went through a number of versions. Finally, a 16-bit multi-user, time sharing “MP/M” was designed with real time capabilities, and a

genuine competition with the minicomputers started. In 1980 CP/NET was released to provide networking capabilities with MP/ M as the server to serve the requests received from other CP/ M machines.

One of the reasons for the popularity of CP/M was its 'user friendliness'. This had a lot of impact on all the subsequent operating systems on microcomputers. After the advent of the IBM-PC based on Intel 8086 and then its subsequent models, the DISK Operating System (DOS) was written. A company called 'Seattle Computer' developed an operating system called QDOS for Intel 8086. The main goal to enable the programs developed under CP/M on Intel 8080 to run on Intel 8086 without any change. Microsoft Corporation acquired the rights for QDOS which later became MS-DOS.

MS-DOS is a single user, user-friendly operating system. In quick succession, a number of other products such as Database systems (dBASE), Word Processing (WORDSTAR), Spreadsheet (LOTUS 1-2-3) and many others were developed under MS-DOS, and the popularity of MS-DOS increased tremendously. The subsequent development of compilers for various High Level Languages such as BASIC, COBOL and C added to this popularity, and, in fact, opened the gates to a more serious software development process. This was to play an important role after the advent of Local Area Networks (LANs). MS-DOS later was influenced by UNIX and it has been evolving towards UNIX over the years. Many features such as hierarchical file system have been introduced in MS-DOS over a period of time.

With the advent of Intel 80286, the IBM PC/AT was announced. The hardware had the power of catering simultaneously to multiple users, despite the name "Personal Computer". Microsoft adopted UNIX on this platform to announce 'XENIX'. IBM joined hands with Microsoft again to produce a new Operating system called 'OS/2'. Both of these run on 286 and 386 based machines and are multi-user systems.

With 386 and 486 computers, bit mapped graphic displays became faster and therefore, more realistic. Therefore, Graphical User Interfaces (GUI's) became possible and in fact necessary for every application. With the advent of GUI's some kind of standardization was necessary to reduce development and training time. Microsoft produced MS-WINDOWS.

Windows

The windows operating system, developed by Microsoft, expands on the Dos operating system; users can activate programs from windows using icons (or symbols). An icon is a

picture on the screen that represents an action or application that the computer can implement. Windows is a graphical user interface (GUI), that uses the point-and-click method (i.e., the use of a mouse to point at the file command open, close, delete, and move. In Windows, each applications appears in its own window. For example word processing can appear in one window a spreadsheet in another window, and a graphics program in a third. A user can easily move between windows to switch different applications. Further windows is a highly integrated environment, in which different applications have the same “look and feel”, so user familiar with one application can easily work in other applications. For example in windows the symbol for closing a file or a document is the same in a word-processing application or in a spreadsheet application. Many companies now Use windows as their operating system. For example, Boston Chicken uses Windows-based PCs for applications such as customized forecasting, scheduling and inventory management.

Windows 95, also known as windows Version 4.0 is in many ways similar to windows. Yet it is radical departure from windows in that it is independent of Dos and allows for “plug and Play” unlike windows. Like Windows, Windows 95 represents programs with icons; when a user clicks on an icon, the system recognize and open the application associated with it. For example suppose you have a file called RESUME.DOC in Windows 95. If you simply click on the file, windows 95 automatically recognises that this is a word processing file and opens the word processing program. Windows 95 is powerful; operating system that enhances the speed and performance of the Pc. It takes up about 20 MB of hard disk storage and uses 8 MB of RAM.

Windows NT

Windows NT is another new and powerful operating system from Microsoft with multitasking and multiprocessing capabilities. It process data in 32-bit chunks (unlike earlier versions of windows, which process data in 16-bit chunks), resulting in increased speed and efficiency. Windows NT is ideal for large business applications that run in a net worked environment it provides mainframe like capabilities on microcomputer. It can support multiple processors and has excellent I/O device support. Although windows NT can run on 486 (or more powerful) PCs, it is better suited for workstations, because it requires 20 MB of RAM and occupies 40 to 45 MB of disk space.

UNIX

Although the UNIX operating system was developed ATSLTS Bell labs in 1969. It is only in the last decade or so that it has become popular. Today it is widely used in a number of important

business applications. UNIX is a powerful, interactive multiprogramming, multitasking (allows users to perform multiple tasks simultaneously), multiuser operating system that is highly portable (i.e. it works on a number of types of computers).

One of the disadvantages of UNIX is that there are many different versions of UNIX and this can sometimes get confusing. Also, UNIX compared to other operating systems is cryptic and not very user-friendly. However its advantages far outweigh its drawbacks and UNIX has become a main stream operating system for many businesses.

OS/2

IBM's OS/2 (Operating System/ 2) is a 32-bit operating system that supports multitasking and can run programs written for OS/2 as well as for other operating systems such as DOS and Microsoft Windows thus reducing the need to learn several operating systems. Its 32-bit capability makes it faster than DOS and it is an ideal, sophisticated operating system for applications that require networking and multimedia features, such as playing sound files or movies. OS/2 offers a number of small applications, called applets, such as time scheduling, appointment calendars and card games. The OS/2 version of Windows has most of the features found in Windows 95 yet requires only 4 megabytes of RAM.

In late 1994, IBM introduced its long-awaited new version of OS/2, dubbed OS/2 Warp. Since a dearth of brand-name applications that can run on OS/2 has always been OS/2's Achilles heel, IBM had effectively addressed the problem by ensuring that there are more than 2,500 applications that run on OS/2. Further, Warp can also run all the applications written for DOS and Windows thus greatly increasing the number of applications available to users who choose the OS/2 operating system. OS/2 Warp comes bundled with 12 OS/2 applications, collectively known as Bonus Pak which includes a word processor, spreadsheet, personal information manager, and access to the Internet and other on-line services.

In recent years, several companies are using OS/2 for developing company-wide applications. For example, Travelers developed an OS/2 based case processing application for insurance claims and First Union National Bank is a user of OS/2.

Table: 1.1 Summary of different operating system	
Operating System	Characteristics
Dos	A single user system. Commands are somewhat Cryptic and very user-friendly.
Windows	Uses icons to execute programs. Uses pointand click method to execute commands. Applications appear in windows on the monitor: A highly intergreated environment, in which different applications. have the same “look and feel”.
Windows NT	Has multitasking and multi- processing capabilities. Processes data in 32-bit chunks. Ideal for large business applications that run in a network environment.
UNIX	Powerful, interactive multi-programming multitasking, multiuser, operating system. Highly portable.
OS/2	This 32-bit operating system supports multitasking making it faster than Dos. It can run programs written for OS/2 or for other operating systems such as Dos and windows.

Table 1.1 provides a brief summary of the different types of operating systems. Clearly, no one operating system is superior to another, since number of factors, such as number and criticality of applications, number of users and network requirements, must be taken into account in selecting an operating system. Also, note that it is possible to use more than one operating system on a computer. A user may run some applications using, the windows operating system, and other applications under some other operating system such as UNIX.

Graphical User Interface

Whenever user interact with a computer even a microcomputer, the interaction is controlled by an operating system. The user interface is the part of an information system that user interact

with. User communicate with an operating systems through the interface of that operating system. Early microcomputer operating systems were command-driven which required the user to type in text based commands using a keyboards, but the graphical user interface, often called GUI, make extensive use of icons, buttons, bars, and boxes to perform the same task. For example, DOS is command driven operating system, whereas windows (version 3.1, 3.11,96,98) are GUI based which uses graphical symbols called icons to depict a mouse on the screen. Icons are symbolic pictures and they are also used in GUIs to represent program, files and activities. Commands can be activated by dragging and clicking a mouse on the screen. Icons are symbolic picture and they are also used in GUIs to represent programs and files. For examples file could be deleted by dragging the cursor to a “Recycle Bin” (icon look like basket and used for deleted) icon. Many graphical user interfaces use a system of pulldown menus to help users select commands and pop-up boxes to help users select among various command options. Windowing features allow users to create. Size, stack and move around boxes of information.

Graphical user interfaces softwares save learning time because computing novices do not have to learn different commands for each application. Common functions such as getting help, saving files, or printing are performed the same way. A complex services of commands can be issued simply by linking icons. On the other hand. Graphical symbols themselves are not always easy to understand unless the GUI is well-designed.

Object Linking and Embedding

Object linking and embedding means your document can contain information that was created in different applications, and you edit any of this information from with another, make your changes and keep on working.

To understand linking and embedding you should be familiar with the following terms;
application.

where the object originates.

where you place the object.

Some applications may be both a server and a client.

When you embed an object, you are inserting information from the source document into a destination document in a different application. To make changes to an embedded object, you simply choose the object in the destination document. The application where the object was created opens. Because the object is embedded, there is no connection to the document where you have transferred the information. So when you edit an embedded object, the source document is not affected.

When you link an object, you are creating a reference, for link, to the source document. So when you edit a linked object, you are actually, editing the information in the source document. The destination document only contains a link to where the objects exists in the source document.

Many document can contain a link to a single source document, which must be saved first. You have access to the object from any document that contains a link to it, and you can change the object from within any of them. The updated version appears in all the documents. Linking makes it easy to track information that appears in more than one place and that must be identical.

1.8 Summary

We discussed about the origin of the operating system and their subsequent developments. The first mainframes initially had no protection hardware and no support for multiprogramming, so they ran simple operating systems that handled one manually-loaded program at a time. Later they acquired the hardware and operating system support to handle multiple programs at once, and then full timesharing capabilities.

When minicomputers first appeared, they also had no protection hardware and ran one manually-loaded program at a time, even though multiprogramming was well established in the mainframe world by then. Gradually, they acquired protection hardware and the ability to run two or more programs at once. The first microcomputers were also capable of running only one program at a time, but later acquired the ability to multiprogram.

1.9 Review Questions

1. Discuss about the origin of operating system and their subsequent developments.
2. What is meant by teardown?
3. What is meant by setup?
4. How did they affect the throughput and CPU utilization?
5. What was the use of a Satellite Computer?
6. What is IOCS?
7. What is spooling?
8. How did UNIX originate?
9. Discuss multiprogramming versus single User Systems in terms of throughput and CPU utilization?
10. Contrast Multiprogramming batch and time sharing operating system?

LESSON - 2

THE FILE SYSTEM, DEVICE DRIVER AND TERMINAL I/O

Structure

- 2.1 Introduction**
- 2.2 Objectives**
- 2.3 The File System**
- 2.4 Device Driver**
- 2.5 Terminal I/O**
- 2.6 Summary**
- 2.7 Review Questions**

2.1 Introduction

We use files in our daily lives. Normally, a file contains records of a similar type of information. e.g. Employee file or Sales file or Electricity bills files. If we want to automate various manual functions, the computer must support a facility for a user to define and manipulate files. The operating system does precisely that.

The user/Application programmer needs to define various files to facilitate his work at the computer. As the number of files at any installation increases, another need arises: that of putting various files of the same type of usage under one directory. eg. all files containing data about finance could be put under "Finance" directory. All files containing data about sales could be put under "Sales" directory. A directory can be conceived as a "file of files". The user/application programmer obviously needs various services for these files/directories such as "Open files", "Create files", "Delete a directory" or "Set certain access controls on a file", etc. This is done by the file system, again, using a series of system calls or services, each one catering to a particular need. Some of the system calls are used by the compiler to generate them at the appropriate places within the compiled object code for the corresponding HLL source instructions such as "open files", whereas others are used by the Command Interpreter (CI) while executing commands such as "DELETE a file" or "Create a link" issued by the user sitting at a terminal.

The file system in the IM allows the user to define tiles and directories and allocate/deallocate the disk space to each file. It uses various data structures to achieve this, which is the subject of this section. We have already seen how the operating system uses a concept of a block in manipulating these data structures.

2.2 Objectives

After studying this Lesson, you should be able to

- * File Management System
- * Hierarchical File System
- * Disk Space Allocation Methods
- * Input Output Procedures
- * Terminals Input Output
- * Terminals Buffers and Data Structures
- * Volume Table of Controls

2.3 The File System

2.3.1 Block and Block Numbering Scheme

The operating system looks at a hard disk as a series of sectors, and numbers them serially starting from 0. One of the possible ways of doing this is shown in Fig. 2.1 A which depicts a hard disk.

If we consider all the tracks on different surfaces, which are of the same size, we can think cylinder due to the obvious similarity in shape. In such a case, a sector address can be thought of as having three components such as: Cylinder number, Surface number, Sector number. In this case, Cylinder number is same as track number used in the earlier scheme, where the address consisted of Surface number, Track number and Sector number. Therefore, both these schemes are equivalent. The figure shows four platters and therefore, eight surfaces, with ten sectors per track. The numbering starts with 0 (may be aligned with index hole in case of a floppy), at the outermost cylinder and topmost surface.

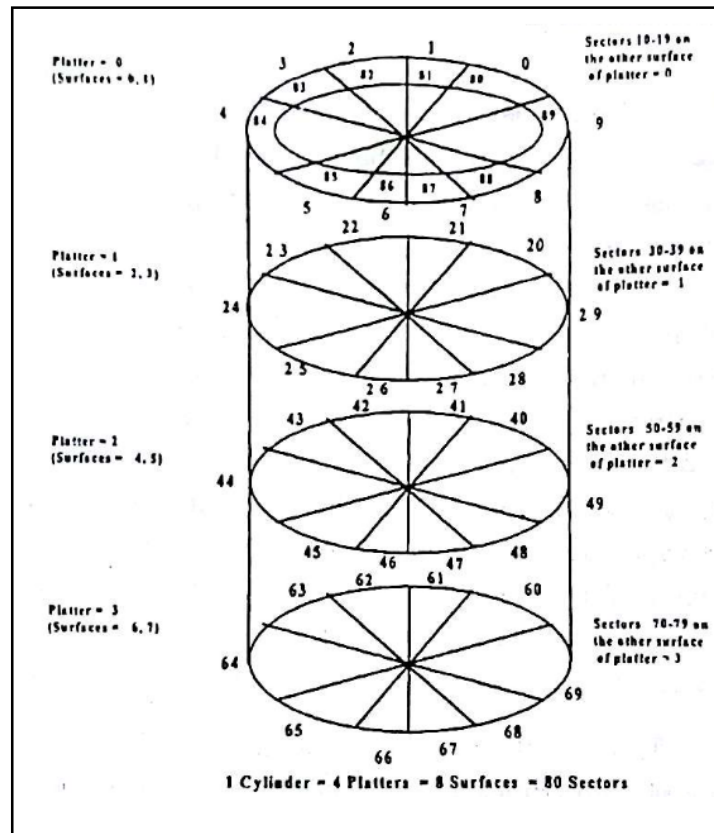
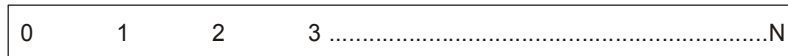


Fig 2A – A Scheme to number sectors

When all the sectors on that surface on that cylinder are numbered, we got to the next surface below on the same platter and on the same cylinder. This surface is akin to the other side of a coin. After both the surfaces of one platter are over, we continue with other platters for the same cylinder in the same fashion. After the full cylinder is over, we go to the inner cylinder, and continue from the top surface again. By this scheme, Sectors 0 to 9 will be on the topmost surface (i.e. surface number=0) of the outermost cylinder (i.e. cylinder number: 0). Sectors 10 to 19 will be on the next surface (at the back) below (i.e. surface number= 1), but on the same platter and the same cylinder (i.e., Cylinder number surface number=0). Continuing this, with 8 surfaces. (i.e. 8 tracks/cylinder), we will have Sectors 0-79 on the outermost cylinder (i.e. Cylinder =0). When the full cylinder is over, we start with the inner cylinder, but from the top surface and repeat the procedure. Therefore, the next cylinder (Cylinder=1) will have Sectors 80 to 159, and so on.

With this scheme, We can now view the entire disk as a series of sectors starting from 0 to N as Sector Numbers (SN) as shown.



For example, if SN=7 what will be its physical address? We know that a track in our example contains 10 sectors. Therefore, the first 10 sectors with SN=0 to SN=9 have to be on the outermost cylinder (cylinder gm and the uppermost surface (surface =0). Therefore, SN=7 has to be equivalent to cylinder=0, surface 0 and sector = 7. By the same logic, the sectors with SN= 10 to SN=19 will be on cylinder =0 and surface = 1. Therefore, if SN=12, it has to be cylinder=0, surface = 1 and sector =2. Similarly, if SN is between 80 and 159, the cylinder or track number I will be 1, and so on. By a similar logic, given a three dimensional address of a sector, the operating system can convert it into a one dimensional abstract address, viz. Sector Number or SN.

The formatting program discussed earlier maintains a list of bad sectors which the operating system refers to. Therefore, these bad sectors are not taken into account for allocating, deallocating of disk space for various files by the operating system.

As we know, the operating system deals with the block number for all the internal manipulation. A block may consist of one or more contiguous sectors. If a block for the operating system is the same as one physical sector, SN discussed above will be the same as Block Number or BN. If a block consists of 2 sectors or 1024 bytes, the view of the disk by the operating system will be as shown 1 Fig. 4.12. In this case, if BN=x, this block consists of sector numbers $2x$ and $2x+ 1$. For instance, Block number 2 consists of Sector 4 and 5 as the figure depicts. Similarly, or a sector number SN, $BN = \text{integer value of } SN/Z$ after ignoring the fraction part if any. For instance, Sector 3 must be in Block 1 because the integer value of $3/2$ is 1.

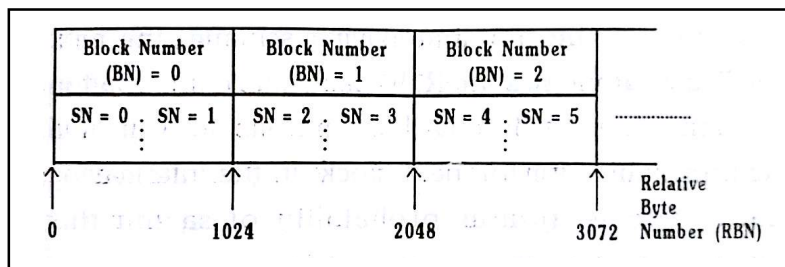


Fig. 2 B A block numbering scheme

Therefore, given a Block Number (BN), you could calculate the one dimensional abstract Sector Numbers (SN) and then calculate the actual three dimensional sector addresses for both the sectors quite easily and vice versa.

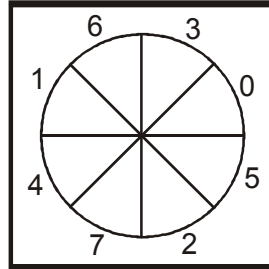


Fig 2 C - Interleaving with factor = 3

Some operating systems follow the technique of interleaving. This is illustrated in Fig.2.C After starting from sector 0, you skip two sectors and then number the sector as 1, then again skip two sectors and call the next sector as 2, and so on. We call this interleaving with factor=3. Generally, this factor is programmable, i.e, adjustable. This helps in reducing the rotational delay.

The idea here is simple. While processing a file sequentially, after reading a block, the program requesting it will take some time to process it before wanting to read the next one. In the non-interleaving scheme, the next block will have gone past the R/W head due to the rotation by that time, thereby forcing the controller to wait until the next revolution for the next block. In the interleaving a scheme, there is greater probability of saving this revolution, if the timings are appropriate.

2.3.2 Writing a Record

A. Arrangement of logical records

Let us assume that a customer record (also referred to in our discussion as logical record) consists of 700 bytes. The application program responsible for creating these records written in HLL has instructions such as “WRITE GUST-REC” to achieve this. As we know, at the time of execution, this results in a system call to the operating system to write a record.

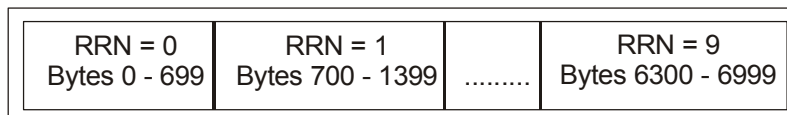


Fig 2 D - The RRN and RBN

For instance, if 10 customer records are written, $700 \times 10 = 7000$ bytes will be written onto the customer file for RRB= 0 to 9. You can, in fact, imagine all the customer records put on after the other like carpets as shown in Fig.2 D. It is important to know that this is a logical view of the file as seen by the application programmer. This is the view the operating system would like the application programmer to have. It does not however mean that in actual practice, the operating system will put these records one after the other in a physical sense of contiguity, as given by the sector/block numbering scheme. The operating system may scatter the records in variety of ways, hiding these details from the application programmer, each time providing him the address translation facilities and making him feel that the file is written and therefore, read contiguously.

RRN	RBN
0	0
1	700
2	1400
-	
9	6300

Fig. 2E- The relationship between RRN and RBN

The operating system can calculate a Relative Byte Number (RBN) for each record. This is the starting byte number for a given record. RBN is calculated with respect to 0 as the starting byte number of the file and again assuming that the logical records are put one after the other. For instance, Fig. 2.E shows the relationship between RRN and RBN for the records shown in Fig.2.D

It is clear from Fig 2 E that $RBN = RRN \times RL$, where $RL = \text{Record Length}$. This means that if a record with $RRN=10$ is to be written (which actually will be the 11th record. The operating system concludes that it has to write 700 bytes, starting from Relative Byte Number (RBN) - 7000. Therefore, if an operating system recognizes a definition of a record, it can be supplied with only the RRN and the record length. It then can calculate the RBN as seen earlier. For an operating system like UNIX which considers a file as only a stream of bytes, it has to be supplied with the RBN itself along with the number of bytes (typically equal to RL) to be read.

B. Arrangement of blocks

The next step is to actually write these logical records onto various blocks. Let us assume that we have a disk as shown in Fig. 4.11 with 8 surfaces (0 to 7), each surface having 80 tracks (0 to 79) and each track has 10 sectors (0 to 9). Therefore, the disk has $8 \times 80 \times 10 = 6400$ sectors of 512 bytes each. Let us also assume that one block.1 sector = 512 bytes and all these blocks are numbered as discussed earlier. Therefore, the operating system will look at the disk as consisting of 6400 logical blocks' (0 to 6399) each of 512 bytes, as shown in Fig 2.F

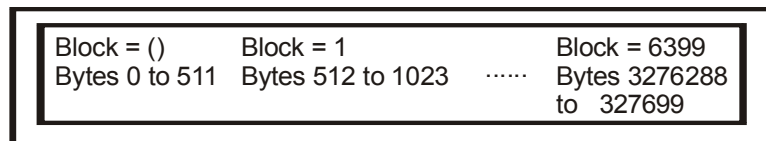


Fig 2 F The operating system view of the disk

C. The allocation of blocks to a file

In this scheme, the user has to specify at the time of creating a file, the maximum number of contiguous blocks required for each file. For instance, if a user knows that he has currently 500 customers, but will never have more than 730 customers, he must ask for 730 I700 bytes or $730 \times 700 / 512$ blocks = 998.05 or approximately 1000 blocks. Let us assume that the Operating System has already allocated block numbers 0 to 99 for some other file and that blocks 100 and thereafter are free. (The Operating System has to maintain a list of free blocks to know this!) Therefore, let us assume that the Operating System allocates block numbers 100 to 1099 for our customer file.

2.3.3 Reading a Record

Let us now see how records are read by the operating system on the request of the AP. An unstructured AP for processing all the records sequentially from a file would be as shown in Fig, 4.19. During the processing, every time it executes the "READ" instruction the AP gives a call to the operating system which then reads a logical record on behalf of the AP as we know. The operating system blocks the AP during this time, after which it is woken up.

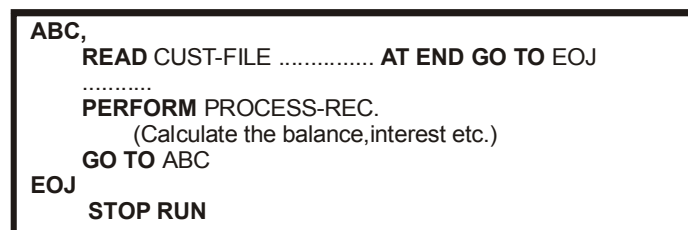


Fig 2. G – A 3GL program to Read and Process records

If there are 50 customer records, the AP requests the operating system to read records from RRN =0 to 49, one by one, for READ CUST-FILE instruction by using it in a loop.

- (i) The operating system maintains a cursor which gives the “next” RRN to be read at any time. This cursor is initially 0, as the very first record to be read is with RRN =0. After each record is read, the operating system increments it by 1 for the next record to be read. This is done by the operating system itself and not by the AP.
- (ii) When the AP requests the operating system to read a record by a system call at the “READ CUST FILE” instruction, the operating system calculates RBN (Relative Byte number) as $RBN = RL \mid RRN$, where this RRN is given by the cursor.

Therefore, initially RBN will be 0, because, RRN =0. For the next record, RRN will be 1 and RBN will be $1 \mid 700 = 700$ and for RRN=2, RBN will be $2 \mid 700 = 1400$. RBN tells the operating system the logical starting byte number from which to read 700 bytes.

- (iii) The file system calculates the logical block number as the integer value of $RBN/512$. For instance, for RRN=2 and RBN= 1400, $1400/512=2+(376/512)$ Therefore, logical block number (LBN)=2, offset=376. This means that the file system has to start reading from byte number 376 of LBN=2. But if only this is done, the operating system will get only $(511-375)=136$ bytes out of this block. This is less than 700.
- (iv) The file system will have to read the next block with LBN=3 fully to get the additional 512 bytes to achieve $136 + 512 = 648$ bytes in all. This is still less than 700. The operating system will have to read the next block with LBN=4 and extract the first 52 bytes to finally make it $648 + 52 = 700$ bytes. Therefore, for this instruction to read one logical record, the operating system has to translate it into reading a sequence of logical blocks first, as shown in Fig. 2 H

LBN	Starting Byte #	Ending Byte #	# of Bytes
2	376	511	136
3	0	511	512
4	0	51	52
TOTAL			700

Fig 2 H Logical to Physical translation

- (v) At this stage, the file system does the conversion from LBN to PBN by adding 100 to LBN, because, the starting block number is 100 and all allocated blocks are contiguous, as per our assumption.

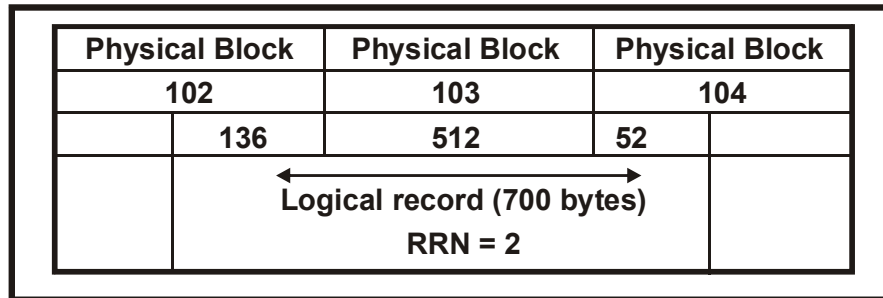


Fig 2 | Address translation for Read

- (vi) Therefore, the file system decides to read 136 bytes (376 to 511) in PBN 102+all (512) bytes in PBN 103 and 52 bytes (0-51) from PBN 104. This is shown in Fig. 2.1. The file system issues an instruction to the DD to read blocks 102 to 104.
- (vii) As before the DD translates the PBNs into three dimensional physical sector addresses are given below.
- Block 102 = Surface 2, Track 1, Sector 2
- Block 103 = Surface 2, Track 1, Sector 3
- Block 104 = Surface 2, Track 1, Sector 4
- (viii) The DD now directs the controller to read these sectors one by one in the controller's memory first and then to transfer the required bytes into the buffer memory of the operating system by setting the appropriate DMA register as studied earlier.
- (ix) After all the data is read, the file system picks up the relevant bytes as shown in Fig. 2.1 to form the required logical record and finally transfers it to the I/O area of the AP.

2.3.4 Disk Space Allocation Methods

2.3.4.1 Contiguous allocation

A. Introduction

This is what most of the earlier IBM operating systems such as DOS/VSE used to follow. We have assumed contiguous allocation up to now in our examples in Sec. 4.2.4. In this scheme,

the user estimates the maximum file size that the file will grow to, considering the future expansion and then requests the operating system to allocate those many blocks through a command at the time of creation of that file.

B. Block allocation list

There are two other methods, viz Best fit and Worst fit methods to choose an entry from the free blocks list for the allocation of free blocks. Both of these methods would require the free blocks list to be sorted by number of free blocks.

The best fit method would choose an entry which is the smallest amongst all the entries which are equal to or bigger than the required one. To achieve this, (this sorted table is used. In our case, where we want 7 blocks, the first entry in the sorted list is such an entry. Therefore, blocks 41-47 will be allocated. The resulting two tables, similar to the file, we would have to use the second entry of 16 blocks in the sorted list and allocate blocks 5 to 14 to the new file. After this allocation, there would be only $16 - 10 = 6$ free blocks left in this hole. As 6 is less than 8, which is the number of free blocks in the first entry, the list obviously would need resorting - therefore, consuming more time.

Block Numbers			File
From	To	#	
0	4	5	ABC
5	20	16	Free
21	40	20	CDE
41	48	8	Free
49	100	52	CUSTOMER
101	200	100	Free
201	280	80	EMP
281	350	70	Free
351	500	150	STOCK
501	730	230	PURCHASE
731	1070	340	Free
1071	2000	930	TRANS
2001	6339	4339	Free

Fig 2 J - Block allocation table (or list)

However, the best fit method claims to reduce the wastage due to fragmentation. i.e. the situation where blocks are free, but the holes are not large enough to enable any allocation. This is because, this method uses a larger hole unnecessarily. Therefore, if subsequently, a request for a very large allocation arrives, it is more likely to be fulfilled.

The advocates of worst fit method do not agree. In fact, they argue that after allocating blocks 41 to 47, block number 48 which is free in the example above cannot be allocated at all. This is because it is far less likely to encounter a file requiring only one block. Therefore, they recommend that the required 7 blocks should be taken from the largest philosophy, blocks 2001 to 2007 will be allocated, thereby leaving enough to cater to other large demands. At some point, however, in the end, it is likely to have very few free blocks remaining and those would most probably be unallocable even in the worst fit scenario. But by then, some other blocks are likely to be freed, thereby creating larger usable chunks after coalescing. It is fairly straight forward to arrive at the resulting two tables after the allocation using this philosophy.

2.3.5 Bit map

A bit map is another method of keeping track of free/ allocated blocks. A bit map maintains one bit for every block on the disk as shown in the Fig. 4.35. Bit 0 at a specific location indicates that the corresponding block is free and bit 1 at a specific location indicates that the corresponding block is allocated to some file.

11111000000000000000000011111111111111110.....

Fig 2 - A bit map

The figure shows a bit map corresponding to the original table in Fig 2J The first 5 blocks are allocated, the next 15 are free, the next 20 are again allocated, and so on.

A bit map is used only to manage the free blocks and therefore, it need not show the file to which a specific block is allocated. In contiguous allocation, the file directory entry contains the file size and the starting block number. This information is sufficient for accessing all the blocks in the file one after the other or at random. You do not need nay help from the bit map in this regard.

When a file is deleted, the file directory entry is consulted and the corresponding blocks are freed, i.e. the corresponding bits in the bit map are set to 0. Let us assume that a file NEW requiring 7 blocks is to be created. Normally the first fit algorithm is chosen and the routine in the File System searches for 7 consecutive zeroes in the bit map, starting from the beginning.

Having found the first such 7 zeroes, it allocated them, i.e. changes them to 1 and creates the corresponding file directory entry with the appropriate starting block number. To implement Best fit and Worst fit strategies using a bit map is obviously tougher unless the operating system also maintains the tables of free blocks in the sequence of hole size

2.4 DEVICE DRIVER (DD)

The basics

We have seen the functions of a disk controller in the previous sections. We have also seen various instructions that this controller understands -i.e. the instruction set of the controller. We had also seen how the DD uses these instructions to construct various routines for read, write operations. In this section, we will look at other aspects of the DD.

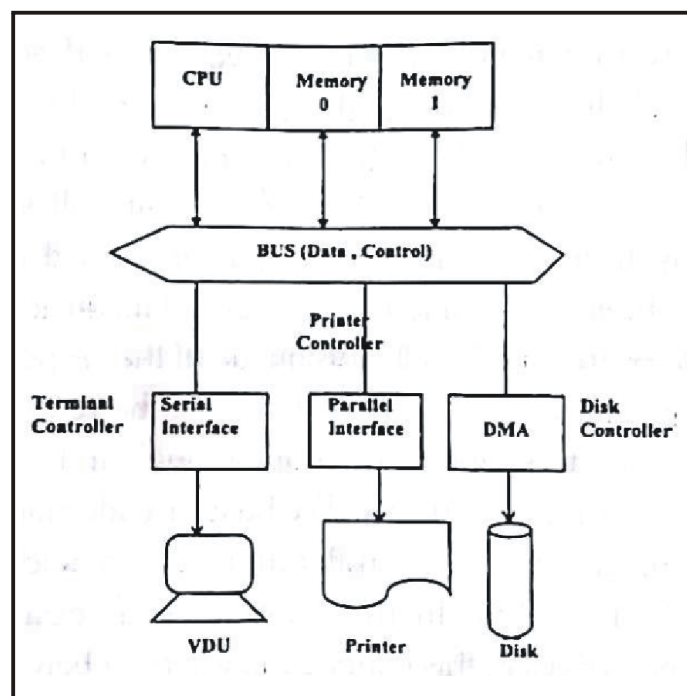


Fig 2.M – Devices, Controllers, Bus and Memory

In most computers, including the IBM-PC family, the devices are attached to the CPU and the main memory through a bus and interfaces or controllers as depicted in Fig. 2.M. The figure

shows a serial interface for the terminals, a parallel interface for the printer and the DMA for the disk.

2.4.1 Path management

Controllers require large memory buffers of their own. They also have complicated electronics and therefore, are expensive. An idea to reduce the cost is to have multiple devices attached to only one controller as shown in Fig.2.N. At any time, a controller can control only one device and therefore, only one device can be active, even if some amount of parallelism is possible due to overlapped seeks. If there are I/O requests from both the devices attached to a controller, one of them will have to wait. All such pending requests for any device are queued in a device request queue by the DD. The DD creates a data structure and has the algorithms to handle this queue. If the response time is ver important, these I/O waits have to be reduced, and if one is ready to spend more money, one can have a separate controller for each device. We have already studied the connections between a controller and a device such as a disk drive in Fig. 2.0. In the scheme of a separate controller for each device, this connection will exist between each controller/device pair. In such a case, the drive select input shown in Fig. 2.0 will not be required. This scheme obviously is faster, but also is more expensive.

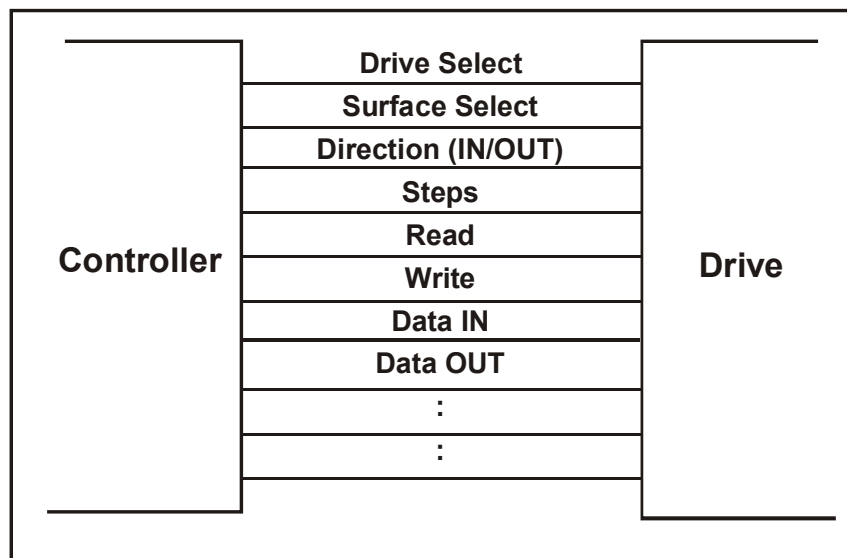


Fig. 2 O - A possible connection between the controller and the disk drive

In some mainframe computers such as IBM-370 family (i.e. IBM 370,43XX,30XX,etc), the functions of a controller are very complex, and they are split into two units. One is called a Channel and the other is called a Control Unit(CU). Channel sounds like a wire or a bus, but it is actually a very small computer with the capability of executing only some specific I/O instructions. If you refer to Fig. 2..N you will notice that one channel can be connected to many controllers and one controller can be connected to many devices. A controller normally controls devices of the same type, but a channel can handle controllers of different types. It is through this hierarchy that finally the data transfer from/to memory/device takes place. It is obvious that there could exist multiple paths between the memory and devices as shown in Fig 2.N. These paths could be symmetrical or asymmetrical as we shall see. Figure 2.P shows a symmetrical arrangement. In this arrangement, any device can be reached through any controller and any channel.

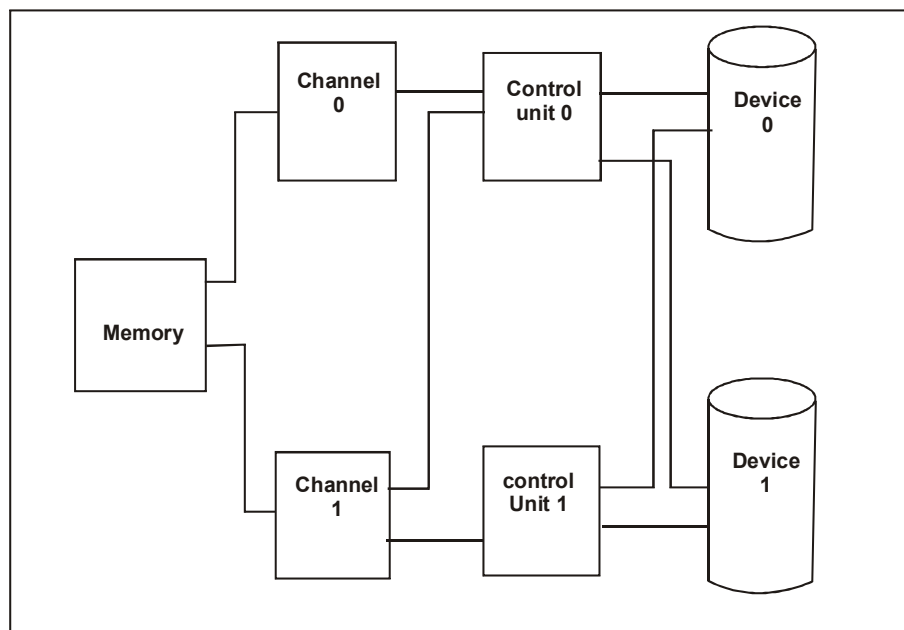


Fig 2.P - Multiple paths to a device (Symmetrical)

We could also have an asymmetrical arrangement as shown in Fig 2.Q. In this scheme any device cannot be reached through any controller and Channel, but only through specific preassigned paths.

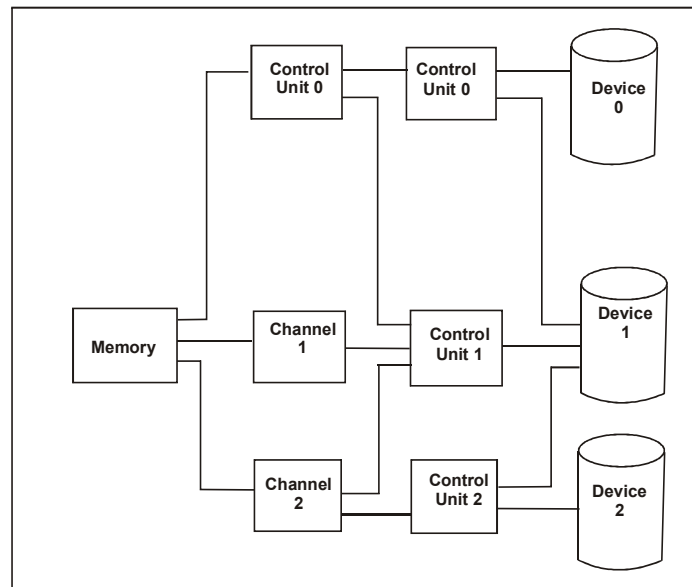


Fig 2.Q - Multiple paths to a device (asymmetrical)

2.4.2. I/O Procedure

In order to perform the path management and to create the pending requests, the I/O procedure maintains the data structures as described below:

- (i) Channel Control Block (CCB)

This data structure maintains the following information:

Channel ID Channel Status (busy, not functioning etc.) List of CUs connected to this Channel List of processes waiting for this channel Current process using this channel Some other information

Channel Control Block (CCB)

(ii) Control Unit Control Block (CUCB)

This data structure maintaining the following information:

Control Unit ID
Control Unit Status (busy, not functioning etc.)
List of devices connected to this CU
List of channels connected to this CU
List of processes waiting for this CU
Current process using this CU
Some other information

Control Unit Control Block (CUCB)

(iii) Device Control Block (DCB)

This data structure maintains the following information:

Device ID
Device Status (busy, not functioning etc.)
Device characteristics
Device Descriptor
List of CUs connected to this Device
List of processes waiting for this Device
Current process using this Device
Some other information

Device Control Block (DCB)

In the Device Control (DCB), we maintain device characteristics and device descriptor to achieve a kind of device independence. The idea is to allow the user to write the I/O routine in a generalized fashion so that it is applicable for any device, once the parameters for that device

are supplied to that routine. These parameters are the same as the device characteristics in the DCB. Therefore, the idea is that whenever an operating systems wants to perform an I/O for a device, it reads the DCB for that device and extracts these device characterisation. It then invokes the “Common I/O routine” and supplies these Device characterisation as parameters.

(iv) Input/Output Request Block (IORB)

These IORB records are chained together with the DCB for that device. For instance, if there are three processes waiting for I/O on a specific device, there will be three IORB records chained to one another as well as to the DCB, as shown in the Fig. 4.61. Similarly, the I/O procedure can create data structures for processes waiting for a control unit or a channel. Again the CCBs, CUs and DCBs can be connected with a pointer chain structure to denote things like a list of CUs connected to a channel, etc. In a situation with no channel, only one controller and one or multiple disks, these structures become very simple.

When some data is required to be read from a specific device, the I/O procedure can use the DCB for the device and check whether it is free. Then, it can trace the pointer chains from the DCB to CUCBs using the field in the DCB “List of CUs connected to this device”. It can access these CUCBs one by one, checking their status (free, busy,...) and choose a CU with a CUCB which is either free or the one with the least number of pending requests. It can use the field in the CUCB “List of processes waiting for this CU” for this purpose.

This selection procedure is not very simple, if one also wants to optimize. For instance, one may choose a controller with the least number of requests, but that controller may be connected to a channel which has a long queue. On the other hand, there could be a controller with a long queue but it could be connected to a channel which is free (at least at that moment). Having chosen the path, it can create IORBs on all the components and chain them in the way as depicted in Fig. 2.R. The figure shows the IORBs connected to the CUCB, and so on. This data structure is maintained in the memory and it is updated as a new request arrives and as it is serviced.

2.5 TERMINAL I/O

2.5.1 Introduction

A terminal or Visual Display Unit (VDU) is an extremely common I/O medium. It would be hard to find any programmer or user who has not seen and used a terminal. Ironically, there is not much of popular literature available explaining how terminals work and how the operating system handles them. We want to provide an introduction to the subject to uncover the mysteries around it.

2.5.2 Terminal Hardware

Terminal hardware can be considered to be divided into two parts: the keyboard, which is used as an input medium and the video screen which is used as an output medium.

The terminal can be a dumb terminal or an intelligent terminal. Even the dumb terminal has a microprocessor in it on which can run some rudimentary software,. It also can have a very limited memory. The dumb terminal is responsible for the basic input and output of characters. Even then, it is called 'dumb' because, it does no processing on the input characters. As against this, the intelligent terminal can also carry out some processing (e.g validation) on the input. This requires a more powerful hardware and software for it.

When a character is keyed in, the electronics in the keyboard generates an 8 bit ASCII or EBCDIC code from the keyboard. This character is stored temporarily in the memory of the terminal itself. Every key depression causes an interrupt to the CPU. The ISR for that terminal picks up that character and moves it into the buffers maintained by the operating system for that terminal. It is from this buffer that the character is sent to the video RAM if the character is also be displayed (i.e. echoed). We will shortly see the need for these buffers maintained by the operating system for various terminals. Normally, the operating system has one buffer for each terminal. Again, the operating system can maintain two separate buffers for input and output operations. However these are purely design considerations.

When the user finishes keying in the data, i.e. he presses the carriage return or the new line, the data stored in the operating system buffer for that terminal is flushed out to the I/O area of the Application Program which wants that data and to which the terminal is connected (e.g. the data entry program).

Therefore, there are multiple memory locations involved in the operation. These are:

- * A very small memory within the keyboard itself
- * The video-RAM (data and attribute bytes)
- * The Operating system buffers
- * The I/O area of the AP (FD, working storage, etc.).

2.5.3 Terminal Software

2.5.3.1 Terminal Buffers and Data Structures

The operating System reserves a large input memory buffer to store the data input from various terminals and before it is sent to the respective APs controlling these terminals.

For large systems, there could be dozens if not hundreds of users logging on and off various terminals throughout the day. The Operating system needs a large area to hold the data for all these terminals for the purpose of input (the data keyed in by the users) and the output (the data to be displayed). These are the buffers which are 'the most volatile in nature. They get allocated and deallocated to various terminals by the operating system, a number of times throughout the day.

(A) Dedicated buffer for each terminal

This scheme estimates the maximum buffer that a terminal will require, and reserves a buffer of that size of that terminal. This is depicted in Fig. 2. U

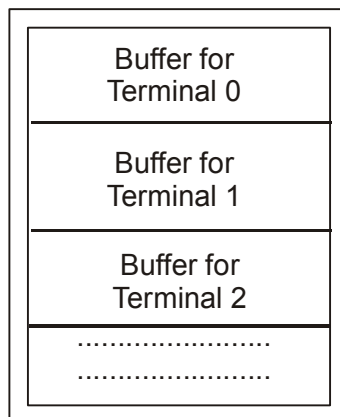


Fig.2 U Dedicated buffers for terminals

The advantage of this scheme is that the algorithm for allocation/deallocation of buffers are far simpler and faster. However, the main disadvantage is that, it can waste a lot of memory, because this scheme is not very flexible.

For instance, it is quite possible in this scheme that one terminal requires a larger buffer than the one allocated to it, whereas some other terminal grossly underutilized its allocated buffer. This scheme is rigid in the sense that the operating system cannot dynamically take away a part of a terminal's buffer and allocate it to some other.

(B) Central pool of buffers

In this scheme, the operating system maintains a central pool of buffers and allocates them to various terminals as and when required. This scheme obviously is more flexible and it reduces memory wastage, but then the algorithms are more complex and time consuming.

The buffer is divided into a number of small physical entities called Character blocks (Cblocks). A Cblock is fixed in length. A logical entity such as Character List (Clist) consists of one or more Cblocks. For instance, for each terminal, there would be a Clist to hold the data input through a keyboard as it was keyed in. This Clist would also actually store the ASCII or EBCDIC codes for even the control characters, such as "TAB", "DEL", etc. along with those for data characters in the same sequence that they were keyed in. If a Cblock is, say, 10 bytes long, and if a user keys in a customer name which is 14 characters, it will be held in a Clist requiring 2 Cblocks. In this case, only 6 bytes would be wasted because, the allocation! deallocation takes place in units of full Cblocks. if a user keys in an address which is 46 characters long, that Clist will require 5 Cblocks, thereby wasting only 4 bytes. All Cblockss in a buffer are numbered serially. Therefore, the terminal buffer can be viewed as consisting of a series of Cblocks 0 to n of fixed length. When the clist is created or when it expand because its already allocated Cblock is full, the operating system allocates another free Cblock to that Clist, The Cblocks assigned to a Clist need not be the adjacent ones, as they are dynamically allocated and deallocated to a Clist.

The size of the Cblock is a design parameter. If this size is large, the allocation/deallocation of Cblock will be faster, because the list of free and allocated Cblocks will be shorter, therefore, enhancing the speed of the search routines. But in this case, the memory wastage will be high. Even if one character is required, a full Cblock has to be allocated. Therefore, the average memory wastage is $(\text{Cblock size} - 1)/2$ for each Clist. If the size of the Cblock is reduced, the

allocation/deallocation routines will become slower as the list of free and allocated Cblock will be longer, and also the allocation/deallocation routines will be called more often, thereby reducing the speed. Therefore, there is a trade-off involved again similar to the one involved in the case of deciding the page size in memory management or the size of a cluster or an element used in the disk space allocation.

2.6 Summary

We have studied about the various file management systems and the allocation methods for input and output devices. Programmed I/O is simple but inefficient. Interrupt mechanism supports overlap of CPU with I/O. Asynchronous I/O allows user code to perform overlapping. Device drivers dominate the code size of OS. Dynamic binding is desirable for many devices and device drivers can introduce security holes. Progress on secure code for device drivers but completely removing device driver security is still an open problem.

2.7 Review Questions

1. Explain the block and the block numbering scheme?
2. Explain the Writing a record?
3. Explain the Read a record?
4. Describe broadly the device management functions with the Operating System?
5. Explain the I/O requests for a busy device are queued and scheduled?
6. How is Path Management device?
7. What data structures help in this task?
8. What is an Interrupt Service?
9. List out the I/O procedures?
10. What is dumb terminal?

LESSON - 3

PROCESS MANAGEMENT AND INTER-PROCESS MANAGEMENT

Structure

- 3.1 Introduction
- 3.2 Objectives
- 3.3 Process Management
- 3.4 Inter-Process Communication
- 3.5 Summary
- 3.6 Review Questions

3.1 Introduction

In order to understand process management, let us first understand what a process is and how it is different from a program as far as operating system is concerned. In simple terms, a program does not compete for the computing resources like the CPU or the memory, whereas a Process does. A program may exist on paper or reside on the disk. It may be compiled or tested but it still does not compete for CPU time and other resources. Once a user wants to execute a program, it is located on the disk and loaded in the main memory at which time it becomes a process, because it is then an entity which competes for the CPU time. Many definitions of a process have been put forth, but we will call a process “a program under execution, which competes for the CPU time and other resources”.

3.2 Objectives

After studying this Lesson, you should be able to

- * Process Management.
- * Process Control Block (PCB)
- * Different Process State Transitions
- * Process Scheduling
- * Multitasking
- * Inter Process Communication
- * Primitive for Mutual Exclusion

3.3 Process Management

3.3.1 What is a process?

In order to understand process management, let us first understand what a process is and how it is different from a program as far as operating system is concerned. In simple terms, a program does not compete for the computing resources like the CPU or the memory, whereas a Process does. A program may exist on paper or reside on the disk. It may be compiled or tested but it still does not compete for CPU time and other resources. Once a user wants to execute a program, it is located on the disk and loaded in the main memory at which time it becomes a process, because it is then an entity which competes for the CPU time. Many definitions of a process have been put forth, but we will call a process “a program under execution, which competes for the CPU time and other resources”.

3.3.2 PROCESS CONTROL BLOCK (PCB)

Process-id
Process state
Process priority
Register Save Area for PC,IR,SP,....
Pointers to process's memory
Pointers to other resources
List of open files
Accounting information
Other info if required (current Dir)
Pointers to other PCBs

Fig. 3 A - Process Control Block

The operating system maintains the information about each process in a record or a data structure called Process Control Block (PCB) as shown in Fig. 3A. Each user process has a PCB. It is created when a user creates a process and it is removed from the system when the process is killed. All these PCBs are kept in the memory reserved for the operating system.

A. Process-id (PID) :

This is a number allocated by the operating system to the process on creation. This is the number which is used subsequently for carrying out any operation on the process as is clear from Fig 3.B. The operating system normally sets a limit on the maximum number of processes that it can handle and schedule. Let us assume that this number is n . This means that the PID can take on values between 0 and $n-1$.

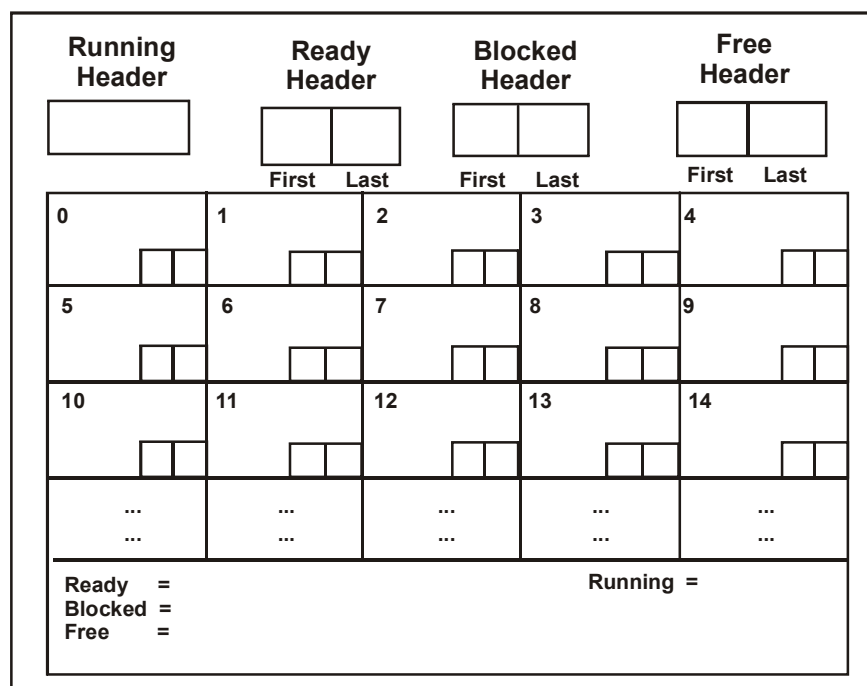


Fig 3.3 PCBs in the memory

The Operating system starts allocating Pids from number 0. The next process is given Pid as 1, and so on. This continues till $(n-1)$ processes are created. At this starts again with 0. This is done on the assumption that at this juncture, the process with $\text{Pid}=0$ would have terminated. UNIX follows this scheme.

B. Process priority:

Some processes are urgently required to be completed (higher priority) than others (lower priority). This priority can be set externally by the user/ system manager, or it can be decided by

the operating system internally, depending on various parameters. You could also have a combination of these. Regardless of the method of computation, the PCB contains the final resultant value of the priority for the process.

C. Accounting information:

This gives the account of the usage of resources such as CPU time, connect time, disk I/O used, etc. by the process. This information is used especially in a data centre environment or cost centre environment where different users are to be charged for their system usage.

D. Other information:

As an example, with regard to the directory, this contains the pathname or the BFD number of the current directory. As we know, at the time of logging in, the home directory mentioned in the system file (e.g. user profile in ADS/VS or / etc/passwd in UNIX) also becomes the current directory. Therefore, at the time of logging in, this home directory is moved in this field as current directory in the PCB.

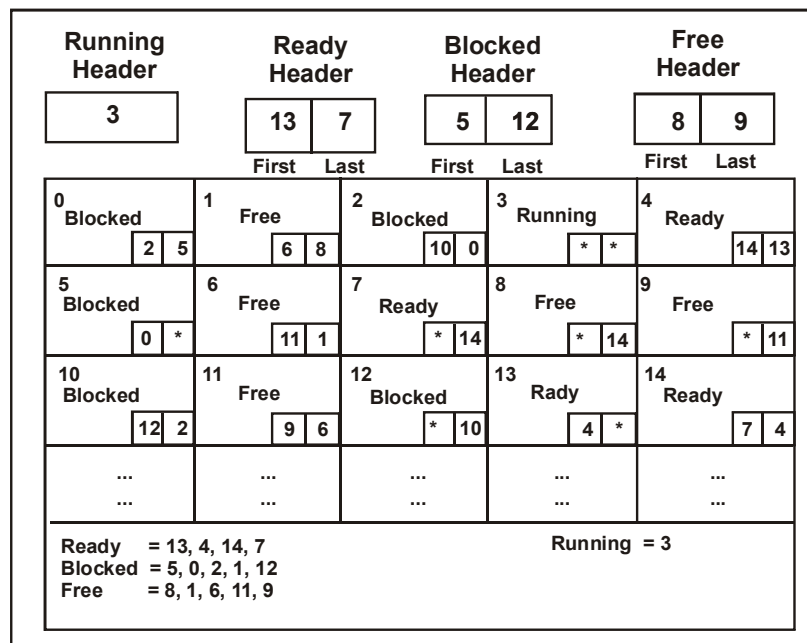


Fig 3.C - PCB chains

Figure 3.C shows the area reserved by the operating system for all the PCBs. If an operating system allows for a maximum of n processes and the PCB requires x bytes of memory each,

the operating system will have to reserve nx bytes for this purpose. Each box in the figure denotes a PCB with the PCB-id or number in the top left corner.

Any PCB will be allocated either to a running process or a ready process or a blocked process (we ignore the new and halted processes for simplicity). If the PCB is not allocated to any of these three possible states, then it has to be unallocated or free. In order to manage all this, we can imagine that the operating system also maintains four queue or lists with their corresponding headers as follows: One for a running process, one for the ready processes, one for the blocked processes and one for free PCBs. Therefore, we assume for our current discussion, that a process is admitted to the ready queue directly after its creation. We also know that there can be only one running process at a time. Therefore, its header shows only one slot. But all other headers have two slots each. One slot is for the PCB number of the first PCB for a process in that state, and the second one is for the PCB number of the one in the same state.

Each PCB itself has two pointer slots. These are for the forward and backward chains. The first slot is for the PCB number of the next process in the same state. The second one is for the PCB number of the previous process in the same state. In both the cases, '*' means the end of the chain.

- (i) Access the ready header. Access the first slot in the header. It says 13. Hence, PCB number 13 is the first PCB in the ready state (i.e. with $Pid=13$).
- (ii) We can now access PCB number 13. We confirm that the state is ready (written in the box). Actually the process state is one of the data items in the PCB which gives us this information
- (iii) We access the next pointer in the PCB 13. It says 4. It means that PCB number 4 is the one for the next process in the ready list.
- (iv) We now access PCB 4 and again confirm that it is also a ready process.
- (v) The next pointer in PCB 4 gives 14,
- (vi) We can now access PCB 14 as the PCB for the next ready process, and confirm that it is for a ready process
- (vii) The next pointer in PCB 14 is 7.
- (viii) We can access PCB 7 and confirm that it is for a ready process.

- (ix) The next pointer in PCB 7 is “*”. It means that this is the end of this chain.
- (x) This tallies with the ready header which says that the Last PCB in the ready list is PCB 7.

3.3.3 KILL A PROCESS

This system call is executed either after the logical completion of a program (typically at the STOP RUN statement in the COBOL program) or after forcibly terminating or aborting a program.

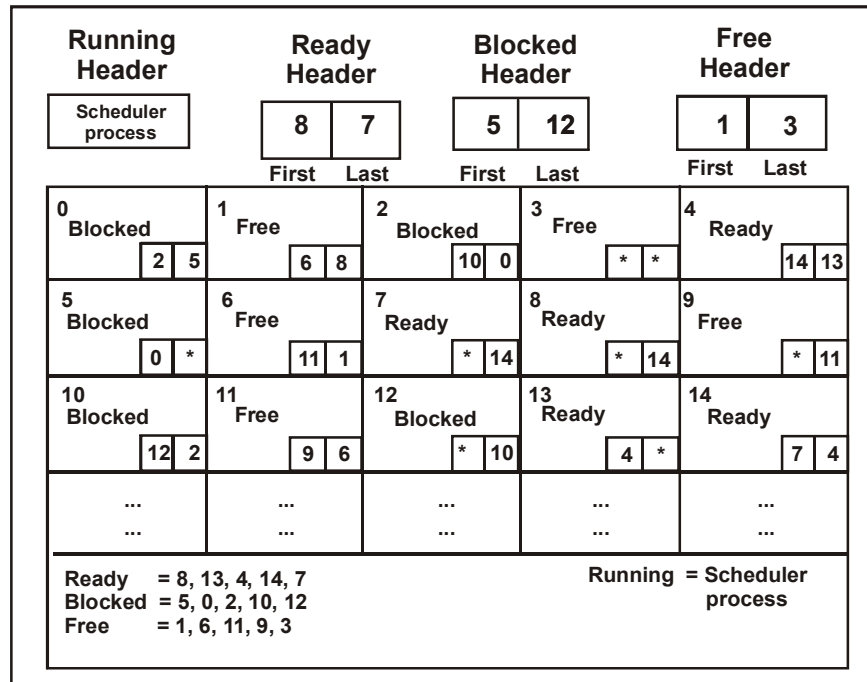


Fig 3.D- PCBs after killing a process iwht PID = 3

Pid is supplied as a parameter to this system call.

The system call causes a software interrupt and the operating system routine starts executing. No user process is in the running state at this juncture.

The operating system stores the Pid (in this case 3) for future use.

The operating system accesses the PCB with Pid= 3 It refers to the data items such as 'pointers to the process 3 memory and 'pointers to the other resources' in the PCB for Pid =3 and frees those resources with the help of the MM and IM routines.

The operating system refers to the list of open files in the PCB 3 and ensures that they are closed (especially if the killing of the process is due to CTRL-C).

The operating system now adds PCB 3 to the list of free PCBs at the end.

The master list of know processes as shown in Fig 3.D is updated.

The PCB chain now looks as shown in Fig 3.F (the free PCBS now are 1,6,1 1,9 and 3).

3.3.4 DISPATCH A PROCESS

We will simply assume that the currently running process gets over (e.g. process with Pid =3 as in the last example) and therefore, there IS a need to dispatch a new process.

3.3.5 CHANGE THE PRIORITY or A PROCESS

This system call is very simple and can be executed after the operating system is supplies with Pid and the new priority as parameters.

The Operating system accesses the PCB for that Pid.

If the scheduling algorithm is based on the process priorities, then the ready list and the corresponding header is updated to reflect the new change. We will assume in our example for simplicity that after the change of the priority of a process, the sequence of ready processes remains unchanged. Even then, reflecting the changes in the priorities in the PCBs is essential, because it may affect the placement of any PCBs added to the ready queue later on, depending upon their priorities.

3.3.6 BLOCK A PROCESS

Let us now assume that the running process with Pid=8 issues a system call to read a record. The process with Pid=8 will have to be blocked by a system call.

All CPU registers and other pointers in the context for Pid=8 are stored in the register save area of the PCB with Pid= 8.

The status field in the PCB with Pid=8 is updated to blocked.

PCB 8 is now added at the end of the blocked list. We have seen why it is not necessary to link the blocked processes in any order such as by priority.

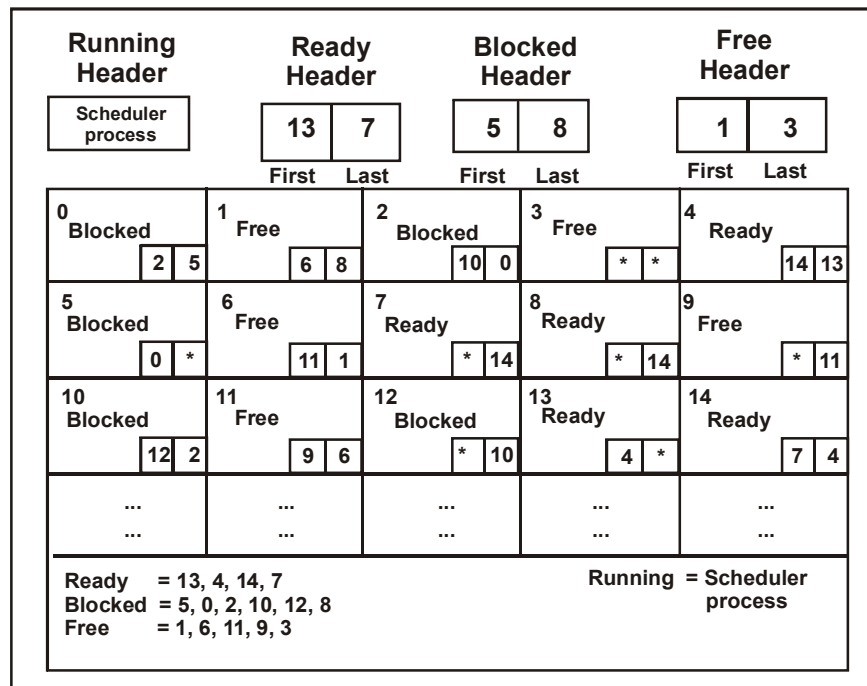


Fig 3.E-PCBs after blocking a process with PDI = 8

The running header is updated to reflect the change. We know that the scheduler process within the operating system is executing at this juncture.

The master list of known processes, as shown in Fig. 3. D is updated accordingly. The PCBs will look as shown in Fig. 3. E

3.3.7 TIME UP A PROCESS

The operating system uses this interrupt to switch between processes so that a process is prevented from grabbing the CPU endlessly. At this juncture, the operating system executes

a system call: “process time up”, given the Pid. Let us assume that the time slice is up for our running process with Pid= 13. The operating system now proceeds in the following fashion.

The operating system saves the CPU registers and other details of the context in the register save area of the PCB with Pid= 13.

It now updates the status of that PCB to ready. It may be noted that the process is not waiting for any external event, and so it is not blocked.

The process with Pid = 13 now is linked to the chain of ready processes. This is done as per the scheduling philosophy as discussed before. Meanwhile, let us assume that, externally, the priorities of all other ready processes have been increased more than that of 13, and hence, the PCB with Pid=13 is added at the end of the ready queue. The ready header is also changed accordingly.

The running header is updated to denote that the scheduler process is executing.

The master list of known processes as shown in Fig. 3E is now updated to reflect this change. The PCBs now look as shown in Fig. 3.G

3.3.8 WAKE UP A PROCESS

The hardware itself generates an interrupt.

The device which has generated this interrupt and the corresponding Interrupt Service Routine (ISR) are identified first (directly by hardware in the case of vectored interrupts or by software routine otherwise).

The interrupt Service Routine (ISR) for that device is executed.

The ISR accesses the Device Control Block (DCB) for that device. We have discussed the DCB in the information Management (IM) module. As we know, the DCB maintains a list of processes waiting on that device. The Operating System also knows from the DCB, the current process for which I/O is completed. The Pid of this process is of importance.

Now, the ISR executes the wake up system call for that specific process. Let us assume that in our example, the I/O is completed on behalf of a process with Pid =2 and therefore, it needs to be woken up.

The operating system changes the status of a process with Pid=2 to ready.

It removes the process with Pid=2 from the blocked list and also updates the blocked header if needed (in this case, it is not necessary).

It chains the process with Pid =2 in the list of ready processes. As we know, this is done as per the scheduling philosophy. We assume that this PCB is added at the end of the ready list. It also updates the ready header if necessary (in this case, it is necessary).

It updates the master list of known processes.

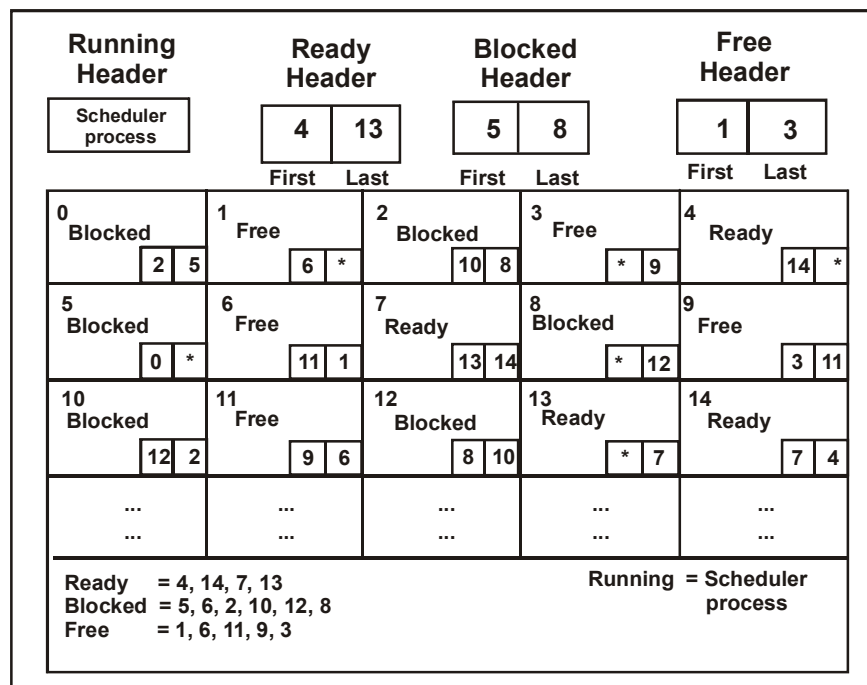


Fig. 3.H - Time up a process with PID = 13

The PCBs now look as shown in Fig. 3.H assuming that the process with PCB =4, shown in the Fig. 3.G which is at the head of the ready list, is already dispatched.

This completes our description of process state transitions. As noted before, the model was based on only a few important process states. We had ignored the process states such as new and halted. There are also a few more process states which complicate the matters further. Some of these are created because of operations such as Suspend or Resume. We will now study these.

3.3.9 PROCESS SCHEDULING

3.3.9.1 Scheduling Objectives

While scheduling various processes, there are many objectives for the operating system to choose from. Some of these objectives are to be achieved, before designing an operating system. Some of the objectives are listed below:

- Fairness
- Good throughput
- Low turnaround time
- Low waiting time
- Good response time

Some of these objectives are conflicting. We will illustrate this by considering fairness and throughput.

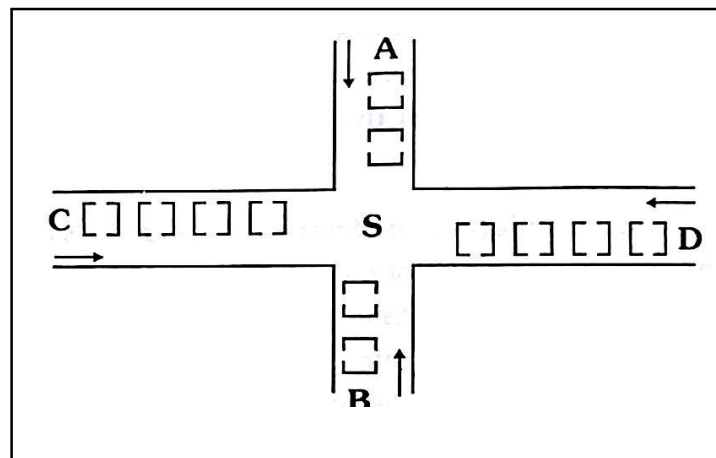


Fig 3.1 Traffic Signals

Fairness refers to being fair to every user in terms of CPU time that he gets. Throughput refers to the total productive work done by all the users put together. Let us consider traffic signals as an example (Fig 3.1) to understand these concepts first and then see how they conflict.

There is a signal at the central point S which allows traffic in the direction of AB, BA or CD and DC. We assume the British method of driving and signals in our examples. Imagine that

there are a number of cars at point S, standing in all the four directions. The signaling system gives a time-slice for traffic in every direction. This is common knowledge. We define throughput as the total number of cars passed in all the directions put together in a given time. Every time the signal at S changes the direction, there is some time wasted in the context switch for changing the lights from green to amber and then subsequently to red. For instance, when the signal is amber, only the cars which have already started and are halfway through are supposed to continue. During this period, no new car is supposed to start (at least in principle) and hence, the throughput during this period is very low.

If the time slice is very high, say 4 hours each, the throughput will be very high, assuming that there are sufficient cars wanting to travel in that direction. This is true, because there will be no time lost in the context switch procedure during these 4 hours. But then, this scheme will not be fair to the cars in all the other directions at least during this time. If this time slice is only 1 hour, the scheme becomes fairer to others, but the throughput falls because, the signals are changing direction more often. Therefore, the time wasted in the context switch is more. Waiting for 1 to 4 hours at a signal is still not practical. If this time slice is 5 minutes, the scheme becomes still fairer, but the throughput drops still further. At the other extreme, if the time slice is only 10 seconds, which is approximately equal to the time that is required for the context switch itself, the scheme will be fairest, but the throughput will be almost 0. This is because, almost all the time will be wasted in the context switch itself. Hence, fairness and throughput are conflicting objectives. Therefore, a good policy is to increase the throughput without being unduly unfair.

The operating system also is presented with similar choices as in the case of street signals. When the operating system switches from one process to the next, the CPU registers have to be saved/restored in addition to doing this period, totally useless work is being done from the point of view of the user processes. If the operating system switches from one process to the next too fast, it may be more fair to various processes, but then the throughput may fall. Similarly, if the time slice is far much, the throughput will increase (assuming there are a sufficient number of processes waiting and which can make use of the time slice), but then it may not be a very fair policy.

Let us briefly discuss the meaning of other objectives. CPU utilization is the fraction of the time that the CPU is busy on the average, executing either the user processes or the operating system. If the time slice is very small, the context switches will be more frequent. Hence, the CPU will be busy executing the operating system instructions more than those of the user

processes. Therefore the throughput will be low, but the CPU utilization will be very high, as this objective does not care what is being executed, and whether it is useful. The CPU utilization will be low, only if the CPU remains idle.

Turnaround time is the elapsed time between the time a program or job is submitted and the time when it is completed. It is obviously related to other objectives.

Waiting time is the time a job spends waiting in the queue of the newly admitted processes for the Operating system to allocate resources to it before commencing its execution. This waiting is necessary due to the competition from other jobs/processes in a multiprogramming system. It should be clear by now that the waiting time is included in the turnaround time.

The concept of response time is very useful in time-sharing or real-time systems. Its connotation in these two systems is different and therefore, they are called terminal response time and event response time respectively, in these two systems. Essentially, it means the time to respond with an answer or result to a question or an event and is dependent on the degree of multiprogramming, the efficiency of the hardware along with the operating system and the policy of the operating system to allocate resources.

3.3.9.2 Concepts of Priority and Time Slice

An external priority is specified by the user externally at the time of initiating the process. In many cases, the operating system allows the user to change the priority externally even during its execution. If the user does not specify any external priority at all, the operating system assumes a certain priority called the default priority. In many in-house situations, most of the processes run at the same default priority, but when an urgent job needs to be done (say for the chairman), the system manager permits that process to be created with a higher priority.

In data centre situations where each user pays for the time used, normally higher priority processes are charged at a higher rate to prevent each user from tiring his job at the highest priority. This is known as the scheme of purchased priorities. It is the function of the operating system to keep track of the time used by each process and the priority at which it was used, so that it can then perform its accounting function.

The concept of internal priority is used by some scheduling algorithms. They base their calculation on the current state of the process. For example, each user, while firing a process, can be forced to also specify the expected time that the process is likely to take for completion.

The operating system can then set an internal priority which is highest for the shortest job (Shortest job First or SJF algorithm), so that at only a little extra cost to large jobs, many short jobs will complete. This has two advantages. If short jobs are finished faster, at any time, the number of processes competing for the CPU will decrease. This will result in a smaller number of PCBs in the ready or blocked queues. The search times will be smaller, thus improving the response time. The second advantage is that if smaller processes are finished faster, the number of satisfied users will increase. However, this scheme has one disadvantage. If a stream of small jobs keeps on coming in, a large job may suffer from indefinite postponement. This can be avoided by setting a higher external priority to those important large jobs. The operating system at any time calculates a resultant priority based on both external and internal priorities, using some algorithm chosen by the designer of the operating system.

§ A process consumes the full time slice or

§ A process requests an I/O before the time slice is over. In this case also, a process switch is done, because, there is no sense in wasting the remaining time slice just waiting for the I/O to complete. The context switch time consisting of all CPU/memory related instructions within an operating system routine are far less time consuming than the I/O that a process is waiting for.

3.3.9.3 Scheduling Philosophies

There are basically two scheduling philosophies: Non-Preemptive and Preemptive. Depending upon the need, the Operating System designers have to decide upon one of them.

A Non preemptive philosophy means that a running process retains the control of the CPU and all the allocated resources, until it surrenders control to the operating system (on its own.) This means that, even if a higher priority process enters the system, the running process cannot be forced to give up the control. However, if the running process becomes blocked due to any I/O request, another process can be scheduled because, the waiting time for the I/O completion is too high. This philosophy is better suited for getting a higher throughput due to less overheads incurred in context switching, but it is not suited for real time systems, where higher priority events need an immediate attention and therefore, need to interrupt the currently running process.

A pre-emptive philosophy on the other hand allows a higher priority process to replace a currently running process, even if its time slice is not over or it has not requested for any I/O. This requires context switching more frequently, thus reducing the throughput, but then it is

The scheme works as follows:

If the number at ready processes in the ready queue becomes very high, the overhead on the operating system for maintaining long lists, context switching and dispatching increases. Therefore, it is wise to let in only a limited number at processes in the ready queue to compete for the CPU. The long term scheduler manage this. It disallows processes beyond a certain limit for batch processes first and in the end also the interactive ones.

At any time, the main memory at the computer is limited and can hold only a certain number of processes. If the availability of the main memory becomes a great problem, and a process gets blocked, it may be worthwhile to swap it out on the disk and put it in yet mother queue for a process state called swapped out and blocked which is different from a queue of only blocked processes, hence, requiring a separate PCB chain.

One Option is to retain the data in the memory buffer of the operating system and transfer it to the I/O area of the process after it gets swapped in. this requires a large memory butter for the operating system because the operating system has to define these buffers for every process as a similar situation could arise in the case of every process. Another Option is to transfer the data to the disk in the process image at the exact location (e.g. I/O area), so that when the process is swapped in, it does so along with the data record in the proper place. After this, it can be scheduled eventually. This requires less memory but more I/O time.

When some memory gets freed, the operating system looks at the list of swapped but ready processes, decides which one is to be swapped in (depending upon priority, memory and other resources required, etc.) and after swapping it in, links that PCB in the chain of ready processes for dispatching. This is the function of the medium term scheduler as shown in Fig 3.J. It is obvious that this scheduler has to work in close conjunction with the long term scheduler. For instance, when some memory gets freed, there could be competition for it from the processes managed by these two schedulers.

The short term scheduler decides which of the ready processes is to be scheduled or dispatched next.

3.3.10 MULTITASKING

3.3.10.1 Introduction

A task can be defined as an asynchronous code path within a process. Hence, in operating system which support multitasking, a process can consist of multiple tasks, which can run simultaneously in the same way that a multiuser operating system supports multiple processes at the same time. In essence, multiple tasks should be able to run concurrently within a process.

Let us illustrate the concept of multiple tasks using an example without multitasking first and then using one with it. Let us consider a utility which reads records from a tape with a blocking factor of 5, processes them (may be selects or reformats them) and writes them onto a disk one by one. Obviously, the speed of the input or read operation may be quite different from the speed of output or write operation. The logic of the program is given in Fig. 3.K.

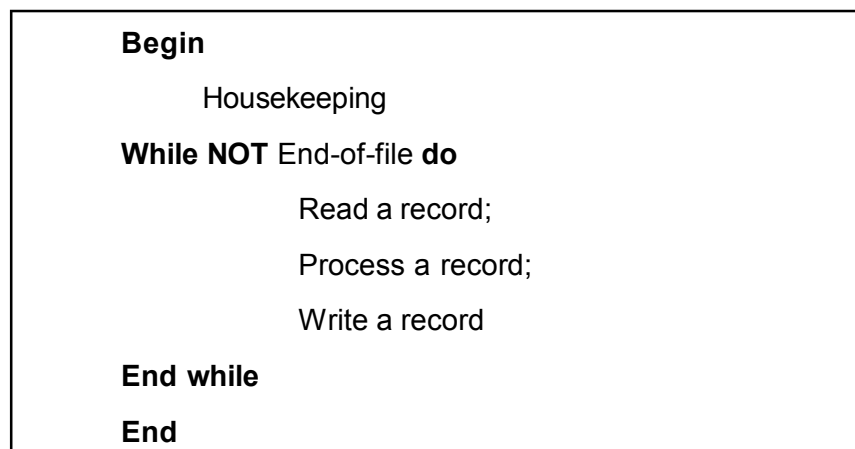


Fig. 3.K A typical program with I/O

The programmer (in this case the one who is writing this tape to disk copy utility) defines two tasks within the same process as shown in Fig 3.L. The advantage is that they can run concurrently within the same process, if synchronized properly. We need not bother about the exact syntax with which a programmer can define a task within a process. Let us just assume that it is possible.

The computer recognizes these as different tasks and maintains their identity as such in the executable code. In our example in Fig.3.L, a task encapsulated between Task-N-Begin and Task-N-End statements for task N.

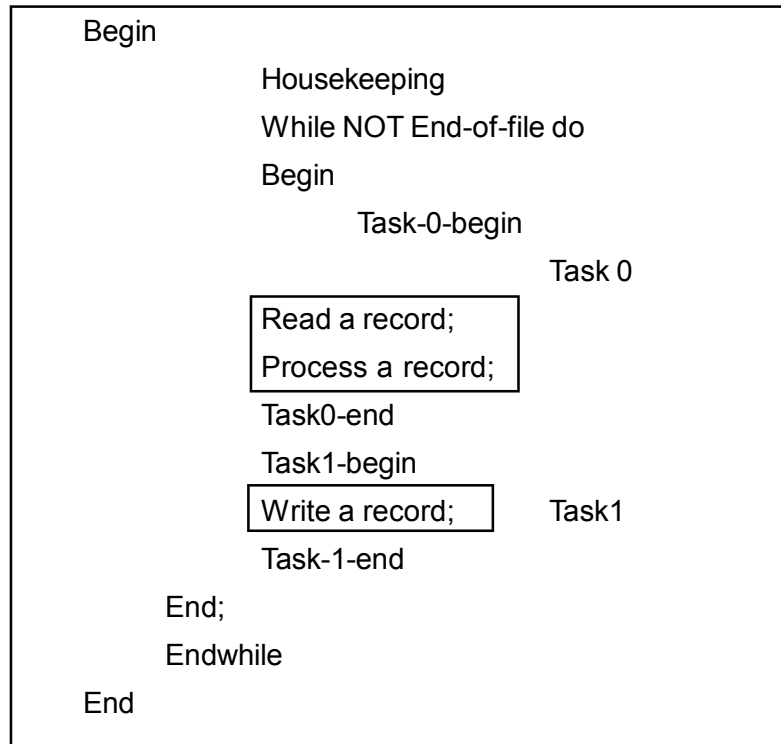


Fig. 3.L – Concept of multiple tasks

When the process starts executing, the Operating system creates PCB as usual, but now in addition, it also creates a Task Control Block (TCB) for each of the recognized and declared tasks within that process. The TCB contains apart from other things, the register save area to store the registers at context switch of a task instead of process.

The tasks also could have priorities and states. A task can be in a ready, blocked or running state, and accordingly all the TCBs are linked together in the same way that PCBs are linked in different queues with their separate headers.

When the operating system schedules a process with multiple tasks and all allocated a time slice to it, the following happens:

The operating system selects the highest priority ready task within that process and schedules it.

At any time, if the process's time slice is over, the operating system turns the process as well as currently running task into ready state from running state.

If the process time slice is not over but the current task is either over or blocked, the operating system chooses the next highest priority ready task within that process and schedules it. But, if there is no ready task left to be scheduled within that process, only then does the Operating system turn the process state into a blocked state.

Different tasks need to communicate with each other like different processes do. Our example can be treated as a producer consumer problem. The tape-read is the producer task and disk-write is the consumer task. Hence, in multitasking, both inter Task Communication and Task Synchronization are involved and the operating system has to solve the problems of race conditions through mutual exclusion.

3.3.10.2 Multithreading

OS/2 uses a concept by the name multithreading. It means the same thing as multitasking, where, a thread is same as a task. Each thread for a process has a program counter, stack and registers save are (as in TCB), but it shares the same address space with others. As different threads read and write the same memory locations, interthread communication can be achieved easily. Dynamically new threads can be created, priorities can be changed and so on.

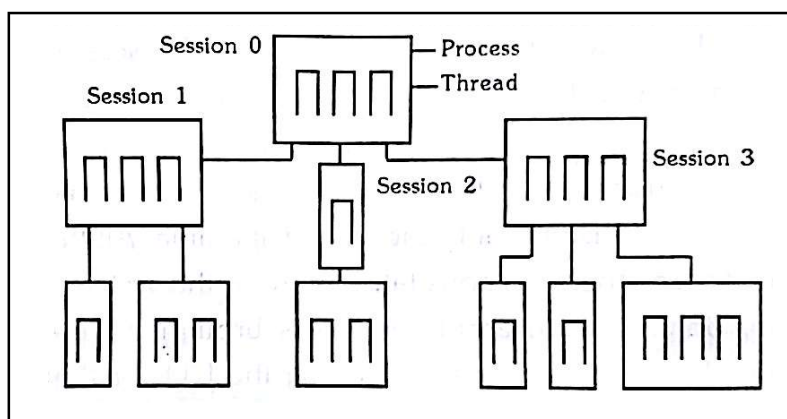


Fig. 3.M – OS/2 sessions, processes and threads

In fact, OS/2 has another concept called session which consists of multiple processes and each process consists of multiple threads. The hierarchy has an interesting property. A session can give rise to a child session in the same way that a process can give rise to a child process. This is shown in Fig. 3.M, which shows four sessions, ten processes shown as rectangles and nineteen threads shown as discrete figures within the rectangle. It is easy to imagine how the scheduling algorithm can become complicated in such a scenario.

The idea of providing multiple sessions is to simulate multiwindow environment on a dumb terminal. Each session has its own virtual terminal with a keyboard, mouse and screen.

The idea is simple. Each session will have some screen I/O instructions. They are executed by simulating the screen in the memory, i.e. as if there were a number of video RAMs one for each session. When the session outputs something to the screen, it is actually output to these memory locations. However, there is only one actual physical screen and the output of only one session running in the foreground is output to the screen at a time. All the screen I/O for others sessions takes place in the system's memory only. If a different session is brought to the foreground, the screen starts displaying the I/O for that session and all others continue in the background. For example, a user can start a session in the background copying selected records onto another file based on certain criteria. A third background session could be carrying out a compilation. At the same time, the user could run an editor in the foreground.

If the user presses the "ALT" and "ESC" keys together known as the hot key, OS/2 moves the editor into a background session temporarily, and brings the first background session (in this case SORT) into foreground. This means that all the messages displayed by that session will be displayed on the actual screen now. If that session needs any input, that prompt will appear on the screen (virtual terminal for session 1 becomes the real terminal). At this juncture, the screen I/O for the restwhile foreground process can go on in the memory area o the virtual terminal.

3.4 INTER-PROCESS COMMUNICATION

3.4.1 THE PRODUCER-CONSUMER PROBLEMS

Let us assume that there are multiple users at different terminals running different processes, but each one running the same program. This program prompts for a number from the user and on receiving it, deposits it in a shared variable at some common memory location. As these processes produce some data, they are called 'Producer processes'. Let us also imagine that there is another process which picks up this number as soon as any producer process outputs it and prints it. This process which uses or consumes the data produced by the producer process is called 'Consumer process'. We can, therefore, see that all producer processes communicate with the consumer process through a shared variable where the shared data is deposited.

The beauty of this process is that the user is not aware of this shared file. UNIX manages it for him. Thus, it becomes a vehicle for the 'Inter Process Communication (IPC)'. However, one thing should be remembered. A pipe connects only two processes, i.e. it is shared only between two processes, and it has a "direction" of data flow. A shared variable is a much more general concept. It can be shared amongst many processes and it can be written/read arbitrarily.

Another example of a producer-consumer situation and the IPC is the spooler process within the operating system. The operating system maintains a shared list of files to be printed for the spooler process to pick up one by one and print. At any time, any process waiting to print a file adds the file name to this list. Thus, this shared list becomes a medium of IPC. This is depicted in Fig. 3.N

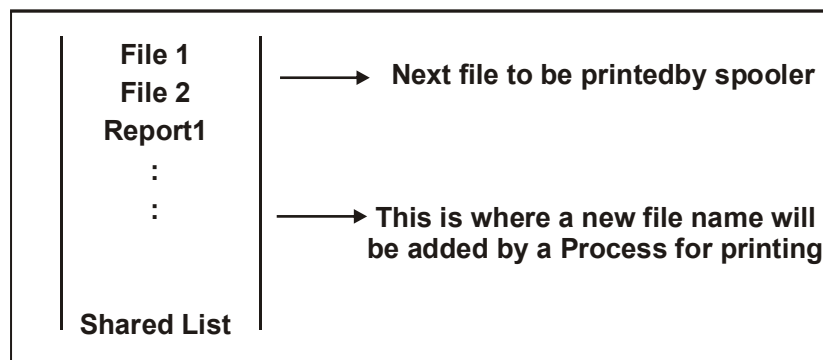


Fig 3.N - IPC in spooling

Sometimes, two or more processes need to be synchronized based on something. The common or shared variables again provide the means for such synchronization. For instance, let us imagine that process A has to perform a task only after process B has finished a certain other task. In this case, a shared variable (say, a flag) could be used to communicate the completion of the task by process B. After this, process A can check this flag and proceed, depending on the flag, in the end resetting the flag for future.

In a sense, all the examples discussed above are typical of both 'Process Synchronization' and IPC, because both are closely related. For example, in the first case, unless any of the producer processes outputs a number, the consumer process should not try to print anything. Again, unless the consumer process prints it, none of the producer processes should output the next number if overwriting is to be avoided (assuming that there is only one shared variable). Thus, it is a problem of process synchronization again!

There is however, a serious problem in implementing these schemes. Let us again go back to our first example. Let us see how we can achieve this synchronization to avoid overwriting. Let us imagine that apart from a shared variable to hold the number, we also have another flag variable which takes on the value of 0 or 1. The value of the flag is 1, if any of the producer processes has output a number. Hence, no producer process should output a new number if this flag=1, to avoid overwriting. Similarly, the consumer process will print the number only if the flag=1 and will set the flag to 0, thereafter. Again, the consumer process should not print anything, if this flag=0 (i.e. nothing is ready (or printing)). We illustrate this in the program in Fig. 3.O.

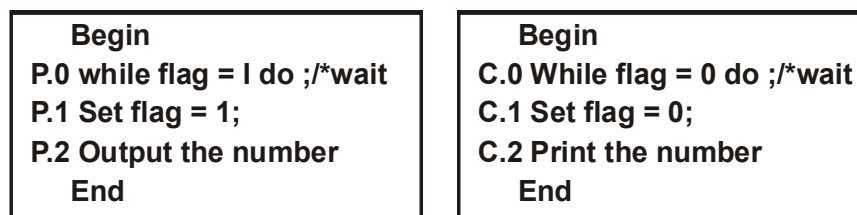


Fig 3.O Programs in producer/ consumer processes

In this scheme, as we know, instruction P.0 i.e. “while flag =1 do;” is a wait loop so long as the flag continues to be = 1. The very moment the flag becomes 0, the program goes down to step P.1 and thereafter to P.2 whereupon the flag is set to 1. When a process reaches instruction P.2, it means that some producer process has output the number in a shared variable at instruction P.1. At this juncture, if another producer process tries to output number, it should be prevented from doing so, in order to avoid overwriting. That is the reason, the flag is set to 1 in the instruction P.2. After this, the while-do wait loop precisely achieves this prevention. This is because, as long as the flag=1, the new producer process cannot proceed. A similar philosophy is applicable for instructions C.0, C.1 and C.2 of the consumer process.

For instance, the consumer process does not proceed beyond C.0 as long as the flag continues to be =0, which indicated that there is nothing to print. As soon as the flag becomes 1, indicating that something is output and is ready for printing, the consumer process executes C.1 and C.2, whereupon the number is printed and the flag is again set to 0, so that subsequently the consumer process does not print non-existing numbers, but keeps on looping at C.0.

Everything looks fine; where then is the problem? The problem will become apparent, if we consider the following sequence of events.

Let us assume that initially the flag = 0.

One of the producer processes (PA) executes Instruction P.0. Because the flag = 0, it does not wait at P.0. but it goes to instruction P.1.

PA outputs a number in the shared variable by executing instruction P.1.

At this moment, the time slice allocated to PA gets over and that process is moved from running to ready state. The flag is still = 0.

Another producer process PB is now scheduled. (It is not necessary that a consume process is scheduled always after a producer process is executed once.)

PB also executes its P.0 and finds the flag as 0, and therefore, goes to its P.1

PB overwrites on the shared variable by instruction P.1 therefore, causing the previous data to be lost.

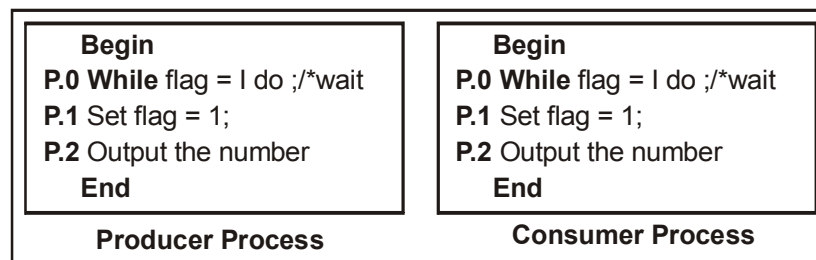


Fig 3.P Modified producer/consumer algorithm

Hence, there is a problem. An apparent problem is that setting of the flag to 1 in the producer process is delayed. If the flag is set to 1 as soon as a decision is made to output the number, but before actually outputting it, what will happen? can it solve the problem? Let us examine this further. The modified algorithms of the producer and consumer processes will be as shown in Fig. 3.P.

However, in this scheme, the problem does not get solved. Let us consider the following sequence of events to see why:

Initially flag = 0

PA executes instruction P.0 and falls through to P.1, as the flag = 0.

PA sets flag to 1 by instruction P.1.

The time slice for PA is over and the processor is allocated to another Producer.

Process PB

PB keeps waiting at its instruction P.O because flag is now = 1. This continues until its time slice also is over, without doing anything useful. Hence, even if the shared data item (i.e. the number in this case) is empty, PB cannot output the number. This is clearly wasteful, though it may not be a serious problem. Let us proceed further.

A consumer process CA is now scheduled. It will through C.O because flag = 1.

(it was set by PA in step (PA sets flag to 1 by instruction P. 1))

CA will set flag to 0 by instruction C.1.

CA will print the number by instruction C.2 before the producer has output it (may be the earlier number will get printed again!). This is certainly wrong!

Therefor, just proponing the setting of the flags does not work. What then is the solution?

Before going into the solution, let us understand the problem correctly. The portion in any program which accesses a shared resource (such as a shared variable in the memory) is called as 'Critical Section' or 'Critical Region'. In our example, instructions P.1 and P.2 of producer process or instructions C.1 and C.2 of consumer process constitute the critical region. This is because, both the flag and the data item where the number is output by producer process are shared variables. The problem that we were facing was caused by what is called 'race condition'. When two or more processes are reading or writing some shared data and the outcome is dependent upon which process runs precisely when, the situation can re called 'race condition'. We were clearly facing his problem in our example. This is obviously undesirable, because, the results are unpredictable. What we need is a highly accurate and predictable environment. How can we avoid race conditions?

A closer look will reveal that the race conditions across: because more than one process was in the critical region at the same time. One point must be remembered. A critical region here actually means a critical region of any program. It does not have to be of the same program. In the first example (Fig. 3.O), the problem across because both PA and PB were in the critical region of the same program at the same time. However, PA and PB were two producer processes running the same program. In the second example (Fig. 3P), the problem arose even if processes PA and CA were running separate programs and both were in their respective critical regions simultaneously. This should be clear by going through our example with both alternatives as in Figs. 3.O and 3.P. What is the solution to this problem then?

If we could guarantee that only one process is allowed to enter any critical region (i.e. of any process) at a given time, the problem of race condition will vanish. For instance, in any one of the two cases depicted in Figs 6.4 and 6.5, when PA has executed instruction P.! and is timed out (i.e. without completing and getting out of its critical region), and if we find some mechanism to disallow any other process (producer or consumer) to enter its respective critical regions, the problem will be solved. This is because no other producer process such as PB would be able to execute its Instructions C.1 or C.2. After PA is scheduled again, only PA would then be allowed to complete the execution of the critical region. Until that happens, all the other processes wanting to enter their own critical regions would keep waiting. When PA gets out of its critical region one of the other processes can now enter its critical region; and that is just fine. Therefore, what we want is 'mutual exclusion' which could turn out to be a complex design exercise. We will outline the major issues involved in implementing this strategy in the next section.

It is important to remember that mutual exclusion is a necessary, though not a sufficient condition, for a good operating system. This is because, the process for achieving mutual exclusion could be very expensive. We do not want an operating system which achieves mutual exclusion at the cost of being extremely slow, or by making processes wait for a very long time.

In fact, we can list five conditions which can make any solution acceptable. They are:

- (i) No two processes should be allowed to be inside their critical regions at the same time (mutual exclusion).
- (ii) The solution should be implemented only in the software, without assuming any special feature of the machine such as specially designed mutual exclusion instructions. This is not strictly a precondition but a preference, as it enhances portability to other hardware platforms, which may or may not have this facility.
- (iii) No process should be made to wait for a very long time before it enters its critical region (indefinite postponement)
- (iv) The solution should not be based on any assumption about the number of CPUs or the relative speeds of the processors.
- (v) Any process operating outside its critical region should not be able to prevent another process from entering its critical region.

We will now proceed to seek solutions which satisfy all these conditions.

3.4.2 SOLUTIONS

3.4.2. 1 Interrupt Disabling/Enabling

We know that we can have instructions to disable or enable interrupts. One solution is to disable all interrupts before any process enters the critical region. As we know, a process switch after a certain time slice happens due to an interrupt generated by the timer hardware. If all interrupts are disabled, the time slice of the process which has entered its critical region will never get over until it has executed and come out of its critical region completely. Hence, no other process will be able to enter into the Critical region simultaneously.

3.4.2.2 Primitives for Mutual Exclusion

Let us imagine that we have two primitives, Begin-Critical-Region and End-Critical-Region. We can use them as a boundary of a critical region in any process. They really act like security guards. The system uses these primitives to recognize the critical region and allows only one process to be in any critical region at any given time. Let us, for now, not bother about how these primitives are implemented, but assume that they exist.

The idea is that for any process when Begin- Critical-Region is encountered, the system checks if there is any other process in the critical region and if yes, no other process is allowed to enter into it. This guarantees mutual exclusion. If we retrace steps (i) to (vii) discussed in connection with Fig 6.4, we will realize this. For cross-reference, we have retained the numbers such as P.0.p.1, etc. We have added only P.S1.P.S2 and C.S1 and C.S2 as mutual exclusion primitives in Fig. 3.Q. The whole scheme will work in the following way:

- (i) Let us assume that initially the flag = 0.
- (ii) A producer process PA executes P.0. Because flag=0, it falls through to P.S1. Again, assuming that there is no other process in the critical region, it will fall through to P.
- (iii) PA outputs the number in a shared variable by executing P.1.
- (iv) Let us assume that at this moment the time slice for PA gets over, and it is moved into the 'Ready' state from the "Running' state. The flag is still 0.
- (v) Another producer process PB now executes P.0. It finds that flag=0 and so falls through to P.S.1.
- (vi) Because PA is in the critical region already, PB is not allowed to proceed further, thereby, avoiding the problem of race conditions. This is our assumption about the mutual exclusion primitives.

We can verify that the scheme works in many different conditions. The problem that remains now is only that of implementing these primitives.

3.4.2.3 Alternating Policy

This was the first attempt to arrive at the mutual exclusion primitives. It is based on the assumption that there are only two processes: A and B, and the CPU strictly alternates between them.

- (i) Let us assume that initially Process-ID is set to "A" and process A is scheduled. This is done by the operating system.
- (ii) Process A will execute instruction AD and fall through to A.1, because Process-ID="A",
- (iii) Process A will execute the critical region and only then Process-ID is set to "B" at instruction A2. Hence, even if the context switch takes place after A.0 or even after A.1. but before A2, and if process B is then scheduled (remember, we have assumed that there are only two processes!), process B will continue to loop at instruction B.0 and will not enter the critical region. This is because process-ID is still "A". Process B can enter its critical region only if process-ID = "B". And this can happen only in instruction A.2 which in turn can happen only after Process A has executed its critical region in instruction A.1.
- a) If there are more than two processes, the system can fail. Imagine three processes: PA1, PA2 executing the algorithm shown for Process A and FBI executing the algorithm shown for Process B. Now consider the following sequence of events when Process-ID = "A".

- § PA1 starts executing. It executes A.0 and falls through to A.1. Before it executes A.1, process switch takes place.
 - § PA2 starts executing, and because Process ID is still “A”, it also goes through A.0 to A1, and during A.1 (i.e. while in the critical region), again process switch takes place.
 - § PA1 resumes from. instruction A.1 and enters its critical region, thereby failing the scheme of mutual exclusion. Both PA1 and PA2 are in the critical region simultaneously.
- (b) This algorithm also involves busy waiting, and wastes the CPU time. If process B is ready and dispatched, it may waste the full time slice, waiting at instruction B.0, if Process-ID = “A”.
- (c) This algorithm forces processes A and B to alternate in a strict sequence. If the speeds of these two processes are such that Process A wants to execute again before process B takes over, it is not possible.

3.4.3 PETERSON'S ALGORITHM

This algorithm also is based on two processes only. It uses three variables. The first one called chosen-process takes the value of “A” or “B” depending upon the process chosen. This is as in the earlier case. PA-TO-ENTER and PB-TO-ENTER are two flags which take the value of either “yes” or “no”. For instance, if PA wants to enter the critical region, PA-TO-ENTER is set to “YES” to let PB know about PA's desire. Similarly PB-TO-ENTER is set to “YES”, if PB wants to enter its critical region so that PA can know about it, if it tests this flag. The following algorithm (Fig. 3.R) will clarify the concepts.

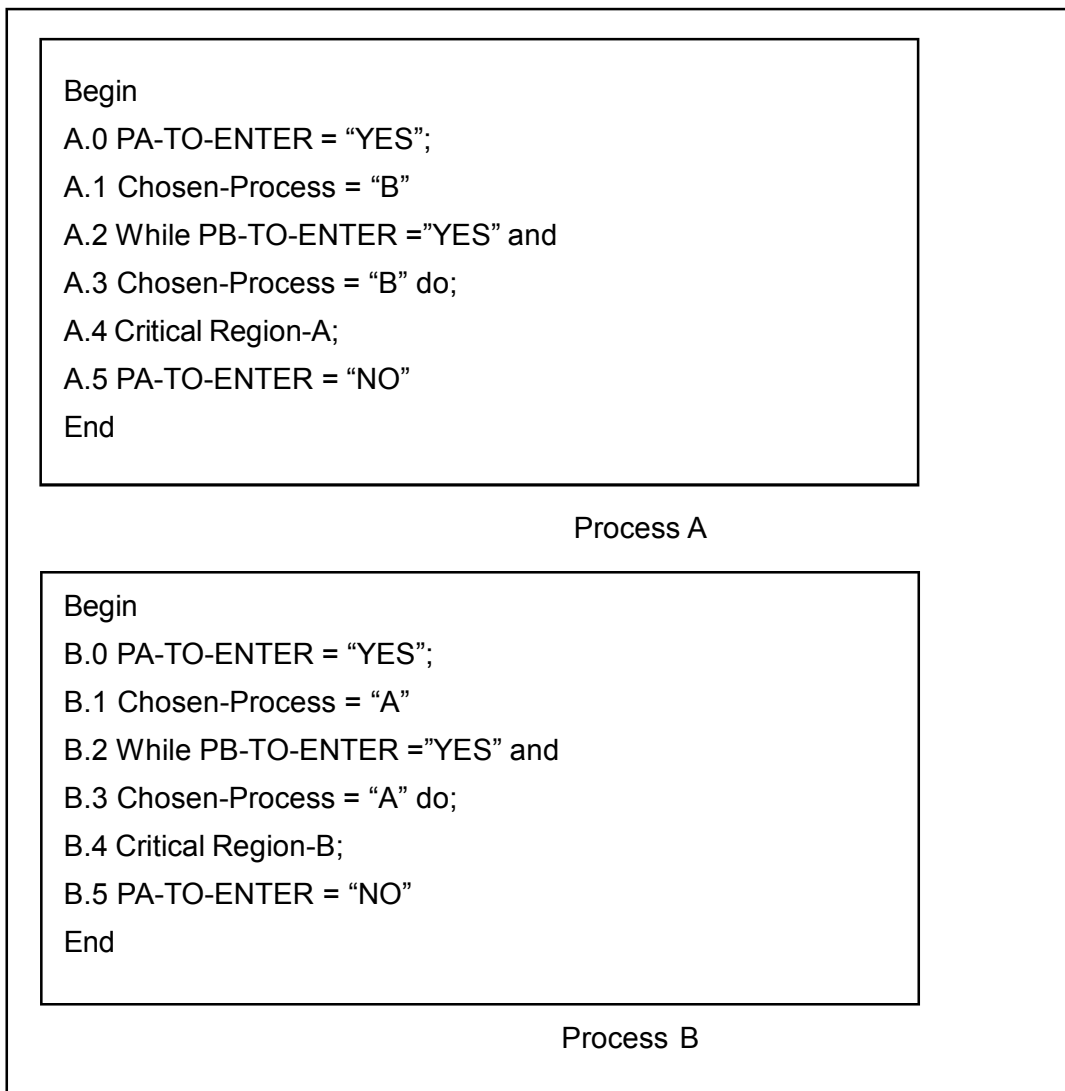


Fig. 3.R Perterson's algorithm

Let us assume that we start with the following values and trace the sequence of events
 PA-TO-ENTER = "NO",
 PB-TO-ENTER = "NO", and events.

Chosen-Process = "A"

- (i) Let us say PA is scheduled first.
- (ii) After executing A.0 and A.1, PA-TO-ENTER will become "YES", and Chosen Process will be "B", PB-TO-ENTER will continue to be "NO".
- (iii) At A.2 and A.3 (it is one statement only!), because PB-TO-ENTER is "NO", it will fail through to A.4. This is because of the "AND" condition in A.2. It will now start executing the Critical Region-A at A.4.

- (iv) Let us assume that at this time, process switch takes place and PB is scheduled. PA is still in its critical region.
- (v) PB will execute B.0 and B.1 to set PB-TO-ENTER to "YES" and Chosen-Process to "A". PA-ENTER will continue to be "YES".
- (vi) But at B.2, it will wait, because, both the conditions are met, i.e. PA-TO-ENTER = "YES" (in step(ii)) and Chosen-Process = "A" (in step (v)). Thus PB will be prevented from entering its critical region.
- (vii) Eventually when PA is scheduled again, it completes instruction A.5 to set PA-TO-ENTER to "NO", but only after coming and out of its critical region.
- (viii) Now, if PB is scheduled again, it will resume at instruction B.2, it will fall through to B.2 and B.3 (because PA-TO-ENTER is "NO" in step (vii)) to execute B.4 and enter the critical region of B.

However, this has happened only after PA has come out of its critical region.

3.4.4 SEMAPHORES

Concepts -Semaphores represent an abstraction of many important ideas in mutual exclusion. A semaphore is a protected variable which can be accessed and changed only by operations such as "DOWN" (or P) and "UP" (or V). It can be a "Counting Semaphore" or a "General Semaphore", where it can take on any positive value. Alternatively, it can be a "Binary Semaphore" which can take on the values of only 0 or 1. Semaphores can be implemented in software as well as hardware.

"DOWN and UP" form the mutual exclusion primitives for any process. Hence, if a process has a critical region, it has to be encapsulated between these DOWN and UP instructions. The general structure of any such process then becomes as shown in Fig 3.S

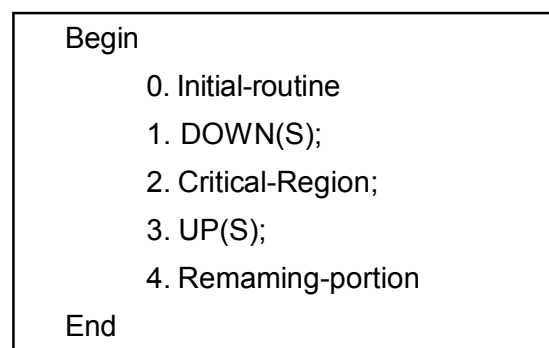


Fig. 3.S General structure of a process with Semaphores

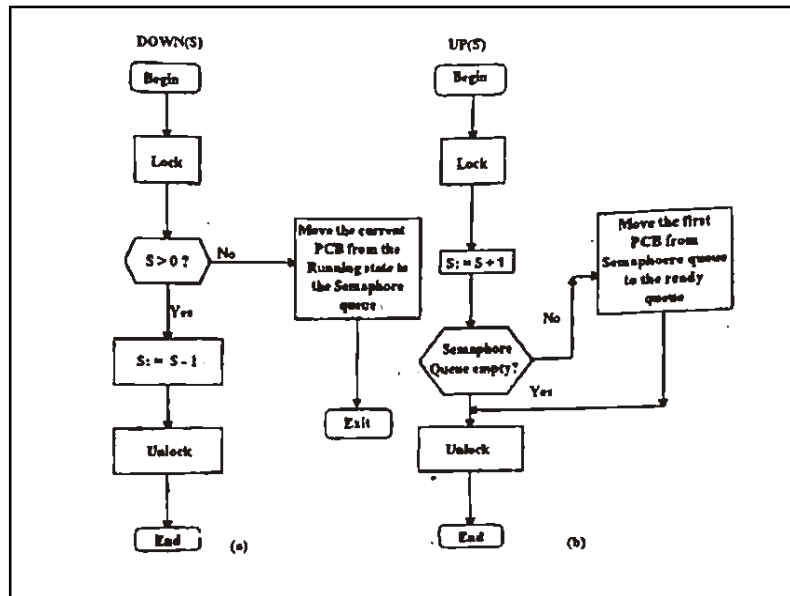


Fig. 3.T – DOWN(s) and UP(s) instructions

The “DOWN(S)” and “UP(S)” primitives ensure that only one process is in its critical region. All other processes wanting to enter their respective critical regions are kept waiting in a queue called a “Semaphore queue”. The queue also requires a queue header and all the PCBs in this queue also need to be chained in the same way as the ready and blocked queues. Hence, the operating system can traverse through all the PCBs for all the processes waiting on the Semaphore (i.e. waiting for the critical region to get free). Only when a process which is in its critical region comes out of it, should the operating system allow a new process to be released from the semaphore queue. Fig 3.T shows the flowcharts for “DOWN(S)” and “UP(S)” routines.

- (i) As is clear from Fig. 3.S, unless a process executes a DOWN(S) routine successfully without getting added to the semaphore queue at instruction 1, it cannot get into its critical region at instruction 2. Thus, if a process is in its critical region, we can safely assume that its DOWN(S) instruction must have been executed.

We now present the algorithms for DOWN(S) and UP(S) on a uniprocessor system. These are shown in Fig. 3.U.1 and Fig. 3.U.2 which tally with Fig. 6.14.

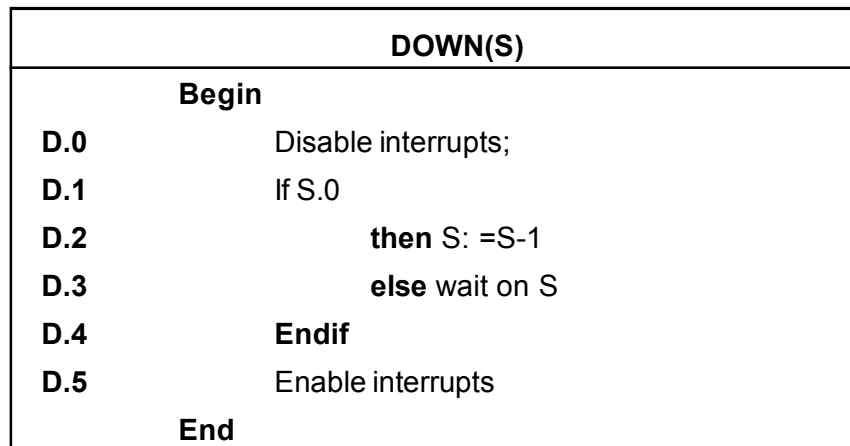


Fig 3.U.1 Down(s) operation

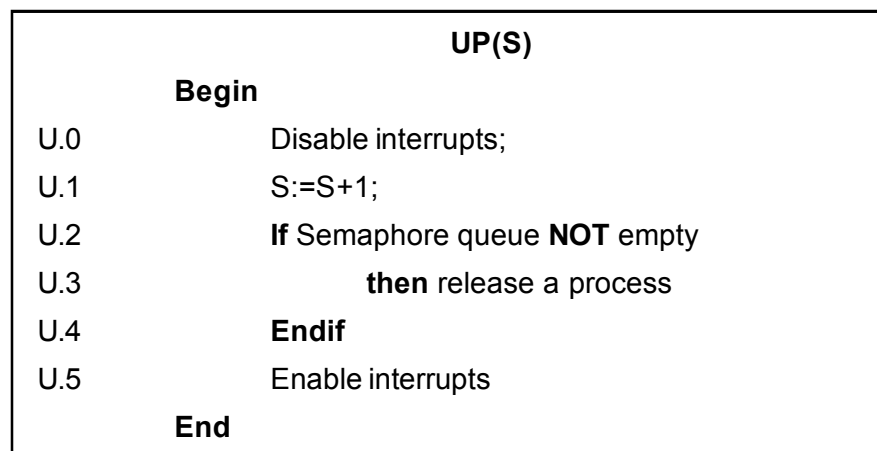


Fig. 3.U.2 UP(S) operation

In the figures above, "Wait on S" in D3 means moving the PCB of the running process in the semaphore queue.

"Release a process" in U.3 means moving the first PCB from the semaphore queue to the ready queue.

As we have seen, $S=0$ indicates that the DOWN(S) operation has been performed but UP(S) has not been completed. This means that there is a process in a critical region. At this time, there could be other processes wanting to enter their critical regions. They cannot be put in a blocked state. This is the reason why they are put in a semaphore queue, if S is not > 0 in the DOWN(S) routine. Thus, the semaphore queue is a list of PCBs for all the processes which are waiting for the critical region to get free. As soon as it gets free and UP(S) is performed, S is made=1 and a process at a time is admitted from the semaphore queue to the ready queue in UP(S) routine.

3.5 Summary

A process is a program in execution. As a process executes, it changes state. The state of a process is defined by that process's current activity. Each process may be in one of the following states: new, ready, running, waiting, or terminated. Each process is represented in the operating system by its own process control block (PCB). A process, when it is not executing, is placed in some waiting queue.

Communication in client-server systems may use (1) sockets, (2) remote procedure calls (RPCs), or (3) pipes. A socket is defined as an endpoint for communication. A connection between a pair of applications consists of a pair of sockets, one at each end of the communication channel. RPCs are another form of distributed communication. An RPC occurs when a process (or thread) calls a procedure on a remote application. Pipes provide a relatively simple way for processes to communicate with one another. Ordinary pipes allow communication between parent and child processes, while named pipes permit unrelated processes to communicate.

3.6 REVIEW QUESTIONS

1. What is Process?
2. Where are the various State and Process State Transition?
3. What are the benefits of a Multi-user Operating System?
4. Explain the use of the Process Control Block?
5. What are different Process Scheduling levels?
6. What is meant by Race Conditions?
7. What is Inter-Process Communication?
8. What is meant by Critical Region?
9. What is meant by Mutual Exclusion?
10. Give the algorithm for alternating Policy?
11. What are semaphores?

LESSON - 4

DEADLOCKS

Structure

- 4.1 Introduction
- 4.2 Objectives
- 4.3 Graphical Representation of a Deadlock
- 4.4 Deadlock Prerequisites
- 4.5 Deadlock Strategies
- 4.6 Summary
- 4.7 Review Questions

4.1 Introduction

In a multiprogramming environment, several processes may compete for a finite number of resources. A process requests resources; if the resources are not available at that time, the process enters a waiting state. Sometimes, a waiting process is never again able to change state, because the resources it has requested are held by other waiting processes. This situation is called a deadlock.

In this lesson, we describe methods that an operating system can use to prevent or deal with deadlocks. Operating systems typically do not provide deadlock-prevention facilities, and it remains the responsibility of programmers to ensure that they design deadlock-free programs.

4.2 Objectives

After studying this Lesson, you should be able to understand

- * Deadlocks
- * Graphical Representation of a deadlock
- * Deadlock Prerequisites
- * Deadlock Strategies
- * Avoid a Deadlock
- * Banker's algorithm

4.3 GRAPHICAL REPRESENTATION OF A DEADLOCK

To represent the relationship between processes and resources, a certain graphical notation is used.

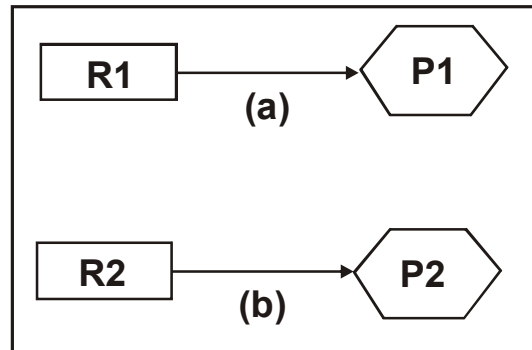


Fig 4.A - Relationship between resources and processes

Figure 4.A shows square boxes as resources named R1 and R2. Similarly, processes shown as hexagons are named P1 and P2. The arrows show the relationship. For instance, in part (a) of the figure, resource R1 is allocated to process P1, or in other words, P1 holds R1. In part (b) of the figure, process P2 wants resource R2, but it has not yet got it. It is waiting for it. (The moment it gets the resource, the direction of the arrow will change.)

These graphs are called 'Directed Resource Allocation Graphs(DRAG)'. They help us in understanding the process of detection of a deadlock, as we shall see.

Now let us imagine a typical scenario for the deadlock.

- * P1 holds R1 but demands R2
- * P2 holds R2 but demands R1

You will notice that there is a closed loop involved. Therefore, this situation is called a 'circular wait' condition. We should not get confused by the shape of the graph. For instance, the same DRAG can be drawn as shown in Fig. 4.B

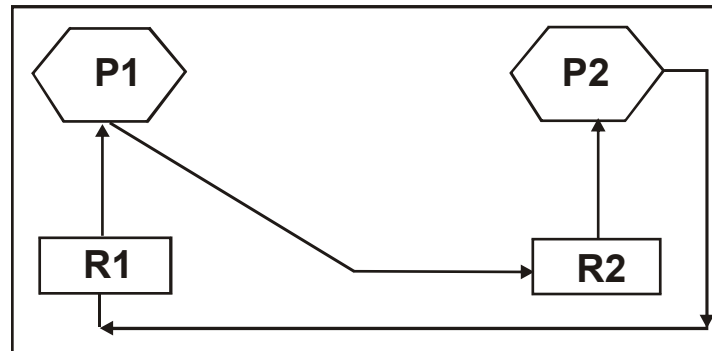


Fig. 4-B. Another version of circular wait

If you start from any node and follow all the arrows, you must return to the original node. This is what it is a circular wait or a deadlock situation. The shape is immaterial.

This principle is used by the operating system to detect deadlocks. However, what we have presented is a simplistic picture. In practice, the DRAGS can get very complicated, and therefore, the detection of a deadlock is never so simple! At any moment, when the operating system realizes that the existing processes are not finishing for an unduly long time, it can find out whether there is a deadlock situation or not. When any process waits for a resource, it is again the operating system which keeps track of this situation of waiting. Therefore, the operating system knows which processes are holding which resources and which resources these processes are waiting on. In order to detect a deadlock, the Operating system can give some imaginary coordinate to the nodes, R and P. Depending upon the relationships between resources and processes (i.e. directions of the arrows), it can keep traversing, each time checking if it has returned to a node it has already travelled by, to detect the incidence of a deadlock.

4.4 DEADLOCK PREREQUISITES

A. Mutual Exclusion Condition

Resources must be allocated to processes at any time in an exclusive manner and not on a shared basis for a deadlock to be possible. For instance, a disk drive can be shared by two processes simultaneously. This will not cause a deadlock. But printers, tape drives, plotters, etc. have to be allocated to a process in an exclusive manner until the process completely finishes its work with it (which normally happens when the process ends). This is the cause of trouble.

B. Wait for Condition

Even if a process holds certain resources at any moment, it should be possible for it to request for new ones. It should not have to give up the already held resources to be able to request for new ones. If this is not true, a deadlock can never take place.

C. No Preemption Condition

If a process holds certain resources, no other process should be able to take them away from it forcibly. Only the process holding them should be able to release them explicitly.

D. Circular Wait Condition

Processes (P1, P2,.....) and Resources (R1,R2,.....) should form a circular list as expressed the form of a graph (DRAG). In short, there must be a circular (logically, and not in terms of the shape) chain of multiple resources and multiple processes forming a closed loop as discussed earlier.

It is necessary to understand that all these four conditions have to be satisfied simultaneously for the existence of a deadlock. If any one of them does not exist, a deadlock can be avoided.

4.5 DEADLOCK STRATEGIES

Various strategies have been followed by different operating systems to deal with the problem of a deadlock. These are listed below:

- * Ignore it
- * Detect it
- * Recover from it
- * Prevent it
- * Avoid it.

4.5.1 Ignore a Deadlock

There are many approaches one can take to deal with deadlocks. One of them, and of course the simplest, is to ignore them. Pretend as if you are totally unaware of them:

People who like exactitude and predictability do not like this approach, but there is a very valid reason to ignore a deadlock. Firstly, the deadlock detection, recovery and prevention algorithms are complex to write, test and debug. Secondly, they slow down the system considerably. As against that, if a deadlock occurs very rarely, you may have to restart the jobs, but then the time may be lost quite infrequently and may not be significantly large. UNIX follows this approach on the assumption that most users would prefer an occasional deadlock to a very restrictive, inconvenient, complex and slow system.

4.5.2 Detect a Deadlock

The graphs (DRAG) provide good help in doing this, as we have seen. However, normally, a realistic DRAG is not as straightforward as a DRAG between two processes (PLPZ) and two resources (R1 and R2) as depicted in 4.C. In reality, there could be a number of resource types such as printers, plotters, tapes and so on. For instance, the system could have two identical printers, and the operating system must be told about it at time of system generation. It could well be that a specific process could do with either of the printers when requested. The complexity arises due to the fact that allocation to a process is made of a specific resource by the operating system, depending upon the availability, but the request is normally made by the process to the operating system for only a resource type (i.e. any resource belonging to that type). A very large number of processes can make this DRAG look more complex and the deadlock detection more time consuming.

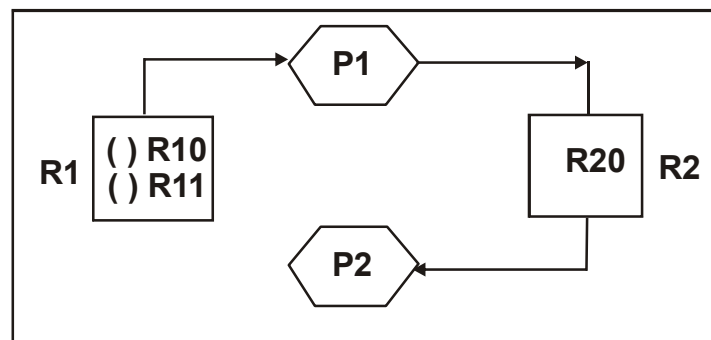


Fig 4.C - Multiple resources for a type

4.5.3 Recover from a Deadlock

Deadlock recovery becomes more complex due to the fact that some processes definitely lose something in the bargain. Basically, there are two approaches to solve this problem: suspending a process or killing it.

(i) Suspend/Resume a Process

In this method, a process is selected based on a variety of criteria (low priority, for instance) and it is suspended for a long time. The resources are reclaimed from that process and then allocated to other processes that are waiting for them. When one of the waiting processes gets over, the original suspended process is resumed.

This scheme looks attractive on the face of it, but there are several problems in its implementation. These are listed below:

Not all Operating Systems support the suspend/resume operations due to the overheads involved in maintaining so many more PCB chains for added process states and also due to the added system calls.

This strategy cannot be used in any on-line or real-time systems, because, the response time of some processes then becomes unpredictable, and clearly this is unacceptable.

Suspend/Resume operations are not easy to manage physically/programmatically for this purpose. Imagine that a tape is read half way through and then a process holding the tape drive is suspended. The operator will have to dismount that tape, mount the new tape for the new process to which the tape drive is now to be allocated. After this new process is over, when the old process is resumed, the tape for the original process will have to be mounted again and, more importantly, it will have to be exactly positioned. The problem with the printer is worse and can be easily imagined.

Therefore, this solution is not normally implemented.

(ii) Kill a Process

The Operating System decides to kill a process and reclaim all its resources after ensuring that such action will solve the deadlock. (The Operating System can use the DRAG and deadlock detection algorithms to ensure that after killing a specific process, there will not be a deadlock.) This solution is simple, but involves loss of at least one process.

Choosing a process to be killed, again, depends on the scheduling policy and the process priority. It is safest to kill a lowest priority process which has just begun, so that the loss is not very heavy. However, the matter becomes more complex when one thinks of a database recovery. (The process which is killed may have already updated some databases on-line) or Inter-Process-Communications.

4.5.4 Prevent a Deadlock

(i) Mutual Exclusion Condition

If every resource in the system were sharable by multiple processes, deadlocks would never occur. However, such sharing is not practicable. For instance, a tape drive, a plotter or a printer cannot be shared amongst several processes. At best, what one can do is to use the spooling techniques for the printer, where all the printing requests are handled by a separate program, thereby, eliminating the very need for sharing. When the spooler is holding the printer, no other process is even allowed to request for a printer, leave alone get it. All that a process is allowed to do is to add the data to the spooler to be printed subsequently.

(ii) Wait for Condition

A process to wait for more resources while already holding certain resources, we can prevent a deadlock.

This can be achieved by demanding that at the very beginning, a process must declare all the resources that it is expected to use. The operating system should find out at the outset if all these are available and only if available, allow the process to-commence. In such a case, the operating system obviously must update its list of free, available resources immediately after this allocation. This is an attractive solution, but obviously, it is inefficient and wasteful. If a process does calculations for 8 hours, updating some files and at the end, uses the tape drive for updating the control totals record only for one minute, the tape drive has to be allocated to that process for the entire duration and it will, therefore, be idle for 8 hours. Despite this, no other process can use it during this period.

(iii) No Preemption Condition

Guaranteeing a situation so that the “no preemption” condition is not met is very difficult. If we allow the resources allocated to a process to be taken away forcibly from it, it may solve the

problem of a deadlock, but it will give rise to worse problems. Taking away the tape drive forcibly from an incomplete process which has processed only part of the records on a tape drive, because, some other process requires it, will definitely be an unacceptable situation due to the problems of mounting/dismounting, positioning and so on. With printers, the situation is worse.

(iv) Circular Wait Condition

It is obvious that attacking the first three conditions is very difficult. Only the last one remains. If the circular wait condition is prevented, the problem of the deadlock can be prevented too.

One way in which this can be achieved is to force a process to hold only one resource at a time. If it requires another resource, it must first give up the one that is held by it and then request for another. This obviously has the same flaws as discussed above while preventing condition (iii). If a process P1 holds R1 and wants R2, it must give up R1 first because another process P2 should be able to get it (R1). We are again faced with a problem of assigning a tape drive to P2 after P1 has processed only half the records. This, therefore, is also an unacceptable solution.

There is a better solution to the problem, in which all resources are numbered as shown in Fig. 4.D

Number	Resource Name
0	Tape drive
1	Printer
2	Plotter
3	Card Reader
4	Card Punch

Fig. 4.D – Numbering of resources

A simple rule can tackle the circular wait condition now. Any process has to request for all the required resources in a numerically ascending order during its execution, assuming again that grabbing all the required resources at the beginning is not an acceptable solution. For

instance, if a process P1 requires a printer and a plotter at some time during its execution, it has to request for a printer first and then only for a plotter, because $1 < 2$.

4.5.5 Avoid a deadlock

Banker's algorithm maintains two matrices on a dynamic basis. Matrix A consists of the resources allocated to different processes at a given time. Matrix B maintains the resources still needed by different processes at the same time. These resources could be needed one after the other or simultaneously. The operating system has no way of knowing this.

Process	Tape Drivers	Printers	Plotters	Process	Tape Drivers	Printers	Plotters
P0	2	0	0	P0	1	0	0
P1	0	1	0	P1	1	1	0
P2	1	2	1	P2	2	1	1
P3	1	0	1	P3	1	1	1

Matrix A

Resources assigned

Matrix B

Resources still required

Vectors

Total Resources (T) = 543

Held Resources (H) = 432

Free Resources (F) = 111

Fig. 4-E. Banker's algorithm

Matrix A shows that process P0 is holding 2 tape drives at a given time. At the same moment, process P1 is holding 1 printer and so on. if we add these figures vertically, we get a vector of Held Resources (H) = 432. This is shown as the second row in the rows for vectors.

This says that at a given moment, total resources held by various processes are: 4 tape drives, 3 printers and 2 plotters. This should not be confused with the decimal number 432. That is why it is called a vector. By the same logic, the figure shows that the vector for the Total Resources (T) is 543. This means that in the whole system, there are physically 5 tape drives, 4 printers and 3 plotters. These resources are made known to the operating system at the time of system generation. By subtraction of (H) from (T) columnwise, we get a vector (F) of free resources which is 111. This means that the resources available to the operating system for further allocation are: 1 tape drive, 1 printer and 1 plotter at that juncture.

Matrix B gives processwise additional resources that are expected to be required in due course during the execution of these processes. For instance, process P2 will require 2 tape drives, 1 printers and 1 plotter, in addition to the resources already held by it. It means that process P2 requires in all $1 + 2 = 3$ tape drives, $2 + 1 = 3$ printers and $1 + 1 = 2$ plotters. If the vector of all the resources required by all the processes (vector addition of Matrix A and Matrix B) is less than the vector T for each of the resources, there will be no contention and therefore, no deadlock. However, if that is not so, a deadlock has to be avoided.

4.6 Summary

In this lesson, we have studied about the deadlocks and deadlock strategies. A deadlocked state occurs when two or more processes are waiting indefinitely for an event that can be caused only by one of the waiting processes. A deadlock can occur only if four necessary conditions hold simultaneously in the system: mutual exclusion, hold and wait, no preemption, and circular wait. To prevent deadlocks, we can ensure that at least one of the necessary conditions never holds. To avoid deadlock, we have studied about the Banker's algorithm. Various synchronization problems are important mainly because they are examples of a large class of concurrency-control problems. These problems are used to test nearly every newly proposed synchronization scheme. Operating systems also provide support for synchronization.

4.7 Review Questions

1. What is a Deadlock? Give an example
2. What is meant by Circular Wait?
3. What are Coffman's Prerequisites for a Deadlock?
4. Explain how a Deadlock can be represented graphically for two processes and two resources?
5. What is meant by Mutual Exclusion?
6. What is a Safe State?
7. What is a UnsafeState?
8. Discuss the Merits/Demerits of two Ways in which the Operating System can recover from a Deadlock?
9. What is a DRAG?
10. Describe Banker's algorithm to avoid a deadlock, What are the problems in its implementation.

LESSON - 5

MEMORY MANAGEMENT (MM)

Structure

- 5.1 Introduction**
- 5.2 Objectives**
- 5.3 Single Contiguous Memory Management**
- 5.4 Fixed Partioned Memory Management**
- 5.5 Swapping**
- 5.6 Relocation and Address Translation**
- 5.7 Variable Partitions**
- 5.8 Non-Contiguous Allocation – General Concepts**
- 5.9 Summary**
- 5.10 Review Questions**

5.1 Introduction

The main purpose of a computer system is to execute programs. These programs, together with the data they access, must be at least partially in main memory during execution. To improve both the utilization of the CPU and the speed of its response to users, a general-purpose computer must keep several processes in memory. Many memory-management schemes exist, reflecting various approaches, and the effectiveness of each algorithm depends on the situation. Selection of a memory-management scheme for a system depends on many factors, especially on the hardware design of the system.

In this lesson, we discuss various ways to manage memory. The memory management algorithms vary from a primitive bare-machine approach to paging and segmentation strategies. Each approach has its own advantages and disadvantages.

5.2 Objectives

After studying this Lesson, you should able to

- * Memory Management
- * Contiguous and Non-Contiguous Memory Allocation Schemes

- * Fixed Partitioned Memory Management
- * Variable Partitions
- * Protection and Sharing

5.3 SINGLE CONTIGUOUS MEMORY MANAGEMENT

In the scheme of single contiguous memory management, the physical memory is divided into two contiguous areas. One of them is permanently allocated to, the resident portion of the operating system (monitor) as shown in Fig. 5.A (CP/M and MS-DOS fall in this category.)

The operating system may be loaded at the lower addresses (0 to P as shown in Fig 8.2) or it can be loaded at the higher addresses. This choice is normally based on where the vectored interrupt service routines are located, because these addresses are determined at the time of hardware design in such computers.

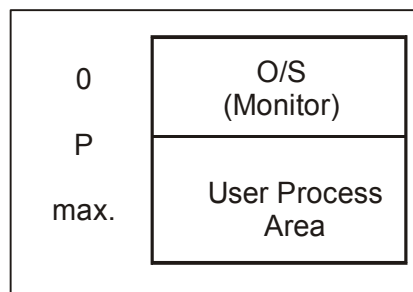


Fig. 5.A - Single Contiguous memory management

At any time, only one user process is in the memory. This process is run to completion and then the next process is brought in the memory. This scheme works as follows:

- * All the 'ready' processes are held on the disk as executable images-whereas the operating system holds their PCBs in the memory in the order of priority.
- * At any time one of them runs in the main memory.
- * When this process is blocked, it is 'swapped out' from the main memory to the disk.
- * The next highest priority process is 'swapped in' the main memory from the disk and it starts running.
- * Thus, there is only one process in the main memory, even if conceptually, it is a multi-programming system.

(i) Relocation/Address Translation

In this scheme, the starting physical address of the program is known at the time of compilation. Therefore, the problem of relocation or address translation does not exist. The executable machine program contains absolute addresses only. They do not need to be changed or translated at the time of execution.

(ii) Protection and Sharing

In 'Protection bits', a bit is associated with each memory block, because a memory block could belong either to the operating system or to the application process. Since there could be only these two possibilities, only 1 bit is sufficient for each block. However, the size of the memory block must be known. A memory block can be as small as a word or it could be a very large unit consisting of a number of words. Imagine a scheme in which a computer has a word length of 32 bits and 1 bit is reserved for every word for protection. This bit could be 0, if the word belongs to the operating system and it could be 1 if it belongs to the user process. At any moment, the machine is in the supervisor (or privileged) mode executing an instruction within the operating system, or it is in the 'user' mode executing a user process.

However, normally the operating system is allowed unrestricted access to all the memory locations, regardless of whether they belong to the operating system or a user process. (i.e. when the mode is privileged and the operating system makes any memory reference, this protection bit is not checked at all!) If a block is as small as a word of say 32 bits, protection bits continue $(1/32) \times 100 = 3.1\%$ overhead on the memory. As the block size increases, this overhead percentage decreases, but then the allocation unit increases. This has its own demerits such as the memory wastage due to the internal fragmentation.

The use of a fence register is another method of protection. This is like any other register in the CPU. It contains the address of the fence between the operating system and the user process as depicted in Fig. 5.B, where the fence register value = P.

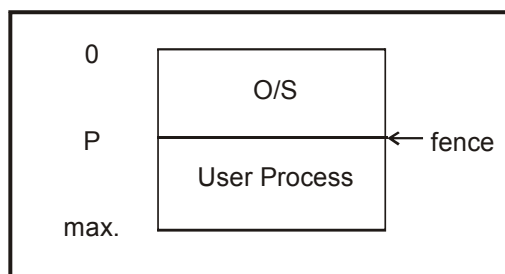


Fig. 5.B Fence register

5.4 FIXED PARTITIONED MEMORY MANAGEMENT

Introduction

To change the partitions, the operations have to be stopped and the operating system has to be generated (i.e. loaded and created), with different partition specifications. That is the reason why, these partitions are also called 'static partitions'. On declaring static partitions, the operating system creates a Partition Description Table (PDT) for future use. This table is shown in Fig. 5.C. Initially all the entries are marked as "FREE". However, as and when a process is loaded into one of the partitions, the status entry for that partition is changed to "ALLOCATED". Fig. 8.4 shows the static partitions and their corresponding PDT at a given time.

- (i) The long term process scheduler of the PM decides which process is to be brought into the memory next.
- (ii) It then finds out the size of the program to be loaded by consulting the IM portion of the operating system. As seen earlier, the compiler keeps the size of the "program in the header of the executable file.
- (iii) It then makes a request to the partition allocation routine of the MM to allocate a free partition with the appropriate size. This routine can use of the several algorithms for such allocations, as described later. The PDT is very helpful in this procedure.
- (iv) With the help of the IM module, it now loads the binary program in the allocated partition (Note that it could be loaded in an unpredictable partition, unlike the previous case, making Address Translation necessary at the run time).
- (v) It then makes an entry of the partition ID in the PCB before the PCB is linked to the chain of ready processes by using the PM module of the operating system.
- (vi) The routine in the MM now the status of that partition as "allocated" (ALLC).
- (vii) The PM eventually schedules this process.

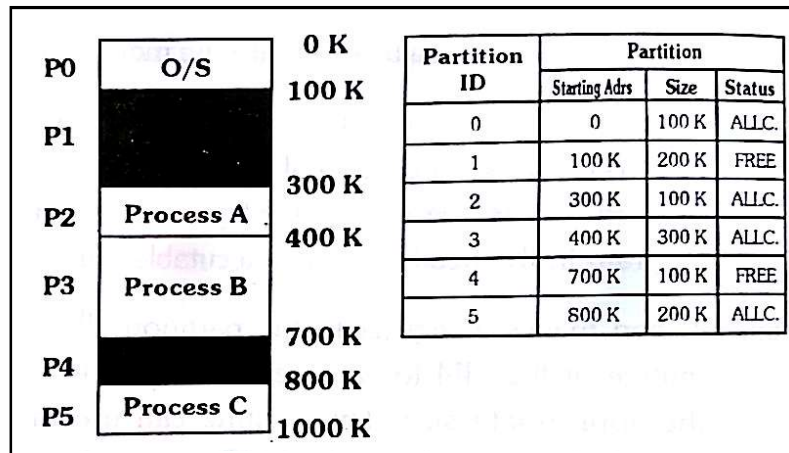


Fig. 5.C- Fixed partitions & Partition Description Table (PDT)

5.4.1 ALLOCATION ALGORITHMS

The operating system maintains and uses the PDT as shown in Fig. 5C. In this case, partition 0 is occupied by the Operating system and is thus unallocable. The “FREE” partitions are only 1 and 4. Thus, if a new process has to be loaded, we have to choose from these two partitions. The strategies of partition allocation are the same as discussed in disk space allocation, viz., first fit, best fit and worst fit. For instance, if the size of a program to be executed is 50k, both the first fit and the worst fit strategies would give partition ID =1 in the situation depicted by Fig. BC. This is because, the size of the partition with partition ID =1 is 200k which is > 50K. Also. it is the first free partition to accommodate this program. The best fit strategy for the same task would yield partition ID = 4. This is because the size of this partition is 100K, which is the smallest partition capable of holding this program. The best fit and the worst fit strategies would be relatively faster, if the PDT was sorted on partition size and if the number of partitions was very high.

The processes waiting to be loaded in the memory (ready for execution, but for the fact that they are on the disk or swapped out) are held in a queue by the operating system. There are two methods at maintaining this queue, viz., Multiple queues and Single queue.

5.4.2 MULTIPLE QUEUES

In multiple queues, there is one separate queue for each partition as shown in Fig.5.D. In essence, the linked list of PCBs in “ready but not in memory” state is split into multiple lists one for each partition, each corresponding to a different size of the partition. For instance, queue 0 will hold processes with size of 0-2K, queue 2 will be for processes with size between 2K and 5K (the process with exact size of 2K will be in this queue) and queue 1 will take care of processes with size between 5K and 8K. etc.

When a process wants to occupy memory, it is added to proper queue depending upon the size of the process. If the scheduling method is round robin within each queue, the processes are added at the end of the proper queue and they move ahead in the strict FIFO manner within each queue. If the scheduling method is priority driven, the PCBs in each queue are chained in the sorted order of priority.

An advantage of this scheme is that a very small process is not loaded in a very large partitions. It thus avoids memory wastage. It is instead added to a queue for smaller partitions. A disadvantage is obvious. You could have a long queue for a smaller partition whereas the queue for the bigger partition could be empty as shown in Fig. 8.6

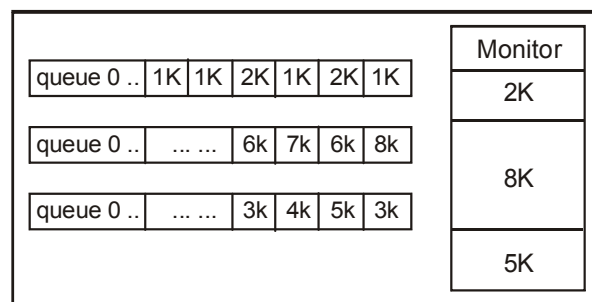


Fig. 5.D Multiple queues

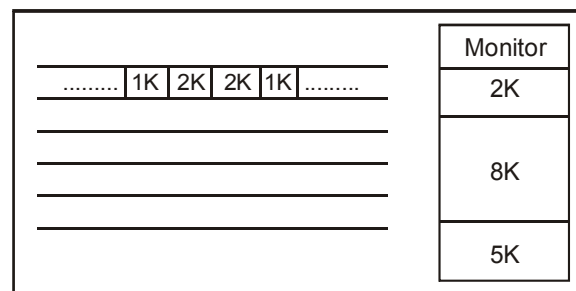


Fig. 5.E Memory utilization with multiple queues

5.4.3 SINGLE QUEUE

In the single queue method, only one unified queue is maintained of all the ready processes. This is shown in Fig. 5.F Again, the order in which the PCBs of ready processes are chained depends upon the scheduling algorithm. For instance, in priority based scheduling, the PCBs are chained in the order of priority. When a new process is to be loaded in the memory, the unified queue is consulted and the PCB at the head of the queue is selected for dispatching. The PCB contains the program size which is copied from the header of the executable file at the time a process is created. A free partition is found based on either first, best or worst fit algorithms. Normally, the first fit algorithm is found to be the most effective and the quickest.

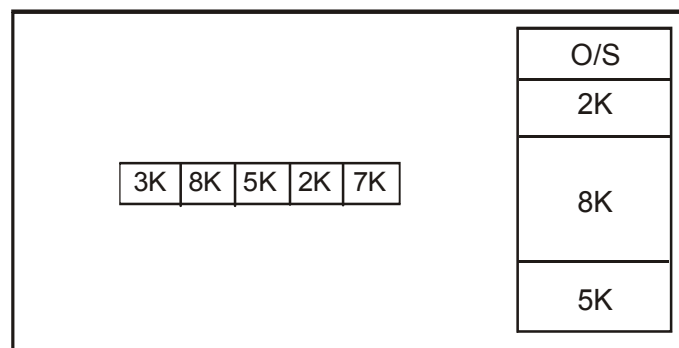


Fig. 5.F A single queue

After the process with size 7K is loaded, if the Operating system chooses a simple but relatively less intelligent solution and loads the process with size 2K in the partition with size 5K, the process with size 5K keeps waiting. After a while, even if the 2K partition gets free, it cannot be used, thus causing memory wastage. This is called external fragmentation. Contrast this with internal fragmentation in which there is memory wastage within the partition cannot be utilized. This wastage is due to internal fragmentation. This discussion shows that the MM and the PM modules are interdependent and that they have to cooperate with each other.

5.5 SWAPPING

One more way in which the partitioned memory management scheme is categorized is based on whether it supports swapping or not. Lifting the program from the memory and placing it on the disk is called 'swapping out'. To bring the program again from the disk into the main memory is called 'swapping in'. Normally, a blocked process is swapped out to make room for

a ready process to improve the CPU utilization. If more than one process is blocked, the swapper chooses a process with the lowest priority, or a process waiting for a slow I/O event for swapping out. As discussed earlier, a running process also can be swapped out.

Swapping algorithm has to coordinate amongst information, process and memory management systems. If the operating system completes the I/O on behalf of a blocked process which was swapped out, it keeps the data read recently in its own buffer. When the process is swapped in again, the operating system moves the data into the I/O are of the process and then makes the process 'ready'. In demand paging, some portion of the memory where the record is to be read can be 'locked' or 'bound' to the main memory. The remaining portion can be swapped out, if necessary. In this case, even if the process is 'blocked' and 'swapped out', the I/O can directly take place in the AP's memory. This is not possible in the scheme of 'fixed partition' because, in this case, the entire process image has to be in the memory or swapped out on the disk.

The operating system has to find a place on the disk for the swapped out process image. There are two alternatives. One is to create a separate swap file for each process. This method is very flexible, but can be very inefficient due to the increased number of, files and directory entries thereby deteriorating the search times for any I/O operation. The other alternative is to keep a common swap file on the disk and note the location of each swapped out process image within that file. In this scheme, an estimate of the swap file size has to be made initially. If a smaller area is reserved for this file, the operating system may not be able to swap out processes beyond a certain limit, thus affecting the performance.

5.6 RELOCATION AND ADDRESS TRANSLATION

5.6.1 Introduction

Imagine a program which is compiled with 0 as the starting word address. The addresses that this program refers to are called 'virtual addresses or logical addresses'. In reality, that program may be loaded at different memory locations which are called 'physical addresses'. In a sense, therefore, in all memory management systems, the problem of relocation and address translation is essentially to find a way to map the virtual addresses onto the physical addresses.

Address Translation (AT) must be done for all the addresses in all the instructions except for constant, physical I/O port addresses and offsets which are related to a Program Counter

(PC) in the PC relative addressing mode, because all these do not change depending upon where the instruction is located. There are two ways to achieve this relocation and AT: Static and Dynamic.

5.6.2 STATIC RELOCATION AND AT

This is performed before or during the loading the program in the memory, by the relocating linker or a relocating loader. In this scheme too, the compiler compiles the program assuming that the program is to be loaded in the main memory at the starting address 0. In this scheme, this relocating linker / loader uses this compiled object program (with 0 as the starting address) essentially as a source program and the starting address of the partition (where the program is to in loaded) as a parameter, as shown in Fig. 5.G. the relocating linker/loader goes through each instruction and changes the addresses in each instruction of the program before it is loaded and executed.

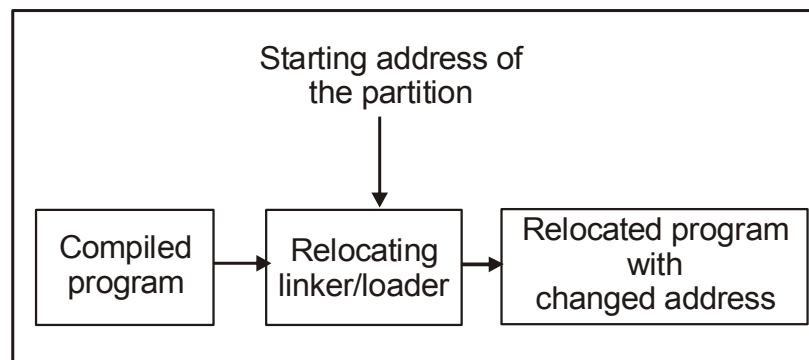


Fig. 5.G Static relocation

Obviously, the relocating linker/loader will have to know which portion of the instruction is an address, and depending upon the type of instruction and addressing mode, it will have to decide whether to change it or not (e.g. do not change PC relative addresses); and this is not very trivial.

This scheme was used in earlier IBM systems. It has two problems. Firstly, it is a slow process because it is a software translation. The software routine for relocation is also not trivial. Secondly, because it is slow, it is used only once before the initial loading of the program. Each time a process is swapped out which then needs to be swapped in, it becomes fairly expensive to carry out this relocation.

5.6.3 DYNAMIC RELOCATION

Dynamic relocation is used at the run time for each instruction. It is normally done by a special piece of hardware. It is faster, though somewhat more expensive. This is because, it uses a special register called “base register”. This register contains the value of relocation.

In this case, the compiled program is loaded at a starting memory location different than 0 (say, starting from 1000) without any change to any instruction in the program. For instance, Fig 8.9 shows the instruction “LDA 500” actually loaded at some memory locations between 1000 and 1500. The address 500 in this instruction is obviously invalid, if the instruction is executed directly. Hence, the address in the instruction has to be changed at the time of execution from 500 to 1500.

Normally, any instruction such as “LDA 500” when executed, is fetched to Instruction Register (IR) first, where the address portion is separated and sent to Memory Address Register (MAR). In this scheme however, before this is done, this address in the instruction is sent to the special adder hardware where the base register value of 1000 is added to it, and only the resulting address of 1500 finally goes to MAR as depicted by Fig. 8.9. As MAR contains 1500, it refers to the correct physical location. For every address needing translation, this addition is made by the hardware. Hence, it is very fast, despite the fact that it has to be done for every instruction.

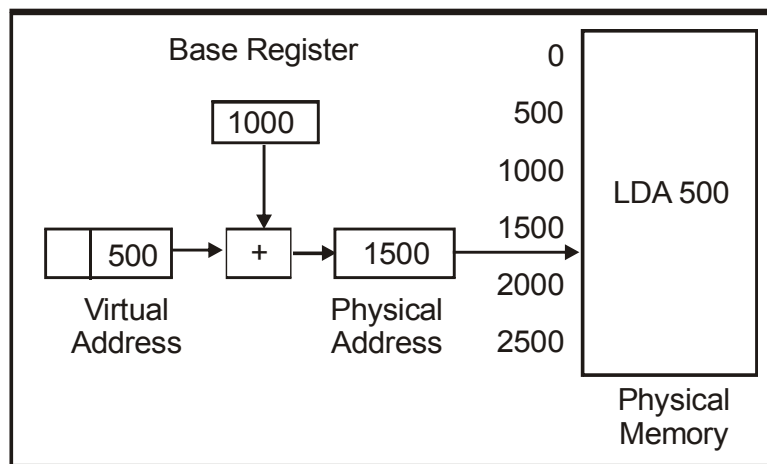


Fig. 5.H Dynamic relocation

Imagine a program with a size of 1000 words. The ‘virtual address space’ or ‘logical address space’ for this program comprises words from 0 to 999. If it is loaded in a partition starting with

the address 1000, 1000 to 1999 will be its 'physical address space' as shown in Fig. 8.9 though the virtual address space still continues to be 0 to 999. At the time of execution of that process, the value of 1000 is loaded into the base register. That is why, when this instruction "LDA 500" is executed, actually it executes the instruction "LDA 1500", as shown in Fig 5.H

5.6.4 PROTECTION AND SHARING

Protection bits

This scheme, however, is expensive. If the word length is 32 bits, 4 bit overhead for every word would mean $4/32 = 1/8$ or 12.5% increase in the overheads. Hence, IBM 360 series of computers divided the memory into 2KB blocks and reserved four protection bits called the 'key' for each such block and could not be any arbitrary number. This resulted in memory wastage due to the internal fragmentation. Imagine that the block size is 2 KB, and the process size is 10 KB + 1 byte. If two of the partitions are of size of 10 KB and 12 KB, the operating system will have to allocate a partition of 12 KB for this process. The one with 10 KB size will not do. Hence, an area of 2KB-1 will be wasted in that partition. It can easily be seen that the maximum internal fragmentation per partition is equal to block size -1, the minimum is 0, and the average is equal to $(\text{block size} - 1) / 2$ per process.

This scheme has four major drawbacks:

- (i) It results in memory wastage because the partition size has to be in multiples of a block size (Internal fragmentation)
- (ii) It limits the maximum number of partitions or resident processes (due to the key length)
- (iii) It does not allow sharing easily. This is because the operating system would have to allow two possible keys for a shared partition, if that partition belongs to two processes simultaneously. Thus, each block in that partition should have two keys, which is cumbersome. Checking the keys by hardware itself will also be difficult to implement.
- (iv) If hardware malfunction generates a different address but in the same partition, the scheme cannot detect it, because the keys would still tally.

LIMIT REGISTER

Another method of providing protection is by using a limit register, which ensures that the virtual address present in the original instruction, moved into IR before any relocation/address translation, is within the bounds of the process address space.

SHARING

Sharing poses a serious problem in fixed partitions, because it might compromise on protection. One approach to sharing any code or data is to go through the operating system for any such request. Because the Operating system has access to the entire memory space, it could mediate.

Another approach is to keep copies of the sharable code/data in all partitions where required. Obviously, it is wasteful, apart from giving rise to possible inconsistencies, if for instance, the same pieces of data are updated differently in two different partitions.

5.7 VARIABLE PARTITIONS

5.7.1 Introduction

- (i) The operating system is loaded in the memory. All the rest of the memory is free.
- (ii) A program P1 is loaded in the memory and it starts executing (after which it becomes a process).
- (iii) A program P2 is loaded in the memory and it starts executing (after which it becomes a process).
- (iv) A program P3 is loaded in the memory and it starts executing (after which it becomes a process).
- (v) The process P1 is blocked. After a while, a new high priority program P4 wants to occupy the memory. The existing free space is less than the size of P4. Let us assume that P4 is smaller than P1 but bigger than the free area available at the bottom. Assuming that the process scheduling is based on priorities and swapping, P1 is swapped out. There are now two chunks of free space in the memory.

- (vi) P4 is now loaded in the memory and it starts executing (after which it becomes a process.) Note that P4 is loaded, However, as the size of P4 is less than that of P1, still some free space remains. Hence there are still two separate free areas in the memory.
- (vii) P2 terminates. Only P4 and P3 continue. The free area at the top and the one released by P2 can now be joined together. There is now a large free space in the middle, in addition to a free chunk at the bottom
- (viii) P1 is swapped in as the operating system has completed the I/O on its behalf and the data is already in the buffer of the operating system. Also, the free space in the middle is sufficient to hold P1 now. Another process P5 is also loaded in the memory. At this stage, there is only a little free space left.

5.7.2 COMPACTION

This technique shifts the necessary process images to bring the free chunks to adjacent positions in order to coalesce. There could be different ways to achieve compaction. Each one results in the movement of different chunks of memory.

Obviously, whenever a process terminates, the operating system would do the following:

- (i) Free that memory.
- (ii) Coalesce, if necessary
- (iii) Check if there is another free space which is not contiguous and if yes, go through the compaction process
- (iv) Create a new bit imp/linked list as per the new memory allocations.
- (v) Store the starting addresses of the partitions in the PCBs of the corresponding processes. This will be loaded from the appropriate PCB into the base register at the time the process is dispatched. The base register will be used for address translation of every instruction at the run time as seen earlier.

SWAPPING

5.7.3 Relocation and Address Translation

This is substantially the same as in fixed partitions. This scheme also depends upon the base register which is saved in and restored from the PCB at the context switch. The physical address is calculated by adding the base register to the virtual address as before, and the resulting address goes to MAR for decoding. After swapping or compaction operations, if the processes change their memory locations, these values also need to be changed as discussed earlier.

5.7.4 PROTECTION AND SHARING

Protection is achieved with the help of the limit register. Before calculating the resultant physical address, the virtual address is checked to ensure that it is equal to or less than the limit register. This register is loaded from the PCB when that process is dispatched for execution. As this value of limit register does not undergo any change during the execution of a process, it does not need to be saved back in the PCB at the context switch.

5.7.5 EVALUATION

(i) Wasted Memory

This scheme wastes less memory than the fixed partitions, because there is theoretically no internal fragmentation if the partition size can be of any length. In practice, however, the partition size is normally a multiple of some fixed number of bytes giving rise to a small internal fragmentation. If the operating system adopts the policy of compaction, external fragmentation can also be done away with, but at some extra processing cost.

(ii) Access Time

Access time is not different from that in fixed partitions due to the same scheme of address translation using the base register.

(iii) Time Complexity

Time complexity is certainly higher with the variable partition than that in the scheme of fixed partitions, due to various data structures and algorithms used.

5.8 Non-Contiguous Allocation -General Concepts

Up to now, various contiguous memory allocation schemes and the problem of fragmentation that arises thereof have been studied. Compaction provides a method to reduce this problem, but at the expense of a lost of computer time in shifting many process images to and fro. Non-contiguous allocation provides a better method to solve this problem.

Consider Fig 5.1 for instance. Before compaction, there are holes of sizes 1k and 2k.

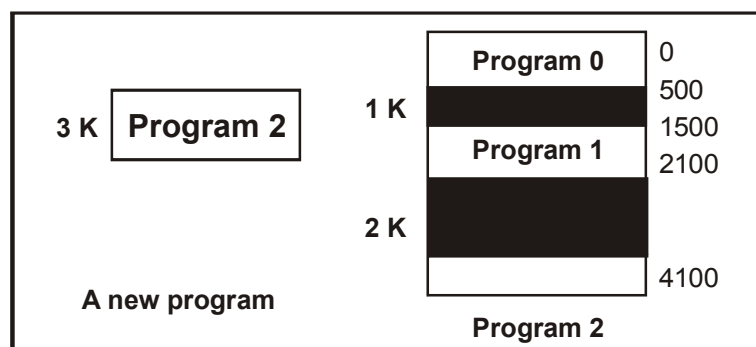


Fig 5.1-A problem of fragmentation revisited

If a new program of size =3k is to be run next, it could not be run without compaction in the earlier schemes. However, compaction would force most of the existing processes also to stop running for a while. A solution to this has to be found.

The problem of fragmentation involves answers to the following questions:

- (a) Can the program be broken into two chunks of 114 and 2k to be able to load them into tow holes at different places? This will make the process image in the memory non-contiguous. This raises several questions
- (b) How can such a scheme be managed?

- (c) How can the addresses generated by compiler be mapped into those of the two separate non-contiguous chunks of physical memory by the address translation mechanism?
- (d) How can the problem of protection and sharing be solved?

Approaches used for solving this are listed here:

- * If the chunks have to be of the same size for all the processes, the method is called 'paging'. In this case, the process image is divided in fixed sized pages.
- * If the chunks can be of different sizes, the method is called segmentation. In this case, the process image is divided into logical segments of different sizes.
- * All the methods discussed or mentioned above use real memory management system, where the entire process image (all chunks) has to reside in the main memory before execution can commence.
- * If the method can work with some chunks in the main memory and the remaining on the disk which can be brought into the main memory as and when required, the system is called 'virtual memory management system'.
- * The virtual memory management system can be implemented using mainly two popular methods. One is called 'demand paging'. The other is called 'working set method'. These methods differ in how and when the chunks are brought from the disk into the main memory.
- * There are other hybrid methods such as 'segmented paged method', in which each process image is divided into a number of segments of different sizes, and each segment in turn is divided into a number of fixed sized pages. Again, this scheme can be implemented using virtual memory; though it is possible to implement it using 'real' memory.

5.9 Summary

Memory-management algorithms for multiprogrammed operating systems range from the simple single-user system approach to segmentation and paging. The most important determinant of the method used in a particular system is the hardware provided. Every memory address generated by the CPU must be checked for legality and possibly mapped to a physical address. Swapping can be added to any algorithm. At intervals determined by the operating system, usually dictated by CPU-scheduling policies, processes are copied from main memory to a backing store and later are copied back to main memory. This scheme allows more processes to be run than can be fit into memory at one time. In general, PC operating systems support paging, and operating systems for mobile devices do not.

5.10 Review Questions

1. What is Memory Management?
2. What is meant by Access Time?
3. Name different Memory Management schemes?
4. Contrast Contiguous versus non-contiguous Memory Management schemes?
5. What is meant by Associative Memory?
6. What is an Address Translation?
7. Contrast Real versus Virtual Memory Management Systems?
8. What is a Demand Feeding?
9. What is meant by Compaction?
10. What is a cache?

LESSON - 6

PAGING

Structure

- 6.1 Introduction**
- 6.2 Objectives**
- 6.3 Paging**
- 6.4 Segmentation**
- 6.5 Combined Systems**
- 6.6 Virtual Memory Management Systems**
- 6.7 Summary**
- 6.8 Review Questions**

6.1 INTRODUCTION

The logical or virtual address space of a program is divided into equal sized pages, and the physical main memory also is divided into equal sized page frames. The size of a page is the same as that of the page frame, so that a page can exactly fit into a page frame and therefore, it can be assigned to any page frame, which is free.

6.2 Objectives

After studying this lesson, you should be able to understand

- Paging and Swapping
- Page Map Tables
- Combined Systems
- Virtual Memory Management

6.3 Paging

- (i) The process address space of a program is thought of as consisting of a number of fixed sized contiguous pages

- (ii) Any virtual address within this program consists of two parameters: a logical or virtual page number (P) and a displacement (D) within the page.
- (iii) The memory is divided into a number of fixed sized page frames. The size of a page frame is the same as that of a logical page. The operating system keeps track of the free page frames and allocates a free page frame to a process when it wants it.
- (iv) Any logical page can be placed in any free available page frame. After the page (P) is loaded in a page frame (F), the operating system marks that page as “Not free”,
- (v) Any logical address in the original program is two dimensional (P,D), as we know. After loading, the address becomes a two dimensional physical address (F,D). As the sizes of the page and the page frame are the same, the same displacement D appears in both the addresses.
- (vi) During execution, for every address, the address translation mechanism has to find out the physical page frame number (F), given the virtual page number (P). After this it has to append or concatenate D to it to arrive at the final physical address (ED). Hence, in the virtual address, it must be possible to separate out the bits for the page (P) and the ones for D, in order to carry out this translation.

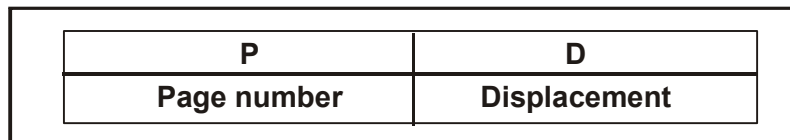


Fig.6.A A two dimensional address

Any virtual address produced by the compiler can be thought of as made up of two components-page number (P) and displacement (D). An interesting point of this scheme is that when a page is loaded into any available page frames (the sizes of both are the same), the displacement for any address (D) is the same in virtual as well as physical address.

This index is called a ‘Page Map Table (PMT)’ which is the key to the address translation. At the execution time, all that is needed is to separate the high order bits in the address reserved for the page number (P), and convert them into page frame number (F) using this PMT, concatenate F and D and arrive at the physical address as we know that D remains same. This is the essence of address translation.

6.3.1. Swapping

The considerations for swapping are similar to those discussed earlier. In paging, if a process is swapped, it is swapped entirely. Keeping only a few pages in the main memory is useless because, a process can run only if all the pages are present in the main memory. AOS running on 16 bit Data General machine follows the pure paging philosophy. The process cannot run unless the entire process image is in the memory, even if the process image is divided into pages, and a few of them are in the main memory. Therefore, the entire process image is swapped out, if required.

While process is to be swapped out depends upon the priorities and states of the processes already existing in the main memory and the size of the new process to be accommodated. Normally, a blocked process with very low priority can be swapped out, if space is to be created in the memory for a new process. These issues are handled by the medium level process scheduler.

When a process is swapped out, its are in the memory which holds the PMT for that process is also released. When it is swapped in again, it may be loaded in different page frames depending upon which are free at that time. At that time a new PMT is created, as the PCB for the process is chained to the ready processes.

6.3.2 Relocation and Address Translation

6.3.2.1 General Methodology

Fetch cycle, when the program counter (PC) gets incremented from 49 to 50 (i.e. 0001 10010), this address is transferred to MAR by the microinstruction PC $\rightarrow$ MAR. The bits in MAR act as the control signals for the address decoder which activate the desired memory location. It is at this stage that we need to modify the address so that the resulting address can finally be put on the address bus which can access the physical memory. The PMT in Fig. 8.21 shows that page 1 is mapped onto page frame 4, and thus, the physical address at which you will find the instruction "LDA 107" will be within page frame 4, at a displacement of 18. Page frame 4 will contain physical address of 128 to 159. Therefore, displacement of 18 within that page frame would mean physical address of $128 + 18 = 146$ in decimal or, 010010010 in binary. Hence, we need to fetch the instruction not at location 50 but at location 146. To achieve this, the address coming out of MAR which is 50 in decimal or 000110010 in binary is split into two parts. The page number P is used to find the corresponding page frame F using the PMT. Fig. 8.21 shows

that P=0001 corresponds to F=0100. F+D now is used as the address which is used as the control signal to the memory decoder.

At the execute cycle, the hardware “knows” that it is an LDA instruction using direct addressing. It therefore, copies the address portion 107 i.e. 001101011 (P=0011=3, D=01011=11) to MAR for fetching the data by giving a ‘read’ signal. The figure shows that page 3 is mapped onto page frame 2. Hence, the data at virtual address decimal 107 will now be found at physical address with page frame (F) =2=0010 and Displacement (D) =11=01011 or binary address= 001001011 i.e. 75 instead of 107 in decimal. Again, this address translation is done using PMT on the address in MAR and the resultant translated address is put on the address bus, so that actually the correct addresses are used for address decoding. This is shown in Fig. 6.B

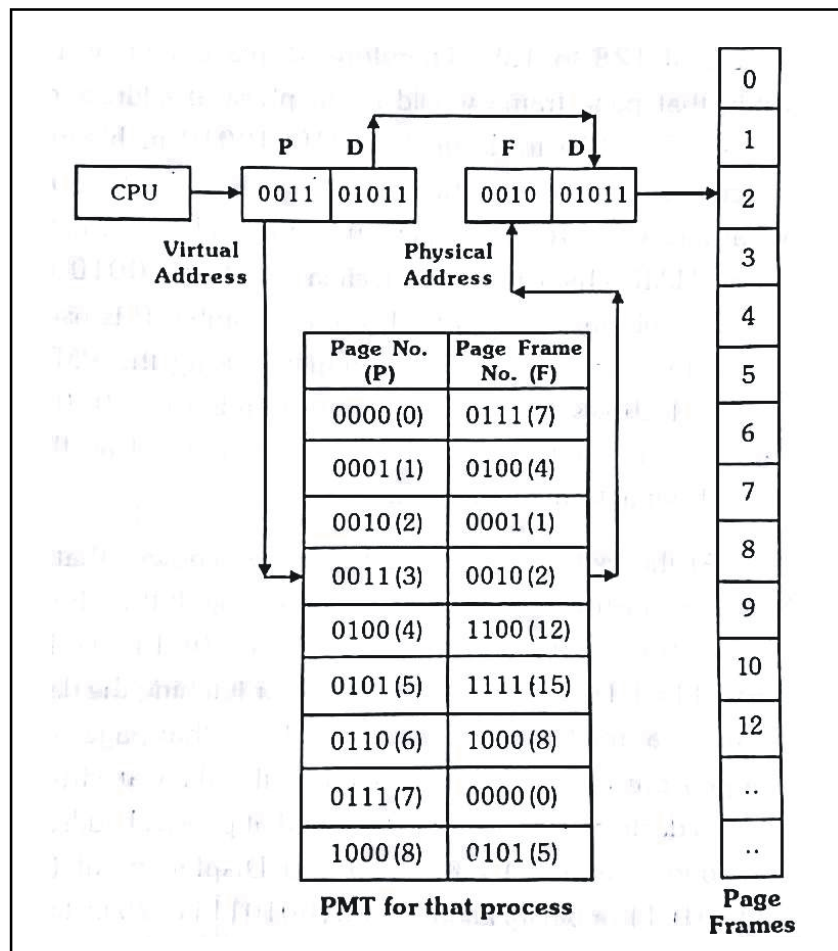


Fig. 6B

6.3.2.2 Implementation of Page Map Tables

There are basically three methods of actually implementing PMTs:

- Software method
- Hardware method
- Hybrid method

(i) **Software Method**

In this method, the Operating system keeps all the PMTs in the main memory. The starting word address of the PMT for a process is known at the time the process is created, its pages are loaded and a PMT is created and stored in the main memory. This address is also stored in the PCB. At the context switch, this address is loaded from the PCB into another hardware register in the CPU called 'Page Map Table Base Register (PMTBR)'. This register is used to locate the PMT itself in the memory. If a process is swapped out and sometime later it is swapped in again in different page frames, a new PMT may be created and that too may be loaded at different memory locations. PMTBR also will change accordingly in this case.

If we reserve one word for each PMT entry, word 0 within the PMT would correspond to the virtual page number 0, word 1 would correspond to virtual page number 1, etc. for that process. Therefore, given a virtual page number (P), the entry for P in the PMT can be easily found out by computing $\text{PMTBR} + P$.

- (a) The logical or virtual address is divided into two parts: Page number (P) and Displacement (D) as discussed earlier.
- (b) The page number P is now checked for its Validity by ensuring that it is PMTLR . This comparison takes place in the CPU registers only, hence, taking negligible time.
- (c) If P is valid, it is used as an index into the PMT whose starting address is given by PMTBR. Therefore, $P + \text{PMTBR}$ gives the entry number of PMT. This addition takes place in the ALU, and hence, takes very little time.
- (d) The selected entry of PMT is fetched into the CPU register. This operation requires one memory access because, PMT resides in the memory.

- (e) The page frame number (F) is extracted from the selected PMT entry, already brought into the CPU register. This again takes negligible time.
- (f) Original displacement (D) is concatenated to F obtained in step(s), to get the final physical address $F + D$. This takes virtually no time as this is done by the hardware itself in the CPU register.
- (g) This address is put on the address bus to locate the desired data item in the memory. This, again, requires a memory access.

(ii) Hardware Method

A pure hardware method would use “associative registers”. They are also known by other names such as “associative memory”, “look ahead memory”, “look ahead buffer”, “content addressable memory”, or “Cache” (This cache is different from the normal cache memory). The essence of this method is really that the “table search” is done in the hardware itself, and hence, it is very fast. For instance, if P is supplied to this associative memory, F is output directly by hardware itself in one shot. This F then is concatenated by D to give the resultant physical address.

As a process is dispatched from “Ready” to the “Running” state, the PMT for that process is located using its starting address (PMTBR) stored in the PCB and the entries in the PMT are loaded into these associative registers as a part of context switch. Each associative register now contains a virtual page number and its corresponding page frame number, ready to carry out the hardware table-search.

(iii) Hybrid Method

The hybrid method provides such a via media. In this method, associative memory is present, but it consists of only 8, 16 or some other manageably small number of registers.

The address translation in the hybrid method is carried ‘ out in the following fashion:

- (a) The virtual address is divided into two parts: page number (P) and displacement (D) as discussed earlier. This is a timeless Operation.

- (b) The page number (P) is checked for its validity by ensuring that $P \leq \text{PMTLR}$. This takes virtually no time, as this comparison takes place in the hardware itself.

* Found in associative memory

- (c) If P is valid, a check is made to see if P is in the associative registers, and if it exists there, the corresponding page frame number (F) is extracted directly from the associative registers. This operation takes some time t_{ama} as seen earlier. This time is required regardless whether P exists in associative registers or not.
- (d) The original displacement (D) is concatenated to F , to get the final physical address $F+D$. This, again, takes virtually no time.
- (e) Using the address, the desired item in the main memory is finally accessed. This requires the time t_{ma} as discussed earlier.

Thus, if found in a associative memory, the total time required $t_{ama} + t_{ma}$, as in pure hardware method. If P is not found in associative registers, the method followed is the same as the pure software method.

* Not found in associative memory

- (f) In the software method, P is used as an index into PMT. PMTBR is added to P (requiring very negligible time) to directly find out the desired entry number of PMT.
- (g) The selected entry of PMT is fetched into the CPU register. This operation requires one memory access time = t_{ma} because full PMT is in the main memory.
- (h) The page frame number (F) is extracted from the selected PMT entry already brought into the CPU register. This, again, is almost a timeless operation.
- (i) The original displacement (D) is now concatenated to F requiring negligible time to get the final physical address $F + D$.
- (j) Using this physical address, the desired data item in the main memory is now accessed. This requires time t_{ma} as seen before.

Thus, the total time required if P is not found in associative registers = $t_{ama} + 2t_{ma}$

$$EAT = h(t_{ama} + t_{ma}) + (1-h)(t_{ama} + 2t_{ma})$$

Let us assume that $t_{ama}=40\text{ns}$, $t_{ma}=800\text{ ns}$ and $h=0.8$

$$\text{We get } EAT = 0.8(40 + 800) + 0.2(40 + 1600)$$

$$= 1000\text{ ns}$$

Therefore, we see the following degradations.

- Pure time to reference a memory location (if this Were possible) = 800 ns.
- Time to reference a memory location using pure hardware method = $40 + 800 = 840\text{ ns}$ (5% degradation).
- Time to reference a memory location using hybrid method with hit ratio = $0.8 = 1000\text{ns}$ (25% degradation).

The hit ratio (h) can be increased by having more associative registers. We have to do it in a cost effective manner by studying the cost and benefits of increasing h.

if $h=0.9$, we get:

$$EAT = (0.9 \times 840) + (0.1 \times 1640) = 920\text{ ns}.$$

6.3.3 Evaluation

(i) Wasted memory and Page Size

In this scheme, a page is a minimum unit of allocation. Therefore, in the best case when the size of the process image is an exact multiple of page size, each page frame will be fully loaded and utilized, and wastage will be 0. IN the worst case if you require only one word to be allocated in the last page (2.9. program size = $[m \times \text{page size}] + 1\text{ word}$, where m is an integer), full page frame will have to be allocated for the last word in the program and hence, the entire page frame will be almost completely wasted. Thus, in paging, the average memory waste is $(\text{Page size}-1)/2$ per process. This is internal fragmentation. In paging, there is no external fragmentation. This is because in paging, it cannot happen that the total memory required for a

process is 6 pages and they are actually available, but they cannot be allocated because they are scattered and not contiguous. We have already removed the precondition of contiguity in paging. Thus, the total average wastage (due to internal +external fragmentation) = (Page size —1)/2 per process, and it is higher for higher page size.

(ii) Access Time

Access time is generally faster than in contiguous schemes as it obviates the need for compaction. They are dependent upon the time for address translation which, in turn, is governed by page size and the address translation method (hardware, software, hybrid).

(iii) Time Complexity

This is very low because generally the algorithms are simple.

6.4 SEGMENTATION

6.4.1 Introduction

Segmentation and paging share a lot of common principles of operations, except that pages are physical in nature and hence, are of fixed size, whereas segments are logical divisions of a program and hence, are normally of variable sizes.

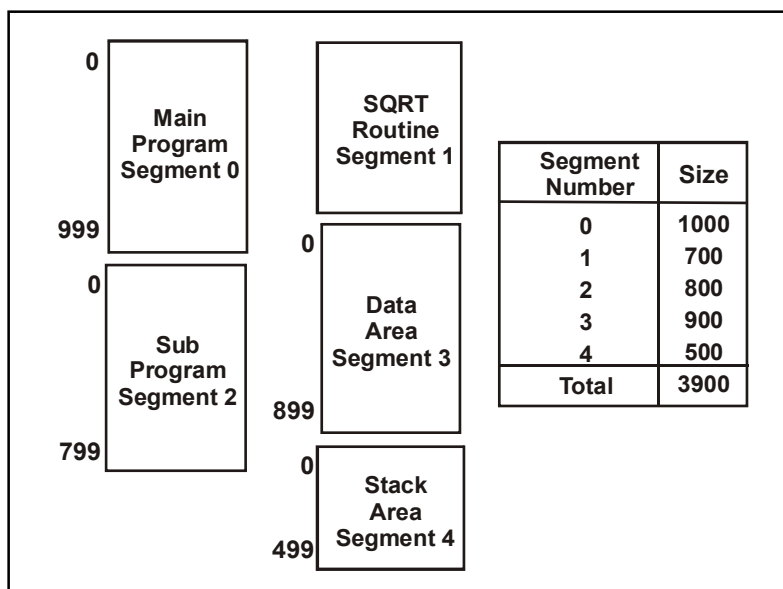


Fig 6 C An example of Segments

For instance, each program in its executable form can be considered to be consisting of three major segments code, data and stack.

Each segment is compiled with respect to 0 as the starting address for that segment.

The compiler/linkage editor thus, does the following:

- * Recognizes different segments
- * Numbers them.
- * Builds the segment table as shown in Fig. 6.D

Produces an executable image by assigning two dimensional addresses.

Virtual Address Limits	Virtual Segment
0-999	0
1000-1699	1
1700-2499	2
2500-3399	3
2400-3899	4

Fig. 6.D Virtual addresses in Segmentation

The steps followed during the initiation of a process are traced below:

- (i) At any time, there is possibly some physical memory which is free and allocable. New processes are loaded after being allocated some of it. At the same time; some old processes are terminated, releasing or freeing some memory. The operating system has to keep track of the chunks of free memory starting time. It will have to know their sizes and starting addresses. There are several algorithms to keep of those. These algorithms using data structures such as bit maps, linked lists or tables are similar to the ones used in the scheme of variable partitions. There are

again the same considerations for coalescing and compaction. We have already covered these and will not repeat them here.

- (ii) At any time, when a new process is to be created, the Process Management (PM) module talks to the Information Management (IM) module and accesses the executable image for that program. The header of this executable file gives the information about the number of segments and their sizes in that program. This is passed on to the Memory Management (MM) module.
- (iii) The Memory Management (MM) now consults the information about the free memory as given in (1) above and allocates it for different segments. It can use a number of algorithms such as first fit, best fit, worst fit, etc. to carry this out.
- (iv) After the memory is allocated, it builds a 'Segment Map Table (SMT)' or 'Segment Descriptor Table (SDT)' as shown in Fig. 8.31. The SMT contains the following information.
 - Segment Number: This is a serial number of the segment. This is shown in the figure for clarity, but actually it need not be maintained. This is because, the size of each entry in the SMT is fixed. The operating system, therefore, can access the entry for any segment directly, given its segment number.
 - Size (Z) : This is the size of the segment.
 - Base (B): This is the starting address of the segment loaded in the physical memory.
 - Access Rights: This says who is allowed to do what with that segment. This is used for protection

6.4.2 Swapping

If some segments are rolled out on the disk from the memory, relocation and the process of bringing it back to the main memory is fairly straightforward. You really do not need to change anything except that if a rolled in segment occupies a different physical location, the SMT is appropriately updated.

In systems without virtual memory support, all segments belonging to a process (excepting the shared ones) are swapped out and swapped in as necessary depending upon the philosophy

used by the process scheduler and based on the process priority. When a process is swapped out, the memory is freed and is added to the pool of free memory. The ideas of compaction and coalescing.

6.4.3 Address Translation and Relocation

- (i) The high order bits representing S are taken out from the IR to form the input to the SMT as an index. For instance, the entry for S=3 in the SMT can be directly accessed.
- (ii) The operating system now stores the data from that entry (in this case the one with S=3). Hence, it will store the segment size $Z=900$ and the starting address $B=6100$ for future use.
- (iii) We know that displacement (D) is a virtual address within that segment. Hence, it has to be less than segment size. Therefore, the displacement (D) is compared with Z to ensure that $D \leq Z$. If not the hardware itself generates an error for illegal address. This is evidently a protection requirement. In our example, D which is 20, is less than Z which is 900, which is acceptable.
- (iv) If the displacement (i.e. D) is legal, then the operating system checks the access rights and ensures that these are not violated. We will discuss the access rights under “protection and sharing”, later.
- (v) Now, the effective address is calculated as $(B+D)$, as shown in the figure. This will be computed as $6100 + 20 = 6120$ in our example.
- (vi) This address is used as the actual address and is pushed into the address bus to access the physical memory.

6.4.4 Sharing and Protection

Sharing will now be discussed, followed by protection in segmentation.

Sharing of different segments is fairly simple and is quite similar to that in the paging systems. We again illustrate this by an editor program. Assume that this editor has two code segments and one data segment.

Also assume that the code is written to be reentrant, and hence, can be shared. If this editor is to be used by two users simultaneously, the two SMTs must map these two code segments onto the same physical locations as shown in the figure. It will be noticed that the size and base values for code segments 0 and 1 are the same in both SMTs.

Protection is achieved by defining access rights to each segment such as 'Read Only (RO)' or 'Read/Write (RW)', etc. and by defining certain bit codes to denote those e.g. 01=RO, 10=RW. These are the access rights bits in any SMT entry. When a user logs on and wants to execute a process, the operating system sets up these bits in the SMT entries while creating the SMT, depending upon the user privileges. As discussed before, if there are 4 processes sharing a segment, all the 4 SMTs for those 4 processes will have the same entry for that segment. In all the entries for a shared segment, base (B), Size (2) will be the same, but access rights can be different in different SMTs for different sharing processes. Because of this scheme, the shared segments also can be well protected, For instance, a shared segment can have only access rights "RO" for process-A and can have "RW" for process -B.

Protection in terms of restricting the accesses to only the address space of that process is achieved during address translation as shown in Fig. 8.32 using the "size (2)" field in the SMT entry, and ensuring that the displacement (D) is not more than Z.

6.5 COMBINED SYSTEMS

The combined systems are systems which combine segmentation and paging. This requires a three-dimensional address consisting of a segment number (S), a page number (P) and displacement (D). There are two possible schemes in these systems.

- Segmented paging as in IBM 360/67 or Motorola 68000 systems is one such scheme. In this scheme, the virtual address space of a program is divided into a number of logical segments of varying sizes. Each segment, in turn, is divided into a number of pages of the same size.
- Paged segmentation as in GE645/MULTICS

The tables required for this scheme are organized as follows:

- (a) The program consists of various segments, as given by the SMT. The SMT contains different entries-one for each segment. Each segment is divided into number of

pages of equal size, whose information is maintained in a separate PMT. Thus, there are as many PMTs as there are entries in the SMT. If a process has 4 segments 0-3, there will be 4 PMTs for that process, one for each segment. (The Fig. shows only one of them to avoid cluttering.)

- (b) The interpretations of various fields in the SMT in this combined scheme is different from those discussed earlier. For instance, the size field in SMT gives the starting from 0, instead of the size of the segment itself. Hence, if PMT for a segment has entries for pages 0-5, the size (Z) field in the corresponding SMT for that segment will be maintained as 5. This is shown in Fig. 8.36. Similarly, the base (B) in the SMT now gives the starting address of the segment itself in the physical memory. Therefore, assuming again that one PMT entry can be accommodated in one word, the address of the entry in the PMT for the desired page (P) in a given segment (S) can be obtained by $(B+P)$, where B can be obtained from the entry in the SMT for that segment with segment number = S. Using this address $(B+P)$, as an index into PMT, the page frame F can be obtained. And, finally, the physical address can be obtained by concatenating D to F.

The scheme now works as follows:

- (i) The virtual or logical address coming from the instruction consists of three components or dimensions: Segment number (S), page number (P), and displacement (D) as shown in the Fig. 6E.
- (ii) The system has a pair of registers viz. SMTLR (Limit register) and SMTBR (Base register). Their values will be different for the SMT of each process. Therefore, their values are maintained in the respective PCBs and are restored at the context switch.
- (iii) The segment number (S) is validated against SMTLT (a protection requirement) and then added to SMTBR to form an index into SMT

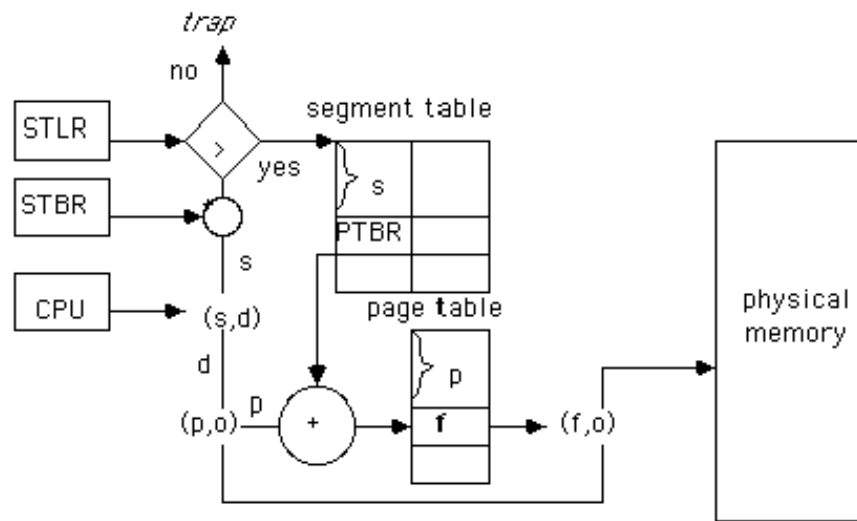


Fig. 6.E- Segmented paging

- (iv) A proper SMT entry is picked up and then the page number (P) is validated against the segment size (Z) which gives the maximum number of pages in that segment.
- (v) Now access rights are validated against the ones in the chosen SMT entry. For instance, if the current instruction is for writing into that segment and the access rights for that user process as mentioned in the SMT are "RO (i.e. Read Only)", then an error results.
- (vi) If everything is fine, the base (B) of that SMT entry is added to P to directly index into the PMT for that segment. (Base in the SMT in this case means the beginning word address of the PMT for that segment). When you add B to P, you go into the PMT entry directly for the required page within the desired segment.
- (vii) The page frame number (F) is extracted from the selected PMT entry.
- (viii) The displacement (D) is concatenated to this page frame number (F) to get the actual physical address.

6.6 VIRTUAL MEMORY MANAGEMENT SYSTEMS

6.6.1 Motivation

Upto now, all the systems that we have considered (contiguous or non-contiguous) were based on the assumption that the entire process image was in the main memory at the time of execution. Due to low priority, if a process had to be removed temporarily from the main memory, the entire process image was swapped out. When it was reactivated, the entire process image was swapped in.

This scheme is simple to implement but it has one major drawback. If the physical memory is limited (which is the case normally), then the number of processes it can hold at any time, and hence, the degree of multiprogramming becomes limited. Also, the entire process image needs to be swapped, thus decreasing efficiency.

What will happen if we can keep only a part of the process image in the memory and the other part on the disk and still are able to execute it? If we can do this, we have a 'virtual memory system'. Virtual memory systems also can be implemented using paging, segmentation or combined schemes. Thus, all that we learnt upto now is still valid, with only one difference. The process can start executing with only part of the process image in the memory. For our discussions and examples, we will consider virtual memory systems using only paging because, this is most common. In this case, a program consists of a number of logical or Virtual pages which are loaded into specific page frames. This is as discussed earlier in "paging". A question now arises: If a page is not loaded in the memory and a location within that page is referenced, what will happen?

The idea is that when a page not currently in the memory is referenced, only that page can be brought from the disk into the memory. This, of course, is more time consuming, but the trade off has to be considered between this extra time or cost and the degree of multiprogramming that can result in increased throughput. The idea is to bring in only zero, one or a few pages of a process to begin with, and continue as long as these pages suffice. As soon as a reference is made outside these pages, the required page is located on the disk and brought into the memory.

If there is no page frame free in the physical memory to accommodate this new page, the operating system may have to overwrite an existing page in the physical memory. Which page should be overwritten? This is governed by 'page replacement policy'. However, if this page had been modified after last being brought in from the disk, the operating system could not just

overwrite it. It would lose all the updates otherwise. If the same page is later brought in from the disk, it will not serve the purpose. In this case, the operating system has to copy this “dirty page” back on the disk first before overwriting the new one onto the page frame holding that page.

In the case of virtual memory systems, a programmer can write a very big program. The size is restricted only by the number of bits reserved for the address in the machine instruction (or Memory Address Register (MAR) width), e.g. in a VAX computer, the maximum size of the program can be 4 GB (4 Gigabytes = 4×1024 Megabytes = $4 \times 1024 \times 1024$ kilobytes = $4 \times 1024 \times 1024 \times 1024$ Bytes = 2^{32} to the power of 32 bytes corresponding to 32 bits in the MAR) However, many VAX computers support a maximum of 8 MB of physical memory which is far less than 4 GB. In such a case, you can still run a program of size bigger than 8 MB, but smaller than 4 GB, because the system supports virtual memory. This reduces or almost removes the size restrictions which the programmers used to face in earlier days.

6.6.2 Jargon

(i) Locality of Reference

The basic principle behind the virtual memory is called ‘locality of reference’. If you study the behaviour of any program, unless it is a very badly written one, jumping from one location to another haphazardly, it will be seen that, in the beginning, it normally uses the pages containing the code for the housekeeping routines using some other ‘pages. The earlier pages are hardly used ever again. Towards the end; the program refers to locus of page references during the program execution. This clustering of page references in a certain time zone is called the principle of ‘locality of reference’.

(i) Page Fault

In many systems, when a process is executing with only a few pages in memory, and when an instruction is encountered which refers to any instruction or data in some other page which is outside the main memory (i.e. on the disk), a ‘page fault’ occurs. At this juncture, the operating system must bring the required page into the memory before the execution of that instruction can restart. In this fashion, the number of pages in the physical memory for that process gradually increases with the page faults. After a while, when a sufficient number of required pages build up, the pages are normally found in the memory and then the frequency of page fault reduces.

(ii) Working Set

At any time, a process has a number of pages in the physical memory. Not all of these are actively referred to at that point in time, according to the locality of reference. The set of pages in the physical memory actively referred to at any moment is called 'working set'. This has a significant bearing on the policy of bringing in pages from the disk to the main memory, if the operating system follows the 'working set model' for this purpose. As discussed earlier, there exist different working sets in housekeeping, during the main routine and during the End-of-job routines. In this scheme, the operating system maintains a list of actively referred pages as a working set at any given time. If due to low priority, the entire process is swapped out and swapped in again, the pages corresponding to the working set can be directly swapped in, instead of having to start from the scratch. The working set, therefore, signifies the knowledge about the current page references and, therefore, has to be stored at the context switch.

(iii) Page Replacement Policy

As the number of processes and the number of pages in the main memory for each process increase, at some point in time, all the page frames become occupied. At this time, if a new page is to be brought in, the operating system has to overwrite some existing page in the memory. The page to be chosen is selected by the 'page replacement policy' as discussed earlier. There are a number of ways in which the operating system selects a page for overwriting. The operating system designer chooses amongst one of many such policies and writes a corresponding algorithm for it.

(iv) Dirty Page/Dirty Bit

Before overwriting a page in the memory, the Operating system has to check if that page has been modified after it was loaded from the disk. For instance, there may be a page containing the I/O area where an employee record is to be read. Initially, that page will be loaded with the I/O area as blank. After reading the record, that page gets modified. Such a modified page is called 'dirty page'. Normally, these days the compilers produce reentrant code which does not modify itself. Hence, the pages reserved for the code portion of a program normally never get dirty, but the ones for the data portion can. The operating system maintains one bit for each physical page frame to denote whether a page has become dirty or not. This bit is called 'dirty bit'.

6.6.3 Demand Paging v/s Demand Segmentation

The virtual memory system can be implemented with both demand paging and demand segmentation. At any time, only a portion of the virtual address space resides in the main memory. If the management and transfer of the parts of the program between the secondary and primary memory takes place in equal sized chunks, it is called 'demand paging'; otherwise it is called 'demand segmentation'.

6.7 Summary

A multiprogrammed system will generally perform more efficiently if it has a higher level of multiprogramming. For a given set of processes, we can increase the multiprogramming level only by packing more processes into memory. To accomplish this task, we must reduce memory waste, or fragmentation. Systems with fixed-sized allocation units, such as the single-partition scheme and paging, suffer from internal fragmentation. Systems with variable-sized allocation units, such as the multiple-partition scheme and segmentation, suffer from external fragmentation. Virtual memory is a technique that enables us to map a large logical address space onto a smaller physical memory. Virtual memory allows us to run extremely large processes and to raise the degree of multiprogramming, increasing CPU utilization. Virtual memory is commonly implemented by demand paging.

6.8 Review Questions

1. What is a Paging?
2. Discuss the impact of Page Size on the overall system performance?
3. What is dirty page, dirty bit?
4. What is meant by reentrant code?
5. What is meant by segmentation?
6. Name various algorithms for page replacement and Belady's anomaly?
7. What is thrashing?
8. Compare various page replacement strategies
9. How can LRU approximation algorithms implemented?
10. Differentiate between static and dynamic relocation
11. What is fragmentation?

LESSON - 7

SECURITY PROTECTION

Structure

- 7.1 Introduction**
- 7.2 Objectives**
- 7.3 Security Threats**
- 7.4 Attack on Security**
- 7.5 Security Violation Through Parameters**
- 7.6 Computer Virus**
- 7.7 Security Design Principles**
- 7.8 Authentication**
- 7.9 Protection Mechanism**
- 7.10 Summary**
- 7.11 Review Questions**

7.1 Introduction

Protection mechanisms control access to a system by limiting the types of file access permitted to users. In addition, protection must ensure that only processes that have gained proper authorization from the operating system can operate on memory segments, the CPU, and other resources.

Protection is provided by a mechanism that controls the access of programs, processes, or users to the resources defined by a computer system. This mechanism must provide a means for specifying the controls to be imposed, together with a means of enforcing them.

Security ensures the authentication of system users to protect the integrity of the information stored in the system (both data and code), as well as the physical resources of the computer system. The security system prevents unauthorized access, malicious destruction or alteration of data, and accidental introduction of inconsistency.

7.2 Objectives

After studying this Lesson, you should be able to

- * Security
- * Security Violation
- * Security Threats
- * Attacks on Security
- * Computer Worms
- * Computer Viruses
- * Security Design Principle
- * Authentication
- * Protection Mechanisms

7.3 Security Threats

Now that the cost of hardware is falling at a rapid rate, millions of ordinary users and programmers have access to small or large computing equipment. With a trend towards networking, the user/programmer has access to data and programs at different remote locations. This has increased the threat to the security of computing environments in general and the operating systems in particular.

Sharing and protection are requirements of any modern computing, but ironically they imply contradictory goals. More sharing gives rise to more possibility of security threats, or penetration, thus requiring higher protection. When the Personal Computer (PC) was designed, it was intended strictly for individual use. This is the reason why MS DOS was not very strong in the security/protection areas.

The major threats to security in any computing environment can be categorized as follows:

- (i) **Unauthorized use of service** (Tapping)
- (ii) **Unauthorized disclosure of information** (Disclosure)
- (iii) **Unauthorized alteration or deletion of information** (Amendment)
- (iv) **Unauthorized fabrication of information** (Fabrication)
- (v) **Denial of service to authorized users** (Denial)

7.4 Attack on Security

The security system can be attacked and penetrated in a number of ways.

7.4.1 Authentication

Authentication means verification of access to the system resources.

- (i) An intruder may guess or steal somebody else's password and then use it.
- (ii) An intruder may use the vendor-supplied password which is expected to be used for the purpose of system generation and maintenance by only the system administrators.
- (iii) An intruder may find out the password by trial and error method. It is fairly well known that names, surnames, initials or some other common identifiers are generally used as passwords by many users. A program could also be written to assist this trial and error method.
- (iv) If a user logs on to a terminal and then goes off for a cup of coffee, an intruder can use the terminal to access, or even modify, sensitive and confidential information.
- (v) An intruder can write a dummy login program to fool the user. The intruder, in this case, can write a program to throw a screen, prompting for the username and the password in the same way that the operating system would do. When a user keys in the username and password for logging in, this dummy program collects this information for the use by the intruder later on. It may, then terminate after throwing back some misleading message like "system down..." etc. This collected information is used for future intrusion.

7.4.2 Browsing

Quite often, in some system, there exist files with access controls which are very permissive. An intruder can browse through the system files to get this information, after which unprotected files/databases could easily be accessed. Confidential information could be read or even modified which is more dangerous.

7.4.3 LINE TAPPING

A special terminal can be used in this case to tap into the communications line and access, or even modify, confidential data. The security threat could be in the form of tapping, amendment or fabrication, once the intruder gets hold of the line. The intruder needs different hardware and techniques depending upon whether the attack is passive or active.

7.4.4 ELECTRONIC DATA CAPTURE

A penetrator can use active or passive wire-taps, or a mechanism to pick up the screen radiation and recognize what is displayed on the screen.

7.4.5 LOST LINE

In networking (or even timesharing) environments, a line can get lost. In such a case, a sturdy operating system can log out a user and allow an access only after reestablishing the identity of the user. Some operating systems cannot do this. In such a case, the process created before losing the line just floats about, and hence, an intruder can gain control of this floating process and access the resources which were accessible by that process.

7.4.6 IMPROPER ACCESS CONTROLS

In some operating system, the system itself does not allow the planning of a meticulous, rigorous access control mechanism (UNIX has, at times, faced this criticism). On top of this, the system administrator may not plan his access controls properly. This may lead to some users having far too many privileges and some others, very few.

7.4.7 ROGUE SOFTWARE

- (i) Ordinary Software Bomb: This is piece of code which “explodes” as soon as it is executed, without delay or warning.
- (ii) Timed Software Bomb: This is like an ordinary software bomb, except that it becomes active only at a specific time or frequency.

- (i) Logical Software Bomb: Again, this is like an ordinary software bomb, except that it is activated only if a logical condition is satisfied (e.g. destroy the Employee Master data only if gross salary exceeds, say, 5000).
- (ii) Worm: These are programs attacking the nodes on the network and spreading to other nodes. They consume all the resources on the network and affect the response time. This will be discussed at length in a separate section.
- (iii) Virus : This is only a part of a program which gets attached to other programs to definitely cause damage.

7.5 SECURITY VIOLATION THROUGH PARAMETERS

When one program calls another program or procedure, it may need some parameters to be passed from the caller routine to the called routine. After the routine is called, a different process is executed and it could have a set of access rights over different objects (such as files and directories) in the system, different-from those of the caller routine and this can cause problems if not handled carefully

7.5.1 DENIAL OF SERVICE

The parameters can be passed either directly (by value) or indirectly (by reference). In 'call by value' method, a user program passes the parameter values themselves. For example, 'it can load certain CPU registers with these values which are then picked up by the called routine. If the called routine does not have access to those registers, it may result in the denial of this service.

In 'call by reference', the actual values of the parameters may not be passed but the pointers or addresses of the locations where the actual values of the parameters are stored may be passed. The denial of service can take place in this case too. For instance, a call may pass parameters, referencing Objects that cannot be accessed by the called routine (called 'target domain'), though the calling routine (called 'Source domain') has the access rights to those. Figure 7.A depicts this scenario.

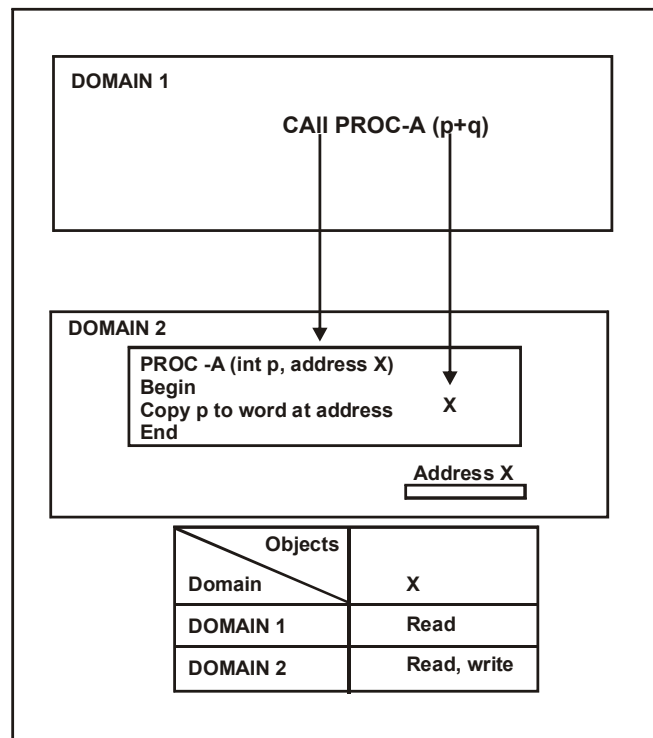


Fig. 7A Unauthorised access

The figure shows the source and target domains : DOMAIN 1 and DOMAIN 2 respectively. DOMAIN 1 contains a call to the procedure PROC-A which has parameters `p` and `q` which are only the addresses. DOMAIN 2 is supposed to execute this procedure by reading the data at an address `p` and then writing it at an address `q`, Note that what are passed are not the data items but the pointers to the data items.

The figure also shows the 'Access Control Matrix (ACM)' for these two domains. It shows that DOMAIN 1 has both Read and Write accesses to both the addresses `p` and `q`. However, DOMAIN 2 has Read and Write access only to `p`. It has only Read access to `q`.

A call will be made as usual, but at the time of execution, when the data read at address `p` has to be written at address `q`, the access will be denied. This is because DOMAIN 2 does not have a Write access for `q`. Thus, the denial of the service results.

7.5.2 A MORE SERIOUS VIOLATION

A more serious violation than denial can occur if data gets accessed, or even modified, by unauthorized processes. This is not possible in case the parameters are passed by value. It can take place only if they are passed by reference. Figure 9.3 depicts this scenario.

As before, DOMAIN 1 (source domain) contains a call to a procedure called PROC-A. This procedure has two parameters p and q. Let us assume that p is a direct parameter and q is an indirect one. The procedure (given in DOMAIN 2) is supposed to pick up the value p which is an integer and then write it at the address X. (At the time of execution the value 'X' is substituted for q). Let us assume that DOMAIN 2 is a more privileged one than DOMAIN 1 as is depicted by the Access Control Matrix (ACM) in the figure. For instance, DOMAIN 2 can perform both Read and Write operations on X, whereas DOMAIN 1 can only Read it. In this case, the operation will be successfully performed in DOMAIN 2; but it should NOT have been allowed! The reason is that if we allow this to happen, in effect, DOMAIN 1 will successfully write at address X which is contrary to the ACM for DOMAIN 1. And this definitely is a serious access violation!

7.5.3 THE CAUSE

The cause for these problems is the excessive mutual trust and the failure to check the validity of the parameters at every stage. As a result, a caller may gain access to unauthorized information.

7.5.4 COMPUTER WORMS

A computer worm is full program by itself. It is written in such a way that it spread to other computers over a network, but while doing this, it consumes the network resources to a very large extent.

(I) ORIGINS

The invention of computer worms had, in fact, quite good motivations. It all began at Xerox PARC research centre. The research scientists at the centre wanted to carry out large computations. Having identified different pieces of computations which can be independently carried out, they designed small programs (worms) which could by themselves spread to other computers. This worm would execute on a machine if idle capacity was available on that machine;

otherwise it would continue to hunt for other machines 'in search of idleness'. This was the original purpose.

(II) MODE OF OPERATION

A computer worm usually operates on a network. Each node on a network maintains a list of all other nodes on the network. It also maintains a 'mailing list' which contains the names and addresses of the reachable machines on the network, the worm gets an access into this list and using this, sends a copy of itself to all those addresses.

If a worm is more intelligent and less harmful, after reaching there, it checks at a new node whether its copy already exists, and if it does, it does not create another one. If it is dumb as well as malefactor, it copies itself to all the nodes on the mailing list regardless.

(III) THE INTERNET WORM

Even if a worm is normally not harmful to the existing programs and data, it can be extremely harmful to the organizations and governments operating over a network, as a case in 1988 demonstrated.

It all began on November 2, 1988 when Robert Tappen Morris introduced a worm into internet. Internet is a network connecting thousands of computers at hundreds of corporations, universities, laboratories and government organizations in the world. The worm brought down the network causing tremendous havoc and generating major controversies.

A friend of Morris let out this secret to John Markoff a New York Times reporter. The following day, the story hit the front page of the newspaper, gaining more prominence than the news of the presidential election which was three days later. Morris was finally tried, convicted and sentenced to a 3 year custody, 400 hours of community service and a fine of 10,000 US Dollars.

(IV) SAFEGUARDS AGAINST WORMS

- (a) Prevent its creation: This can be achieved by having very strong security and protection policies and mechanisms.
- (b) Prevent its spreading: This can be done by introducing various checkpoints in the communications system. You can disallow the transfer of executable files

over a network. But then, this may prove to be an operational hindrance. An option could be to force the user or an Operator to “sanction” such a transfer. Many corporate gateways to public networks employ this technique.

7.6 COMPUTER VIRUS

A computer virus is written with a clear intention of infecting other programs. It is a part of a program which normally piggybacks onto an otherwise valid useful program. In this regard, it differs from a worm, which is a complete program by itself. On the other hand, the computer virus normally cannot, and does not, operate independently.

7.6.1 TYPES OF VIRUSES

There are several types of computer viruses. We give below a list of types that have been encountered so far. As time goes on, new varieties are likely to be added to this list.

- Boot Sector Infectors
- Memory Resident Infectors
- File Specific Infectors
- Command Processor Infector
- General Purpose Infectors

This classification is done based on what is affected (e.g. boot sector) or where the virus resides (eg. memory resident). These types are self explanatory and need no further explanation. However, this categorization is not mutually exclusive. For example, a file specific infector or command processor infector could also be memory resident.

A. INFECTION METHODS

There are five well known methods by which a virus can infect other programs. There are discussed below:

- (i) **Append** : In this method; the viral code appends itself to” the unaffected program.
- (ii) **Replace** : In this case, the viral code replaces the original executable program completely or partially.

- (iii) **Insert:** In this case, the viral code is inserted in the body of an executable code to carry out some funny or undesirable actions.
- (iv) **Delete :** In this case, the viral code deletes some code from the executable program
- (v) **Redirect :** This is an advanced approach employed by the authors of Sophisticated viruses. The normal control flow of a program is changed to execute some other (normally viral) code which could exist as an appended portion of an otherwise normal program.

B. MODE OF OPERATION

- (i) Find out another unaffected program on the disk. Say, it was the Order Entry (OE) program.
- (ii) Append only the harmful viral code (i.e. without the useful host game/utility) at the end of the “OE” program.
- (i) Change the first executable instruction: in the “OE” program to jump to this appended viral code. This is “Instruction-1” in the OE program as shown in Fig 7.B. At the end of the viral code, add one more instruction to execute the overwritten first instruction and then jump back to the second instruction of the OE program.

For instance. Fig 7.C shows the viral code appended to the host program. It starts with the label “VIRUS” in the program. You will notice at label “LLL” that the first executable instruction has been changed to jump to the viral code. You will also notice that after executing the viral code, it executes the overwritten instruction “Instruction-1” at ZZZ and then jumps back to “BACK” in the host program to continue from where it had left.

- (ii) The actual viral code contains the harmful instructions. The least harmful virus would just spread to other programs as described above. A very harmful virus may affect the boot sector or the FAT or some other operating system tables so that the entire system is crippled.

- (iii) In order to further fool the users and to prevent its detection, when a virus is attached to a new program, normally it is not completely executed in one shot. For instance, spreading to all other programs on the disk and then executing the actual harmful viral code for each program (e.g. encrypt to corrupt the executable files) one by one would be quite time consuming. The user/programmer who is running this original useful program such as OE would be quite alarmed by the unusually slow response. This will help its detection. In order to catch the user unawares, the virus designer normally puts the virus through three states, viz.,

- * Nascent
- * Spread
- * Harm

We will study these with an example. Let us again look at figures 7. B and 7 C (This is one of the possible ways of designing a virus)

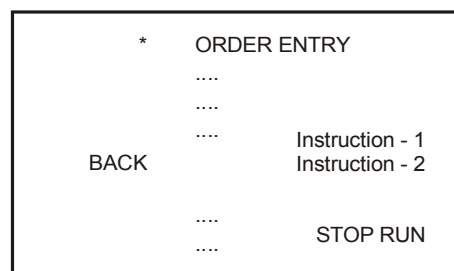


Fig. 7.B The program OE before the attack

Our sample virus defines a flag variable called “Status”. This described the status of the virus. Initially, at the time of creation, it is set to “Nascent”. At the subsequent execution, the viral code itself contains instructions to check the status flag and make it “Spread” if it is already “Nascent”. At this stage, the virus looks for another unaffected program and appends itself to it. The status flag in the newly appended viral code at this time is set “Nascent” only. At still subsequent execution of the original viral code, the viral code itself contains instructions to check the status flag and to change it to “Harm” if it is already “Spread”.

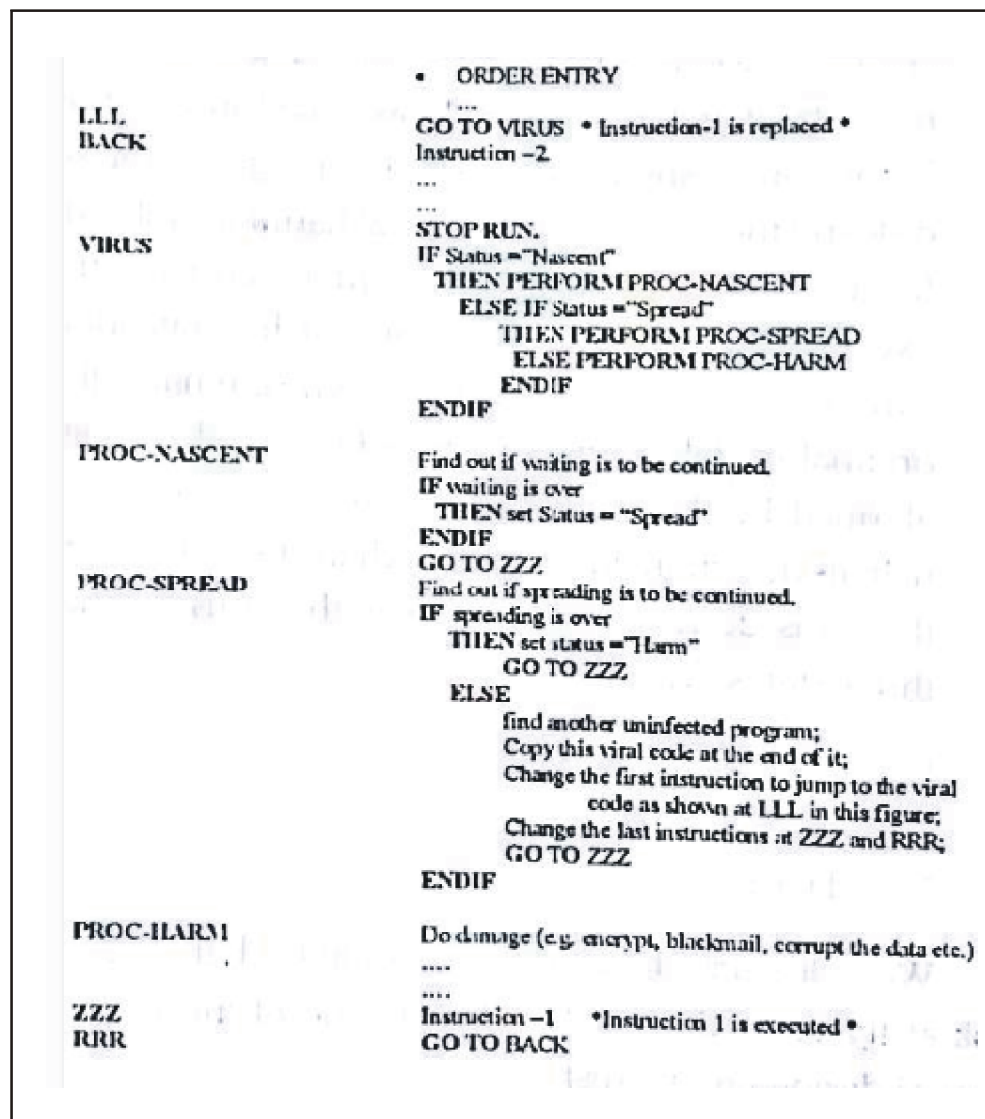


Fig. 7 C The program OE after the attack

This scheme works as follows

- (i) Assume that the virus is already attached to the Order Entry (OE) program. At this time, the status flag of the virus is set as "Nascent".

- (ii) When the first time the OE program is executed, it will jump to the viral code from LLL (shown as “VIRUS” in the figure).
- (iii) At this stage, it will check the status flag of the virus and jump to PROC-NASCENT due to its ‘status flag.
- (iv) At PROC-NASCENT, it will execute its waiting condition. For instance, some virus may wait for a few hours or days before it spreads. Some may wait until a specific calendar time. Some others may wait until the OE program itself is executed n number of times. This condition depends upon the creativity of the designer and is basically meant to fool the users to prevent its detection. If waiting is to be continued, the program jumps to “BACK” without changing its status flag, but only after executing the Instruction 1 belonging to OE itself, so that the execution of OE can proceed without any problems at least the first time after the virus infection.

B. VIRUS DETECTION

Normally, the virus detection program checks for the integrity of the binary files. The program maintains a checksum on each file or for better accuracy, on subsections of each file, in addition. At regular frequency, and for better control before each execution, the detection program again calculates this checksum and matches it with the one originally calculated and stored. A mismatch indicates some tampering with the executable file.

There are also some programs available which normally reside in the memory permanently and continuously monitor certain memory and I/O operations for guarding against any suspicious behavior.

C. VIRAL REMOVAL

A generalized virus removal program is very difficult to imagine due to the multiplicity of the viruses and the creativity with which they can be constructed. However, there are some viruses whose bit pattern in the code can be “predicted”. In this case, the virus removal program scans the disk files for the patterns of the known viruses. On detection, it removes them. However, if that virus has already damaged some other data, it would be almost impossible to recover the old data which could have had any values with different bit patterns.

D. VIRUS PREVENTION

The best way to tackle the virus problem is to prevent it, as there is no good cure available after the infection. One of the safest ways is to buy official, legal copies of software from reliable stores or sources. One should be extremely careful about picking up free, unreliable or illegal software. Frequent backups and running of monitoring programs also help in detection, and thus-subsequent prevention of different viruses.

7.7 SECURITY DESIGN PRINCIPLES

7.7.1 Public Design

The design of the security system should not be a secret. The designer should, in fact, assume that the penetrator will know about it. For instance, the penetrator may know the algorithms of cryptographic systems. However, security can still be maintained, because he may not know the keys.

7.7.2 Least Privilege

Every process should be given the least possible privileges that are necessary for its execution. For instance, a word processor program should be given access to only the file being manipulated and which is specified at the beginning. This principle helps in countering attacks such as Trojan Horse. This means that each protection domain is normally very small but then switching between the domains may be needed more frequently.

7.7.3 Explicit Demand

No access rights should be granted to a process as a default. Each subject should have to demand the access rights explicitly. The only thing that the designer has to keep in mind is that in this case, a legitimate user can be denied access at times. But, this is less dangerous than granting of unauthorized access. Also, the denial of access is reported or detected, and therefore, can be corrected.

7.7.4 Continuous Verification

The access rights should be verified at every request from the subject. Checking for the access rights only at the beginning and not checking subsequently is a wrong policy. For instance,

it is not sufficient to verify access rights only at the time of opening a file, the verification must be made at every read/write request too. This will take care of a possibility of somebody changing the access rights after a file is opened.

7.7.5 Simple Design

The design of the security system should be simple and uniform, so that it is not difficult to verify its correct implementation. Security has to be built in all the layers including the lowest ones. It has to be built in the heart of the system as an integral part of it. It cannot be an additional new feature.

7.7.6 User Acceptance

The design should be simple and easy to use to facilitate acceptance by the users. Users should not have to spend a lot of efforts merely to learn how to protect their files.

7.7.7 Multiple Conditions

Whenever possible, the system should be designed in such a fashion that the access depends on fulfilling more than one condition, e.g. the system could demand two passwords, or, in a cryptographic system, it should ask for two keys.

7.8 AUTHENTICATION

Authentication is a process of verifying whether a person is a legitimate user or not. There are two types of authentication that are possible, and in fact, necessary. The first is the verification of users logging into a centralised system. The second is the authentication of computers that are required to cooperate in a network or distributed environment. We will consider both of these scenarios one by one.

7.8.1 Authentication in Centralised Environment

Authentication in this environment can be achieved in the following three ways

- * A secret, known only to that user (e.g. password).

- * Something possessed only by that user (eg. An artifact such as a magnetic badge).
- * Some human (morphological, physiological or behavioral) characteristics of the user (e.g handwritten signatures, fingerprints, retina images)

These three ways are considered one by one.

(1) PASSWORD

The password is the most commonly used scheme which is also easy to implement. The operating system associates a password alongwith the username of each user, and stores it after encryption in a system file (e-g. / etc./passwd file in UNIX). When a user wants to log onto the system, the operating system demands that the user keys in both his user name and password. The operating system then encrypts this keyed in password using the same encryption technique and then matches it with the one stored in system file.

The password scheme is very easy to implement, but it is just as easy to break. All you need to do is to know somebody else's password. In order to counter this threat, the designers of the password systems make use of a number of techniques. Some of these are listed below.

7.8.2 Echo Suppression and Encryption

The password is normally not echoed onto the terminal after keying in. It is also stored in an encrypted form, so that even if somebody reads the password file, the password cannot be deciphered from it. In this case again, even if the encryption algorithm is known to the penetrator, the key is not known thus making penetration more difficult.

7.8.3 Choice of the Password

Three methods can be used in choosing a password. Either the operating system itself selects the password for the user or the system administrator decides the password for each user or allows a user to select it. The system -selected password may not be easy to guess for an intruder, but then the problem is that the user himself may not remember it. As the password is not chosen by the user, it may not have any significance for him, even remotely. MULTICS tried to improve upon this scheme by employing a password generator that produced random

combinations of pronounceable characters, eg. “Notally” or “Nipinzy”, which were relatively easier to remember.

Morris and Thompson made a study of passwords on UNIX systems in 1979. They collected a list of likely passwords by choosing various name, street names, city names and so on. They encrypted them with known encryption algorithms and then stored them in a ‘password-pool’ in an alphabetically sorted order to facilitate the search. They then found out how many users actually chose the passwords from this pool. They found that 86% did choose from the pool.

7.8.4 Password Length

The password length plays an important role in the effectiveness of the password scheme. If the password is short, it is easy to remember (Berman found that a password of length of five characters could be remembered easily in 98% of cases). But the, a short password is easy to break. If a password is too long, it is difficult to penetrate; but it is also difficult to remember by legitimate users. Therefore, a tradeoff is necessary. It is normally kept 6-8 characters long.

7.8.5 Pass Phrases

This scheme is used, especially if the password length is very short. Along with each password, a long but meaningful message or phrase is predetermined. For instance, for a specific password, it could be a full sentence such as:

“I AM READING A GOOD BOOK”

The operating system would then apply some algorithm on the bits of this message to arrive at a shorter derived bit pattern or additional shorter password. It then stores this additional shorter password alongwith the original password.

7.8.6 Salting

Salting is a technique suggested by Morris and Thompson to make it difficult to break somebody’s password. This breaking usually implies maintaining a large list of possible passwords and then encrypting and storing them in a sorted order. The salting technique appends a random number n to the password before the encryption is done.

7.8.7 Additional Password

Some operating systems ask for multiple passwords at different levels. This makes penetration more difficult. This additional password could be demanded at the very beginning or intermittently at a random frequency. This might irritate a legitimate user, but it provides additional security. Assume that a user has logged on to a machine and gone for a cup of coffee, and an intruder is trying to access some information at his terminal. He certainly would be quite stumped to see a password demanded at an unexpected time.

7.8.8 Continuous Challenge

An operating system, at random intervals, may ask predetermined questions to the user challenging him to prove his identity. For instance, the system may display a random number and expect the user to key in its square or cube as per the predecided convention between the legitimate user and the operating system. This convention will of course differ from user to user. A variation of this is to ask questions to the users on the lines given below:

- * Where were you born?
- * What was the name of your Maths teacher?
- * What is your exact height in inches?

7.8.9 Computer Dial Back

In this case, the operating system maintains a list of all the legitimate users and their work telephone numbers. After a user keys the username, the operating system consults this list and dials back the telephone number automatically to ensure that it is the same user. At this juncture, a prerecorded voice message could ask certain questions to the user who has picked up the phone. A question could be: "key in your date of birth now". The user has to key this in, to get an entry into the system. An alternative to this is to have a voice recognition system along with the operating system, and to validate the user when he speaks on the phone. This scheme is fairly good, but it can be expensive in terms of extra equipment and telephone charges especially if a user is at a remote location. This scheme also does not work satisfactorily if the unauthorised users start making use of the 'Call forwarding' facility from other authorised users?

7.8.10 Force Password Changes

In this scheme, the operating system forces the user to change the password at a regular frequency. This is done so that even if an intruder has found out a password, it would not be valid for a long time, thus reducing the window of damage.

7.8.11 One-Time Password

An extreme case of changing the passwords at a regular frequency is to force the user to use a different password each time. For each user, a list of passwords can be prepared by the operating system or the system administrator, and stored in the system. A user keeps one copy of the same with him. For the first time, the first password in the list has to be keyed in for the successful login. The user is forced to key in the 'next' password from the list each time he logs onto the system. The operating system as well as the user have to keep track of the 'next' password that is valid. When the list is exhausted, a new list can be generated or one could start from the beginning. This scheme has only one major drawback. It is not exactly a very safe 'strategy to lose this list.

7.8.12 Disable Users

Many operating systems allow a user to try a few guesses. After these unsuccessful attempts, the operating system logs the user off.

(a) Artifact-based

Some systems make use of artifacts such as machine readable badges with magnetic stripes or some kinds of electronic smart cards. The readers for these badges or cards are kept near the terminal from which the user is going to login. Only on the supply of the correct artifact that the user possesses, he is allowed to use the system. In many cases, this method is coupled with the use of a password. This method is popular in Automatic Teller Machines (ATMs).

(b) Human Characteristics

Each of these techniques measures something human about a user, which is normally unique to him. These can be subdivided into further categories as given below:

(c) **Physiological**

For example, fingerprints, patterns in the retina of the eye, hand shapes, facial characteristics. These techniques are also called “biometric” technique for obvious reasons.

In the case of fingerprints, for instance, the computer uses scanners to capture and store a database of the bit patterns of fingerprints for different users. When the user wants to access the machine, he inputs his ID into the computer, and gives the fingerprints again.

(d) **Behavioral:**

For example, voice pattern, timings of keystrokes, signature analysis, and so on.

In the case of voice pattern matching, the system requires the user to speak out something at the time of creating a user profile for him. The system digitizes these spoken passwords and creates a database of them for future use.

When the user wants to login, he is asked to speak out his password before he keys it in. The operating system can match the voice pattern for the ‘spoken password’ with the bit patterns of the voices stored for all legitimate user for “spoken passwords”.

7.9 **PROTECTION MECHANISM**

7.9.1 **Protection Framework**

One of the main problems that any file system has to tackle is to protect the files from unauthorized users. Confidential information from a very sensitive file should not be accessible to any ordinary user for reading, let alone for changing or deleting. In some cases, it may be necessary to prevent a user, or users belonging to a certain group from accessing a complete directory, i.e all the files and directories underneath it. This function of ‘protection’ is not treated as important in single-user systems such as CP/M or MS/DOS. However, in multiuser systems, this assumes enormous importance. In fact, like files, it may be necessary that certain devices are accessible only to certain users. The same thing may also be true about processes, databases or semaphores. The operating system has to have a generalized strategy to deal with all of them. All such items are called ‘objects’, which need to be protected by giving certain access rights to known ‘subjects’ who want to access them. A subject, in reality, could be and normally is a process created by either a user or the operating system.

(a) Access Rights

For various objects, the operating system allows different 'Access Rights' for different subjects. For example, for a file, these access rights can be Own, Write, Append, Read, Execute (OWARE), as in ADS/VS of Data General machines. UNIX has only Read, Write and execute (RWX) access rights. For example, for a printer as a device, the access rights can be 'Write' or 'None' only.

Serial No.	Access Rights	Code
0	No Access	N
1	Execute Only	E
2	Read Only	R
3	Append Only	A
4	Update	U
5	Modify Protection	M
6	Delete	D

Fig. 7.D Different access rights

An interesting alternative could be to organize the access rights in the form of a table as shown in Fig. 9.6 in such a way that the presence of any access right in the table implies the presence of all the ones preceding it in the table. For instance, if a process is allowed to delete a file (D), it should be certainly allowed to execute it (E), read it (R), append to it (A), update it (U) or modify its protection (M). Similarly, if a process is allowed to update a file (U), it should be allowed to read it (R) but not allowed to delete it (D).

Thus, one could specify only one code against a file to specify all the access rights for it according to this scheme. In this case, we could associate only one code for a subject (user) and object (file) combination. If a user creates a process, the process inherits this code from the user. When it tries to access a file F1, this access right code could be checked before granting any access to that process on F1. This scheme appears deceptively simple, but it is

not very easy to create this hierarchy of access rights in a strict sense that only one code implies all the rest above it.

(b) Domains and Domain Switching

The operating system defines another concept called domain which is a combination of the objects and a set of different access rights for each of the objects. You could then associate these domains with a subject such as a user or a process created by him. This is depicted in Fig. 7E

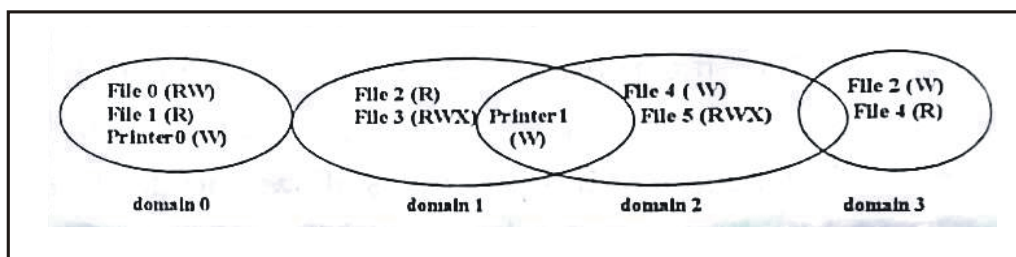


Fig 7.E. The concepts of a domain

A user process executing in domain 0 has an access right to Read from or Write to file 0, Read from file 1 and Write onto the printer 0. Domains 1, 2 and 3 are similarly defined. It will be noticed that domains 1 and 2 intersect. This means that an object (in this case printer 1 with access right = Write) can belong to two domains simultaneously. It also means that a user process executing in either domain 1 or domain 2 can write onto printer 1.

(c) Access Hierarchies

A variation of this scheme could be to organize these domains into a number of 'access hierarchies'. MULTICS used this concept. AOS/VS is also based on the same idea. Pictorially the entire protection space comprising a number of domains could be divided into a number of protection rings.

Each domain is again a set of access rights for a set of objects, but the entire protection space is divided into n domains 0 to $n-1$ in such a way that domain 0 has the maximum access rights and domain $n-1$ has the least. A subject such as a process executing in a specific ring can access all the objects permissible within that ring. If it changes its domain, a domain switch results. A domain switch to an outer domain is easily possible because it is less privileged than the inner ring, but domain switch to an inner domain requires strict permissions. These protection barriers are called 'gates'.

(d) Block Structured Languages

The concept of access hierarchies can easily be seen in the way the block structured languages such as C or Pascal are organized.

7.9.2 Access Control List (ACL)

In Access Control List (ACL), the operating system stores the data by column. For each file, it maintains the information about which users have which access rights. It, of course, skips the blank columns from the matrix, i.e. for those files, the ACL will be a null. It also skips the blank row entries in each column. It is obvious that the best place to maintain the ACL is in the directory entry for that file.

(a) ACL in AOS/VS

In AOS/VS, each user has a username and password combination. This combination is stored in a record for that user called 'user profile'. Each file in ADS/VS has an entry in the BFD. This entry contains the user name of the owner i.e. the user who created the file. Also, the BFD for that file contains a pointer to a block containing the ACL which specifies the access rights for that file.

7.9.3 Capability List

(a) General Working

In the ACL method, we had sliced the Access Control Matrix (ACM) by the column. If we slice the access control matrix horizontally by a row, we get a 'capability list'. This means that for each user, the operating system will have to maintain a list of the files or devices that he can access and the way in which they can be accessed.

(b) Comparison

ACL mechanisms "allow rigorous enforcement of authorization state changes through the centralization of data and mechanisms. As against that, the capability systems allow the protected data to be distributed. In capability systems, it is however very difficult to cancel any access rights already granted.

(c) Implementation

One of the ways in which the capability mechanism can be implemented is to use a technique of indirection and to have a central or global segment of capabilities.

(d) Advantages

Capability based method has the following advantages:

- (i) Small protection domains can be constructed in keeping with the principle of least privileges.
- (ii) The binding between high level names and capabilities may be done using the directories to hold the pairs of names versus capabilities.
- (iii) Object naming and protection can be provided at runtime.

7.9.4 Combined Schemes

It is possible to design protection mechanisms that employ a combination of ACL and capability list techniques. In such cases, initially a permission is verified using the ACL technique. At this juncture, a capability is granted, which is stored in the memory and then used for subsequent operations with the principles of the capability lists.

Let us take an example of operations on a file to see how this works. In this case, the following sequence of events takes place:

- (i) The Basic File Directory (BED) of file, or I-node in the case of UNIX contains the ACL for the file giving the information as to which user is allowed to carry out which operation on that file.
- (ii) When a user process opens a file for a specific operation (say read), this ACL is consulted to find out what access rights that process has on that file.
- (iii) If permission is granted, the operating system generates a capability in the memory and grants it to this user process.
- (iv) Thus, during its execution, each user process possesses different capabilities for different objects which it stores in its address space. This really then becomes its local capability segment.

It must be noted that here too, the operating system normally generates a global capability segment in the main memory, and keeps adding to it when a process requests a new capability. It can then maintain local capability segments for each

process pointing to the slots in the global capability list. The u-area or the PCB points to only the local capability list.

- (v) While performing subsequent operations on that file, the user process consults its local capability list, traverses to the correct slot in the global capability list to verify the specific access right required and then carries out the Operation, if permitted.

7.10 Summary

Protection is an internal problem. Security, in contrast, must consider both the computer system and the environment—people, buildings, businesses, valuable objects, and threats—within which the system is used. The data stored in the computer system must be protected from unauthorized access, malicious destruction or alteration, and accidental introduction of inconsistency. User authentication methods are used to identify legitimate users of a system. In addition to standard user-name and password protection, several authentication methods are used. Methods of preventing or detecting security incidents include intrusion detection systems, antivirus software, auditing and logging of system events, monitoring of system software changes, system-call monitoring, and firewalls.

7.11 Review questions

1. What is Atomic Verifications?
2. What is the difference between a worm and a virus
3. What is a virus?
4. Explain in detail how a virus operates
5. What is Convert Channel?
6. Discuss Virus detection.
7. What is removal?
8. What is meant by Authentication?
9. Discuss Access Rights?
10. What is Arbitrated Digital Signature?
11. What is meant by Authentication Code?
12. What is Capability list?

LESSON - 8

ENCRYPTION

Structure

- 8.1 Introduction
- 8.2 Objectives
- 8.3 Security in Distributed Environment
- 8.4 Summary
- 8.5 Review Questions

8.1 Introduction

Encryption is one of the most important tools in protection, security and authentication. The process involves two things: Encryption which means changing the original data to some other form so that nobody can make out anything about it, and Decryption which means recovering the data in the original form. Figure 8.A depicts this process. The figure shows A and B as the source and destination nodes.

The data before encryption is called plaintext and the data after encryption is called ciphertext. The encryption and decryption are normally performed by hardware devices for the reason speed. They can be combined into one device too. The hardware performs these function according to a predetermined algorithm.

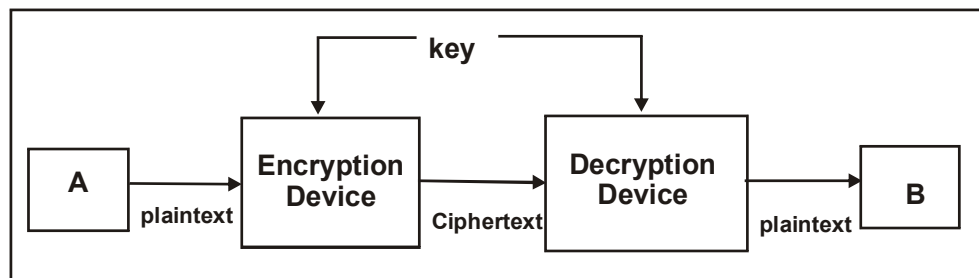


Fig. 8.A Conventional encryption

8.1.1 Encryption Algorithm and the Key

Various algorithms are used for encryption and decryption. There are two basic methods (ciphers) used for encryption. These are

- * Transposition ciphers
- * Substitution ciphers

In the transposition ciphers, the letter in the message are not changed but their order is changed. A simple example of this type could be a message sent in the reverse order. "I am fine" would become "enif ma I". 'Railfence cipher' is a popular and simple method which belongs to this class. This method obviously requires more memory and is slower, because the full message has to be stored before it is encrypted and sent.

Substitution ciphers work by sending a set of characters different from the original ones. For instance, you could send the next alphabet, "I am fine" would become "J bn gjof". Caesar cipher is a popular and simple example of this type. This method is relatively faster. It also requires less memory because, as each character arrives, the next character could be immediately determined and sent. No storage will be required for this operation.

Thus, every encryption has an algorithm and a key. An important point is that the encryption process must be restorable. For instance, if the algorithm is to perform the "OR" operation on every 8 bits in the message and the key is "11110000", decryption would be impossible and therefore, this process will not be restorable. If a bit at a specific position in the ciphertext and the bit in the corresponding position in the key are both equal to 1, it would be impossible to guess what the original corresponding bit in the plaintext was. It could have been either 0 or 1. In both cases, the result would be the same and would be equal to 1. For instance, using the key "11110000", if the result of the encryption process (ciphertext) is "11111100" by using the "OR" logic for every bit, what would be the first bit in the original message? As is clear, it could be 0 or 1. Hence, this method is not restorable.

In actual practice, both encryption and decryption are done by a hardware device wherein one has to feed in the key. Also, normally only one piece of hardware is responsible for both encryption and decryption. Hence, having this device at both ends serves the necessary purpose in the duplex communications, wherein the message traverse in both directions.

In conventional encryption scheme, both parties A and B have to agree upon a key. Somebody (A, B or a third party) has to decide upon this common key, get the concurrence of all concerned parties, and then initiate the communication. This process is called key distribution. In the conventional encryption method, each pair of nodes need a unique key. Thus, if there are n nodes, you will need $n(n-1)/2$ keys. For 100 nodes this number is $100 \times 99/2 = 4950$. Deciding, agreeing upon, conveying and storing of these keys can become a mammoth task. Tapping or leaking can take place, during this process. This problem is known as key distribution problem.

An alternative method is called Public Key Encryption. This is based on a principle that the keys used for encryption and decryption should not be the same. In fact, there are possible algorithms and hardware, in which one key K_1 is used for encryption and another key K_2 is used for decryption. In fact, only K_2 can decrypt the message and NOT K_1 . This is important because one of these keys is publicly known (hence, the name) and another one is known only to the person who does decryption thereby ensuring that the information can not leak out. There is an interesting property of the public key encryption system, Even if you interchange the keys, the encryption/ decryption is still possible. For instance, you can use K_2 for encryption and K_1 for decryption. The scheme will still work.

Therefore, in this scheme, for each user, two keys are defined. One is called 'private key' and the other 'public key'. Each user node keeps its private key secret, but publishes the public key to the central key database.

The following steps are taken to achieve the encryption/decryption

- (i) when A wants to send the data to B, A consults the database for public keys and accesses B's public key. Because it is a public key, it is accessible to A even if it is B's public key.
- (ii) A encrypts the message using B's public key and produces the ciphertext.
- (iii) The ciphertext now traverses to node B.
- (iv) Node B decrypts the message using B's private key, which is known to it anyway.

8.1.2 Data Encryption Standard and CCEP

As in everything else, a standard evolved in 1977 for data encryption, called 'Data Encryption Standard (DES)' was adopted by Nation Bureau of Standards (NBS). In DES the data is divided

into 64 bit blocks, and a 56 bit key is used to carry out the encryption. The output again consists of different blocks of 64 bits each. The length of the key in DES has sparked off an interesting controversy.

NBS had asked for proposals from various parties in 1973 to prepare an encryption standard. IBM, at that time had almost finalized a scheme of their own, known as Lucifer which used a key of 128 bit length. Ultimately, Lucifer was adopted but the key length was dropped to 56. The reasons for dropping 72 bits of the key were interesting. Ironically, One of the reasons was, in fact, that a 56 bit key was easier to break (with 2^{56} possibilities) than a 128 bit key (with 2^{128} possibilities).

DES has come to be widely accepted today in the world as a standard, especially in the commercial and financial applications. The discussion on the exact algorithms followed by DES is beyond the scope of the current text.

Apart from DES, there are many different standards Commercial Communications Endorsement Program (CCEP) is one such standard developed by the National Security Agency (NSA). The CCEP algorithms themselves are also secret in addition to the keys. There are again two different types of CCEP but the discussion about them is also beyond the scope of the current text.

One point, however, has to be remembered. One important advantage of these standards is that since the encryption/decryption is done by hardware, standard hardware chips can be mass produced economical.

8.2 Objectives

After studying this Lesson, you should able to

- Encryption
- Encryption Algorithm and the Key
- Security In Distributed Environment
- Traffic Padding
- Windows Technology
- GUI
- Components of GUI

8.3 SECURITY IN DISTRIBUTED ENVIRONMENT

Problems of security are compounded in the distributed environment, because a message passing through the network can potentially be tapped at multiple locations. In non-distributed environments, if an intruder wants to access the data from a file, the password and the ACL mechanism of the file system have to be broken into. This can happen in the distributed systems too. However, in this case, the communication links can be tapped while the file is being transferred to another node and the data from the file can be accessed without having to know the password or the ACL for that file.

For active attack, the intruder has to get control over the link to be able to insert or modify/delete, any data. For passive attack, the intruder merely needs to “listen” to the link. In twisted pair and coaxial cable, invasive taps or inductive devices that monitor electromagnetic emanation can be used for penetration. Invasive taps can be used for both active as well as passive attacks, whereas inductive devices can be used, only for passive attack. Optical fiber, however, is a good communication medium, where tapping is almost impossible. This is because, it does not generate electromagnetic emanation. In comparison, the microwave or satellite communication can be easily tapped due to its wider geographic coverage.

8.3.1 Locations of Encryption Devices

There are basically two forms of encryption:

- * End to end encryption
- * Link encryption

In Fig 8.3, the dark box stands for end-to-end encryption and the empty box stands for line encryption. The boxes actually stand for the encryption devices used in the system. If end-to-end encryption is used by two nodes X and Y as shown in Fig 9.19, We need only two encryption/decryption devices at both these ends and we do not need to bother about any other encryption. The entire data moves about throughout the network in an encrypted form. However, this scheme has problems in packet switched networks. In this case, data is sent in the form of packets where a packet consists of a header containing, among other things, source and destination addresses and the actual data.

This header containing the routing address cannot be encrypted since the nodes involved in routing will not be able to decrypt and understand the routing addresses and, therefore, will not be able to route the data properly. This is because, the intermediate nodes are not supposed to know the key used for encryption by the node X. To surmount this problem, only data portion of a packet is encrypted while the header is transmitted without any change. However, this leaves the system vulnerable. An intruder can tap the information from the headers to carry out the traffic analysis.

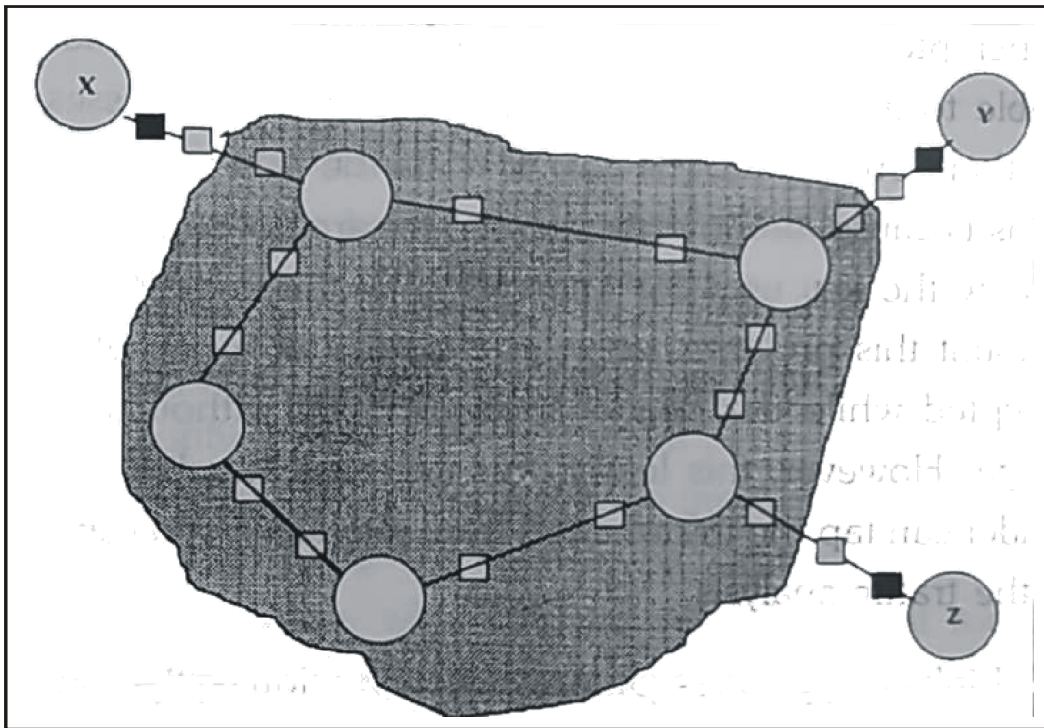


Figure 8.B-Two encryption matrices

Link encryption requires more encryption equipment for each line-obviously two devices, one at each end. This prevents any tapping on the line. Also in this method, the entire packet including the header can be encrypted. As the figure shows, just before the packet enters a node, decryption is carried out, so that the node will receive the plaintext. This is a far better method in terms of security. However, it is more expensive and time consuming.

In order to make the system sturdy, of both of these methods can be used.

8.3.2 Key Distribution

If two parties want to communicate with one another, they should send the data in an encrypted form. There are two ways of doing this:

Permanent Key: In this case, a key is predetermined permanently between the two parties and it is used by both of them for all the communication.

Sessions Key: In this case, every time a party wants to start a dialogue with the other party, a session is started and a separate key is agreed upon for each session.

Combination of both of these methods is also possible. Having agreed upon the key, the encryption and decryption can continue smoothly. The point is: who decides upon the key and how is it conveyed to these parties? If party X conveys to party Y a key that he intends to use for a specific session, what will happen if this first message about the key itself is tapped? The intruder will know about the key which is going to be used between X and Y for a specific session. The entire dialogue during that session can then be tapped thereafter. This is the problem that we need to surmount.

Key distribution can be achieved in a number of ways as described below:

- (i) Party X selects a key and physically hand delivers it to Y.
- (ii) A third party chooses a key and physically hand delivers it to both X and Y. Obviously, the third party is assumed to be trustworthy.
- (iii) Assume that X and Y have been communicating with one another using a key say K1. At some juncture, X chooses a new key (say K2) and communicates it to Y on the network after encrypting K2 using K1. This will make it more difficult for the intruder to penetrate. In a sense, it is another form of method (1) where the key is sent over a network instead of hand delivering.
- (iv) A third party decides a key for X and Y as in (ii) but sends it to both parties X and Y on the network using already encrypted lines, if they exist. This, again, is another form of method (ii) where the key is communicated over a network instead of hand delivering.

Of these four methods, the first two involve physical key distribution. Apart from the possibility of its getting lost, there is also a problem of managing the system when there are a number of parties wanting to communicate with a number of other parties. Therefore, they are not particularly suitable in a large network.

The third method is an appealing one, but a closer look would reveal its drawback. If an intruder has the first key, all the subsequent keys can be derived easily by tapping. Therefore, normally the fourth method is used in a distributed environment.

In fact, in this method, you could have two servers, as listed below:

- * Authentication Server (AS)

- * Distribution Server (DS)

Authentication server (AS) is responsible for allowing or disallowing a party to communicate with the other. Distribution server is actually responsible for the key distribution.

These two pieces of hardware/software could be combined in one node, or they could be split into two nodes. The way this scheme functions is as follows:

- (i) When X wants to communicate with Y, X applies to AS for permission to communicate with Y.
- (ii) AS checks for permissions using the access control mechanism, as discussed earlier.
- (iii) Having checked the permissions, AS makes a request to the DS to generate a key and distribute it to X and Y.
- (iv) Both X and Y receive the key from DS over the network using the encrypted lines if they exist.
- (v) X and Y communicate with one another using that key.

This procedure could be repeated for every session between X and Y, if sessions key is used, or it could be followed only once initially, if a permanent key is used. The problem is : How does the communication among X,Y, As and DS take place during steps (i) to (v)? What steps

can be taken to avoid tapping, when this key distribution itself is taking place? These are the problems that need to be solved by the designer.

8.3.3 Traffic Padding

We have seen that if we use only end-to-end encryption, an intruder can carry out the traffic analysis because packet headers are not encrypted to enable routing. Usage of link encryption can overcome this problem, but this can be an expensive solution. An alternative is to use the technique of 'traffic padding' as depicted in Fig. 8C

If the input is not continuous, we can have a random data generator which would output some random data on the input line before the encryption. At any time, if the genuine input data is present, it is encrypted and sent without this random data. If there is no input data, the random data is encrypted and sent, which is obviously removed subsequently. This technique is essentially used to mislead the intruder's traffic analysis.

8.3.4 Message Authentication

Message authentication is a process whereby communicating parties can verify that the data received is authentic. Normally the following attributes of the message need to be verified (i.e. authenticated) to achieve this.

- * Actual message sent
- * The time at which it was sent
- * The sequence in which it was sent
- * The source from where it has arrived

Out of these, authenticating the source is probably the most critical issue. Common methods used to achieve message authentication are:

- * Authentication code
- * Encryption
- * Digital signatures

(a) Authentication code

Authentication code is used as follows:

- (i) A secret key say K is selected and using this, some calculations are performed using some algorithm. This is used by the source node X on the data to be sent, to produce some code called authentication code. We will call it $AC-X$ in our discussions. The philosophy is similar to the Cyclic Redundancy Check (CRC).
- (ii) Node X sends the original data as well as $AC-X$ to Y
- (iii) Node Y performs the same calculations using the same secret key and the same predetermined algorithm as are used by the source node X on the received data, to arrive at its own authentication code. We will call this $AC-Y$. If everything is OK. $AC-Y$ should be same as $AC-X$.
- (iv) Node Y compares $AC-X$ and $AC-Y$.
- (v) If they are the same, Node Y is assured that Node X has sent it, because the secret key is known only to X and Y .

This method can also be used to detect errors. e.g. AC can be as an additional checksum mechanism.

The point is: How do we produce the AC using the data and the Key? The DES algorithm suggests some ways. You could use your own methods too. However, the function of authentication is quite important, and it is widely used to send important financial messages or data. SWIFT uses it quite widely.

(b) Encryption and Message Authentication

Encryption can be used to authenticate messages. In this case, the “algorithm” mentioned in the previous section would actually be the encryption algorithm itself.

However, we know that there are two ways in which encryption can be performed. Conventional encryption can certainly be used for message authentication, because it is based on key. Hence, if Y receives a message from X and it is able to decrypt it successfully using the same key, it can be sure that the message must have arrived from X (except when the key is stolen).

The public key encryption, however, cannot be used for message authentication as the source node X sends the data encrypted with a key which is publicly known (The destination's public key). Hence, anybody could have sent it. Thus, public key encryption provides protection but not authentication. If you want protection as well as authentication, you have to use a new technique called 'digital signature'.

(c) Digital Signature

Digital signature is like any other human signature on a plain paper. Let us assume that a person A sends a signed letter to person B. This serves two purposes.

- (i) A cannot deny having sent a letter to B (B can produce it as proof).
- (ii) B cannot refuse having got it, obviously assuming that the post office is a perfect institution and A can produce the proof of its being received by B.

The same should also happen with electronic message using digital signatures. That is the ultimate aim.

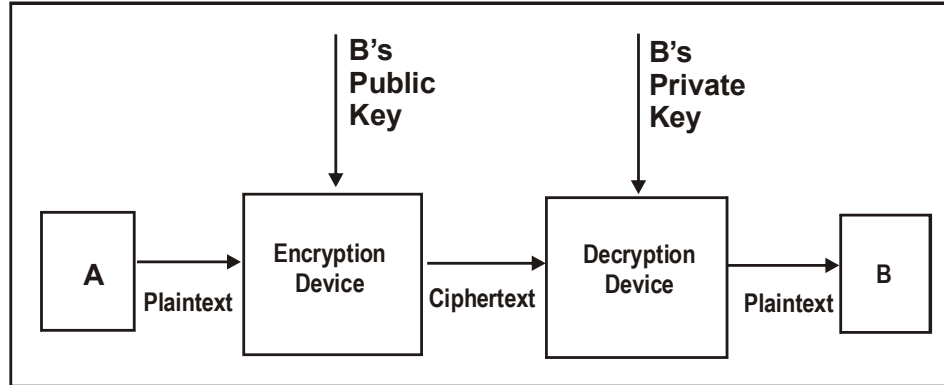


Fig. 8.C - Public key encryption

We have seen that the public key encryption scheme provides protection but not authentication as seen in Fig. 8.C. If we want to use public key encryption to get authentication without protection, the sequence in which the keys are applied is just reversed as depicted in Fig 8.E. Note that difference between Fig. 8.D and 8.B. In Fig 8.E, we use the keys of A, because the purpose is authentication.

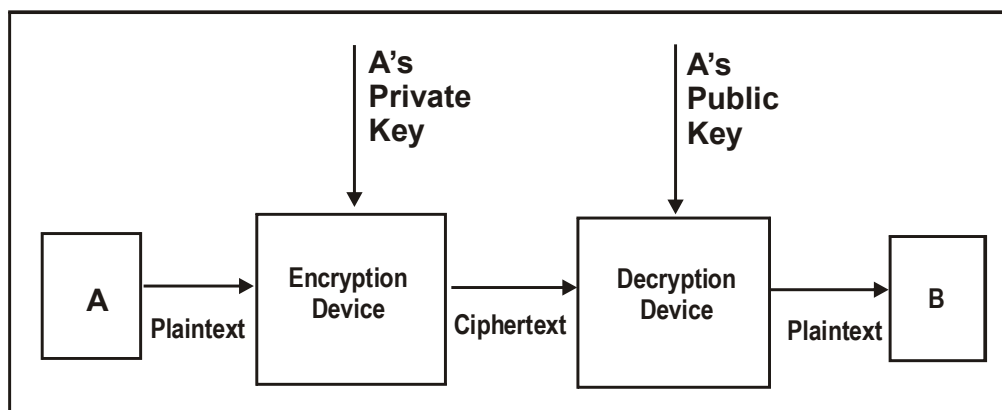


Fig 8.D Public key encryption for authentication with protection

This method is based on the principle that the public key encryption algorithm is such that you could use either one of the public and private keys for encryption and the other one for decryption. As the figure shows, the plaintext from A is encrypted using the private key of A which only A knows. At the other end, it is decrypted using the public key of A, which everybody knows. After decryption, the user B can be sure that only A could have sent that message.

Hence, this scheme is good for authentication, but it cannot be used for protection, because anybody can access A's public key and decrypt the message to get the information. The problem is: How can we achieve both authentication and protection? There is a way out of this. If you want to provide both authentication and protection, you have to use the public and private keys of both A and B in a specific sequence.

This scheme works on the principle that the two keys (public and private) can be used in any order. Hence, the ciphertext at points 2 and 4 in the figure is the same because it is encrypted by B's public key and decrypted by B's private key. Therefore, conceptually, we can remove the portion between points 2 and 4. Now applying the same principle, the plaintext at points 1 and 5 is also the same because it is encrypted using A's private key and decrypted using A's public key. This is how the original data is restored.

The important point is that this scheme provides authentication as well as protection. Authentication, because between points 4 and 5, the decryption is done by A's public key, and only A could have encrypted it with its private key. Hence, the message could have been sent only by A, assuming that A has not leaked out its private key. This guarantees authentication. It

also provides protection, because point 3 onwards, only B can decrypt it with its private key. None other than B is supposed to know B's private key. Hence, nobody else can access it apart from B. The only weak points are the links depicted by number 2 and 4 where anybody can decrypt the message with A's public key. Hence, tapping at points 2 or 4 can lead to security violation. However, so long as those links are kept minimum (or nil by pipelining two encryption and decryption devices respectively), this scheme achieves both authentication and security. This is how digital signatures work.

What we have discussed so far is direct digital signatures. We can also use 'arbitrated digital signatures' whereby A sends a message to an arbitrator who sends it to B after keeping a record of it. In case of a dispute of whether A had sent a message, or whether, B had received it, the arbitrator's decision is regarded as final.

8.4 Summary

The data stored in the computer system must be protected from unauthorized access, malicious destruction or alteration, and accidental introduction of inconsistency. Several types of attacks can be launched against programs and against individual computers or the masses. Encryption limits the domain of receivers of data, while authentication limits the domain of senders. Encryption is used to provide confidentiality of data being stored or transferred. Symmetric encryption requires a shared key, while asymmetric encryption provides a public key and a private key. Authentication, when combined with hashing, can prove that data have not been changed.

8.5 Review Questions

1. Why do we need encryption?
2. Explain about various algorithms in encryption?
3. How to implement encryption in distributed environment?
4. What is digital signature?

LESSON - 9

WINDOWING TECHNOLOGY

Structure

- 9.1 Introduction**
- 9.2 Objectives**
- 9.3 Graphical User Interface**
- 9.4 Components of GUI**
- 9.5 Summary**
- 9.6 Review Questions**

9.1 Introduction

Today's computers on the desktop are very powerful and can do more than one job at a time. With any Windows product (e.g. MS-Windows, X-Windows), the screen can be split into different partitions as shown in Fig. 8.G. Each partition can be of a different size. We can run different applications in each partition of the screen and watch the progress of each application in them. Each of these partitions is called a window and hence, the technology is known as windowing technology. Each window is independent of the others and can be of any dimensions. Some of the characteristics of a window are its title, borders, work area and command area. There can be scroll bars at two of the borders to allow the user to scroll horizontally and vertically. The typical scroll bars are shown in Fig 9.A.

Today's computers on the desktop are very powerful and can do more than one job at a time. With any Windows product (e.g. MS-Windows, X-Windows), the screen can be split into different partitions as shown in Fig. 8.G. Each partition can be of a different size. We can run different applications in each partition of the screen and watch the progress of each application in them. Each of these partitions is called a window and hence, the technology is known as windowing technology. Each window is independent of the others and can be of any dimensions. Some of the characteristics of a window are its title, borders, work area and command area. There can be scroll bars at two of the borders to allow the user to scroll horizontally and vertically. The typical scroll bars are shown in Fig 9.A.

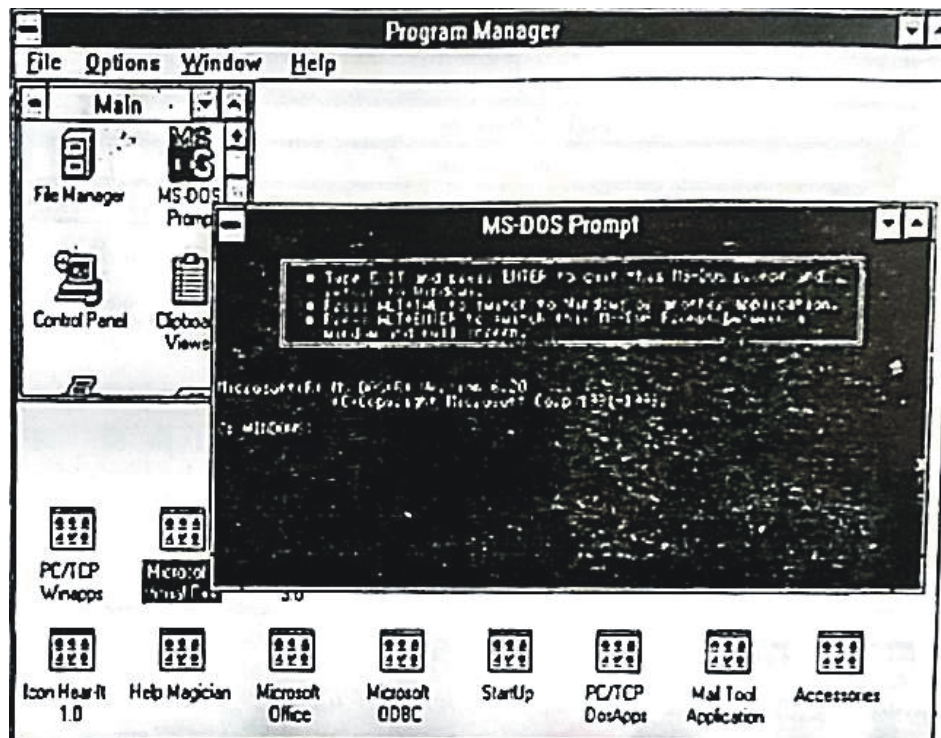


Fig. 9.A – A typical GUI environment

The title helps to identify each window separately. Users can configure each application to have specific settings. These will include the background and foreground colors of the window work area, size of each window, the border colors and the fonts to be used in the window.

These can normally be configured for each application or at a global level. On the screen, windows can overlap each other as shown in Fig.9.B In this way, the screen can be divided into many windows of different sizes. The Windowing software will have to reserve separate areas in the memory for different windows as per their sizes and allow all the operations on them as if each were executed on a separate terminal. While displaying a screen, it will have to display all of them simultaneously. This complicates the I/O handling portion of the software.

However, the computer has only one keyboard and only one of each type of input devices. Having multiple input devices will not help, because, the user can handle only one at a time anyway. So a question arises: Where does the user input go to? User can control where the input goes to, by bringing the window of his choice to the top of the screen. This can be done by

using either specific key combinations or an alternative input device (normally a mouse). The window thus brought forward is said to be “in focus” and henceforth, will receive all the user input. In a single-tasking system like MS-DOS, the application in the window in focus will be the one which will be executed by the CPU.

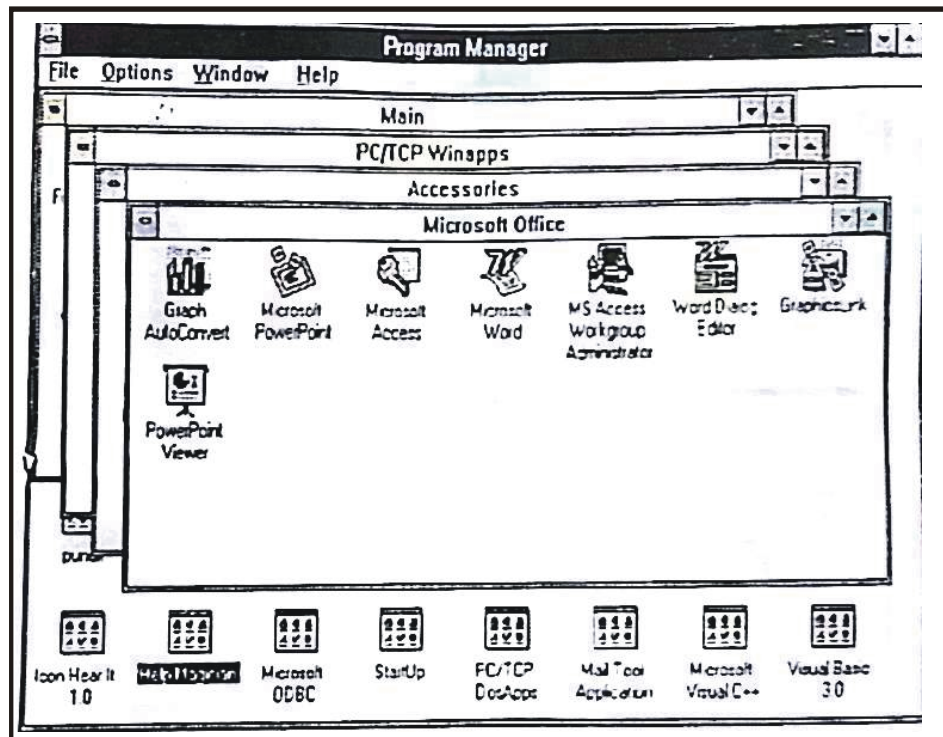


Fig. 9.B- A program manager

9.2 Objectives

After studying this lesson, you should be able to understand

- Windows
- GUI
- Components of GUI

9.3 Graphical User Interface

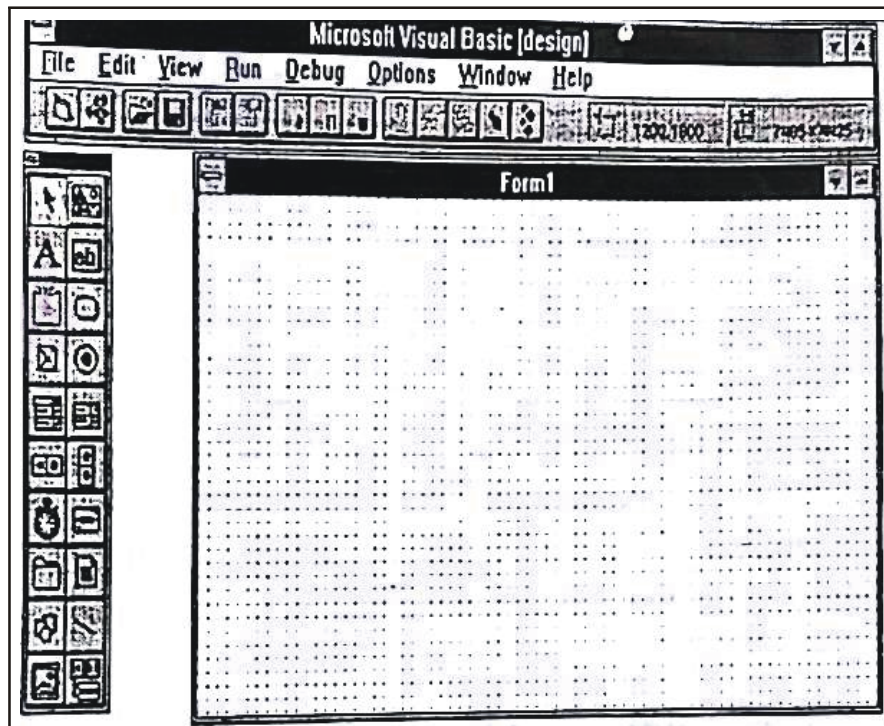


Fig. 9.C. Icons

In the past, computer-user interface consisted of some cryptic acronyms. It was very difficult to remember all the commands. Each command had several options. (e.g. the Unix. "ls" command which has over 25 options.) Current trend is to display various commands on the screen, and allow the user to select a command using a cursor and simply clicking on it. The problem in this scheme is that with hundreds of commands available, it may not be possible to display all the commands at one time. Therefore, commands are grouped together in various levels of hierarchy, and when the user selects a group, further commands in that group are displayed. This is shown in Fig. 9.C and 9.D

On selecting a specific command, if there are any options to the command, these are also displayed. This allows the user to select the command he wants and use the application without first having to know about the computer and its working. The display of these command sequences takes place graphically.

A very important feature that has always been supported with the GUI-based application is “HELP” about various features of the application. HELP can assist the user in knowing everything about the application. This makes it more efficient for may one to use the system without knowing much about it. GUI-based applications are designed in such a way that the user is in full command of the application. A fundamental concept of windowing environment is ‘event driven programming’. The application responds to each command from the user and waits for the next. The events can be mouse movement, clicking of mouse or keyboard input. If the user gets confused at any point, a HELP is readily available. This makes GUI-based applications more popular and efficient.

The input device which is commonly used with GUI is a device called a mouse which normally actually looks like one and has two or three buttons on it. Navigation on the screen is faster with a mouse than with a keyboard cursor.

9.4 Components of GUI

Some of the common components of GUI environment are;

- * Menu bars with pull down menus
- * Scroll bars
- * Controls
- * Dialogue boxes
- * Feedback pointer

These components are considered one by one;

(a) Menu Bars

Menu bars normally appear at the top of the window under the window title. Some of the commonly used menu bar options are File, Edit, View and Help. Figure 9D shows a typical menu bar with these options. When one of these menus is selected, a pull down menu appears on the screen. A pull down menu is a rectangular box with more specific actions listed in the box. The pull down menu will have an action on the left hand side and a keyboard accelerator combination on the right. The action can be selected from this menu. It can also be selected with the keyboard using the accelerator combination from within the application without using the pull down menu. Similar actions are always placed in one pull down menu. All files related actions such as to save, rename, copy, or delete will be listed under the file menu. The edit

menu will have actions to insert or delete a word or a line, select a block of data and copy or move the block of data to another location within a document.

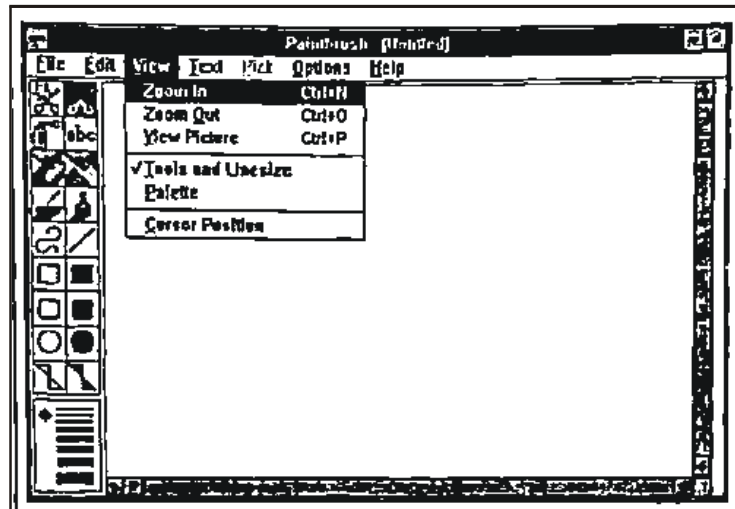


Fig. 9.D – Menu Bars

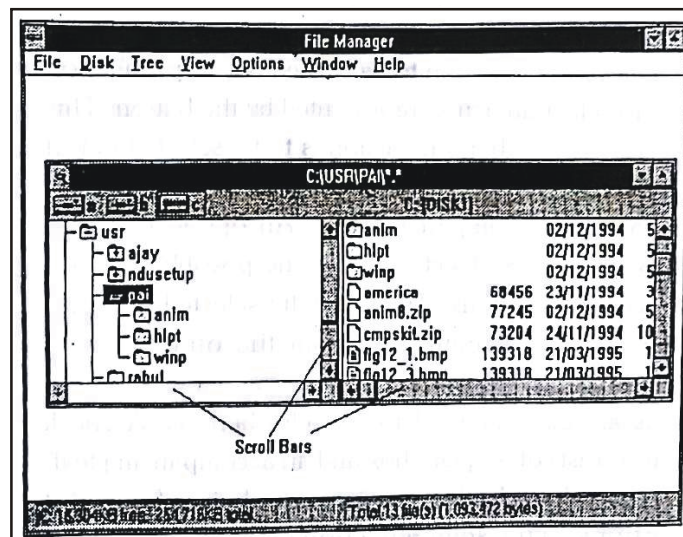


Fig. 9.E. Scroll Bars

(a) Scroll Bars

Scroll bars are used to look at information which is not currently visible in the window. Scroll bars and its usage are shown in Fig. 9E. A scroll bar consists of a horizontal or vertical

scroll area (rectangle) with a slider box and an arrow in a box at each end. The slider box gives a hint on the size and the position of the visible part of the object in relation to the entire object.

(b) Controls

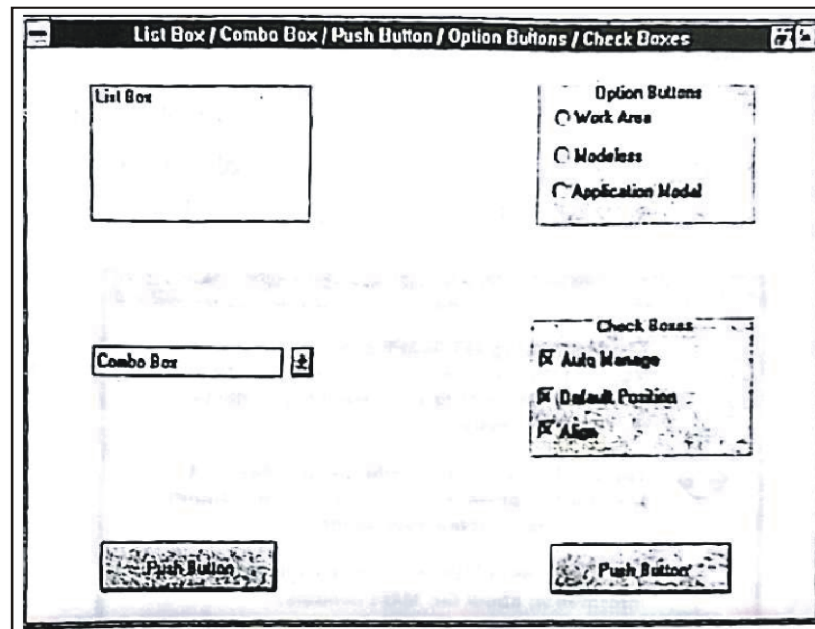


Fig. 9-F. Option Buttons, Check Boxes, List Box, Combo Box and Push Buttons

A variety of controls are used in a GUI to enable users to select type of information. These controls are either buttons or boxes. A Pushbutton consists of a graphical image (rectangle with rounded edges) of a button and a label indicating the action to be carried out. A pushbutton is used to select an action represented by the button. This is normally used when one action is to be selected out of many choices. An option button consists of a graphical image and an accompanying text. An option button is used to select one object out of many possible mutually exclusive combinations. The currently selected or active choice can be distinguished from the others by the highlighting on the graphic image. Sometimes option buttons are also referred to as radio buttons. A check button consists of a square box and an accompanying text. This is used for selecting one or more choices from a list of options. The selected choices are highlighted appropriately on the square box (a cross on the box or graying of the box). Figure 9.F shows the use of these in a typical dialogue box. The box controls are list boxes and entry boxes. A list box is a rectangular box with scroll bars. This allows you to select one item from a scrollable list of

choices. List box is shown in Fig 9.F. Entry box is a rectangular box to allow the user to enter some text. Typical entry box is shown in Fig. 9.G. A cursor gives a hint about the location where the text can be entered. An additional hint about the type of text to be entered is provided near the box. Combo Box is a combination of list box and entry box. A typical combo box is shown in Fig. 9.F.

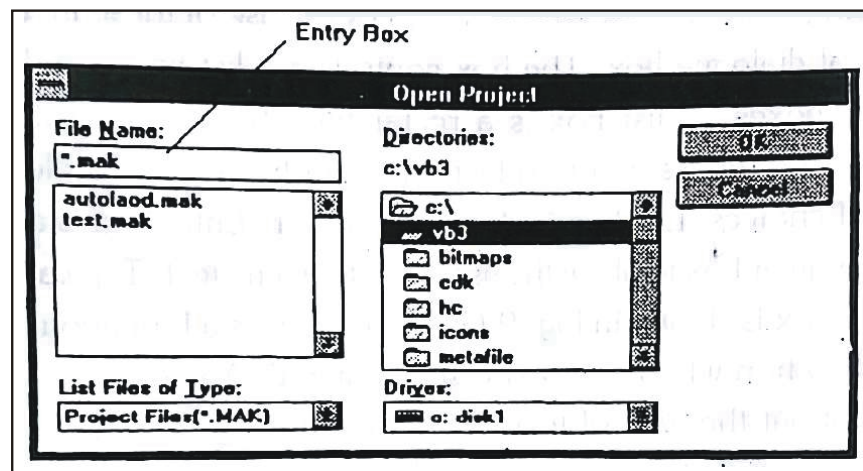


Fig. 9-G Entry Box

(c) Dialogue Boxes

Dialogue box is a window used by the application to interact with the user. A dialogue box can also be used to display information or to get user input and also for a combination of these two functions. A typical dialogue box is shown in Fig. 9.H.

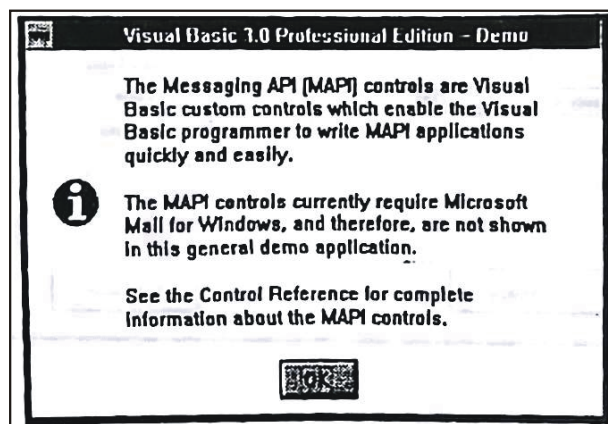


Fig. 9-H. Model Dialogue Box

A dialogue box where an application can continue only after the user has responded to the dialogue is called modal dialogue box. A dialogue box which allow the user to continue without responding to it is called modeless dialogue box. Such a dialogue box is shown in Fig 8.P. Dialogue boxes usually combine some of the controls already discussed above to interact with the user.

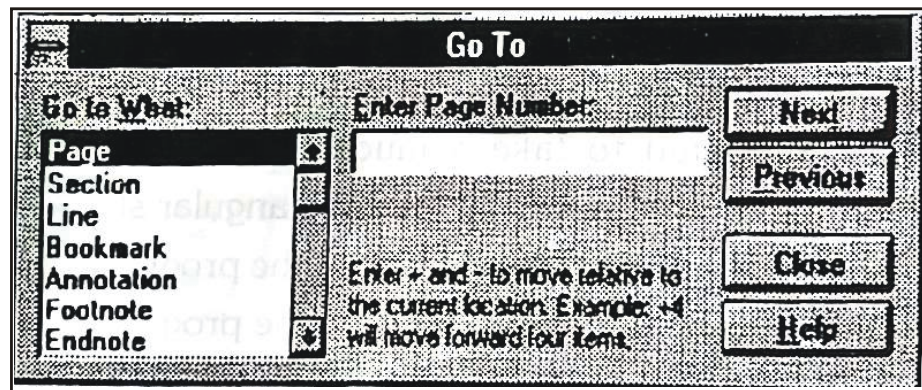


Fig 9.1 - A modeless dialogue box

(d) Feedback

Feedback is provided in two common ways to the user by using an hour glass pointer or a progress indicator. An hour glass pointer is shown in Fig. 9.J. These are used when the system cannot complete the user request immediately but takes a long time to complete it. The hour glass pointer is used by an application when the system is performing some simple operations. The convention is to change the mouse pointer to hour glass shape. This tells the user that the requested function is in progress. When the request is completed, the hour glass pointer changes back to the shape of the original pointer.

A progress indicator is used when the requested action is expected to take a much longer time. This progress is indicated normally on a rectangular strip with a scroll bar indicating the percentage of the progress made. This is periodically updated to indicate the progress. Since this is used for time consuming tasks, a provision is normally made to abort the requested operation.

A GUI consists of and makes use of a number of objects. One of the very important GUI objects is an 'Icon'. An icon is a graphical representation of an application or a utility. Creating good graphical icons which transcend the language and cultural boundaries is a very important aspect of a good GUI design. A good icon should be able to help identify and even invite the user

to an application. Some sample icons are a trash bin to delete files, a mailbox to open the mail application. Icons are the navigational aids for a computer user to find his way around the maze of applications. Icons which are generally used in the various GUI based applications are shown in Fig.9.K and Fig.9.L. Figure 9.K shows the icons that are used to display various types of files or application programs that are contained in the directory, whereas Fig. 8.S shows how icons can be used to represent various applications in the window.

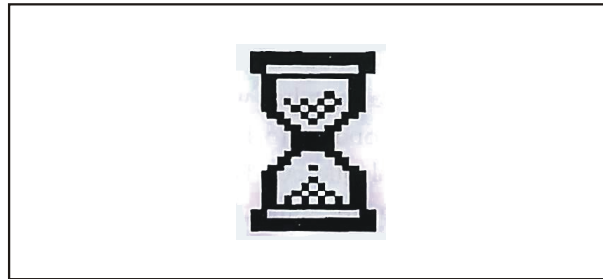


Fig 9.J Hourglass Feedback

Windows based applications are always designed to be event-driven. The windowing system and the applications wait for an event and then respond to that event. Some events can be: user input from keyboard, clicking of the mouse, movements of the mouse, movement of the mouse from one window to another, arrival of messages, and closing of windows.

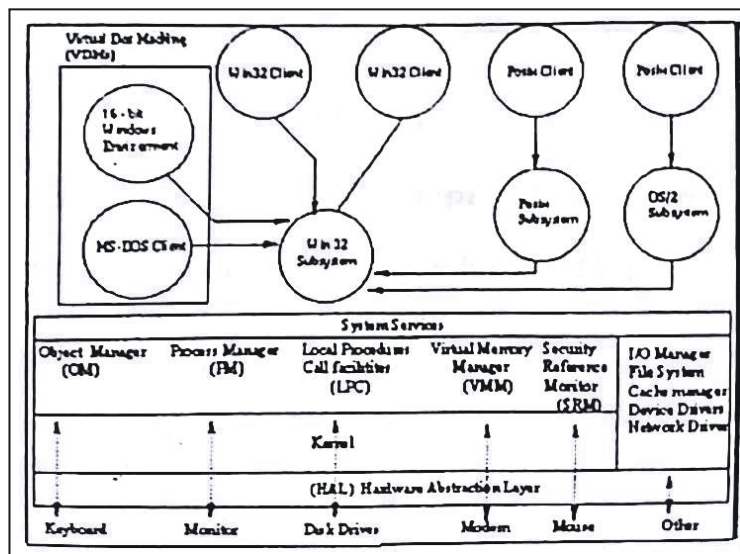


Fig. 9.K.

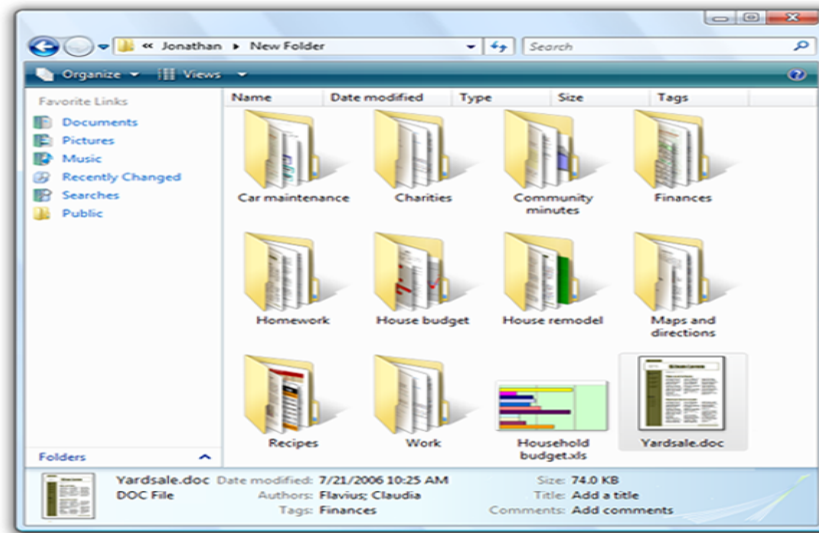


Fig. 9.L

(a) Program Manager

Under WIS-Windows, the Program Manager starts executing along with MS-Windows. This provides user interface to start and stop applications. It is an organizational tool, which is used to organize various applications into different groups. This can also indicate how each group contents are controlled and displayed on the screen. The Program Manager is also used to end the MS-Windows session.

(b) File Manager

This is as tool to help organize user files and directories. The File manager can also be used to traverse through the file system and change drives, and also to search, copy, move, create or delete files and directories. Applications can be started directly from the File Manager by clicking on an application icon.

(c) Control Panel

The control panel can be used to choose or change the color schemes in the applications, select and display the background of the screen (called the Desktop Wallpaper), select border width and other border characteristics, cursor size and shape, etc. Fonts are also managed by the control panel. We can add or remove fonts and view fonts on the screen. The control panel can also be used to configure printers and other ports on the PC though the control panel under

the X-Windows is not used for the same. In X-windows parlance control panel is known as the Resource Manager.

9.5 Summary

Microsoft designed Windows to be an extensible, portable operating system—one able to take advantage of new techniques and hardware. Windows supports multiple operating environments and symmetric multiprocessing, including both 32-bit and 64-bit processors and NUMA computers. The use of kernel objects to provide basic services, along with support for client–server computing, enables Windows to support a wide variety of application environments. Windows provides virtual memory, integrated caching, and preemptive scheduling. It supports elaborate security mechanisms and includes internationalization features. Windows runs on a wide variety of computers, so users can choose and upgrade hardware to match their budgets and performance requirements without needing to alter the applications they run.

9.6 Review Questions

1. What is Encryption?
2. What is Traffic Padding?
3. What are the characteristics of a GUI?
4. What do understand by look?
5. What do understand by feel?
6. How does the GUI depend on the operating System?
7. What is meant by distribution of application, data and control?
8. What is replication data?
9. Where do you use partitioned data?
10. Discuss in detail how remote read operation take place?

LESSON - 10

REQUIREMENTS OF A WINDOWS-BASED GUI, MS WINDOWS AND WINDOWS NT

Structure

- 10.1 Introduction
- 10.2 Objectives
- 10.3 Requirements of a Windows based GUI
- 10.4 MS-Windows and Windows NT
- 10.5 Summary
- 10.6 Review Questions

10.1 Introduction

Microsoft Windows is a group of several graphical operating system families, all of which are developed, marketed, and sold by Microsoft. Each family caters to a certain sector of the computing industry. Active Windows families include Windows NT and Windows Embedded.

Windows NT is a family of operating systems produced by Microsoft, the first version of which was released on July 27, 1993. It is a processor-independent, multiprocessing and multi-user operating system. Since Windows NT manages users, the network administrator (who holds all special privileges on the system) is able to control privileges for each user connected to the system.

10.2 Objectives

After studying this Lesson, you should be able to

- * Requirements of a Windows-Based GUI
- * Inter Client communications
- * Conventions Manual (ICCCM)
- * Dynamic Data Exchange standard
- * Ms Windows
- * Windows NT

10.3 REQUIREMENTS OF A WINDOWS-BASED GUI

Some of the basic requirements of a windows-based GUI are as follows:

- * Consistency
- * Direct Manipulation
- * Flexibility
- * Explicit Destruction

(a) Consistency

All application within one windowing environment must be consistent. This implies that similar controls operate similarly and are used similarly. The visual appearance of controls and their components should be consistent. For example, many applications may prompt the user for a 'yes' or 'no' answer. The placement of yes and no buttons should be consistent across all applications. A different placement can surprise the user and lead to too many errors.

An action always needs some objects to act upon. Selection of action and objects are necessary to complete any action. For example, for delete action, the objects can be the files to be deleted. The selection and action sequence for objects should be consistent. It should not be the case that in one application the selection of objects is made before the selection of the action, whereas in another application, the selection of the action precedes the selection of the objects.

An application should present the menus in an orderly manner. Commonly used actions and objects should be presented first and in some logical order. This order can be alphabetical, if none other can be found in the objects. Actions and objects which cannot be carried out should not be presented to the user or they should be de emphasized.

(b) Direct Manipulation

Direct manipulation allows the user to control his actions better by prompting him to select each command. With direct manipulation, users get a feedback on their actions. Visual responses are provided for each action that is carried out. This also simulates a real world environment on

the workstation. When users can manipulate objects and actions and are in control, they should be able to change their mind and start or stop actions at any time. This reduces the need of users to remember or find information from a reference document.

(c) Flexibility

The interface should be flexible in many ways. Users should be allowed to configure the settings and change configurations to their liking. For example, a right-handed person may want to sue certain mouse buttons for a specific task and a left-handed person may want to use different buttons for the same task. Such changes should be easily possible. Making small changes to colors and borders allows the person to play around with the system and encourages him to learn more about it.

At another level, flexibility would also demand that the application provides multiple ways of performing the same functions. Experienced users may want to use function keys or keyboard accelerator keys to carry out their work. Some may prefer typing file names rather than hunting for them by means of a mouse. As the user gets more experience on an application, he may want to use all available facilities which allow him to work faster. The system should provide the flexibility to the user to allow him to change his working style with experience.

(d) Explicit Destruction

When an action is irreversible and has negative consequences, the user should be able to explicitly confirm it before being carried out. Such confirmation will be appropriate while deleting files or removing blocks of data from documents. When a deleted file cannot be salvaged or the removed block of data cannot be recovered immediately (as the buffer to hold this data is too small), the user should be warned of this consequence and asked explicitly to confirm his intended action. This would minimize user apprehensions and give him confidence to use the application to carry out his work.

Windows allow the user to execute more than one application at a time on the workstation. Frequently, one may need to transfer or migrate data from one application to another. In some cases, this data can be entire files. In many cases, it may be a block of data from one application to another. A good example can be transferring some attention to such details. In the MS-Windows environment, to allow such data migration, application should be designed as per the “Dynamic Data Exchange standard”. In the X-Windows environment, the applications should follow the

guidelines given in the Inter Client communications Conventions Manual (ICCCM). One of the important things to remember will be that applications are no longer to be designed as standalone entities created for a Specific task. Applications should be designed so as to communicate with other applications and benefit from them.

The question arises as to how do windows help the user? When applications are consistent, they represent no surprise to the user. The user is able to learn and use a GUI based application much faster than a character based one. Since the controls and user interfaces across all the applications are similar, the user feels at home with any application, reducing the chances of committing mistakes. The biggest advantage is the ability to use more than one application at a time.

Today, microprocessors which are used in workstations are very powerful. They are capable of doing more than one job at a time without the user feeling any degradation in response. The user also wants the computer to do more than one job at a time. A typical user will want to do word processing, have a scheduler, run an E-mail facility, work on a spreadsheet and database, all at the same time. While doing these tasks, the user may also want to play some games. The reason for the user wanting to have all these applications running at the same time is partly that it normally takes too long for an application to start. While still in the middle of one application, the user may want to use another one to answer some other query. Yet another reason could be the real need to migrate data from one application to another. Very often, data is taken from a spreadsheet or database to a wordprocessor or a presentation program. Before the arrival of MS Windows, this was being done in a clumsy manner. The typical procedure to migrate data from a spreadsheet to a wordprocessor without Windows was as outlined below:

- (i) Start the spreadsheet program.
- (ii) Open the required worksheet and select the required data.
- (iii) Convert the selected data to an ASCII text format and save it in a file.
- (iv) Exit from the spreadsheet program.
- (v) Start the wordprocessor program.
- (vi) Open the files where the data from the spreadsheet is to be copied.
- (vii) Read the ASCII text file created from the spreadsheet program, and manipulate it (eg. copy it fully/ partially into other files opened in step (vi)).

This procedure takes a long time. If the data is to be sourced from multiple applications to one, this can be a very tedious and protracted procedure. This is normally the case when creating presentations and summary reports in an MIS environment. With the advent of windowing interface, these operations are made simpler. Now the procedure for transferring data from a spreadsheet and a database to a wordprocessor will be as follows:

- (i) Open three windows for the three applications; the wordprocessor, the spreadsheet and the database, the start each application in a separate window,
- (ii) Go to the window with the spreadsheet, open the worksheet file and select the data from the spreadsheet. This is normally achieved by keeping the mouse button pressed at the beginning of a block of data to be selected and dragging it across the block and releasing it at the end of the block to be selected. This convention is that the application highlights the selected data area with a yellow color as it marked by a highlighting pen on a real document.
- (iii) Select the window with the wordprocessor. Open the document where the selected data is to be copied and take the cursor to this location.
- (iv) Use the appropriate mouse interface (usually clicking on the center mouse button) to copy (paste) the selected data from the spreadsheet. The highlighted data from the window with the spreadsheet appears in the wordprocessor window. The data from the database can be selected (cut) and pasted in a similar way.

The entire procedure is very easy and natural. The old method needed some technical knowledge and expertise. The new one is more “user-friendly”

10.4 MS-WINDOWS AND WINDOWS NT

In the IBM PC world, since MS-DOS is a single tasking operation system, there are plenty of limitation. MS Windows supports all the above mentioned GUI features, but due to the handicap of MS-DOS, they work very slowly. Under MS-DOS, Windows or no windows, only one application at a time can be executed on the processor. MS-Windows allows one to start many applications. However, only the application in the window which has the focus is actually getting executed. All others are idle.

When the window focus changes, the old application is stopped and the new one is restarted. The time to switch from one application to another is much shorter than to start an application from the beginning. When switching between applications, we resume at the point where we left earlier. All the files opened earlier are again available. To handle such facilities, applications should handover control to MS-Windows. MS-Windows does cooperative multitasking unlike pre-emptive multitasking in a more powerful operating system. This needs a lot of memory.

One of the major shortcomings of MS-Windows in the MS-DOS environment is that many applications are not written as per the requirements of MS-Windows. To be able to work well with MS-Windows, MS-DOS calls should not be used by the application. Memory and video should never be handled directly by the application.

Applications should be developed using the API functions provided by MS-Windows. Otherwise, programs will misbehave and cause a lot of inconvenience to the user.

To overcome these difficulties and also other limitations of MS-DOS, Microsoft came up with Windows NT. Unlike MS-Windows, Windows NT is a full-fledged operating system. It does not need MS-DOS to run underneath. Also, Windows NT is a portable operating system like UNIX and will be available on other hardware platforms. This will ensure consistency of applications across a variety of platforms. The hardware currently supported by Windows NT includes Intel's x86 family, Silicon Graphics' MIPS RS4000 RISC and Digital Equipment Corp's Alpha processor.

Windows NT is a multiuser and multitasking operating system. It has all the features of a modern operating system. These include virtual memory management, threads, preemptive multitasking, symmetric multiprocessing, and built-in networking, in a modular and portable design. A major advantage for PC family users will be a truly 32 bit architecture unlike MS-DOS. Windows NT has a small executive as in modern microkernel based operating systems. On top of the executive are six subsystems and the applications. Diagrammatic representation of organisational structure of Windows NT is shown in Fig. 10.A

These features are similar to those available in an operating system like Mach. A feature MS-DOS users will cherish is built-in security. Even though Windows NT is expected to be used mainly on single-user workstations, a great emphasis is laid on security. Let us study the features in more detail.

As we discussed earlier; one of the major limitations of MS-DOS has been its single tasking feature. Windows NT has not only removed this handicap, but has also improved on this feature. Windows NT is a multiuser, multitasking and multithreading operating system. To a MS-Windows' user, this feature will be most visible in the truly fast response he can get in his applications, when multiple applications are running.

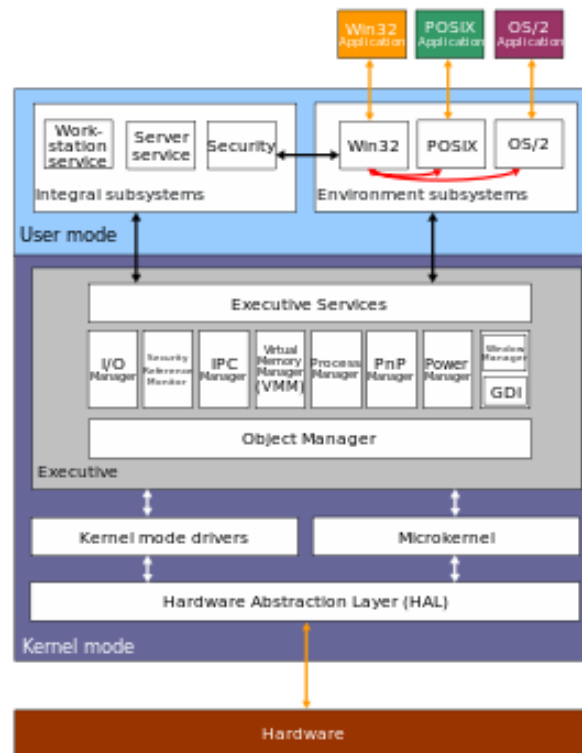


Fig. 10.A Architecture of Windows NT

Windows NT's multitasking is preemptive like UNIX. Tasks are scheduled by the operating system, giving a time slice to each task and when its time slice is over, the next one is started. The previously running task is resumed when its turn comes again. Applications cannot avoid getting preempted and thus, they cannot tie up the CPU for a very long time. This is not the case in MS-Windows.

Multithreading is another new feature which differentiates Windows NT from MS-Windows. A task can be subdivided into smaller units called threads. Each thread has an independent flow of control within a task. Different threads can be scheduled and executed independently.

Symmetric multiprocessing will allow Windows NT to schedule various tasks on any CPU in a multiprocessor system. Currently, workstations are mostly single processor systems. However, there are many applications which will benefit from multiprocessor systems. Some examples are file servers, database servers etc. on a computer network. To be able to process queries from many users for large databases and files, the throughput of a single processor will be a bottleneck. Hardware technology is available to improve performance by using multiprocessors in a single system.

The operating system technology is not widely and economically available to harness this computing power. UNIX and Windows NT are moving in the direction to fill the gap and provide the technology. Symmetric multiprocessing allows tasks to be scheduled on any available processor. When tasks are broken down into independent threads, a larger amount of parallelism can be achieved. This can be used by symmetric multiprocessing to achieve a more efficient use of all processors in a system.

One of the greatest limitations of MS-DOS has been in the area of data handling. MS-DOS has been using 32-bit processors for the last few years. However, due to backward compatibility requirements with the original 8088 processor used in the PC and for MS-DOS designed for the same, MS-Windows uses only 16 bits for data addressing. This limits the access of data to 64 K bytes (which is 2^{16}). MS-DOS has been getting around this problem using very clumsy solutions. These have also been causing a degradation in its performance. Windows NT is built as a 32-bit operating system from the beginning and will overcome this limitation of MS-DOS.

When an operating system is capable of multitasking and it supports multiple users, a virtual memory management system is very useful. It is very costly to provide physical memory in the computer which spans the full address spaces of all the processes executing at a given time. There is also a need for protecting applications from accessing memory area outside their own, thus causing security violations. Under MS-DOS, this was not required. Under MS-Windows, applications have been using some clumsy tricks to get around this problem. Windows NT provides a full featured virtual memory subsystem.

Workstations are used more and more for solving problems in a workgroup. There is a heavy demand for interconnecting workstations on a network. Windows NT has these services built in. MS-DOS machines had to install NetWare for its networking needs. Windows NT supports file transfer, E-mail services and resource sharing on a network. Windows NT can interact with

all existing networking systems like Novell's NetWare, Sun Microsystem's NFS, etc. Other communication facilities include support for sockets, named pipes, messages and remote procedure calls.

Windows NT does not use the File Allocation Table (FAT) file system of MS-DOS. It has a new high performance file system called New Technology File System (NTFS). In NTFS, file names can be upto 256 characters long. NTFS also implements fault tolerance, security and has support for very large files. Disk access is speeded up because of NTFS. However, to keep backward compatibility with MS-DOS, Windows NT can support the FAT file system in a separate partition. Floppy drives will continue to have the FAT file system.

Windows NT has a modular architecture, divided into different subsystems and layers. At the lowest layer is the Hardware Abstraction Layer (HAL). This layer provides all the hardware specific functions. This is the layer which will be different on different hardware platforms. This layer handles interrupts, Direct Memory Access (DMA), memory management at the lowest level and multiprocessor synchronization. On top of the HAL layer is the Windows NT kernel. The kernel provides the basic operating system services like scheduling of threads modules, which together form the executive. These have a well defined interface and one module can be replaced by another to handle the same functions when upgrading the operating systems and when a different functionality is needed, without affecting the other modules. The six subsystems and their functions are listed below:

- (i) The Object Manager creates, manages and deletes objects which are processes, threads, named pipes, files, etc.
- (ii) The Process Manager creates, terminates, suspends and resumes processes, tasks and threads.
- (iii) The Virtual Memory Manager manage the memory for processes. It allocates and frees memory. This subsystem also handles paging and memory protection from other processes. Each process can access upto 2 GB of virtual memory.
- (iv) The Security Reference Monitor enforces security policy and keeps track of file access rights, based on the ownership and permissions for the user. This subsystem can also be used to create alarms for any security violations and also to keep an audit trail.
- (v) The Local Procedure Call Facility is used to pass messages between client systems and subsystems on one system.

- (vi) The I/O subsystem handles the device drivers and passes data to and receives data from the devices of all the subsystems. The file system is also handled by the I/O subsystem.

10.5 Summary

In this lesson, we have studied about the requirements for a windows based GUI and the difference between the MS Windows and Windows NT. Windows NT (1993-1996) is a version of the Windows operating system. Windows NT (New Technology) is a 32-bit operating system that supports preemptive multitasking. There are actually two versions of Windows NT: Windows NT Server, designed to act as a server in networks, and Windows NT Workstation for stand-alone or client workstations.

10.6 Review Questions

1. What is Consistency?
2. What is meant by Direct Manipulation?
3. What is Flexibility?
4. What is meant by Explicit Destruction?
5. What is meant by Preemptive Multitasking?
6. What is Cooperative Multitasking?
7. What is Symmetric Multiprocessing?
8. Define Object Manager
9. What is Local Procedure?
10. Define Security Reference Monitor
11. What is meant by I/O Subsystem?
12. What is Process Manager?

LESSON - 11

UNIX HISTORY & OVERVIEW

Structure

- 11.1 Introduction
- 11.2 Objectives
- 11.3 The History of Unix
- 11.4 Overview of Unix
- 11.5 Unix File System
- 11.6 Summary
- 11.7 Review Questions

11.1 Introduction

Unix is an Operating System which is truly the base of all Operating Systems like Linux, Ubuntu, Solaris, POSIX etc. Unix and Unix-like operating systems are a family of computer operating systems that are derived from the original Unix System from Bell Labs. It was developed in the 1970s by Ken Thompson, Dennis Ritchie, and others in the AT&T Laboratories. It was originally meant for programmers developing software rather than non-programmers.

Unix is the most powerful and popular multi-user and multi-tasking Operating System. Unix programs are designed around some core philosophies that include requirements like single purpose, interoperable, and working with a standardized text interface. Unix systems are built around a core kernel that manages the system and the other processes. Main focus that was brought by the developers in this operating system was the Kernel. Kernel subsystems may include process management, file management, memory management, network management and others. Unix was considered to be the heart of the operating System.

11.2 Objectives

After studying this Lesson, you should be able to

- * The History of Unix
- * Overview of Unix

- * Unix File System
- * Different Types of Files
- * Special Files
- * The Internals of File System

11.3 The History of UNIX

UNIX is one of the most talked about products in the computing world today. One Bell executive has called UNIX as Bell's second most important invention after the transistor. UNIX dominates all the universities and has started dominating the commercial and industrial world in a big way.

The originators of UNIX Dennis Ritchie and Kenneth Thomson would never have predicted or could never have dreamt of it when they started working on UNIX. Their aim was really very simple. They wanted a computing environment for themselves which was pleasant to work with.

It was this 'selfish' interest that led to the development of UNIX and created a major upheaval in the computing world. There was no financial motivation behind this at all. They did not write UNIX with the intention of marketing it and nor did Bell Labs want sell it for a long time.

Denis Ritchie joined Bell Labs in 1968. Ken Thomson had already been working there since 1966. Both were attached to the Computer Science Research Department. Their mandate was vague though ambitious: To investigate interesting problems in computer science, Ritchie had majored in Physics at Harvard and graduated in 1963. He had then pursued his doctorate in applied mathematics which he never completed. He was introduced to computers but was interested more in theoretical problems than in practical applications. He got involved in MULTICS and then worked part time for MAC, the MIT computer time sharing project before joining Bell labs.

Ken Thomson joined the University of California, Berkeley in 1960 and majored in electrical engineering and graduated in 1965. He received his masters degree in electrical engineering from the same school a year later. He liked playing chess. In 1970s, he was to create a chess-

playing computer program that became American champion three times!

When Ritchie and Thomson came together at Bell Labs, only a culture of “mainframe” computing existed there. Microcomputers had not come into existence. Only mini-computers such as PDP-7 or PDP-11 existed, but these were not very popular. They wanted a machine for themselves to work on. After a long search, they found one PDP-7 which nobody was using. Ideally, they would have liked a bigger PDP 11, but it could not be sanctioned. An additional problem on the PDP-7 was that, there was no software (Operating system, assemblers, compilers etc.) on it.

Thomson had written a cross-assembler program running on a large GE machine. Whenever they wanted to run a program on PDP-7, they had to write a program for the large GE machine first and then use the cross assembler on that machine to produce the machine program for PDP-7. The machine program for PDP-7 was punched on a paper tape by the GE machine. They had then to carry that paper tape to the PDP-7 machine, load it and then run the program. This whole procedure, was extremely tedious. Ritchie used to call it ‘horrible’.

Ritchie and Thomson decided to ‘write an operating system for PDP-7. But that could not be ‘done without the approval from Bell Labs. All said and done, it was not a trivial project. If they had approached Bell Labs with that proposal, they would have been shown the door! Bell Labs had burnt their fingers badly Very recently in the MULTICS project, MULTICS (MULTIplexed Information and Computer Service) was an operating system developed by General Electric, MIT and Bell Labs as a combined project. This was a multiuser, time sharing Operating system running on Honeywell mainframes. It was a very ambitious project and hence, the Operating system became massive, bulky and complex.

As MULTICS was mainly written in PL/1 and the assembly language, it became very difficult to maintain, leave alone port it to other machines. It also took far longer to develop. It was a good crucible, however, where all ideas about any modern operating system were tested and tried out. Thus, MULTICS was a good research project, but it could not have been a commercially successful one. Bell Labs had pulled out of the MULTICS project as it proved to be very expensive to continue. Ritchie and Thomson were upset over this decision, even though time shelling offered MULTICS was very inefficient. The two men felt that due to this decision, the computing world would go a step backwards towards batch processing of some kind.

They could not have dared to propose the development of a new operating system on this

background. At this time, Thompson and his colleagues learnt that the patent department of Bell Labs was looking for a word processing system. Thompson decided to take this opportunity. He immediately prepared a proposal to produce an office automation system for the patent department, though Thompson called it 'Editing system of office tasks' in his proposal. The proposal met with negative reaction at first, but was eventually approved in May 1970.

Thomson always wanted a PDP-11 instead of a PDP-7 for the development. The approval for his editing system was really a God-sent opportunity as they got a PDP-11 machine along with a PDP-10 memory management unit. This was the real starting point for UNIX.

In order to do the editing, one needed an editor software. But the editor would need a lot of support from the operating system in terms of file management and terminal management. Whatever a user typed in had to be displayed on the terminal with all the features such as backspace, tab, scroll, etc. The same text had to be stored in files. The files had to be stored on a disk in some form of directory structure. When a user wanted to edit an existing file, the file had to be retrieved in the buffers and the desired chunk had to be displayed on the terminal. Hence, the editor involved a support of all these I/O functions. An operating system, therefore, was a must for this editor to be successful. This really provided the motivation for the development of UNIX (or at least Thomson grabbed this opportunity and used it to his advantage!).

Initially, UNIX was written in assembly language like all other operating systems of the time. Portability was not thought of then. The actual creation of the UNIX system was Thomson's idea. Ritchie made valuable contributions to it though. For instance, in the UNIX system, devices are treated as files in the file system. This idea was Ritchie's. Both X03940 and MULTICS had influenced Thomson and Ritchie a lot in terms of UNIX design. Another researcher at Bell Labs, Brian Kernighan used to call this new operating system as UNICS (i.e. Uniplexed Information and Computing Service). Because UNIX was regarded as a 'castrated' version of MULTICS, some people at the Bell Labs, used to call it, disparagingly, as 'Eunuchs' as well. Despite this, the name UNICS continued for a while. In the end, the spelling was changed to UNIX.

The end of 1971 saw the first version of UNIX, and finally three typists from the patent department were called in to "system test" the product. Ritchie and Thomson worked hard during those days to avoid any errors. Those were the days of anxiety, tension and uncertainty. As THE DAY approached, the tension mounted. Luckily, the experiment was successful. The patent department immediately adopted UNIX as the standard operating system. As the need for computing as well as the popularity of UNIX both increased through the word of mouth, it

spread gradually within the Bell Labs first and then slowly outside. Even then, until 1973, very few people outside Bell Labs had heard about UNIX.

It was only in October 1973 that UNIX was first publicly described at a symposium on operating systems principles held ironically at IBM in Yorktown Heights!

Despite these developments, Bell Labs had no particular interest in UNIX as a commercial product. They only set up a development group to support projects inside Bell. Several research versions were licensed for use outside.

The main problem in the growth of popularity was its lack of portability. UNIX was written in assembly language. In those days, very fast and efficient computers had not come into existence and writing an operating system in a higher level language was considered to be! very impractical by the computing community. Despite this, simplicity rather than speed or sophistication, was always the most important objective of UNIX design. Wanting to rewrite UNIX in a high level language, (HLL), Ritchie himself designed a language for this purpose. He called it "B". B was a simplified version of another language called BCPL, which was very similar to PL/1 but which: never was successful. "B" did not have good control structure. Ritchie designed another language for this purpose which he called C. He also wrote a very good compiler for it.

After this, UNIX was rewritten in C by both Thompson and Ritchie together. This happened in 1977. C combined features of both high and low level languages. It could, therefore, provide a good compromise between the execution speed and development time. UNIX now consisted of 10000 lines of code, of which only 1000 liners were retained in assembly language for the purpose of speed because they controlled the hardware and you would have to rewrite only 10% of assembly code for the new machine!

The portability of UNIX was a tremendous boost to its popularity. UNIX was ported to many hardware platforms from micro to supercomputers quite rapidly. Bell Labs also gave UNIX source, code almost free to many universities. Of course, this was because AT&T was not allowed to enter the computer business due to monopoly regulations. It, Coincidentally, most universities at that time had PDP-11s, on which UNIX worked! The students almost got addicted to it. They liked to play around with it because they also had an access to UNIX source code. In years to come, the same students would be selecting a computer and an operating system for their organizations, where they would be holding key managerial positions.

UNIX generated a lot of interest in the academic world. Many papers were produced and many seminars were held on UNIX. Many articles were published discussing a number of bugs in UNIX. Interestingly, they also gave the fixes for them. Due to all this, most of the major bugs were removed and the product was allowed to mature before it came into the market. This mature version was described in the sixth edition of UNIX programmer's manual. This is why, it was called Version 6. Version 7 replaced Version 6 within a few years.

In order to port UNIX as an operating system, three things have to be carried out.

Write or obtain a C compiler on the target machine and port 90% of the code written in C. This is a very easy task.

Write device drivers (again in C, or if required, in assembly language of the target machine) for the devices that the new machine wants to support.

Rewrite a small amount of machine dependent code such as for memory management routines or interrupt handlers in assembly language of the target machine.

Initially, writing a C compiler for every machine used to be a cumbersome exercise, because the architectures and hence, the machine languages on different machines would be very different. This hurdle was overcome by a portable C Compiler designed by Steve Johnson of Bell Labs. This portable C compiler was retargetable to produce machine code on any machine, given its architecture, with a very little effort.

1984 saw the breaking up of the AT&T, when it was allowed to set up a computer subsidiary and sell UNIX commercially. AT&T called its first product, system III. It had a lot of problems. System IV never came up. AT&T announced directly System V with different releases: 2, 3 and 4. System V, Release 4 is called SVR4. It is a far more comprehensive and complicated product than what UNIX was originally thought of as.

University of California at Berkeley was one of the universities which had the complete source code of UNIX. This is the reason they could modify the code easily. Aided by grants from a defense organization DARPA, Berkeley produced a number of UNIX versions, called 1BSD, 2BSD, 3BSD AND 4BSD. 4BSD ran on the VAX machine. The improvements in this UNIX were called 4.1 BSD, 4.2BSD, 4.3BSD and 4.4BSD, the improved version introduced paging and virtual memory in UNIX. Networking also became an integral part of BSD 4.X. TCP/IP became a de facto standard protocol supported by UNIX. A number of utilities such as the editor (vi), C

shell (CSH), etc. were also added to the Berkeley UNIX. All these improvements made many vendors like SUN, DEC, etc. base their UNIX on Berkeley UNIX.

Around the late 1980s, two major camps in UNIX arose. One, which adopted AT&T's SVR3 (and the subsequent versions of) UNIX and the second which adopted BSD 4.3 (and subsequent versions of) UNIX. They differed in many ways. The utilities available on both varied. In fact, even the file formats also differed. Hence, the file formats of the executable/binary files in these two were different. This is the reason why the object code of any application program could not be ported from one UNIX, to the other. The source code had to be ported and recompiled. AT&T issued a document called SVID (System V Interface Definition) which defined the system calls, file formats, and so on. The document was meant to bring some harmony within the AT&T camp, but it had no impact on the BSD camp. Finally, IEEE took the lead to bring the two camps together and define a standard for UNIX called 'POSIX'. 'POS' stands for Portable Operating Systems, and the 'IX' is borrowed from UNIX.

After a lot of deliberations, the POSIX committee within IEEE came up with the POSIX standards called 1003.0 to 1003.11. Out of these, 1003.0 gives general guidelines, 1003.2 refers to library functions, or system calls, and 1003.1 refers to shell and utilities. For instance, 1003.1 lists all the system calls which are common between the two camps. Therefore, if a software developer uses only system calls which are from 1003.1, the code could be called 'POSIX Compliant' which essentially means that it will be truly portable.

As soon as this gap between these two groups was bridged by IEEE, another split occurred. IBM, DEC, HP and some others formed a consortium called Open Software Foundation (OSF). The idea was to come out with a version of UNIX which conformed to IEEE, and other standards, but which also had a lot of additional features such as X-Windows, a Graphical User Interface called MOTIF based on X-11 and other pieces such as Distributed Computing Environment (DOE) and Distributed Management Environment (DCE) and Distributed Management Environment (DME).

AT&T reacted by forming its own group called UNIX International (UI) to define "their own" UNIX. This UNIX was based on System V of AT & T. As a result, again two camps were generated with more differences. This resulted in a lot of confusion.

Today UNIX has become mature since there are no major bugs left in it. This is because it was allowed to mature. It was not rushed to the marketplace with bugs. This was its strong

point. However, with different camps within UNIX (and also other different subcamps of different vendors), the software developed under any UNIX is far from being portable.

11.4 Overview of UNIX

UNIX consists of a kernel and a number of utility programs: The utility programs act as intermediaries between a user and the UNIX kernel. The utility programs typically written in C are extremely easy to add or change and, therefore, are very useful to “customize” UNIX to the specific needs. The kernel, therefore, is very small and it always resides in the main memory. It consists of about 10000 lines of C code and about 1000 lines of assembly code. The size of the kernel makes it easy to understand, debug or enhance it. Because it is mainly written in C, and almost all vendors provide C compilers, the kernel are easily portable to other machines. Only the 1000 lines of assembly code which basically control the hardware will need to be rewritten at the time of porting.

Operating systems written in assembly languages are hardware dependent. Also, in the other operating systems, quite a few functions are incorporated in the kernel instead of a simple, different utilities. This makes the kernel of these operating systems bulky and difficult to port. Figure 11.A depicts the UNIX environment. As the figure indicates, shell (sh) or the editor (vi) and other utility programs. As the figure indicates, the application program talks to the kernel only through these utilities. It is the kernel, which finally manages and talks to the hardware.

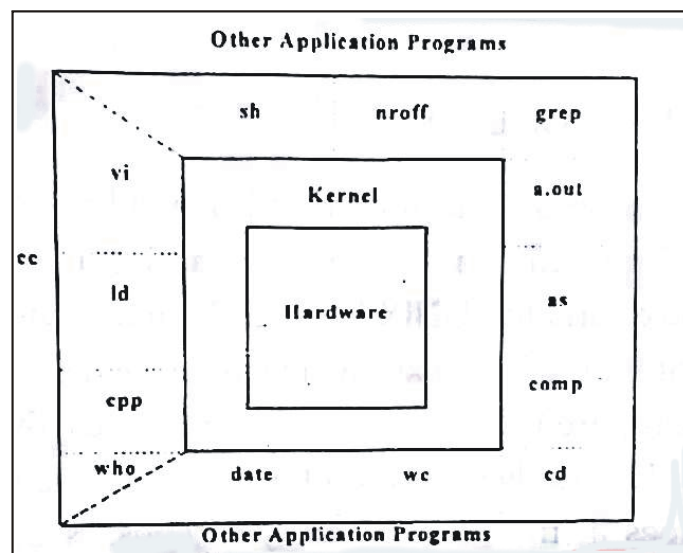


Fig. 11.A- The UNIX environment

Apart from UNIX as an operating system itself which is easily portable, the applications developed around a UNIX machine are also relatively easily portable to another machine (or a “box”) running UNIX. What do we mean by porting an application? An application typically consists of three different components: compiled program, utilities, and the sequence in which these compiled programs or utilities are to be executed. For instance, a payroll application consists of a number of compiled application programs such as payslip printing and tax computations, a number of utilities such as SORT and a Job Step Sequence expressed as Command Language (CL) such as JCL on IBM machines or Shell scripts under UNIX

Porting an application from one UNIX box to another implies porting all these three components. If the architecture of the target machine is the same as that of the source machine, even the object code could be portable. If it is different, the source programs will need to be recompiled. The compiler on the new machine, but it can still use the same system calls while generating the machine code. This is true for both the application programs as well as the utilities.

Shell scripts are like macros in the assembly language. They have been discussed in the earlier chapter. You can define a number of steps that you wish the operating system to follow one after another. Each step could mean the execution of an application program or a utility program. This execution could be based on satisfied too! Thus, because the Shell commands including scripts and the system calls are same on both the UNIX machines (this was our assumption for their being called ‘UNIX’ in first place!), applications such as payroll will be easily portable. If at all, one will have to be careful about the DBMS interface instructions in the programming languages. If the databases used are also the same, then there will be no problem.

All this appears fine. But in reality, there are a number of differences in different versions of UNIX. Unfortunately, these differences are not restricted to the internals or implementation details of UNIX, but they pertain to the system calls and their interfaces as well. In some case, user interface also differs slightly. This increases the porting efforts.

UNIX kernel is supposed to be divided only into two parts. Information management and process management. Memory management is very intimately linked to process management and is almost driven by it. Hence, it is assumed to be a part of process management.

A number of different types of devices can be attached to a computer running UNIX. The information management portion of UNIX kernel has drivers for different devices. As new devices

are manufactured, the device manufacturers or the vendors of UNIX operating system add drivers for these devices to support them. This process is greatly facilitated by a concept called “device independent I/O”. A device in UNIX is treated like a file. What does this statement actually mean? There is a directory in UNIX under the a root directory called “/dev” which consists of a number of files, one for each device. Each files has access rights similarly defined as for normal files.

Each device file is a very small file which contains details of the specific characteristics of that device. You could now think of a generalized device driver in UNIX to which these characteristics can be supplied as parameters to obtain a ‘tailor made’ device driver for a Specific device at a given moment. Things of course are not that simple to be so easily generalized: But you could write generalized drivers for devices which are at least similar. If such a driver is available, adding a new device is simple. You only have to create a file for that device in /dev directory. If the device characteristics differ a lot, you may have to modify the “generalised” device driver or write a new one for it, corresponding the file created in /dev directory.

For instance, the devices can be character devices like terminals, or block devices like disks. Each will require a certain specific driver program to be executed, and a specific memory buffer from/to which data can be read/ written. Hence, for any device I/O you will have to specify the device type, its characteristics and the addresses of the driver program as well as the memory buffer reserved of the device. These and other details about the device are stored in the device file for that device in the /dev directory in a predefined format or layout.

In UNIX, if you want to write to a specific device, you issue a system call to write onto the device file for that device in the /dev directory. The user issues a usual instruction as per the programming language syntax. For instance, he may want to write to a printer. Corresponding to this instruction, the compiler generates a system call to write to a device file for that printer defined in /dev directory. This system call to write to a device file is intelligent. At the time of execution, it extracts the contents of the device file which contains the device characteristics and the address of the device driver. It uses this information to invoke the required device driver and so on. When a new device is added to a system, the device driver for that device is written and a corresponding “device file” is created under /dev. This is what we mean when we say that devices are treated as files in UNIX.

You do not need all the hardware and devices available on your machine all the time. Remember, the more your need, the more will be the size of the operating system because, those many drivers will nee to be included and loaded and those many buffers will have to be

created. Thus, you normally specify to the system only what you need. This process is called “System Generation”.

The UNIX, file system is hierarchical. Its logical implementation will be discussed later. UNIX processes execute in either a user mode or kernel mode. When an interrupt occurs (e.g. due to I/O completion or a timer clock interrupt), or a process executes a system call, the execution mode changes from user to kernel. At this time, the kernel starts processing some routines, but the process running at that time will still be the same process except that it will be executing in the kernel mode.

Processes have different priorities which are “intelligently” heuristically changed, so that I/O bound processes have more priority than, the CPU bound processes with the passage of time. Processes with the same priority get their CPU time slices in a strictly round robin fashion.

If a process with higher priority wants to execute when there is no memory available, two techniques are commonly used across UNIX implementations. One is swapping, where the entire process image is swapped out (except its resident part). Another is demand paging where only some pages are thrown out and the new process is now created. In this case, both the processes continue. These days, demand paging is more popular.

The kernel maintains various data structures to manage the processes. For example, it maintains a data structure called “u-area” which is similar to the Process Control Block (PCB) discussed earlier. There is one “u-area” per process. The kernel also maintains a kernel stack for each process. Thus, for each process, there is user stack as well as kernel stack. These are used respectively in user and kernel modes of a process for storing return addresses or parameters for system calls/functions. An interesting point is that the virtual address space of a user process consists of not only the user portion of the process, but some of the kernel data structures associated with that process as well. If a process is in the kernel mode, it can access the entire address space; but if it is in the user mode, it can access only the “user portion” of the user address space. The “user portion” of a process consists of the following elements or regions.

- * Text (i.e. compiled machine instructions or code e.g. compiled procedure division of COBOL).
- * Data (e.g. defined as FD or working-storage section in the data division of COBOL)
- * Stack (for storing subroutine return addresses and parameters).

This user portion is entirely swappable if memory is to be freed for some other process. the kernel portion of the user address space has a portion (such as u-area) which is also swappable, and a portion which is non-swappable or resident. This resident portion is never swapped. For instance, the kernel keeps a process table which has one entry for each existing process in the system, regardless of its state such as running, ready or blocked (sleeping) This process table is never swapped, because it contains very fundamental information about process scheduling and swapping itself. For instance, this table is required to take decision about which processes are to be swapped in, if some memory space becomes free. There are pointers from an entry in this table to the u-area for that process, as well as the user portion of the address space (e.g. text, data and stack). Figure 11.8 depicts this scenario. The figure is horizontally divided into two portions: Resident and swappable. It is also vertically divided into two portions: Kernel address space and User address space. The process table is in the resident portion of the kernel address space. The table has one entry for every “Known” process to UNIX. The figure shows various pointers from entry in the table as discussed earlier.

Remember that these tables are described only at a logical level. A specific UNIX implementation may decide to split or combine various tables at a physical level. The text, data and stack are treated in UNIX as ‘regions’. A region is quite like a segment. For instance, most of the compilers under UNIX generate ‘reentrant’ text, is more than one user can use it with only one copy of it in the memory. The code is stable and does not modify itself during the course of its execution. In order to implement the ideas of sharing of various regions, various data structures (e.g. per process region tables) need to be shared .

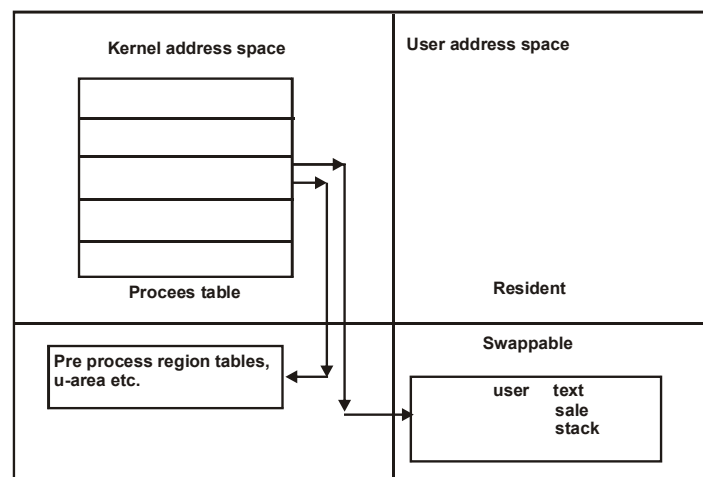


Fig 11.B- Kernel Vs user and resident Vs swappable address spaces

11.5 UNIX FILE SYSTEM

11.5.1 User's View of File System

UNIX implements hierarchical file system. In this scheme, a directory can have a number of files and subdirectories underneath it. A disk can be divided into multiple partitions. Each of the partitions has its own file system. Each file system starts with a root directory at the top the inverted tree. The root directory contains a number of directories, which in turn contain a number of files/ subdirectories, and so on. This is shown in Fig 11.C In the figure, rectangles represent directories and circles represent files.

In UNIX, a file is a stream of bytes i.e. there is no concept of a record known to Unix. The application program reading a record of fixed length of 500 bytes from a file can request the UNIX kernel to read bytes with Relative Byte Numbers (RBN) 0-499, 500-999, 1000-1499, etc. one after the other, when it wants to read the first, second, third record and so on. The kernel will do the translation of the RBNs into Logical Block Numbers (LBNs), then to Physical Block Numbers (PBNs) and then issue the necessary instructions to the disk controllers to position the Read/Write heads and then read the desired sector(s). This translation process has already been studied in detail. Thus, the interpretation of this “stream of bytes” into records, and further into fields, is completely left to the application program.

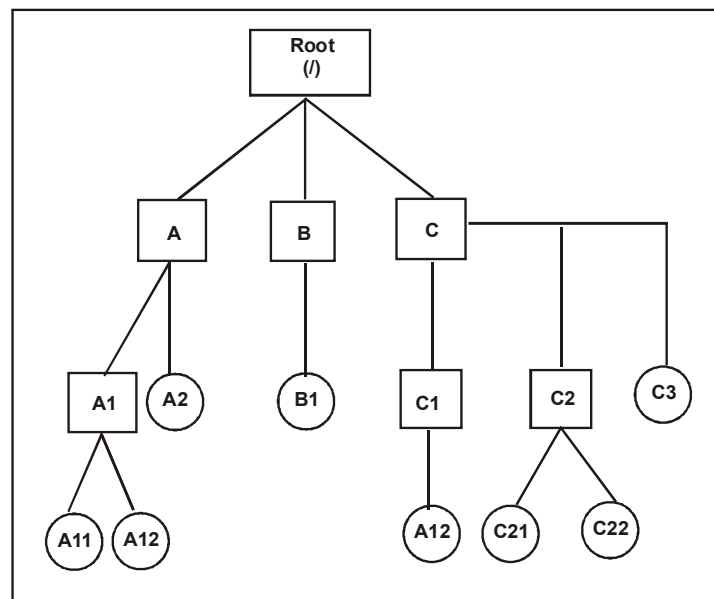


Fig. 11.C - Hierarchical file system

11.5.2 Different Types of Files

UNIX recognizes four types of files, as given below

- * Ordinary files
- * Directory files
- * Special files
- * Fifo files

(a) Ordinary Files

Ordinary files can be the regular text files or binary files. The word 'text' here is used in the way we use English text. It should not be confused with the text region of a process which consists of compiled code. Text files can contain all the source program or the documents prepared using a word processor. These text files normally contain only, 'ASCII' codes UNIX has a command called 'cat' to display the contents of a text file on the screen.

Binary files, on the other hand, contain the compiled programs and all other non-text information. Needless to say that in any byte of the binary file, any of the possible 256 combinations of 8 bits can exist. whether they have any meaning according to the "ASCII convention or otherwise. Therefore, cat command cannot be used to display the binary files (cat command picks up 8 bits at a time and interprets them according to the 'ASCII' table before displaying). UNIX provides another command called 'od' to display the contents of a compiled 'object' or binary file. 'od' is a short form of 'octal dump'. the 'od' utility picks up the binary file, converts the bits within it to printable octal characters by picking up 3 bits at a time. This utility is applied for detailed inspection/ debugging. Under the root directory, there is a file called 'UNIX', which contains the binary or object code for UNIX itself. We can use a command 'od/UNIX" to get an octal dump of UNIX itself!

(b) Directory files

In UNIX, a directory is also treated as a file. A directory can be considered as a file of files. This means that a directory is like a file with a number of records or entries. There is one entry for each file, or a subdirectory under tint directory. The entry contains the symbolic name of the

file/subdirectory underneath and a pointer to another record or data structure called “index node” or “inode” is short. The inode maintains the information about the file/directory such as its owner, access rights, various creation, usage and amendment dates and the addresses used in locating/ retrieving various blocks allocated to that file/ directory. Thus, every file, directory or a subdirectory including the root has an inode record or a fixed length, All inodes are kept together on a disk. Inodes are numbered as 0,1,2 etc. and this number itself is used as a pointer in the symbolic directory to locate the inode which contains the information about the file/directory.

Inode No.	File/Dir Name	Type	...	Access Rights	Dates (Creation, last update)	Pointers to the disk blocks (Address)
0	Root	DIR	..			
1	A	DIR	..			
2	B	DIR	..			
3	C	DIR	..			
4	A1	DIR	..			
5	A2	FILE	..			
6	B1	FILE	..			
7	C1	DIR	..			
8	C2	DIR	..			
9	C3	FILE	..			
10	A11	FILE	..			
11	A12	FILE	..			
12	C11	FILE	..			
13	C21	FILE	..			
14	C22	FILE	..			
..	..	..	..			
..	..	..	..			

* This is only shown for convenience. It actually is not stored in the inode.

Fig 11.D-Inodes (BFD) for 11.C

We can refer to Fig. 11.C again. Figure 11.D shows the inodes corresponding to all the files, directories and subdirectories in Fig. 11.D. This is similar to the Basic File Directory (BFD) studied earlier. The inode as well as the BFD entry does not contain the symbolic file name, though it has been shown in Fig. 11D for better comprehension of the correspondence. The symbolic names are actually stored in a file called Symbolic File Directory (SFD) as shown in Fig 11.E

Figure 11.C shows the directory entries for all the directories in Fig. 11.A. This is similar to the Symbolic File Directory (SFD) studies earlier. This will be clear from the "Type" filed in Fig. 11.B. For instance, inode number 1 corresponds to the directory A shown in Fig 11.A. The "Type" field in Fig. 11.B shows that it is a directory and not a file. For such a directory, the field "address" in Fig. 11.B. is the address of the disk blocks for the file containing the SFD. However, if the inode is for a file instead of a directory, this "address" field is the address of the disk blocks of the file itself.

File/ Subdirectory	Inode no.
"	0
.	0
A	1
B	2
C	3

Directory-Root
(inode=0)
(a)

File/ Subdirectory	Inode no.
"	0
.	1
A1	4
A2	5

Directory-A
(inode=1)
(b)

File/ Subdirectory	Inode no.
"	0
.	2
B1	6

Directory-B
(inode=2)
(c)

File/ Subdirectory	Inode no.
"	0
.	3
C1	7
C2	8
C3	9

Directory-C
(inode=3)
(d)

File/ Subdirectory	Inode no.
"	1
.	4
A11	10
A12	11

Directory-A1
(inode=4)
(e)

File/ Subdirectory	Inode no.
"	3
.	7
C11	12

Directory-C1
(inode=7)
(f)

File/ Subdirectory	Inode No.
:	3
.	8
C21	13
C22	14

Directory-C2
(inode=8)
(g)

Fig 11.E -The directories (SFD)

For instance, in Fig. 11.C the directory for A with inode= 1 shows the entries for A1 and A2 with their inode numbers. If we use these numbers to trace our way to the inode entries in Fig. 1

1.C, we will realize that A1 (inode=4) is a directory and A2(indoe=5) is a file. As we know, the “Type” filed signifies this. For both of these, the disk block addresses or pointer to them are maintained in the inode itself. If we use this address for A1 from the inode with number=4 to read the blocks allocated to that directory file of A1, we will get the directory for A1 as shown in Fig. 11.E. We can now read the contents of directory A1, entry by entry (each entry is of fixed length-14 characters in some implementations) to get to any file or a subdirectory within A1. We can now access the inode number of any directory or file within A1 by doing a table search using the symbolic name of the file or directory as the key.

After this, we can repeat the procedure to read that inode, pick up the pointers or addresses in the inode to get to the actual blocks. If it is a directory within A1, we could repeat this procedure interactively to traverse anywhere within the hierarchy. (This is why we say the directories are treated as files in UNIX). If it is a file within the directory A1, the address filed takes us directly to the data blocks allocated to that file. The kernel now can use this to carry out the address translation to arrive at the sectors that are needed to be read. It then can instruct the controller to read those sectors to read the data.

Every directory has two fixed entries, one for the entry of ‘.’ and the other for ‘..’. The entry is for the same directory. The inode number against it is for the same directory. For instance, in directory for C12 in Fig.11.E, the ‘.’ entry maintains 7. This is the inode number for directory C1 as figures 11.D and 11.E show.

The ‘..’ entry is for the parent directory and its corresponding inode number. For instance, in Fig. 11.E(e) the ‘..’ entry for directory A1 shows the inode number 1. This is the inode number of directory A as Figures. 11.D and 11.E show. Figure 11.E(b) reveals that A is the parent directory of A1. The ‘.’ entry for the root directory in Fig. 11.E(a) shows 0 as the inode number because the root directory is supposed to be a parent of itself. The buck stops at the root directory!

Each user has a ‘home directory’. This directory has to be defined at the time a user is made known to the system. At this time, the user’s home directory is stored in the /etc/passwd file against they username. The program to “create a user” prompts for the home directory. Immediately after logging in, the home directory becomes the working directory or current directory. When the user logs in, this /etc/passwd file is consulted and the user is then placed in the home directory. At any moment, the directory in which the user operates is called ‘Working directory’ or ‘Current directory’. UNIX has a command to “change directory cd”. Using this, a user can move to a new directory. For instance, referring to Fig. 11.C, if A1 is defined as a home directory

for a user, a command 'cd/ C/C1' moves the user to that directory. Incidentally, /C/ C1 is called the absolute pathname. At this time, C1 becomes the user's current directory. The current directory is found using the "pwd" command.

If a file is to be specified by a user sitting at a terminal, there are two ways of specifying it. One is the relative pathname which is specified with respect to the current directory and the other is the absolute pathname which starts from the root directory. Normally, the files in the current directory are directly accessible without mentioning any pathname.

For instance, referring again to Fig. 11.C, if A1 is the current directory, and if it is desired to list the files in that directory, UNIX command "ls" is used :

The system will respond by displaying

A11

A12

Notice that no pathname was necessary in the "ls" command.

On the other hand, if the user wants to list the files in the directory C2 while the current directory is still A1, he will have to issue the command as shown below:

ls/C/C2

The system will respond as follows

C21

C22

In this case, /C/C2 is the absolute pathname. If the user wants to list the contents of the directory A, while still in the current directory A1, "ls.." (relative pathname) or "ls/A" (absolute pathname) can be used. Both will be treated as equivalent. The ".." refers to a parent directory to A1 which is A in our example as shown in Fig 11.C. Notice the usage of "..". Compare it with Fig. 11.T which shows the directory contents of A1. At the top of the table, there is an entry for "..". This is for the parent directory of A1. Figure 11.C shows that this is directory A. This will clarify the usage of ".." if you want to access the parent directory. The commands for relative pathname work in this manner.

Similarly, if from A1 as the current directory, one wants to list the contents of C2 directory, one can say “ls../C/c2” (relative pathname) or “ls/C/C2” (absolute pathname). In the relative pathname, the first “..” will take the system to the directory A and the second “..” will take it to the root directory. After this, “/C/C2” will take it to the C2 directory (in the cases of both relative and absolute pathnames),

In general, a pathname starting with “/” is an absolute pathname and all others are relative pathnames

UNIX provides another facility, viz. “Linking”. Linking allows a user to have two or more pathnames for the same file. Hence, the same file can be viewed as belonging to two or more directories. Put in yet other words, with linking, you could have only one physical file with one inode, with two or more symbolic names of it, and hence, those many entries in the (symbolic) directories pointing to the same inode.

Let us consider Fig 11.C again. If file A12 is to be accessed from A1 (as it exists now) and also from the directory C1, a link needs to be created from C1 to A12. Figure 11.U depicts this scenario. After this linking is done, there are two absolute pathnames to the shared files. They are /A/A1/A12 and /C/C1/A12. In fact, in the directory C1, the same file could be called by another name. If this name is, say F1, the absolute pathnames /A/A1/A12 and /C/C1/F1 would be identical. They would point to only one physical file which has only one inode (which in this case is inode =11 as Fig.11.D depicts).

Linking allows file sharing and obviates the need to copy the same file in the other directories which would have been wasteful. Unlinking removes the link created earlier. UNIX provides the commands as well as equivalent system calls to link and unlink.

Internally, link system call adds a symbolic file to the new directory pointing to the same inode. It also increments the “usage count” or “number of links” field in the inode of the linked file/directory. For instance, the directory for C1 after linking would look as shown in Fig. 11.D. Contrast this with Fig. 11.E to see what has been added.

Note that the same file A12 with inode number 11 is called F1 from the C1 directory. Thus, the resolution of the pathnames /A/A1/A12 and /C/C1/F1 results in the same physical file.

(c) Special Files

A third category of files is Special files. Special files are maintained by UNIX which allow the users to treat I/O devices as files. In general terms, one can say that for each I/O device such as a tape, a disk, a terminal and a printer, there is a special file maintained by UNIX in a directory “/dev”, i.e. a directory called “dev” under the root directory maintains all these special files.

If an application program issues a “write” instruction to the terminal, the system call used by UNIX is the same as for writing to a file. The only difference is that, the file in this case is a special file for that specific device. The address fields in the inodes for the files for various devices maintain pointers to the contents of these special files. The special file itself contains, amongst other things, the address of the memory buffer (character lists or clists in the case of terminals), and the address of the piece of the device driver software for the terminal. The kernel accesses the inode for the required special file, picks up this address and then executes the actual device driver. But to the user or a programmer, it appears that he is writing to a ‘file’.

(d) Filo Files

Fifo files are used by UNIX to implement pipes. It is a file which is also treated as a stream of bytes, but in a FIFO manner, i.e., the byte which is written to this file first is the one which is put out first by the FIFO file. This is the reason it is called “First In First Out (FIFO)”. Let us assume that a message consisting of a number of bytes is written to this FIFO file by a process. Let us also assume that another process reads this file. That process will receive the message exactly in the same way and sequence as it was sent. Thus, “I am hungry” and not as “Yrgnuh ma I”. This is the basic idea of a pipe.

The Fifo file differs from the other files in one important way. Once data is read from a FIFO file or a pipe, it cannot be read again, i.e. the data is transient.

11.5.3 Mounting/Unmounting File Systems

We know that a single disk can be partitioned to house multiple file systems, each starting with its own root directory. Even if a disk is not partitioned but a system has multiple disks/diskettes connected to it, multiple file systems can result. There is sometimes a need to copy a file from one file system to another. How is this then.

Let us assume that there are two disks D1 and D2 connected to a system. D1 could be a hard disk and D2 could be a floppy disk, or vice versa, or both could be hard disks or floppy disks. Let us assume that they have file systems as depicted in Fig. 11.F.

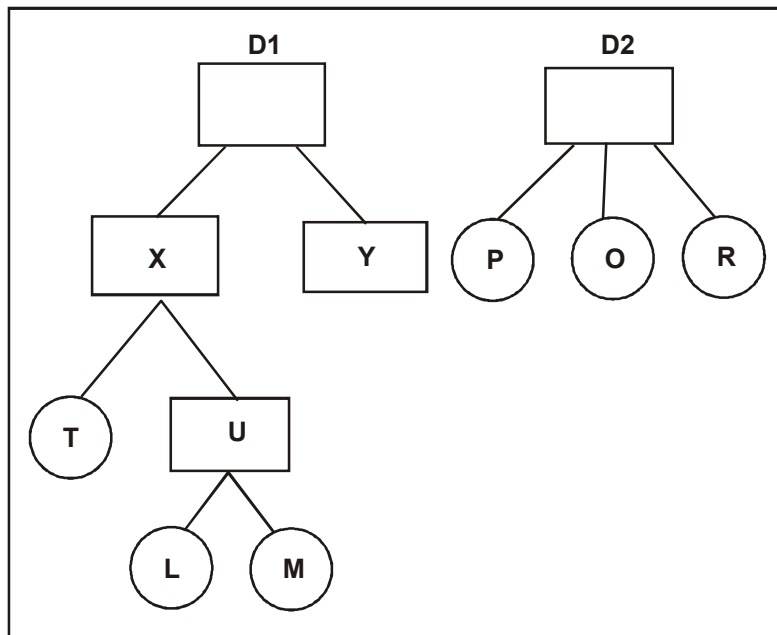


Fig. 11.F. A directory structure after mounting

Let us assume that it is necessary to copy a file from D2 to directory U under D1. MS-DOS or VMS, use a command equivalent to

`cp D2:R D1:/X/U/R` or

`cp D2:R /X/U/R` (if D1: is implicit).

UNIX adopts a different approach. UNIX allows to “uproot” the whole directory structure under D2: so to say, and “mount” or “graft” it on D1: under a specific directory say Y by using a mount command. After mounting, the directory structure looks as shown in Fig 11.G.

Now copying of the file R is a trivial job to be carried out by a simple command within a “unified” file system. Now a command.

`cp/Y/R/X/U/r` could be issued.

Similarly, a file system can be unmounted and separated into another independent file system with a separate root directory.

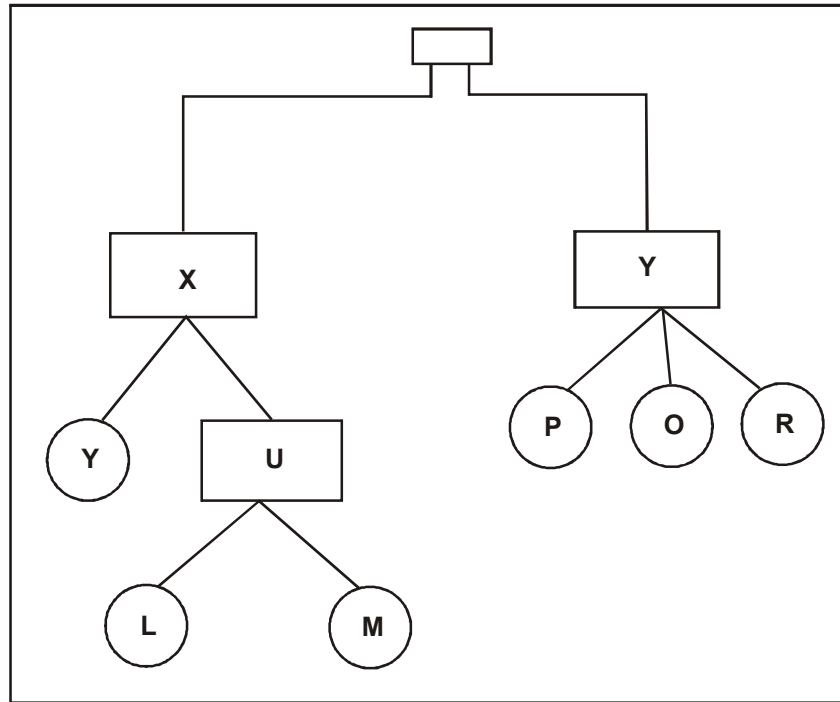


Fig. 11.G – A directory structure after mounting

UNIX gives an option to a user to mount or unmount a file system at a prespecified time (such as only for the night shift). UNIX maintains a configuration file that maintains these times and instructions. A background process (called daemon) continuously checks the stored times and mounts the file system at those specified times. It uses the “mount” or “unmount” system calls to carry out the task. Alternatively a system administration sitting at a terminal can explicitly use “mount” or “unmount” commands to achieve the same thing. These commands issue the required system calls internally, to carry out the desired task.

Mounting is usually done at the time of booting UNIX by the system administrator. If all users with all their directories were to be supported all the time in one file system, it would be a difficult task. Imagine each user or a student having his own removable disk/diskette with his own file system. In this environment by allowing to mount/unmount file system as per the needs of the students UNIX can support large number of such users without the file system size

growing beyond a certain limit. Therefore, with this facility, UNIX can now support a number of removable disks.

The mounting facility provides the system manager the flexibility to change or tune his file system as per the need. It also allows, added security. He can effectively disallow certain users from using the system at a certain time of the day.

11.5.4 Important UNIX Directories/Files

The concept of hierarchical file system in general terms has been studied earlier. Let us now look at some of the actual, important directories and files maintained by UNIX. These are supplied along with the UNIX operating system itself. We will now study their significance. Fig. 11.1 shows some of these directories/files.

- (a) `/` is a root directory for the file system
- (b) `/bin` (binaries)

This `/bin` directory contains the compiled, executable or binary version of essential UNIX utilities that are useful for various users at some time or the other. Not all the utilities are stored in the `/bin` directory. Remember that UNIX utilities can be in two forms. One is the binary (compiled) files. Execution of these utilities is very fast. The other is shell scripts. Shell scripts also are executable files but they are not in a compiled or binary form. Shell scripts are executed at run time, a command at a time. Hence, they can be termed as “interpretive”. Shell scripts are not stored in `/bin` directory. Some of the common binary files maintained under `/bin` are as shown in Fig. 11.1.

We will now study some of these in more detail.

(i) `/bin/ls`

`/bin/ls` is a utility program to list the contents of a directory. For example, if the command is given at a root directory, the command as well as the displayed results will be as shown in Fig. 11.J, which correspond to the subdirectories under the root directory shown in Fig. 11.H

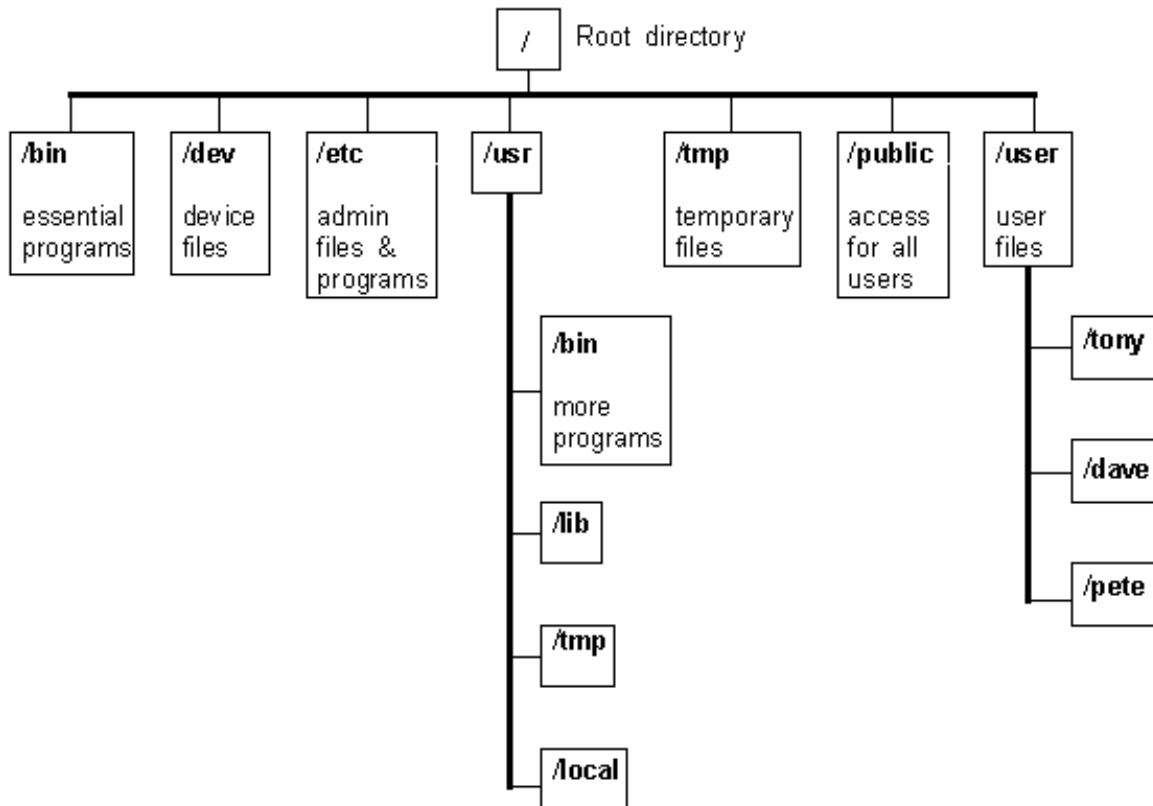


Fig. 11.H - Important UNIX files / directors

This utility is used a number of times by almost every end-user. When this command is given to the shell running on the system as the command interpreter for UNIX the shell process uses the system call to locate the binary executable file for this command. It finds it in / bin directory. The /bin directory will contain various symbolic names and their corresponding inodes. The kernel can search for the name “ls” as the symbolic file name.

Having found it, it can get the inode number, and access the corresponding inode. The inode for the file “ls” contains the address of the disk blocks where the actual executable binary code for “ls” is stored. It then creates another process (forks it), loads the binary compiled code of “ls” utility into the process address space of this newly created or forked process. This new process now takes charge and

executed the command “ls” by again issuing various system calls for searching for different directories under the root directory, and then displaying their names.

“ls” has many options, the discussion of which is beyond the scope of the current text.

(ii) / bin/who

/bin/who utility outputs information about the currently logged on users. It reports user names, the terminals being used, the dates and the times at which they have logged on to the system. A sample command and its output are shown in Fig. 11.1. UNIX maintains these details for every user in its internal data structures which are used to execute this and finally executed using these internal data structures.

```
$who
Emp1  console    Dec 15   10:10
Emp2  try1        Dec 15   11:15
Emp3  try2        Dec 15   11:30
$
```

Fig. 11.1 The “who” command

(iii) Other Utilities

/bin/mv is utility to change the name of a file. It can also be used to move a file to a new directory. The old file and its name do not exist any more in the old directory.

/bin/cp is used to copy a file into a new directory. The old file is retained intact in the old directory. A new physical file has to be created in this case. New disk

blocks have to be allocated and a new inode for this new file is then created. The target directory now shows the symbolic file name and the new inode number for this file, while retaining both in the old(source) directory.

(c) /Unix

/Unix contains the actual code for UNIX kernel in an executable binary form. When the system starts, / Unix is read from the disk into memory and then started. Actually, the file /boot is read first, which, in turn, reads /Unix.

(d) /usr

/usr is a very important directory, to which all users belong. As a default, all the user directories are direct subdirectories of /usr, but one can also organize another level of directory (shown as “users” in Fig.11.1) for the sake of user-a, user-b, user-c, etc. Each one can then have a hierarchy underneath it.

Apart from the user directories, /usr has also three main subdirectories /usr/lib, /usr/bin and /usr/ tmp. “lib” stand for libraries, “bin” stands for binary, and “tmp” stands for temporary files. An interesting point is that all these directories are under /usr as well as directly under the root directory “/”. Thus, there are two bin directories to keep binaries/bin and /usr/bin. There is little reason for both to exist. In fact, one could store all binaries in /usr/bin or in / bin. In SunOS, these two directories are linked together meaning that logically, the two are treated as only one directory by the SunOS. /usr/lib is used to store libraries in languages such as C, FORTRAN, and so on.

/usr/tmp stores short-lived files created during the executing of a program. For example. editing a file using ed creates a temporary tile /tmp/e00512. This temporary file holds a copy of the file being edited, rather than working on the original file. /tmp directory is cleaned up automatically when the system starts: There is however a problem, if a user creates a temporary file with the same name as the one in / trnp directory such as “e00512”. For this reason, utilities such as ed choose strange and unusual names such as “e00512” for the temporary files created.

(e) /lib

The /lib contains mainly parts of the C compiler such as /lib/cpp, the C preprocessor, /lib/libc.a, and C subroutine library.

(f) /etc

This directory contains various files that are important for system administration. Some of these are now discussed below.

(i) /etc/passwd

This is the file, which contains the essential information for required during login. It consists of a number of records, one of each user. Each record in turn consists of the following fields for each user:

- username: This is self-explanatory
- password: This is encrypted form of password the user most supply during login. This field can be empty, but should not be kept so for security reasons. Users can change their own password using the 'passwd' command.
- pw-age: This is an optional aging field. This is used to specify the time (age) after which the user is forced to change his or her password. After login, the system checks if the age is 'ripe' for it to force the user to change the password if so, the system prompts for the new password accepts, it and updates the password field in his record of this file.
- userid: This is a numeric user-id unique to each user
- groupid: This is the group id to which the user belongs
- idstring: This is the full name of the user.
- homedir: This is the home directory in which a user is placed automatically, immediately after logging. At this time, this is copied in his u-area. After this, the movement to any other directory or the listing from the working directory etc. can

be easily accomplished as studied earlier. When a user is created onto the system, his home directory is ascertained, and is input by the system administrator along with his/her user name, password, etc. The `/etc/passwd` then stores it.

- command: This is a command which should be executed immediately after logging in. Typically, the shell to be used such as `/bin/sh` or `/bin/sch` is specified here. After logging, the kernel searches the `/etc.passwd` file, picks this command up and invokes it. This is how most of the time, the user directly sees the shell prompt (\$) after logging in. This could however be tailored, for a specific user to run, say Purchase Order (PO) entry program immediately after logging in directly. The pathname for that program file will need to be specified there. That is all. As soon as that user logs on, the PO entry program will be invoked and started directly. This concept can be used in airlines or banking or retailing business, if specific user is supposed to use only a specific screen throughout the day. This saves a lot of time.

(ii) `/etc/motd`

`/etc/motd` is a file which contains a message that is displayed on a terminal as soon as a user logs onto the system. The process involved in the login procedure extracts this message and displays it. There is a command which can be used by the system administrator to set this message up. This is how a user can see login messages like "Hello! Happy New Year!"

(ii) `etc/fsck`

`/etc/fsck` contains a utility to check the consistency of the file system. This checking is done normally during bootstrapping. The inconsistencies can be caused by power loss or malfunctioning and files to ensure that all the linkages between different directories and files are correct and the linkages between various blocks allocated to those are also correct. It also does that for all the free blocks and bad blocks. In order to do this, it has to go through multiple passes.

(iv) /etc/mount

/etc/mount is a utility program to allow mounting of file systems as discussed earlier.

(v) /etc/unmount

/etc/unmount is a utility program to allow unmounting of file systems as discussed earlier.

(vi) etc/group

/etc/group file contains all the groups that exist in any UNIX installation. There are three types of users for any file: Owner, Group and Others. There could be different read (r), write (w) and execute (x) permissions that can be granted to these categories. For instance, the owner could have all rwx permissions, but the group could have only r and x permissions and others could have only read (r) permission. The /etc/group file serves as a master file for all the groups. If any group is mentioned or created for any file, it has to first exist in this master file of /etc/group to qualify to be a valid group.

(g) /tmp

/tmp directory maintains all the temporary files created while executing different utilities such as editors.

/dev

/dev directory contains different special device files. There is one such file for each device. This file contains the address (directly/indirectly) of the device driver and various parameters. Thus, when some data is required to be written onto a device, the normal system call to "write" on the file is used, except that the file in this case is a special device file for that device. The procedure to execute this system call internally involves the desired device driver with appropriate parameters. This is the way devices are treated as files in UNIX.

11.5.5 The Internals of File System

(a) General overview

Part 1 or the zeroeth block in the file system is reserved for the boot block. For the file system that is involved in booting, the first block contains the boot or bootstrap program. After the machine is powered on, the hardware itself is constructed in such a way that it automatically read in this boot block containing the bootstrap program. Typically, this bootstrap program has instructions to read in a longer bootstrap program, or perhaps the UNIX kernel itself. Only one file system needs to have this boot block, even if multiple file systems may exist on a disk. In all the other file systems, this block is empty.

Part 2 or the second block is called “Superblock”. It acts as a file system header. It contains the information about the file system. Any file system has to be mounted on the root directory before it can be used. When so used by a mount command, the file system’s superblock is read into the kernel’s memory. This all the superblocks of all the file systems are available for the kernel in its buffer memory when UNIX starts running. A superblock need not be exactly of one block in length. It can be and normally is more than one block. The name ‘superblock’ should be treated as a logical name. The superblock contains the fields as shown in table 11.1

Table 11.1 The superblock

The size of the file system
Number of free blocks in the file system
A partial list of free blocks
A pointer to the next free block in the free list
The size of the mode list
The number of free inodes in the system
A partial list of free inodes
A pointer to the next free inode in the free list
Lock / flag fields

We will understand the significance of the fields within the superblock in the discussion that follows.

The allocable blocks in a file system are divided into two portions as shown in Fig. 11.V. These are parts 3 is reserved for inodes. It has already been seen that inode is a fixed length record which stores the basic information about the file. It is like the Basic File Directory (BFD) entry. The inode maintains the information about the owner, various permissions, addresses to locate the data blocks allocated for that file etc., There is one and only one inode for a file. However, the reverse is not true. For instance if the file is known by two symbolic names, i.e., if there are links between them, there is still only one inode for both, even if there are two different pathnames and hence, two symbolic names for that file. As a directory is also treated as a file in UNIX, there is one inode for every directory as well, including the root directory.

A file system has some blocks reserved for the inode entries. This then becomes a limiting factor for the total number of files, directories and subdirectories in that file system. If a file system has fewer valid files/directories, there will be only a few valid inode entries in part 3 of the file system. All the other entries in the part3 of the file system will be free and allocable, if there is a need to create a new file. Newer files are being created and some older ones are being deleted all the time. Therefore, the allocation of free inode entries within part 3 of the file system is a very dynamic process. The kernel must, therefore maintain somewhere the list of free inode entries in part 3 of the file system. Thus, when a new file is created, an entry from this list can be allocated and the list then can be Updated. This list can be fairly long. The kernel normally reserves a small portion be stored entirely in the superblock itself to hold a few free inode entries. If the list is small enough, it can be stored entirely in the superblock itself. If it is larger, the remaining (mines are stored outside the superblock. We will call them 'disk inodes entries' or 'disk inodes'.

When a file is to be created, the kernel first looks for a free inode entry in the superblock itself. When all the free inode entries in the superblock are exhausted, the list of free inode entries kept outside the superblock is consulted. The superblock essentially acts like the 'cache' in this case. When a superblock does not have any free inode entry left, it is updated with the entries in the disk inodes, whereupon the disk inodes also are appropriately updated. There are algorithms in UNIX to allocate a new inode entry to a new file or tie-allocate an inode entry and add it to this free list, if a file is deleted.

For instance, a reference to Fig.11.W will show that there are 26 inodes (0 to 25) in a file system in our example. The figure shows that a number of inodes are free or unallocated. The superblock maintains a partial list of free inodes - e.g. 25,23,21,19,17,16,15 and 12. The list is maintained from the highest to the lowest number. The superblock also maintains an index or pointer to the next free inode entry in the file system which can be allocated next. This pointer is at inode entry 17 in Fig. 11.W. Whenever a user wants to create a new file, the following happens:

- (i) The kernel accesses the superblock. As we have seen earlier, a superblock starts with the second block within the file system. Hence, accessing and reading it poses no problem for the kernel. The superblock maintains a pointer to the next free inode. This is now accessed.
- (ii) The kernel allocates that inode for the new file. It will be 17 in this case.
- (iii) The kernel updates the pointer to point to the next entry -e.g. After allocating inode 17, the pointer will now point at inode 19 in Fig. 13.15
- (iv) When the pointer indicates that there is no free inode number left in the superblock, the kernel searches the disk inodes to find the inode entries that are free and updates the list of free inodes in the superblock. It also updates the value of the pointer to point at the very beginning of the list in the superblock, and then starts processing as before.

The part 4 or the second portion of the allocable blocks in a file system consists of data blocks that can be allocated to different files as shown in Fig.11.V. UNIX does not have a concept of an element or a cluster. Only one block is allocated to a file at a time, on demand. The blocks allocated to a file need not be contiguous. Hence, UNIX has to maintain an index of all the data blocks allocated to a file. There can be multiple levels of these indices and the addresses of the data blocks/indices are maintained in the inode for that file. Thus, after accessing the inode for a file, one can traverse through all the data blocks for a file. The details of this will be studied when we study the structure of inodes.

At any time, some data blocks will be allocated to some files and the remaining will be free. UNIX has a very interesting way of maintaining a list of free data blocks. A partial list of the free data blocks is maintained in the superblock itself. The superblock also maintains a pointer which gives the next free data block in the partial list of free data blocks maintained in the

superblock. The superblock obviously cannot contain the full list of tree data blocks. The remaining entries of the free data blocks list are kept separately. UNIX maintains a pointer from the superblock to other blocks containing the list of remaining tree data blocks. This is quite similar to the way the free inode entries are kept.

11.6 Summary

UNIX is a computer operating system. An operating system is the program that controls all the other parts of a computer system, both the hardware and the software. It allocates the computer's resources and schedules tasks. It allows you to make use of the facilities provided by the system. Every computer requires an operating system. UNIX is a multi-user, multi-tasking operating system. Multiple users may have multiple tasks running simultaneously. This is very different from PC operating systems such as MS-DOS or MS-Windows (which allows multiple tasks to be carried out simultaneously but not multiple users).

11.7 Review Questions

1. What is Multi-user Operating System?
2. What is Ordinary Files?
3. What is meant by Directory Files?
4. What is Special Files
5. What is meant by FIFO Files?
6. What is Mounting File Systems?
7. What is Unmounting Files Systems?
8. What is SuperBlock Cache?

LESSON - 12**DATA STRUCTURES FOR PROCESS / MEMORY
MANAGEMENT****Structure**

- 12.1 Introduction**
- 12.2 Objectives**
- 12.3 Data Structures for Process/Memory Management**
- 12.4 Process States and State Transitions**
- 12.5 Executing and Terminating a Program in Unix**
- 12.6 Summary**
- 12.7 Review Questions**

12.1 Introduction

A process is basically a program in execution. The execution of a process must progress in a sequential fashion. To put it in simple terms, we write our computer programs in a text file and when we execute this program, it becomes a process which performs all the tasks mentioned in the program.

When a program is loaded into the memory and it becomes a process, it can be divided into four sections % stack, heap, text and data. The OS must allocate resources to processes, enable processes to share and exchange information, protect the resources of each process from other processes and enable synchronization among processes. To meet these requirements, the OS must maintain a data structure for each process, which describes the state and resource ownership of that process, and which enables the OS to exert control over each process. In this lesson, we will discuss about the data structures for processes/memory management.

12.2 Objectives

After studying this Lesson, you should able to

- * The Compilation Process

- * The Executable File
- * EI Process Table (PT)
- * Per Process Region Table
- * Page Map Tables (PMT)
- * Process States
- * State Transitions U
- * Executing Programs

12.3 Data Structures for Processes / Memory Management

12.3.1 The Compilation Process

The data structures maintained by UNIX for manipulating the file system have already been discussed. In this section, the data structures maintained by UNIX for manipulating various processes will be studied (memory management is assumed to be a part of process management).

In order to understand various data structures maintained by UNIX, it is necessary to understand the basics of the compilation process. Regardless of where it will be ultimately loaded, the compiler generates the code starting from the address 0. For example, in COBOL, the data division could start at address 0. After all the FD and working storage are defined, it may leave some space and then start assigning addresses to the procedure division. The address of the first instruction in the procedure division then becomes this starting virtual address of the “text” region of that program.

After all instructions are compiled and addresses generated, the compiler may leave some space for expansion and then define some space for the stack region. There are only two restrictions on this procedure. Firstly, the total virtual address space (including the gaps in between) allocated to a process cannot exceed the addressing capability of the hardware. Secondly, the virtual addresses of regions have to be non-overlapping. For instance, the starting virtual address of “text” region has to be higher than the highest virtual address in the “data” region.

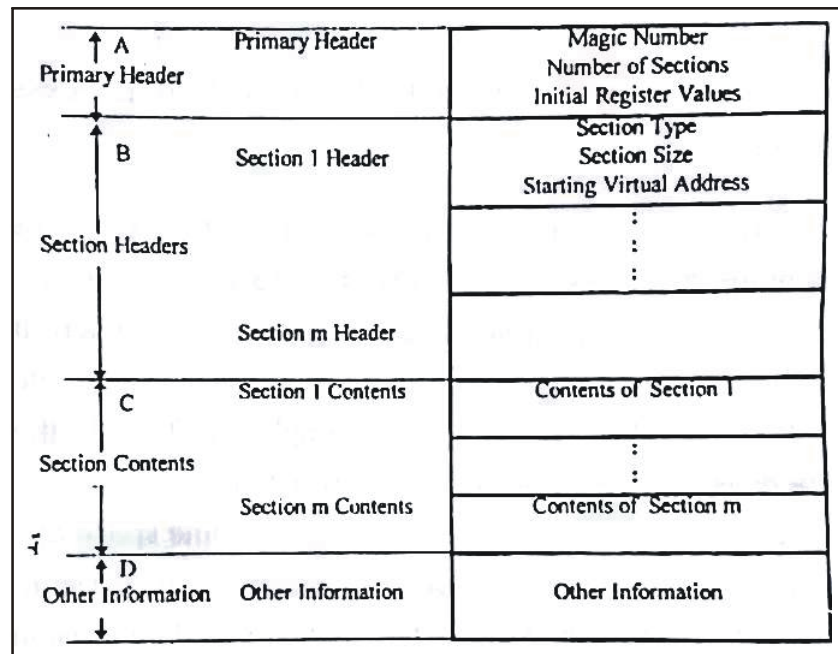


Fig. 12.A – Executable File

The file keeps all this information about starting virtual addresses, etc. in an executable file as shown in Fig 12.A. The file defines a section for each Region. The compiler keeps the information of section type, section size and starting virtual address for each section in the executable file as shown in the figure.

The executable file is divided into four parts as shown in the figure.

- A. Primary Header
- B. Section Headers or Descriptors
- C. Section Contents
- D. Other Information

The 'Primary Header' contains a 'magic number' which specifies the type of the executable file. The primary header also indicates the number of sections in the program. It then stores the initial register values to be loaded, once the process begins execution.

This part will also contain the value for the Program Counter (PC). This value specifies the address from which the execution should begin. It is the virtual address of one of the instruction in the "text" section from where the execution is supposed to commence.

The “Section Header” specifies the type of the section and it also stores the size and starting virtual address of section as discussed earlier.

The “Section Contents” store the actual “meat” of the section. For instance, the sections for the text contain the actual compiled machine instructions. The section for “data” contains various I/O areas and other locations reserved for different tables and variables with their initial values (given by value clauses).

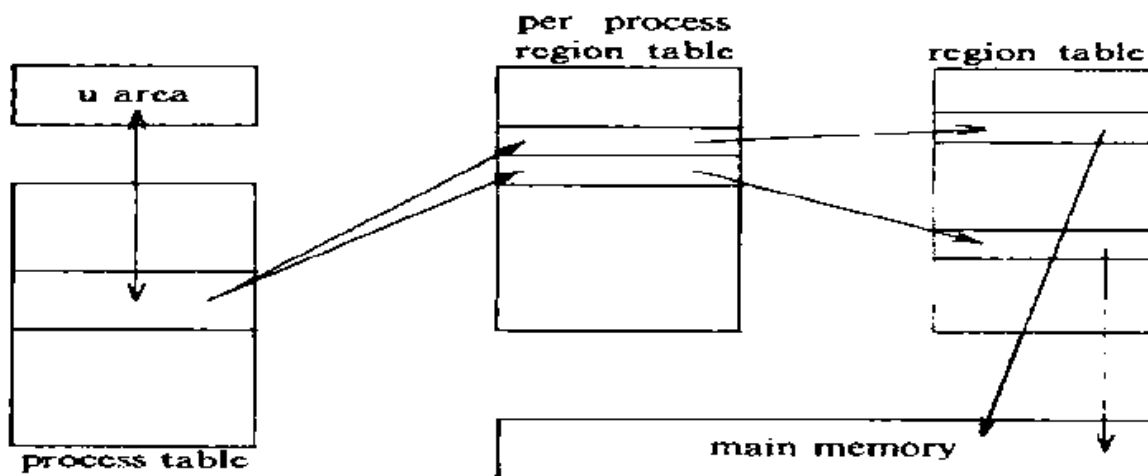


Fig. 12.B – Various UNIX data structures.

The “Other Information” includes symbol tables which are used for debugging a program at a source level. When a program starts executing, this symbol tables is also loaded. It gives various data and paragraph names (labels) and their machine addresses. Thus, when a source level debugger wants to display the value of a counter, the debugger knows where to pick up the value from. When a process is created and it wants to execute a program, the program name is treated as a filename, the kernel resolves the pathname and gets its inode. From the inode, it then reads the actual blocks allocated to this executable file. From the section size, mentioned in the header (part B), a request is made to the memory manager to allocate the desired memory locations and then the actual sections (part C) are loaded into those locations. If paging is used, the page map tables are also set up for address translation. The information from the section headers is used to set up various data structures maintained by UNIX shown in Fig. 12.B.

The data structures maintained by UNIX are listed below:

- * Process Table (PT)
- * u-area
- * Per Process Region Table (Pregion)
- * Region Table(RT)
- * Page Map Tables (PMT)
- * Kernel Stack (KS)

12.3.2 Process Table (PT)

The process table is maintained at a global level, wherein there is one entry per process known to UNIX on the system at any given juncture, regardless of its state. It contains the following information for each process:

- (a) Process Ids which specify the relation ship of processes with one another (e.g. parent-child).
- (b) Process State, i.e. Ready, Asleep, Running
- (c) Scheduling Parameters such as priority.
- (d) Process Accounting Data such as execution time and kernel resources utilization. (These are used to set the process priority).
- (e) Pointers to Per Process Region (Pregion) Tables: normally three in number, one each for text, data and stack for each user process as shown in the figure.
- (f) Pointer to the u-area and the kernel Stack: as shown in the figure. We will study these data structures in the sections that follow.

12.3.3 u-area

The u-area contains information about a process. It could be a part of process table itself, but it is maintained separately, because this portion can be swapped with the process image at the context switch. The entry in the process table for that process is still retained in the main memory, as it is necessary for the scheduling and controlling of processes as well as for the decision about when to swap the process. The name contains the following fields:

- (a) A pointer to the entry in the process table for that process. This is not shown in the figure to avoid cluttering.
- (b) The real and effective user Ids that determine various privileges allowed for the process, such as file access rights.
- (c) The control terminal field identifying then “login terminal” associated with the process, if any.
- (d) Timer fields to accumulate the times that the process has spent executing in the user and kernel modes respectively.
- (e) An array indicating the manner in which the process wishes to react to various signals. When a signal arrives for this process, the array is consulted and the appropriate action is taken.
- (f) The current directory and current root describe the file system environment of the process. When a user moves from one directory to another, essentially, this held is changed. This field is also used to construct absolute path names from the relative pathnames after concatenation.
- (g) The user file descriptor table maintains the files that the process has opened in different modes. This has already been studied.
- (h) Permissions field which is used for masking the permission bits set for the files at the time of creation by this process. It means that, these permission bits in the u-area will be used as a mask on the permission bits supplied by the “creat” system call as a parameter. The resulting permission bits will be set finally as permissions for that file in its inode. Thus, all the files created or used by this process could have certain permission denied as per the mask.
- (i) Limit fields are used to restrict the size of a process and the size of a file that a process can write.
- (j) The I/O parameters describe the source/target addresses, amount of data to be transferred, etc. They normally store the memory address and the relative byte number (RBN) as the file offset. The meaning of this field will be discussed while discussing the system calls.
- (k) A return field stores the result of system calls.

- (l) An error field records errors encountered during the execution of a system call issued by this process. The exact significance of this field will be discussed while discussing the system calls.

The items (j), (k) and (l) are really the work areas reserved in the u-area for the execution of system calls. There is a separate area to store the parameters of the system calls (j) which can be used to transfer the parameters between the u-area and the stack. There is an area to store the result of the system call (k) and another area to store the error codes (l), if any errors are encountered during the execution of the system call. They can then be interpreted and appropriate action can then be taken.

12.3.4 Per Process Region Table (Pregion)

'Pregion' has normally three entries for every process, one each for text, data and stack regions of that process. These entries are located from the process table once the process ID or PID is known. In the process table entry, there are multiple pointers pointing towards the entries in the pregon table. Thus, the entries in the pregon table for text, data and stack regions for any process need not be contiguous, though Fig. 12.B shows them as contiguous for the sake of simplicity. For each process, the figure shows only one pointer instead of three from the process table to the pregon table to avoid cluttering.

Each entry for a process and for a specific region in this program table contains the following information:

- (a) A pointer to the process table: This could be maintained as a PID which could be used as a key to access an entry of the process table from the pregon table. Thus, it is possible to traverse from the-process table to the pregon table and vice versa. This is not shown in the figure to avoid cluttering.
- (b) A pointer to the region table, as shown in Fig. 12.B. The region tables will be studied in the next section.
- (c) Virtual address is the starting virtual address of that region in that process. The compiler generates this address and stores it in the executable file. When the process is created, the pregon tables are created. At this time, these values are moved from the in core image of the executable file to the pregon tables. If a

region is shared by two processes (e.g. text region of a word processor or a compiler), this starting virtual address can be different in two processes sharing that region.

- (d) Access rights info gives the information about the kind of accesses that are allowed for that process on that region, e.g. Read only, Read/Write or Read/Execute.

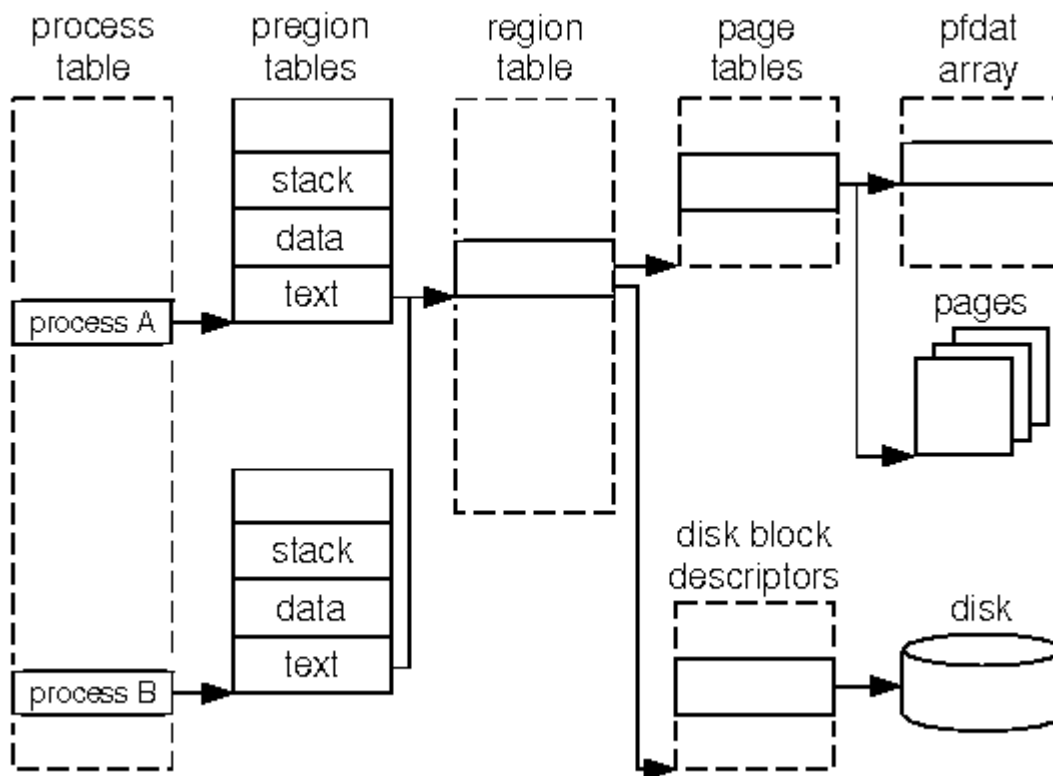


Fig. 12.C – Preregion chains for shared region

- (e) Pointer chain for shared regions could be the address (entry number) of the “next” preregion entry sharing the same region. This is shown by Fig. 12.C. The figure depicts that processes A, B and C share a region with region number = 1, shown with COUNT=3. It also shows that the preregion entries for shared regions are linked together. The entry contains the address of the “next” preregion entry for

the same region. The "" in this address field denotes the end of the chain. Hence, for an unshared region such as the region number =2, this address field is "i as shown for process D in Fig. 12.C. This is because, there is no further "next" entry for this.

12.3.5 Region Table (RT)

The region table allows sharing of regions. It is known that an editor program (e.g. vi) also has text, data and stack regions. The text portion of the editor can be used by many users executing it. For each, user, the data region will, however, vary depending upon the file or the data being edited. Thus, some regions are shared and some are not. This is made possible by the region table. In the region table, there is only one entry for a shared region such as text region of an editor or a compiler. Figure 13.27. depicts that processes 1 and 2 share a region with region number 0. However, for unshared or "private" regions, separate entries are maintained in both process as well as the region tables. It is the region table which maintains a pointer to the memory map tables that help in translation from the logical to physical pages in the main memory. If the contiguous memory management scheme is used instead of paging, region table directly contains a pointer to the main memory address for that region.

A region in the main memory is ultimately loaded from the contents of the executable file (generated by the compiler). This file also is a part of the UNIX file system and is uniquely identified by an inode number. Thus, the region table also contains a pointer to this inode to trace back to the source from which region was loaded.

To summarize, the region table contains the following information:

- (a) Region type (text, data or stack)
- (b) Region size
- (c) Location in the main memory (direct or through page maps)
- (d) Region status such as locked, being loaded, valid (already loaded)
- (e) Reference count giving the number of processes sharing a region. Each time a new process is created which shares that region, the count is incremented. When the process terminates, the count is reduced by one. When this count is 0, the region can be freed.

- (f) Pointer to the inode
- (g) Pointer to pregon table

Region table entries are of fixed size and can, therefore, be accessed directly once the kernel knows the region id or region number. Thus, the region id or number need not actually form a field in the region table. It has been shown only to facilitate comprehension.

At any time, the region table contains some entries which are used and some which are free. The free entries are linked together by a pointer chain and UNIX contains routines to add an entry to this list when a region gets free. Figure 12.D clarifies how the allocation/deallocation region table entries is achieved.

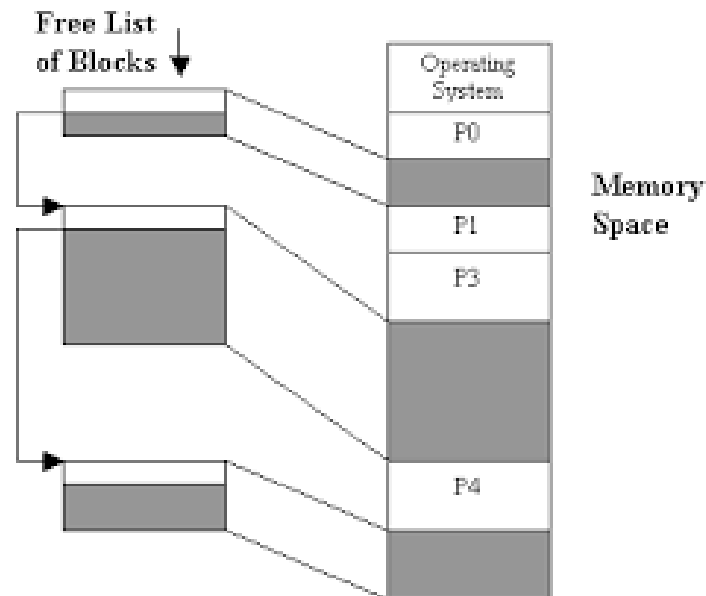


Fig. 12.D – Region table : free list

12.3.6 Page Map Tables (PMT)

The page map tables maintain information about logical pages versus physical page frames for each region. Thus, for a process, there could be three PMTs one for each region of a process. Because a region is divided into virtual or logical pages as 0,1,2,.....etc., the PMT needs to

mention only the corresponding physical page frames. The virtual or logical page numbers are implicit. For example, Fig. 12.E shows a PM for a region, Virtual page numbers are shown there only for better comprehension. In this figure, logical page 3 would reside in physical page frame number 19 and logical page 6 would reside in physical page frame number 83. This PMT is used at the time of address translation.

Logical (Virtual Page # (P)	Page Frame # (F)
0	50
1	24
2	6
3	19
4	73
5	61
6	83

Fig. 12.E. – A Page Map Table (PMT) for a region

The compiler has no idea as to where, in the physical memory, the different pages of a program would be loaded at the time of execution. In paging systems, the compiler, therefore, generates an address consisting of two components, logical page number (p) and displacement (d) within that logical page. The address generated in any compiled instruction would be $p + d$. This has been studied earlier. We have also studied, that, though compiler generates only single dimensional virtual addresses assuming the starting address as 0, the same address can be thought of as a two dimensional address p and d if the page size is a power of 2.

12.3.7 Kernel Stack (KS)

A process executes either in a user mode or a kernel mode. When it is in the user mode, it uses the user stack to store data such as parameters, return addresses of nested subroutines, or procedures. When the process issues a system call, it starts executing in the kernel mode.

At this time, the kernel uses its own stack to store the parameters used for executing that system call as well as for storing the results from that system call, before they are moved to the u-area of that process.

These parameters are copied from the u-area of the process issuing the system call and the return values (or results) as well as the error values from that system call are moved back to the u-area of the process. Thus, the system call picks up the parameters from the u-area (where they are moved from the user stack) and deposits results/error values from the system call into the u-area again. Internally, the kernel uses the kernel stack to execute the system call.

Thus, for each process, there is a user stack and a kernel stack separately. Even if different processes use and share the same kernel routines, each process has its own kernel stack to store parameters and return addresses/error values.

The address of the stack is maintained in the process table as shown in Fig. 12.8 as a pointer.

12.4 PROCESS STATES AND STATE TRANSITIONS

Process states and state transitions for UNIX are similar to the ones discussed earlier in this text in the chapter on processes. It might be a good idea to refresh the material once again before proceeding further. The only additional considerations are given below:

A Process can execute in either user or kernel mode.

A process is not added to the “Ready” state immediately after creation. it is put in a “created” state instead. The idea is to create a process without enough memory being available for it to run. it can subsequently move to either “Ready” (i.e. in memory) state (bubble ‘c’ in the figure), or “Ready, Swapped” sate (bubble ‘e’ in the figure) from “Created” state, depending upon the available memory, See Fig 12.F.

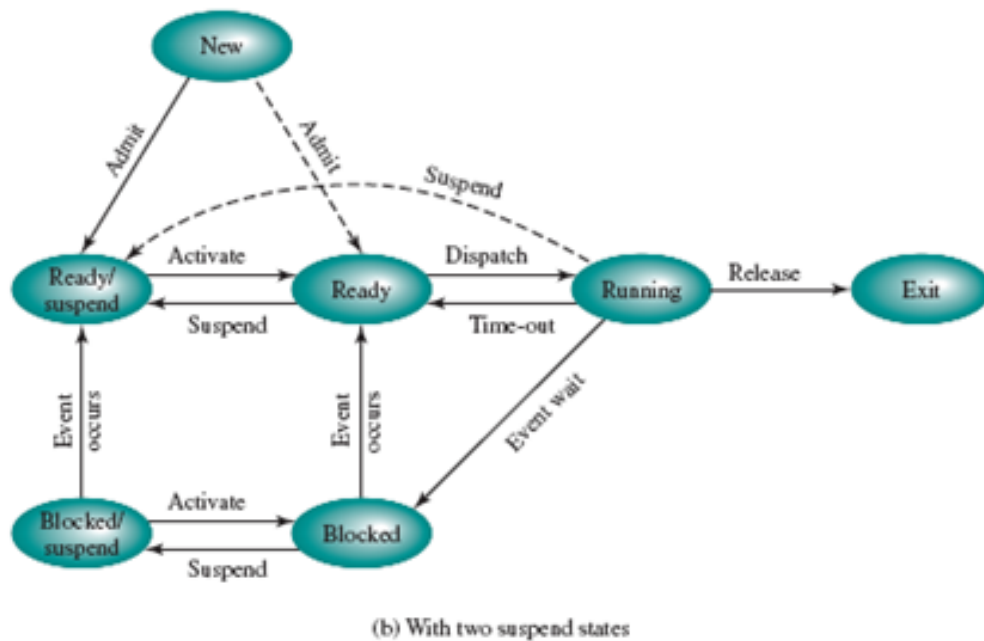


Fig. 12.F – Process states and state transition

When a process gets completed, after running in the user mode, it invokes an “exit” system call shown as “System Call Interrupt” at the top of the figure. At this time, it enters the “kernel running” mode (from bubbles ‘a’ to ‘b’ of Fig. 12.F) and ultimately goes into a “zombie” state as bubble ‘l’ in Fig. 12.F depicts. In this state, all the process details excepting a slot in the process table are removed.

There are two states “Ready to Run” and “Preempted” depicted by bubbles ‘c’ and ‘g’ in Fig. 13.34. These are similar in nature. Both are really Ready in the sense that each one can be scheduled, as none is really waiting for nay event to occur. They are not running only because the CPU is executing: some other process and the CPU can execute only one process at time. However, the major difference is that when a “Ready” process is scheduled, it goes into a “kernel running,” state first (from bubbles ‘c’ to ‘b’) and then it assumes “a state of “user running” (from bubbles ‘b’ to ‘a’). in fact, it does so necessarily.

The reason for this can be found in the history of how that process became preempted in; the first place. when a process is executing in the kernel mode (bubble ‘b’) and it is about to go into the “user running” state (bubble ‘a’) after completing the processing in the kernel mode, and

at that time itself if its time slot is complete, it goes into a preempted state (bubble 'g'). Thus, the operating system knows that such a process should be executed in the "user running" mode directly after being dispatched. All other ready processes have to go the kernel mode before they can start executing in user mode.

Some different terminology is used, in this section then the one studied earlier, e.g. we have used "sleep" instead of "block" and a process is called "asleep" if it is "blocked" i.e. if it is waiting for an event such as the completion of an I/O.

The following process states as identified in Fig. 13.34 can now be listed.

- (a) User Running (UR): A process running in user mode.
- (b) Kernel Running (KR): A process running in kernel mode.
- (c) Ready (RD): A process ready to run: As discussed above.
- (d) Asleep in Memory (AM): A process which is blocked and is waiting for some event such as I/O.
- (e) Ready Swapped (RS): A process which can be scheduled and dispatched only after swapping in but the one which is not waiting for anything else.
- (f) Asleep Swapped (AS): A process which is both swapped out and also waiting for an event. From this state, it makes a transition to state RS first after the occurrence of the awaited event, and then to state RD when it is swapped in. Alternatively, it can go to the AM state, if it is swapped in while it is still sleeping on an event. It can move to RD after that event. After it is in the RD state, eventually it is scheduled after which it goes into state KR and ultimately to a state UR whereupon the actual user process starts executing.
- (g) Preempted (PR): A process not waiting for any event and the one which has given up the control of the CPU only to return to "user mode (UR)" directly when dispatched.
- (h) Created (CR): A process which is created but which is not yet readied.

- (i) Zombie(ZM): A process which has terminated, but not completely removed from the system.

The process table entry for a process has a field called 'process state' which tells you the state of that process. Using pointer chains, all the entries in the process table belonging to the same state can be linked together. For instance, the linked list of ready or preempted states can be maintained in the priority order, where the process priority is one of the fields in the process table. As the processes change their states and the priorities quite frequently during their life cycle, the algorithms and data structures to manage these changes need to be very efficient.

It is known that the data structures such as the process table entries of a process hierarchy are also chained together for speedy traversal. This makes the life of the operating system designers more complex. For instance, the process table slots of all the ready processes will be chained together. All preempted ones, created ones and all ready swapped processes also will be chained together by separate pointer chains. In fact, all of these will be maintained in a priority sequence. All others will be chained in the time sequence. Also, all the processes in a process hierarchy will be linked together, regardless of the states of different processes within it.

12.5 EXECUTING AND TERMINATING A PROGRAM IN UNIX

12.5.1 Introduction

In UNIX, 'fork' is the only way that a process can create another process. Only the very first process called 'init' has to be 'hand created'. All the other processes are created by using the fork system call. The newly created process becomes a child to the process which issues the fork system call. At the time of creation, the child process is identical to the parent process except only the process id or pid. This pid is different for the parent and child processes, and it helps to determine whether it is a child or a parent process. All the other data structures like u-area or region tables are identical. In fact, the child process created as a result of the fork system call shares the same open files and in fact, the same program text. Hence, the fork system call only duplicates the data and stack of the parent process and lets the text region be shared between the two processes.

A question arises as to why the process address space is duplicated and if it is, then what can the child process achieve? The answer is in exec system call, which will be studied later in

more detail. The exec system call enable the child process address space to be loaded by the desired program from the disk and then allows the execution of the new program to start. Thus, the exec system call does not create a new process. It only replaces the address space of the child process by the new program to be executed.

A fork followed by an exec is commonly employed by shell each time a command is issued, to execute a specific utility or a program. There is another system call called 'wait', which allows the parent process to wait until the child process terminates. The death of a child process a send a signal to the parent waiting on the death of the child and awakens it. Hence, when a shell process is executed and a command is given to execute a program in the foreground, the following takes places:

- The shell forks (i.e. a child process to shell process as parent is created)
- The child process execs the required process (by loading the new process image as per the command, into the address space of the child)
- The parent process i.e. the shell in this case waits until the child process terminates.
- After the death of the child process, the shell i.e. parent process continues i.e. it prompts for another command.

It was seen that the parent and child share the text portion which means that at the juncture, they run the same program. But it is known that the child process has to exec the desired program, whereas the parent has to wait until the child terminates. How can a common algorithm for a parent and a child achieve two different things? This is achieved by checking in the common algorithm whether it is parent or child (For the child, pid =0), and then taking the appropriate action. This is illustrated by Fig. 12.G.

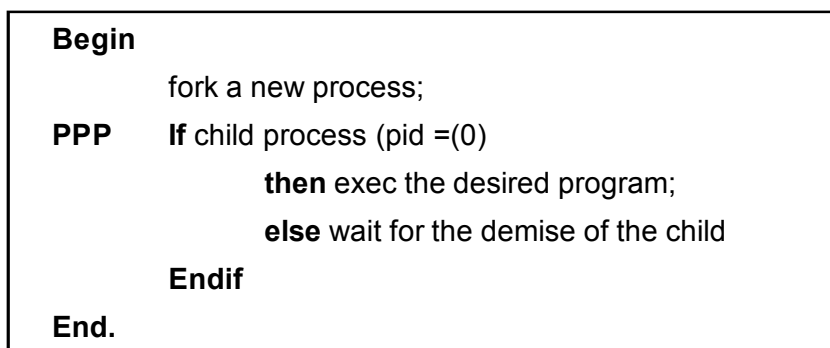


Fig 12.G Execution of a new program by a fork. exec and wait

The sequence of events that takes place is outlined below:

- (i) A new child process is created by the fork system call. It has a new entry in the process table. This child process is identical to the parent process except the pid field in the process table entry. This pid in the process table entry of the child is 0;
- (ii) The child also shares the same text region as the parent. Thus, the parent and the child both execute the same program, the one shown in Fig. 12.G. The program counter (PC) in the parent process after the fork system call was at PPP. Because the context of both processes also is identical (as it is duplicated), the PC in the register save area of the child also is set to the address of PPP in Fig. 12.G.
- (iii) The parent starts executing the process at PPP after the fork. It checks whether it is a parent or a child by checking the pid. As it is not a child process, it waits until it receives a signal indicating the demise of the child.
- (iv) The child process is put in a 'ready and in memory' state as soon as created. It is eventually scheduled and it also starts at PPP. The reason for this is explained in (b) above. It again checks whether it is parent process or a child process by examining its pid. Because it is a child process, it issues a system call for exec with the program name as parameter (e.g. ls or PAY.CAL). The exec talks to the file system, parses the pathname, locates the inode of the object file, checks permission and reads the headers of the executable file, talks to memory management to load all the regions in the address space of the child. The code for the program in Fig.12.G which existed in the child's address space is now overwritten by the code for the program that needs to be executed.
- (v) The child now starts executing the new program with a new id. Before this, the initial register values are loaded into the CPU registers like PC from the header of the executable file of that program. This is how the new program starts at the correct location.
- (vi) Eventually the child terminates i.e. it goes into zombie state and sends a signal to the parent about its demise. The u-area of the child process accumulates the details of resources used by it such as CPU time, etc. for accounting purpose. (Refer to point (d)).

- (vii) The parent wakes up and continues. At this time, the accounting details such as CPU time, etc. from the u-area of the child process are added to those in the u-area of; the parent. In the example, the parent was shell. In this case, the shell prompts for a new command. As soon as a new command is given, it executes the algorithm in Fig. 12.F again. This time if a different program is to be executed, the 'exec' will have to locate and load the code for that program and execute. However, the general scheme remains the same.

A detailed discussion on the 'fork' and 'exec' system calls follows.

12.5.2 The Fork System Call

When the fork system call is executed, the following actions are carried out (refer to Fig. 12.G).

- (a) The kernel ensures that there are enough system resources to create a new process. This is done as follows:
 - (i) It ensures that the system can handle one more process for scheduling and that the load on the scheduler is manageable.
 - (ii) It ensures that this system can handle one more process for scheduling and that monopolizing on the existing resources.
 - (iii) It ensures that there is enough memory to accommodate the new process. It is known that the new process is identical to the parent process in every aspect at this juncture. This also includes the memory requirements. In the swapping system, the memory has to be available to accommodate the entire program size. In pure paging system, the memory for all the pages to hold the entire address space as well as the page map tables is necessary. In demand paging, only the page map tables are necessary at the least to initiate a process. Further pages from the address space can be accumulated with page faults in demand paging.

If space is not available in the main memory, the kernel checks if there is space on the disk such as in the swap are to hold all this. Depending upon this, the state of the child process will be determined as was studied in the process state transitions.

- (b) The kernel now finds a slot in the process table to begin to construct the context of the child process.
- (c) The kernel maintains a systemwide global value of “next ID number available”. Anytime a new process is create by fork, the kernel assigns this ID to the new child process and increments this number by 1. The kernel also maintains a maximum value beyond which the system cannot handle any more processes. If this number becomes equal to or higher than this maximum, the kernel Wraps around and starts assigning numbers from 0, again hoping that the earlier process with pid=0 would have terminated by then.
- (d) The kernel initializes the fields in the slot of process table for the child process as follows:
 - (i) It copies the real and effective user IDs from the slot of the process table of the parent process to that of the child process.
 - (ii) It also copies the nice value (used in process scheduling to be studied later) from the parent process to that of the child process.
 - (iii) It links the child process in the process tree structure by putting the parent process ID in the slot for the child process.
 - (iv) It initializes various scheduling and accounting fields such as initial priority CPU usage, etc. in the slot for the child process.
 - (v) It sets the state of the process to “being created”, for the child process.
- (e) The kernel now searches for the file descriptors in the u-area of the parent, and following the pointers from them (UFDT) to the File Table (FT) entries, it increments the reference counts of those entries in the FT.

- (f) The kernel allocates memory for the u-area, region tables, page tables, etc. for the child process.
- (g) It now copies the parent u-area to the child was except that the child u-area's pointer to the process table slot is properly adjusted. This is because the parent and child process have two separate entries in the process table. Hence, pointers to them will be different. All the others are the same at this juncture.
- (h) The kernel copies the data and stack regions (unshared portions) into another memory area for the child and adjusts the region table entries. it however keeps only one copy of the text region, because this can be shared. As was seen, this text at this juncture contains the code of a program illustrated in Fig. 12.F.
- (i) After the static portion of the child context, the kernel creates the dynamic portion. It copies the parent context layer 1 comprising saved registers and kernel stack of the fork system call itself. The kernel stacks for both the child and the parent at this time are identical.
- (j) The kernel creates a dummy context layer 2 of the child process, containing the saved register context for layer 1. It sets the Program Counter (PC) and other registers in the register save area, so that the child can "resume" execution at the right place.
- (k) It now changes the state of the child process to "ready to run" (either in memory or swapped as per the case). It returns the ID of the child to the user.
- (l) The scheduler finally schedules the child process. At this juncture, the "text" of the child process is similar to that shown in Fig. 12 F and Fig 12.6 In this program, it checks if it is child or not Because it is a child process it executes 'exec' system call, whereby the new program is loaded in the address space of the child process. We will study 'exec' later.

Figure 12.G depicts the scenario. The figure shows the duplicated process address space (with shared text region), u-area (with the same pointers to the tile table), kernel stack, etc.

12.5.3 The Exec System Call

Following actions are carried out by the kernel to implement this system call:

- (a) The exec system call expects a name of an executable file to be supplied as a parameter. This is stored for future use. Along with the file name, other parameters also may be required to be supplied and stored e.g. if the command to the shell is "ls-l", both ls as a file name and l as a option need to be stored for future use.
- (b) The kernel now parses the pathname of that file to get the inode number for it. It then accesses and reads that inode. The kernel knows that for my shell command, it has to search in the / bin directory first. A reference to Fig. 10.1 and point The kernel ascertains the user category (whether it is owner, group or others). It then accesses the execute (x) permission for that category for that executable file from the mode. It checks whether that process has the permission to execute that file. If it does not, the kernel outputs an error message and quits.
- (c) If everything is ok, it accesses the header of the executable file.
- (d) The kernel now has to load the executable file for the desired program such as "ls" in our example into the regions of the child process. But the sizes of various regions required by "ls" will be different from the ones existing for the child process, as they were copied from the parent process. Thus, the kernel frees all the regions attached to the child process. This is to prepare for loading the new program from the executable image into the regions of the child process. This freeing is done after storing the parameters for this system call elsewhere which were stored in this memory only. This storing is done to avoid their being overwritten and getting lost by the executable code of "ls". The storing is done at a convenient place depending upon the implementation. For instance, if "ls" is the command and "-l" is the parameter for it, "-l" is stored in the kernel area. The binary code of the "ls" utility in the /bin directory is what the kernel wants to load into memory of the child process.
- (e) The kernel then allocates new regions of required sizes after consulting the headers of the image of the executable file (e.g. of "ls"). At this stage, the links between the region tables and page map tables are established.

- (f) The kernel attaches these regions to the child process, i.e. it creates the links between the region tables and pregon tables. As Fig. 12.G indicates, the pregon table for the child process will have already been created by the fork.
- (g) The kernel then loads the contents of the actual regions into the allocated memory.
- (h) The kernel creates a saved register context by using the initial register values from the header of the executable file.
- (i) At this stage, the child process (Is" utility) is ready to run. Thus, it is inserted in the list of "ready" processes at the appropriate place depending upon its priority. Eventually, it is dispatched.
- (j) After the child process is dispatched, the context of the process is generated from the saved register context as discussed in point (1) above. The PC, SP, etc. will then have the correct values.
- (k) The kernel then jumps to the address indicated by PC. This will be the address of the first executable instruction in the program to be executed. This commences the execution of the new program such as "Is". The parameters are picked up by the kernel from a predefined area where they were stored in step (e) and the required output is generated. The parent process waits until the child process terminates if the child process is executed in the foreground; else it also continues.
- (l) The child process terminates and goes into a zombie state. The desired program is already complete. It now sends a signal to the parent to denote the 'demise of the child', so that the parent can now wake up. (Refer to Fig. 12.F.)

It this child process opens new files, the UFDT, the FT and the IT structures of the child process differ from those of the parent process thereafter. If the child process calls another subprogram, the process of forking/executing is repeated. Thus, a process hierarchy of various levels of depicts can be created.

12.5.4 Process Termination (exit)

UNIX has an exit system call to terminate a process. For instance} the compiler generates a system call for exit, when it encounters a 'stop run' instruction in a COBOL program. After the termination of a process, all the resources are released, the process state becomes zombie and then it sends a 'death of the child' signal to the parent." The format of this system call is as follows:

The status is a value returned to the parent process for examination and further action. The kernel takes the following steps to carry out this system call execution:

- (a) The kernel disables all signals to the terminating process, because now handling any of them is meaningless. This disabling can be done by setting appropriate values in the u-area of the process for signal handling.
- (b) The kernel goes through all the open files by consulting the UFDT which is part of the u-area. It then closes all the files with the 'close' system call. It follows the pointers to the FT and the IT and reduces the count field by 1 for the appropriate files, and releases the resources (FT entries, inodes) when the count becomes 0.
- (c) The inodes held by the process directly (eg. current directory inode) are also released.
- (d) The kernel frees all the pregon table entries. Before this is done, it" follows the pointers to region table entries, reduces the reference count by 1 and if the count then becomes 0, also releases the region table entry, the corresponding page map tables and the corresponding memory. All these are added to their respective free pools.
- (e) The kernel saves the accounting details of the exiting process (such as CPU usage) from the u-area in the process table entry of that terminating process. This process table entry is not deleted even after the process becomes zombie. This is because, its resource usage details, CPU time, etc. have not been added to those fo the parent process yet.) It then frees the u-are and all other data structures except the process table entry.

The kernel sets the state of the child process to zombie.

12.5.5 The Wait System Call

The wait system call is used by a parent process to sleep until the death of the child after which it can continue. The syntax of this system call is as follows:

Where pid is the process ID of the zombie child, and status-address is the address of the status code of the child in the user space.

The following actions are taken to execute this system call.

- (a) At any moment, there could be a number of child processes to a parent process and some of them could be zombie. The kernel goes through all the children for the parent process (the chain of pids denoting the process hierarchy as well as the process states is maintained in the process table as discussed in points (a) and (b), if there are no zombie children, it outputs an error message and exists. Otherwise, for each zombie child process, it goes through the following steps:
- (b) It has already been seen that when the child process terminates, its accounting details are stored in the process table entry before releasing the u area and other data structures. Hence, for each zombie child, it adds the accumulated accounting details from the process table slot of the zombie child into those in the tram of the parent process. (When the parent process becomes zombie, these u-area details are saved in process table entry of the parent process and the cycle repeats.) The idea is that a process should be responsible for all the resources used by its children, grandchildren. etc, in general, the entire hierarchy under it.
- (c) The kernel then removes the entry in the process table for the zombie child.
- (d) The kernel returns after sending, the child Id and exit code as the parameters to be used by the parent.

12.5.6 USING THE SYSTEM (BOOTING AND LOGIN)

Booting Process: Process 0, Process 1

After studying various data structures and system calls for managing various files and processes, the process of booting in UNIX can be studied. The problem of booting is can be put forward simply as to how to start the kernel.

The action of loading the UNIX kernel system image into the memory and starting its execution so that users can use the system is called booting. The following takes place at the time of booting:

- (a) The first block on every file system is reserved for the boot block. The boot block contains the boot program. Each file system need not have a boot program, but at least one file system on one disk must have it. That disk is called bootable disk.
- (b) There is one or more hardware switches on each machine, which if pressed, automatically generate hardware signals to move the disk Read/Write arm and access the boot block and read it in a predefined memory locations. No program is needed for this. At the start-up time, this is automatically done by the hardware itself. The control is thereafter passed to the first instruction in the boot program, again automatically.
- (c) The boot program contains instructions to locate the executable code of UNIX in the root directory (/Unix). The file /Unix contains the binary machine instructions for the kernel. This file is created by compiling and linking the UNIX source code files.
- (d) The boot program issues instructions to load/Unix in the memory. The control then passes to the first instruction in the kernel.
- (e) The kernel then initializes a number of hardware interfaces and data structures.

This is done as follows:

- (i) The system clock is initialized.
- (ii) The kernel constructs the linked lists of free buffers (for all the devices including main memory depending upon the implementation).
- (iii) The kernel clears pregion, region and page map tables.
- (iv) The kernel mounts the root file system onto the root ("/").
- (v) The kernel creates a u-area for process-0 (we will learn about it shortly).
- (vi) The kernel initializes slot 0 in the process table for process 0.

- (e) The kernel then creates a process 0. This process 0 is not a process at all. It does not execute anything. It does not contain any text. it only contains a uarea. Process 0 is created anonymously and persists for the life of the system. Process 0 is truly a system process. It is active only while the processor is in the kernel mode. Process 0 is called a process only because it has an entry in the process table.
- (f) The kernel then creates process 1 after process 0 is created. Process 1 is a true process. It contains a code segment with only one instruction to perform a system call exec to execute a program called / etc / init i.e. the program called 'init' in the /etc directory. The init program carries out a number of initialization activities; therefore, it is called 'init'.

Process 1 is also created NOT by using a fork system call. All the subsequent processes are generated only by fork. The process 1 is thus, hand-created as a child process of process 0, exactly the way fork would have done. This is achieved as follows:

- (i) Process 1 is created as a copy of process 0, in terms of all data structures such as u-area, etc. This is exactly the manner in which 'fork' would have worked.
- (ii) The kernel expands the size of process 1.
- (iii) The kernel creates an instruction to perform a system call exec to execute /etc/init in the text region of process 1.
- (iv) Process 1 is put in the "Ready (RD)" state. At this time, it is the only process which can run.
- (d) The process scheduler is brought into action. But it finds only process 1 as the candidate for scheduling. Hence, process 1 is scheduled.
- (e) Execution of process 1 leads to that of exec system call. As a result, the original code in process 1 is overlayed with the code contained in the file /etc/ init. Now process 1 can be called the 'init' process as it has reached its final executable form.

- (f) The init process is responsible for setting up the process structure of UNIX. There are two modes in which UNIX can operate. One, a single user mode and the other, a multiuser mode. This mode is specified in `/etc/inittab` file. Depending upon the version, the init takes appropriate actions. In single user mode, only the superuser can log in with root privileges. The single user mode is often used for testing and debugging/repairing file systems and other structures.
- (g) In a multiuser mode, init creates a 'getty' process for every active communication line. The 'getty' is a process which therefore, runs at every terminal, waiting for a user to type in any input. It accepts the input and passes it on to the login process to verify the username/password. We will study this in more detail while studying the login procedure.
- (h) Init creates a shell process to execute the commands in the file '`/etc/rc`'. The `/etc/rc` file contains a shell script which contains commands to mount file systems, start background processes (daemons), remove temporary files and commence the accounting programs. The exact details of `/etc/rc` differ from implementation to implementation. It also displays the copyright messages.

12.5.7 The Login Process

Before a user goes through the login procedure the following steps should take place;

- * UNIX must have been booted and process 0 must have been created
- * The init process (process 1) must have been "hand created"
- * The init must have spawned `/etc/getty` (referred to as only getty) processes one each for all the terminals. The process hierarchy at this stage looks as shown in Fig. 12.1

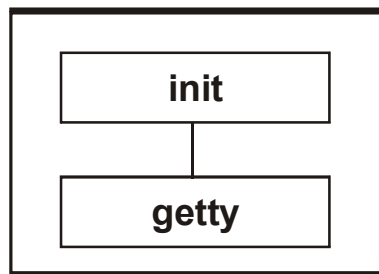


Fig 12.1 The login process hierarchy 1

Assume that the getty process prompts for the username.

During the login process, the following take place:

- (a) The user enters a username. The process getty continuously monitors the communication line for that terminal, and so soon as a character is typed, it picks it up and stores it. This is the manner in which getty received and stores the username input by the user.
- (b) The getty forks and Spawns another process called 'login'. The process hierarchy at this time looks as shown in Fig. 12.1. The getty passes the stored username as an argument to the login process.
- (c) The login process resolves the pathname for /etc/passwd file and accesses its inode and finally the file itself.
- (d) The login process checks the validity of this username keyed in by the user. To be valid, it must exist as one of the valid usernames in the /etc/passwd file. When a user is added to UNIX, its username, password and other details are stored in /etc/passwd file. The table search to ensure this validity must be an efficient one for better response time. It must be remembered, that there could be hundreds of such users and login could take place a number of times every day for each one of them!

If the username does not exist, the login process proceeds as follows:

- (i) The login process prompts a user, for a password (This is again dependent on the implementation. HP-UX does this). The reason for asking for a password, despite a wrong username, is to make it difficult for an intruder to find valid usernames illegally.
- (ii) The login process then displays a message for the invalid login. Thus, the intruder would not know whether the username or the password is invalid.
- (iii) The login process keeps track of invalid login attempts (in HP/UX, this is done by making use of the file /etc/btrnp).

- (iv) After three consecutive invalid login attempts by a user, the login process exists; else it reprompts for a valid username/password.
- (v) If the username exists as a valid username in `/etc/passwd`, the login process checks whether the password field for that user in `etc/passwd` has been set (Le. whether the user wants to have his password checked or not). If not, the login process skips the validation procedure for the password (i.e. it skips step(f))
- (e) If the password field in `fete/passwd` is set, then login process validates it as follows:
 - (f) The login process prompts for a password after forcing the user to change it, if it is time for the change. The times when the password need to be changed is defined in `etc/passwd`.
 - (g) The login process accepts the password keyed in by the user.
 - (h) The login process encrypts it in a predefined manner.
 - (i) The login process compare it with the encrypted password stored in `/etc/passwd`, file for that user.
 - (ii) If the password does not match, it displays an error message, and reprompts for another attempt both for the username and password. After three consecutive unsuccessful attempts, it displays another error message and the login process terminates.

If a user does not login successfully; login exits after a suitable time limit, closes the terminal line opened before. 'Init' at this stage spawns another 'getty' process for that terminal.

- (g) If the username and password both match, the login process copies the user id, group id, and home directory fields from the `/etc/passwd` file to the u-area. As we have seen before, the user and groups ids for that user are assigned and stored in `/etc/passwd` file at the time a user is created in the system.
- (h) Login updates a file keeping track of valid logins (In HP/UX, this file is called `/etc/wtmp`).

- (i) The login process now checks the `/etc/motd` file for the “message of the day”. If it exists, it is displayed. It also checks for the file `/bin/mail` for any messages to this user. If there are any, it displays “You have mail”.
- (j) Now, the initial program should be executed as the last part of login procedure. There is a command field in `/etc/passwd` file which contains the pathname of this initial program which is now to be executed. Login, therefore, runs this command via the `exec` system call. Typically, this field is set to the pathname of the shell the user wants “to execute. There can be different shell programs that can be present in `/bin` directory. e.g. `/bin/ksh` (korn shell), `/bin/sh` (Bourne shell), `/bin/csh` (C shell) and `/bin/pam` (Pam shell). Depending upon the computing environment and the user, this initial program will have been defined at the time of adding a user to the system by the supervisor. The idea is that this initial program should start executing as soon as a user logs in.

If a specific user wants to skip the shell and directly go to execute a specific program, that pathname can be given in the `/etc/passwd` file instead of the Shell's. This is useful for Point Of Sale (POS) or many other applications where after logging in, the user should not have to take the trouble of even executing the shell commands. This is also the case normally in the entry applications, where an untrained operator would like to see the relevant application screen immediately after logging in, so that the data entry could commence fairly mechanically. We will assume that the user is running the shell for further discussion.

- (k) The login process thus, follows the shell by the `fork/exec` procedure as seen earlier. The process hierarchy at this time looks as shown in Fig. 12.J
- (l) The shell runs the appropriate system login scripts which initialize the user's environment. It runs `/etc/csh.login` for C shell and `/etc/profile` for Korn or Bourne shell in HP/UX.
- (m) The shell then will be ready to accept the commands from the user. It is known that the `fork` process copies most of the data structures from the parent to the child. Thus, the u-area of the shell process for that user/terminal will have the user id, group id, etc. inherited from the ancestors. The same thing continues if shell also spawns other processes. This is the way the access rights for any process are transmitted and used for the controlling purposes.

- (n) Assume that a user gives a command “rm file-A”. This command enables the user to remove a file by name file-A from the current directory (which at this stage will be the same as the home directory).
- (o) The shell again goes through a fork/exec cycle to create a new process, to locate the binary executable file for rm utility in /bin or /usr/bin and check the access rights and to load it in the memory of the new child process during ‘exec’. It now passes ‘file-A’ as a parameter to the new process (let it be called N-proc) and the N-proc starts executing.

The Process hierarchy at this stage looks as shown in Fig. 12.K

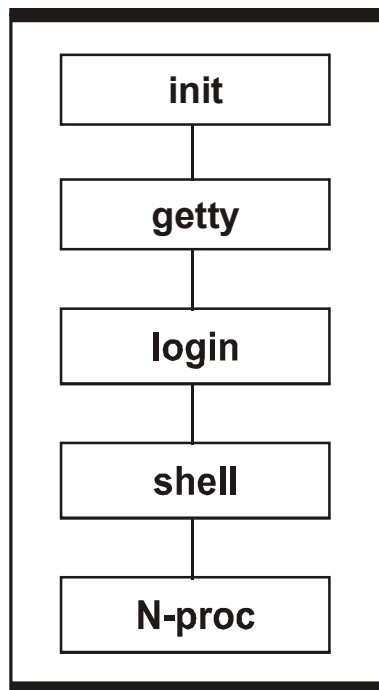


Fig. 12K The login process hierarchy 4

The new process N-proc will contain the executable binary code for the ‘rm’ utility in its address space. The ‘rm’ program then starts executing from the first executable instruction in the ‘rm’, where PC is set. The ‘rm’ carries out the pathname resolution, access verification, ect. for file-A and ultimately removes it by removing its entries in the directories, inodes, do it also release the disk blocks held by file-A. Ultimately it terminates. At this time, it sends a message to

its parent (shell in this case), and wakes it up. The shell now prompts for the new command. The process hierarchy at this stage again looks as shown in Fig. 12.K until the user gives another command. This cycle continues until the user continues to execute any shell command.

12.6 Summary

In this lesson, we have studied about the data structures for process and memory management in operation systems. While creating a process the operating system performs several operations. To identify these process, it must identify each process, hence it assigns a process identification number (PID) to each process. the process control block (PCB) is used to track the process's execution status. A process control block (PCB) contains information about the process, i.e. registers, quantum, priority, etc. Page Table is a data structure used by the virtual memory system to store the mapping between logical addresses and physical addresses.

12.7 Review Questions

1. What is Primary Header?
2. What is meant by Section Headers or Descriptors?
3. What is Section Contents?
4. What is Virtual Address?
5. What is meant by Pointer Chain?
6. What is File Descriptor?
7. What is Process. States?
8. What is State Transitions?
9. What is Pointer?
10. What is File Table?

LESSON - 13

PROCESS SCHEDULING

Structure

- 13.1 Introduction**
- 13.2 Objectives**
- 13.3 Process Scheduling**
- 13.4 Memory Management**
- 13.5 Summary**
- 13.6 Review Questions**

13.1 Introduction

The objective of multiprogramming is to have some process running at all times, to maximize CPU utilization. The objective of timesharing is to switch the CPU among processes so frequently that users can interact with each program while it is running. To meet these objectives, the process scheduler selects an available process (possibly from a set of several available processes) for program execution on the CPU. For a single-processor system, there will never be more than one running process. If there are more processes, the rest will have to wait until the CPU is free and can be rescheduled.

Process scheduling is an essential part of a Multiprogramming operating systems. Such operating systems allow more than one process to be loaded into the executable memory at a time and the loaded process shares the CPU using time multiplexing. The prime aim of the process scheduling system is to keep the CPU busy all the time and to deliver minimum response time for all programs. For achieving this, the scheduler must apply appropriate rules. In this lesson, we will discuss about the concept of process scheduling in detail and it's related techniques.

13.2 Objectives

After studying this Lesson, you should able to

- * Process Scheduling

- * Memory Management
- * Swapping
- * Demand Paging
- * Page Map Tables
- * An Example Using Demand Paging

13.3 Process Scheduling

The algorithm for process scheduling is implementation dependent. Most of the UNIX implementations make use of 'multilevel feedback queues' that were studied earlier. This method tries to do justice to both I/O bound and CPU bound processes.

In the method that was studied earlier, a CPU bound process gets a chance to run less often (by lowering its priority), but when it runs, it can run for longer duration (by increasing the time slice). This method, therefore, maintains a number of queues within ready processes. Each queue has an associated value of priority and a corresponding value of time slice. A process consumes more CPU time out of the allocated time slice, if it is a CPU bound process. On the other hand, an I/O bound process consumes much less portion of the time slice, as it gets blocked after causing an interrupt. The operating system monitors this performance, and depending upon the past behaviour of a process, when a process becomes ready, the operating system computes its new priority and introduces the process to the appropriate queue. If a process consumes full time slice, next time it will be introduced to a queue with lower priority but higher time slice, as it is a CPU bound process. Within any queue, the operating system schedules the processes in a round robin method.

Many UNIX implementations use a variation of the method described above. Take System V Release 3 (SVR3) as an example. In this scheme, the range of priorities is divided into two categories. One category is for the kernel processes and the other for user processes. This is shown in Fig. 13.A.

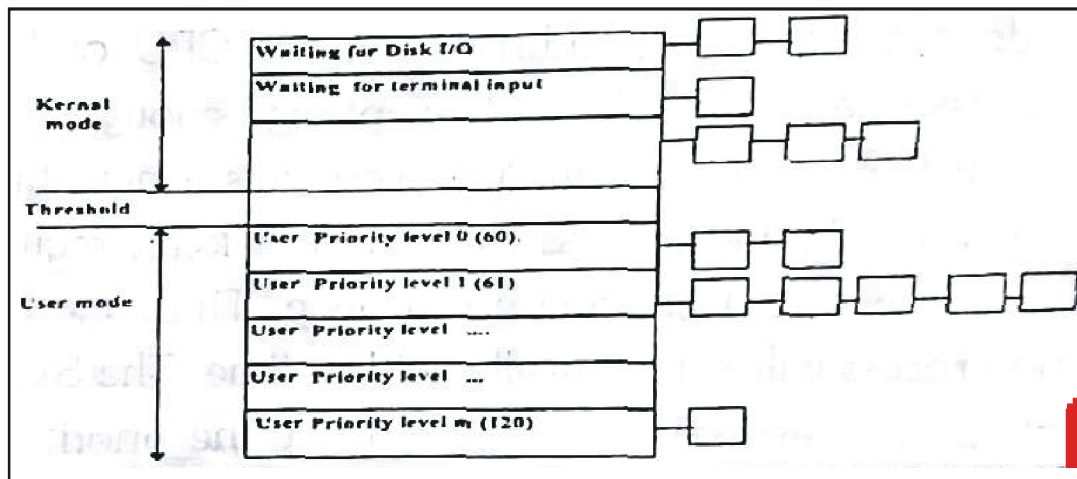


Fig. 13.A – Priority queues for ready processes

The dynamic placement and movement of ready processes in different queues is applicable for only user processes. The kernel processes, though divided into different priority queues, are more 'rigid' in this respect. 'For instance, there is a queue for all the processes which are waiting for a disk I/O. Regardless of the past performance, any process waiting for a disk I/O is always introduced in that queue with a fixed priority level. When a process issues a system call for disk I/O is always introduced in that queue with a fixed priority level. When a process issues a system call for disk I/O and is about to go to sleep, a fixed dependent upon the priority with which it was executing in the user mode before going to "sleep". Within the kernel mode priorities, there are different levels that UNIX maintains, depending upon the criticality or urgency. For instance, there are different priorities between a process waiting for a disk I/O and a process waiting for the terminal input. The question arises as to how the kernel manages the priorities and time slices for user processes in SVR3. The SVR3 adopts a scheme which is slightly different from the one that was studied under "multilevel feedback queues". In this scheme, the time slice does not vary depending upon the CPU or I/O boundness of a process. It can be kept high enough (say, 1 sec) so that the CPU bound process gets a high time slice. The I/O bound process will almost certainly request for an I/O before 1 sec and go to sleep. Thus, the I/O bound process will automatically get less time. The SVR3 adopts an interesting scheme ' to increase the priority of the I/O bound process and lower that of the CPU bound process. There is a priority number (P) associated with a priority level. By convention, the higher priority number denotes lower priority level and vice versa. eg. if there are two processes with priority numbers

10 and 20, the former is a process with actually a higher priority and thus, it will be scheduled earlier. The priority numbers are calculated with this convention in mind as will be seen later.

The threshold priority number (P_0) is normally 60, meaning thereby that priority numbers for all the user processes are 60 or more. (Note that, therefore, a user process with priority number 60 is highest priority user process). The user processes could have priority number 60, 61, 62, ..., etc. upto, say 120. For each number, there can be a queue of processes maintained by creating links between the process table entries of different processes with the same priority number. The kernel processes have priority numbers definitely lower than 60. This is depicted in Fig. 13.A.

The kernel defines a decay function (F) in such a way that if a process uses a lot of CPU time (measured by the number of clock ticks consumed), the value of F becomes high then the kernel calculates priority number (P) of any process in such a way that a higher value of F results in a higher value of P , and thus, actually a lower priority. This is the way a CPU-bound process gets lower priority. Processes with the same priority value are linked together at every context switch. Within the same priority value (P), the kernel selects a process which has not used the CPU for longer duration. This scheme gives the algorithm a kind of fairness which also helps in avoiding the problem of indefinite postponement.

In order to give less priority to the CPU-bound jobs, the kernel adopts an interesting philosophy. This will be illustrated by an example. Figure 13.B shows four processes: P_1, P_2, P_3 and P_4 . The figure also shows a study of the scheduling algorithm, as observed during 4 seconds. The kernel follows the exact steps as described below:

- (i) The kernel allocates a fixed time slice to all the processes. Say, 1 second
- (ii) The clock is adjusted to produce 60 ticks i.e. 60 interrupts in this time slice.
- (iii) The kernel maintains 2 data items for each process. They are priority number (P) and decay function (F). In the beginning (i.e. at time = 0 sec.), the figure shows that the value of P for all the 4 processes is 60 which is the lowest value of P for a user process as we know. The initial value of F for all the processes is 0.
- (iv) The kernel chooses a process with the highest priority, i.e. the lowest value of P . In this case, all the four processes have the same value of P , viz. 60. The kernel

thus, chooses a process which is at the head of the ready queue for that priority. Assume that P1 is such a process.

- (v) Assume that P1 is a CPU-bound process. Hence, it will consume all the 60 clock ticks for the 1 second time slice allocated to it.
- (vi) For each clock tick, the decay function F for that process (P1) is incremented. Hence, at the end of 1 sec., the values of F for the 4 processes are 60, 0 and 0 respectively as shown in row 1 in the figure. At this juncture, the time slice for P1 is over. Hence, the kernel takes over and recalculates F then P for all the processes as given below:

- (i) Now fresh values of F are calculated as follows:
New F is the integer value of (Old F/2).

Hence, the new values of F become 30,0,0 and 0 for the 4 processes respectively. Row 2 reflects these values.

- (ii) Now the new priority number (P) is calculated as follows:

New P is base (i.e. 60) + integer value of (New F/2). Hence, the new values of P for the four processes are 75 [=60+(30/2)],60,60 and 60 respectively. The values of new F as calculated in (vii) are retained as the decay function in the four columns for F shown in Fig. 13.42. Row 3 in the figure depicts this position.

- (iii) At $t=1$, i.e after 1 second, the kernel chooses the next highest priority process with the lowest P. There are again P2,P3 and P4 as 3 candidates all with the value of $P=60$. Remember that the value of P for P1 is 75, and, therefore, it is not the lowest (i.e. the one with the highest priority). Therefore, we have only 3 possible candidates. Assume that P2 is scheduled.

Assume that P2 requests for an I/O after 36 clock ticks after which it goes to sleep. The time slice of 1 second has still 24 more ticks left. The kernel allocates them to the next ready process with the lowest P. (For simplicity, let us assume the context switch time to be 0). There are P3

and P4 as the possible candidates. If P3 is at the head of the chain of ready processes at that priority level, it is chosen, P3 consumes the remaining 24 clock ticks. Row 4 shows the values of F at this juncture (i.e. $t=2$). The F for P1 is still 30 (unchanged from the previous value). Process P2 is still sleeping and thus, it is no more in any queue for ready processes. Thus, F is not calculated for it. F is 24 and 0 for P3 and P4 respectively.

The kernel again calculates the values of new F (by using the formula given in step (vii)) as 15, "Sleeping" 12 and 0. Row 5 depicts this position.

- (iv) The new values of P at $t=2$ are recalculated by the same formula as given in step (viii). These values now are : 67, "Sleeping", 66 and 60 respectively [$67: 60 + \text{integer of } (15/2)$]. Row 6 shows this position.
- (v) After $t=2$, P4 is scheduled because it has the highest priority ($P=60$). It consumes all 60 clock ticks. The values of F at the end of $t=2$ are 15, "Sleeping", 12 and 60 for P1, P2, P3 and P4 respectively (shown in row 7).
- (vi) The kernel calculates the new value of F as 7, "Sleeping", 6 and 30 using the formula discussed earlier. Row 8 depicts this position.
- (vii) The kernel calculates the new values of P using the formula discussed earlier. These values now become 63, NA, 63 and 75 respectively. This is because , $63=60+\text{integer of } (7/2)$, $63=60+\text{integer of } (6/2)$ and $75=60 + \text{integer of } (30/2)$, Row 9 shows this position.
- (viii) At time $t=3$ sec, the kernel has a choice of two processes P1 and P3 because both of them have a priority number of 63. The kernel chooses P1 because it is waiting for the CPU in the "Ready to Run" state longer than P3 (P1 got the CPU at $t=0$, P3 got it at $t= 1$).
- (ix) While P1 is executing, the value of F starts with 7 and keeps on incrementing by 1 with every clock tick. Let us assume that during this period, the I/O for P2 is complete and P2 again enters the queue for ready processes. It has preserved the value of F in the process data structures for it to continue.

- (x) P1 consumes 60 clock ticks. The value of F was 7 in the beginning. It becomes 67 after 60 clock ticks. Therefore, at time $t=4$, the values of old F are 67, 36 (unchanged from before), 6 and 30 (Row 10).
- (xi) The kernel recalculates the new values of F as 33, 18, 3 and 15 as depicted by row 11.

The kernel recalculates the new values of P as shown by row 13-They are 76, 69, 61 and 67. The kernel now chooses P3 and continues.

Note: It is interesting to note that if a process consumes less of CPU time and/ or it does not get the CPU for longer time, its priority number (P) starts approaching the base level of 60. This automatically forces the kernel to schedule it earlier due to its increased priority. Also, if a process consumes larger CPU times and that too more recently, its priority number increases (i.e. priority decreases) and hence, it is scheduled only later.

At any time, there can be only one 'preempted' process, whereas there can be a number of 'ready' processes. Both are candidates for scheduling, as they are not waiting for any I/O. At the context switch, when the kernel recalculates the priorities, it moves the preempted process to the ready process in the appropriate queue before scheduling them. Therefore, the kernel needs to schedule only 'ready' processes hereafter until a process becomes 'preempted' again.

The time slice of 1 second is followed extremely rigidly excepting in one case. If the clock tick occurs while the kernel is executing a code in the critical region which cannot be interrupted, the kernel "remembers" that such a thing had happened and as soon as it comes out of the critical region, it recomputes the new values of F and P.

The above scheme takes care of both CPU and I/O bound processes. It, therefore, gives good response time for on line processes such as text editor: as well as computation bound programs run in the background.

13.4 MEMORY MANAGEMENT

13.4.1 Introduction

Though the memory management scheme varies from implementation to implementation, two schemes are used most extensively in UNIX. One is swapping and the other is demand

paging. The basic principles of memory management were considered in earlier chapters. They are valid for UNIX with only a few variations. Therefore, only these two schemes will be considered in brief.

When the full process image needs to be in the memory before the process can be executed, swapping is used. If a high priority process arrives when the physical memory is insufficient to hold its image, a process with lower priority has to be swapped out. At this time, the entire process image is written onto the portion of the disk known as a 'swap device'. When that process needs to be run again, it has to be 'swapped in' from the swap device to the physical memory.

A point needs to be clarified here. Swapping can co-exist with simple paging. The executable code of a program can still be divided into a number of pages and the physical memory can be correspondingly divided into several page frames the process image in the main memory can spread over a number a physical page frames, which are not necessarily contiguous, and, therefore, requires the page map tables to map the logical or virtual addresses into physical page frames. However, even in this case of Simple paging, the entire process image is swapper out or swapped in. This is what really differentiates swapping or simple paging from demand paging. In the former, the entire process image has to be in the physical memory before execution, whereas in demand paging, execution can start with no page in the memory to begin with The pages are brought in only when 'demanded' through page faults as we have studied earlier.

Initially, UNIX implemented swapping as a memory management system. Swapping limits the maximum size of the process to that of the physical memory. Later implementations such as BSD 4.0 implemented demand paging, thus, removing this limitation. However, it must be remembered that in terms of implementations, swapping is an easier method to implement, whereas demand paging is more difficult. Both swapping and demand paging will now be considered.

13.4.2 SWAPPING

A swap device is a part of the disk. Only the kernel can read data from the swap device or write it back. Normally, the kernel allocates one block at a time to ordinary files, but in the case of a swap device, this allocation is done contiguously to achieve higher speeds for the I/O While performing the functions of swapping in or out. This scheme obviously gives rise to fragmentation and may not use the disk space optimally, but it is still followed to achieve higher speeds.

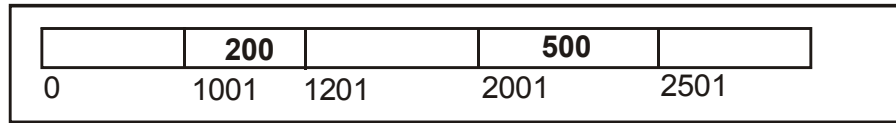


Fig 13.8 Allocated and free blocks

For instance, Fig.13.B shows the allocated (shaded) and free blocks on the swap device. If a process with size=300 blocks is to be swapped out, these blocks will be allocated out of the second free chunk of 500 blocks. The kernel does not decide to allocate 200 blocks from the first free chunk and 100 from the next chunk. The allocation will no longer remain contiguous. If the allocation is non-contiguous, the kernel will have to maintain an index (like inode) to keep track of all the blocks allocated to the swapped process file. Such a process will degrade the I/O performance. When a process has to be swapped in, the kernel will have to go through the index, find the blocks that need to be read, and then read them one by one. As the blocks are likely to be dispersed, disk Read/Write head movement can be erratic and, therefore, time consuming. Even buffering will not be as efficient. In short, if swapping is done very often, the noncontiguous allocation can give rise to tremendous overheads. The swapping is managed by using a simple data structure called 'swap map'. We will now consider a few examples to clarify this.

- (a) At any moment, the kernel maintains a swap map of free blocks available on the swap device. For instance Table T1 shows such a swap map for Fig . 13.B

Starting block no.	No. of free blocks
1001	200
2001	500

Table T1 Initial sap map-(A)

- (ii) After a process of 300 blocks was swapped out as discussed earlier, the swap map will be as shown in Table T2 (Blocks 2001 to 2300 would have been allocated to the swapped file).

Starting block no.	No. of free blocks
1001	200
2201	200

Table TZ Swap map after swapping out (B)

- (i) Assume that the allocated space of 1001 blocks from 0 to 1000 blocks consisted of three processes of 500 blocks (0-499), 400 blocks (500-899) and 101 blocks (900-1000). Assume that the second process of 400 blocks occupying blocks 500 to 899 on the swap device is scheduled and, therefore, is 'swapped in'. Thus, these 400 blocks will become free on the swap, device and allocable for swapping further processes. The swap map now looks as shown below Table T3.

Starting block no.	No. of free blocks
501	400
1001	200
2301	200

Table T3 Swap map after swapping in -(C)

Assume again that the process of 101 blocks (900-1000) is scheduled and therefore, swapped in. The kernel now modifies the swap map as follows Table T4:

Starting block no.	No. of free blocks
501	701
2301	200

Table T4 Swap map after swapping in and coalescing (D)

Note that the kernel has performed the coalescing function as studied earlier.

When the kernel decides to swap a process out in order to make room for a new higher priority process, the kernel follows the steps given below.

- (a) Trace the links from the process table to preregion
- (b) Traverse from preregion to the region tables.
- (c) Decrement reference count of each region.
- (d) Swap out only those regions where this reference count =0 (Typically data, stack and unshared text regions). If a text regions is shared amongst different processes, this count will become 0 only when there is no further user process running that program.
- (e) For such processes where this reference count =0, trace the links to the physical memory addresses (e.g. page map tables in case of paging), and swap out the process, and update the swap maps.
- (f) Figure 12 C shows how a process is swapped out. It is known that the virtual process image can have some gaps between different, regions to allow for growth at run time. These gaps are generated by the compiler itself. In addition, the physical locations where even a single region resides in the physical memory need not be contiguous. However, while swapping, a contiguous image is created after removing all the gaps as shown in the figure. The kernel then updates the swap table.

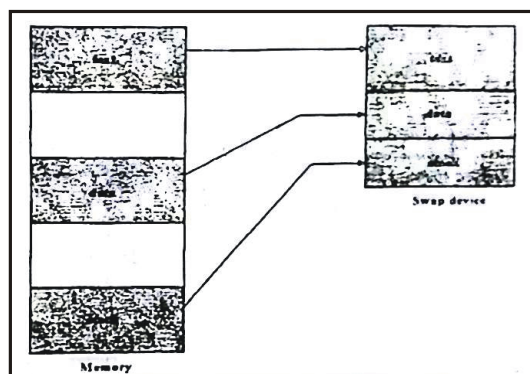


Fig 13.C Swapping out the process image

While swapping in, the kernel consults the memory management data structures (such as page map tables) to find the free memory locations available for swapping in. It then loads the process image from the swap device into the assigned memory locations. While loading the

kernel makes sure that the gaps between the regions kept in the virtual process image for future growth are recreated. The kernel updates the swap map in the end to reflect the swapping in.

13.4.3 DEMAND PAGING

A review of the way demand paging works is given below:

- (i) In demand paging, the process image is divided into equal sized pages and the physical memory is divided into same sized page frames.
- (ii) In the beginning, the entire process image resides on the disk in the 'executable file'. The blocks allocated to this file by the kernel need not be contiguous. When the process starts executing, depending upon the free physical page frames, an equal number of pages are loaded and the execution begins.
- (iii) As the page faults occur, new pages are brought in the physical memory from the executable file.
- (iv) If there is no physical memory left to bring in a new 'demanded' page, an old one has to be replaced. The page replacement algorithm decides which page has to be removed, (There could be a global or local page replacement policy). Normally at 'Least Recently Used (LRU)' algorithm is used to achieve the replacement.
- (v) Having established the page to be replaced, the kernel has to decide whether it can simply be overwritten or it has to be preserved before it is overwritten. If the page was modified (i.e. it was dirty) after loading; (typically pages contain data and stack regions), then it needs to be preserved.
- (vi) If a page is to be preserved, it is written onto the swap file, so that next time the kernel can locate it and load it back.
- (vii) Thus, a page of process can be in the physical memory or on the disk. On the disk, again, it can be in the executable file or the swap file. This information is given by the Page Map Table (PMT). The PMT is, thus used for address translation. The PMT also gives some information about the history of that page e.g. whether it was referenced, whether it was modified and how long it has been a part of the working set. This extra information is useful for the kernel in

implementing the page replacement algorithm.

- (viii) The kernel has to keep track of all the page frames in terms of whether they are free, and if not, the process to which they are allocated. This is done by maintaining another data structure called 'Page Frame Data Table (PFDT)'.

a) Page Map Tables (PMT)

As seen earlier, every region table entry has a pointer to a page map table where the details of all the pages are kept. Each entry in the page map table consists of two parts: Page Table Entry (PTE) and Disk Block Descriptors (DBD).

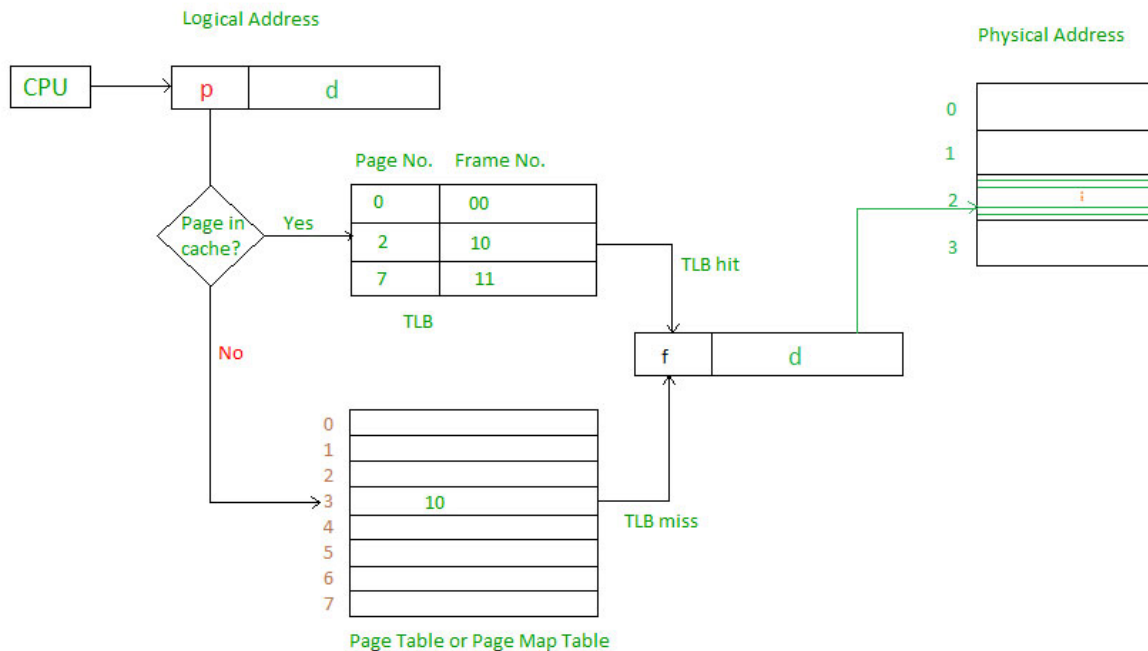


Fig. 13.D Page Map Table

The page map table has as many entries as there are pages in a region. At any time, a page can either be in the physical memory or on the disk. If it is in the physical memory indicated by valid bit =1, the Page Table Entry (PTE) gives the address of the page in the 'page frame number'. If the page is on the disk, the Disk Block Descriptor (DBD) gives its address. The

address on the disk can be specified by the file name and the block number within that file. Again, the desired page can be at two places on the disk. At the very beginning, it will be in the executable file on the disk. When the process is scheduled, it is brought in the memory from the executable file. If and when, subsequently it is swapped out, it will be in the swap file. The swap/executable file indicator in the DBD gives that information. The entries in the PTE and the DBD will now be studied in detail:

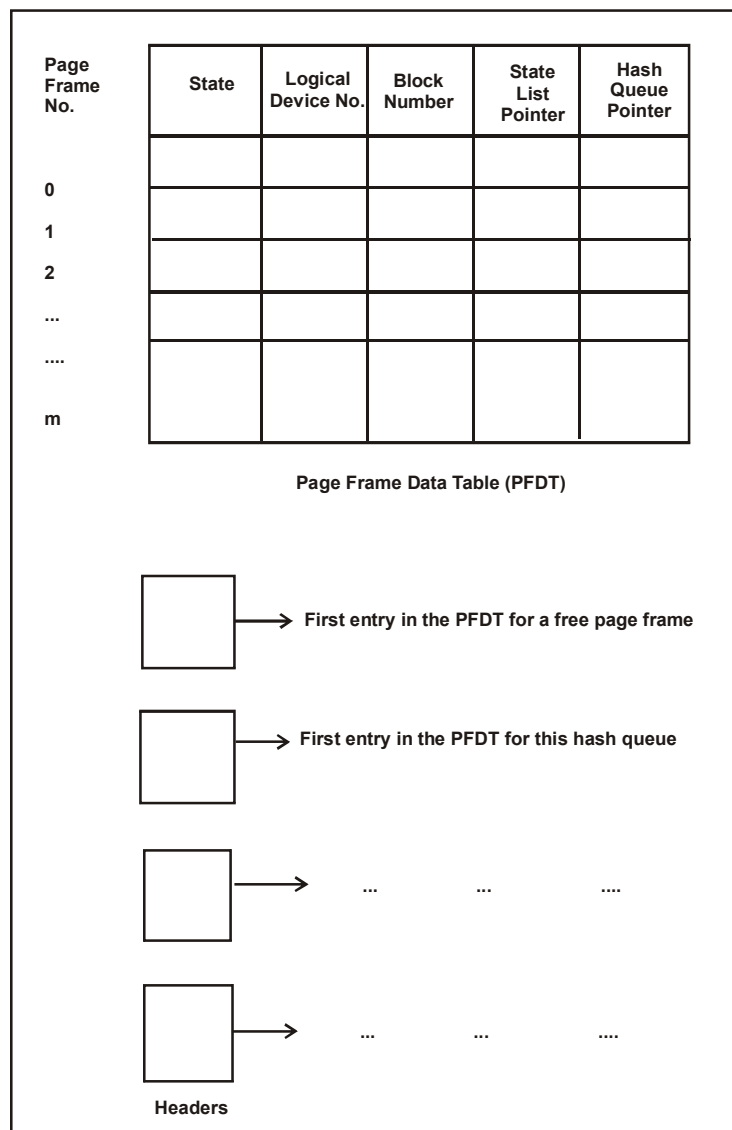


Fig. 13.E depicts the details of PMT and PFDT

(i) Page Table Entry (PTE)

Valid bit: This is ON (= 1) if the page is in the physical memory. It is OFF (=0) if the page is on the disk.

Reference bit: Ideally, whenever a page is referenced, it could be 'time stamped', by additionally defining and using a field in the PTE called 'time of page reference'. The hardware itself could pick up the system time and move it to this field at the time of the page reference. The pages then could be maintained in the ascending order of this time field. To implement Least Recently Used (LRU) algorithm, all that was needed was to pick up the page at the head of the chain. However, this is a time consuming and expensive method.

This is the reason why, another method called 'LRU approximation' is used. The reference bit in the PTE is used for this purpose. Whenever a page is referenced, the hardware, at the time of address translation itself sets this bit ON for that page. At regular intervals, the hardware clears the bits for all the pages. Thus, at any moment, the pages, where this reference bit in the PTE is ON, are the ones most recently referenced. The time duration at which these bits are cleared is again a design issue. If this duration is small, it allows the operating system a fine differentiation amongst the page references, but then the overheads of this method increase. On the other hand, if these bits are cleared after a long interval, the overheads are less but the kernel then cannot differentiate between recently used and most recently used pages during the same interval.

Modify bit: Modify bit is set ON normally by the hardware but as in the reference bit, the software has to take care of this if there is no hardware support. The software implementation is, of course, slow. This bit is set ON, when the contents of a page are changed by any instruction. The hardware detects a write instruction and sets the modify bit ON for the page corresponding to the page number in the address of that instruction.

These days, the compilers usually generate "reentrant" code. This code does not modify itself. Therefore, the pages corresponding to the "text" region of a process normally, will have this bit as OFF permanently as they will never be modified during the execution of a program. However, the pages belonging to the data and the stack regions can change in their contents and therefore, the bits for those changed pages will be set ON.

Modify bit is mainly used while swapping out the pages. If this bit is OFF the page frame

can be made free and can be allocated to a new process without having to write it back to the disk. If the modify bit is ON, the kernel writes back this page to the disk to store the latest copy of that page. At this time, it has to write it back onto the swap file and not the executable file, so that the original contents are not lost. The idea is that next time a program is executed from the beginning, it should start exactly in the same way that it did the first time.

Age bits: Age bits indicate the length of time that a process has been a part of the working set.

Page frame number: This is a physical address of the page frame to be used for address translation, if the valid bit is ON, i.e. the page is in the memory.

13.4.4 DISK BLOCK DESCRIPTOR

The Disk Block Descriptor (DBD) block essentially gives the address on the disk where a block could be found, if it is not in the physical memory.

Swap/Executable file indicator indicates whether the block is on the swap file or executable file.

The Block Number gives the block number within the file as per the indicator. The DBD also contains some other fields as per the implementation.

Using the indicator, the kernel knows which inode to refer to. Using the block number the kernel searches the direct blocks (within the inode itself) or the blocks at different levels of indirection (depending upon the block number) and accesses the block. All this is done only if the valid bit in the PMT is not ON indicating that the desired block is really on the disk.

(b) Page Frame Data Table

The Page Frame Data Table (PFDT) describes the physical page frames. There is one entry in this table for every physical page frame. Given a page frame number, the kernel can directly access the entry for that page frame without any “table search”. The PFDT contains many important fields some of which are described below (refer to Fig.13.E).

- (i) The state of the page frame (free, allocated, on swap or executable file or being loaded, etc).

- (ii) The logical device and block number that contain the copy of that page.
- (iii) A pointers for tree page frame in the list of the PFDT entries i.e. the address of the next free page frame in the PFDT. This is valid only if the page frame is free and thus, allocable. Outside the PFDT there are several headers as shown in Fig. 13.E. One of them is for the “first entry in the PFDT for a free page frame”. The PFDT itself maintains a pointer chain of all the free page frames. Thus, using the header and these pointers, the kernel an access all the tree page frames.
- (iv) A pointer in the list of allocated PFDT entries on a hashed queue, the idea of a bashed queue is simple It helps to improve the search timings. Basically, the kernel needs to maintain a list of all the allocated page frames, and it also needs to know the “source” of that page i.e. where it came from. That is the reason the device number and block number i.e. the address on the disk is kept in the PFDT. Conversely, given the device number and block number of a block, the kernel needs to find out whether that block is already in the -physical memory and if so, where to find it.

For this reason, the device number and block number are used as a “key” to the hashing algorithm. The hashing algorithm could for instance divide this “key” by a predetermined prime number and use the remainder to build the hash queues. All the PFDTs with this remainder =1 could be chained together, those with remainder =2 could be chained together separately, and so on. Each of these pointer chains will have different headers as shown in the figure. The hashing algorithm may vary but the principles and philosophy remain unchanged. Essentially, it helps reduce the search timings. Given a device and block number, the kernel can subject it to the same hashing algorithm. It can find out which has queue that block belongs to. It can then traverse that queue starting with its header to check if that block is still in the physical memory, or it needs to be loaded from the disk after consulting the DBD.

A very important use of this scheme is to avoid extra disk I/O by checking first if the page frame is actually available in the physical memory itself before loading it from the disk. For instance, imagine that an editor is being used by a number of users. As all of them log off or finish their work with the editor, the page frames will be made free and linked into the “free list”.

The page frames associated with the data and stack of each process running that editor will be freed as soon as mat process terminates. However, the page frames of the “text” region

of that editor can be freed only after the last user finished his work. With this editor, making the count field in the text region=0.

Assume that this has happened, but the page frames for the “text” region of that editor are still intact in the main memory. They are actually not been allocated to any other process and consequently they are not overwritten by any other process. These page frames will be linked in both the chains, the chain of free page frames (for further allocation) and the hashed queue. When a new process wants a page frame, and one of these page frames is allocated to it, it is removed from both queues and then it is linked back to the appropriate hash queue, depending upon the new device and block number. However, before that happens, the old hash queue continues.

Imagine that a page frame belonging to the text region of the editor is still not allocated to a new process and is still linked to the old hash queue. Also imagine that a new user now wants to execute this editor again. The kernel will locate the executable file of this editor and the block numbers for the text region for the same. However, before they are actually loaded into the main memory, the kernel goes through the hashing algorithm to check, if the corresponding page frames are still available in the main memory. If any are available, it only reassigns the same and delinks them from the chain of free page frames. This saves a lot of I/O time especially for programs which are used very often.

13.4.5 AN EXAMPLE USING DEMAND PAGING

Let us now take an example of a process to see how the scheme works.

- (a) Assume that a user wants to execute a program
- (b) The kernel creates a process for the same and creates an entry in the process table.
- (c) The kernel accesses the executable file containing the image of that process. It finds its size in terms of required number of pages and creates the page map tables with required number of entries after interfacing with the memory management module. It then creates the region and pregon tables for the process. The PMT entries will be blank at this juncture.

- (d) The kernel then finds out whether the pages belonging to the text regions are already available in the main memory. This is done by using the hashing techniques. From the device number and the block number for each block in the executable file, an appropriate hash queue is searched to check if any page is available. If available, it can be directly assigned to this process. In this case, the page frame numbers in the PTEs of these pages are then updated in the PMT. The valid bits of these pages also are set to ON. It can skip the steps (e), (g) and (h) if the required page is already existing in a page frame.
- (e) If any page does not exist in the main memory, a page has to be loaded from the disk into a free page frame. To do this, the kernel goes through the PFDT chain for free page frames, starting with its header.
- (f) This is the way in which the kernel allocates the page frames to this process (They could amount to less than the total size of the process address space). After the allocation, the kernel removes the page frames from the free list and it chains them in the list of PFDT entries as per the hashing algorithm. It now sets the valid bit ON and other (reference, modify, etc.) bits for these pages OFF in the PTEs.
- (g) It updates the page frame numbers in these PTEs appropriately.
- (h) The kernel then loads the pages from the executable file into those allocated page frames and also updates the DBD entries. It marks the executable/swap file indicator to “executable” for those DBD entries.
- (i) For all the other pages which are still not in the memory, the kernel sets valid bits =OFF, and page frame number =blank to denote that the page is not in the physical memory so that a page fault would occur. It, however, fills up the details of the DBD entries for those pages with the respective device number, block number addresses of the “executable” file, so that those pages could be located on a page fault.

- (j) The process now starts executing. For every instruction, the virtual address consists of two parts: page number (P) and displacement (d). The hardware separates these two parts. The page number (P) is used as an index into the PMT directly. At this juncture, its valid bit is checked. Assume that it is valid (i.e. is in the memory). The page frame number of the PTEs is then extracted and the displacement 'd' is appended to it to get the physical address. The instruction can be completed. The reference bit is then set to ON. If the instruction is a 'write' instruction, the 'modify' bit in the PTE is also set to ON.
- (k) Assume that the CPU encounters an instruction with an address containing an 'invalid' page. This is called "page fault". At this time, the instruction which caused the page fault is abandoned. It is again taken up only when the required page is read into the physical memory. The hardware must have this capability to abandon and restart an instruction (after the page fault processing) to support demand paging. At this juncture, the kernel again finds out whether the page is already in the physical memory by using the hashing algorithm. As discussed earlier, the page may have been used by a process terminated some time ago but the page may not have been removed. If it is not in the physical memory, it goes through the free, list of PFDT entries to check, if a free page frame is available, so that it can be loaded from the disk.
- (l) Assume that a free page frame is available. The kernel grabs it and removes it from the free list. The kernel extracts the device and block number from the DBD of the PMT entry for that page with valid bit OFF. It then loads the contents of that page from that address into the allocated page frame. It also updates the "page frame number" in the PTE slot of that PMT entry. It inserts the device and block numbers in the PFDT entry for that page frame. It then chains that PFDT entry according to the hashing algorithm. It also sets the valid bit to ON and all other bits (reference, modify) to OFF for that page in the PTE slot of the PMT entry.
- (m) At this time, the interrupted and abandoned instruction restarts its execution to find a 'valid' page in the PMT. It can, therefore, consult PTE to extract page frame number and proceed with the address translation as before.

- (n) The kernel monitors the number of free page frames available. When they fall below a minimum level, it invokes a process called a 'Page Stealer Process (PSP)'. The function of this process is to go through all the PMTs and remove the least recently used pages. The PSP also adds these page frames to the free list of PFDT entries. It still maintains such as LRU, so that if that page is referenced again, the page fault processing is faster, as was seen earlier.
- (o) Assume that a new process with a higher priority now wants to run. If that process does not have enough physical memory to run, a few pages from other low priority processes have to be acquired. Assume that our original process loses a few (or all) pages. When this happens, for each page thus removed, the following actions are taken.
 - (i) The valid bit in the PTE of the PMT of that page is set OFF.
 - (ii) The page frames "text" regions are added to the "free" list, still maintaining the hash queue (A copy of the text pages is available on the executable file. Thus, another one is not needed on the swap file).
 - (iii) The unchanged page frames the data and stack regions also can be treated in the same way.
 - (iv) The pages from the data or stack regions, which have been modified, are written onto the swap file. The BBB entry is changed for such pages. The device and block numbers then correspond to those on the swap file. The swap/executable file indicator in the DBD is set to "swap". The page from the data or stack regions, Which have been modified, are written onto the swap file. The DBD entry is changed for such pages. The device and block numbers then correspond to those on the swap file. The swap/executable file indicator in the DBD is set to "swap".
- (p) Assume that after some time, a data page on the swap file is to be accessed. This causes a page fault as seen earlier, since the valid bit in the PTE is OFF. The current instruction is abandoned. The kernel looks at DBD to know that it is on a swap file. It makes sure that the page is already not in any page frame by using the hashing algorithm. If it is not, it grabs a free page frame and reads the

contents into it by using the address in the DBD. After this happens, it updates the valid bit (ON), reference and modify bits (OFF) in the PMT. The kernel removes the page frame from the free list of the PFDT entries. It however, maintains the PFDT entry in the appropriate has queue.

- (q) When the process terminates, the pages belonging to the data and stack regions are released immediately. For pages in the text region, the kernel decrements the reference count. If the count now becomes 0, the pages belonging to the text region are also freed, otherwise they are retained.

13.5 Summary

Process scheduling is the task of selecting a waiting process from the ready queue and allocating the CPU to it. The CPU is allocated to the selected process by the dispatcher. First-come, first-served (FCFS) scheduling is the simplest scheduling algorithm, but it can cause short processes to wait for very long processes. Shortest-job-first (SJF) scheduling is provably optimal, providing the shortest average waiting time. Round-robin (RR) scheduling is more appropriate for a time-shared (interactive) system. RR scheduling allocates the CPU to the first process in the ready queue for q time units, where q is the time quantum. After q time units, if the process has not relinquished the CPU, it is preempted, and the process is put at the tail of the ready queue.

13.6 Review Questions

1. What is Process Scheduling?
2. Define Decay Function
3. What is Priority number?
4. What is meant by kernel?
5. What is swapping?
6. Define Demand Paging
7. What is Page Map Tables?
8. What is PTE?
9. What is DBD?
10. What is meant by valid bit?
11. What is Disk Block Descriptor?

MODEL QUESTION PAPER**PART-A (10 * 2= 20 Marks)****Answer any 10 questions**

1. Define Operating System.
2. What are the functions of a dispatcher?
3. Give two examples for non-sharable resource.
4. What are the two types of semaphores?
5. What is a logical address?
6. What is seek time?
7. What is the role of a directory structure?
8. Mention any two file types.
9. Which system call creates a new process in UNIX?
10. What is name resolution?
11. Distinguish between sequential access and direct file access?
12. What is dynamic loading?

PART-B (5 * 6= 30 Marks)**Answer any 5 questions**

13. Briefly discuss about the two common models of communication.
14. Discuss the following:
 - (a) Deadlock
 - (b) Starvation
15. Briefly discuss about the types of fragmentation. Also write the solution for solving the problem of external fragmentation.
16. Write short note on thrashing.
17. List out various file attributes.
18. Explain any one of the disk scheduling algorithms with an example.
19. Briefly explain the file I/O structure of Windows NT.

PART-C (3 *10 = 30 Marks)**Answer any 3 questions**

20. Describe the shortest – job-first CPU scheduling algorithm and compare this with priority scheduling.
21. What are the methods used for handling deadlocks? Explain any one in detail.
22. Explain the internal and external fragmentation with respect to memory management.
23. Give an overview of paging.
24. Explain the kernel I/O subsystem.